© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

NÄTVERK

Av Robin Bräck

Utgåva 2026-1

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Licensinformation

© 2025 Robin Bräck. Detta verk, inklusive text och illustrationer, är skyddat enligt lag om upphovsrätt. Illustrationerna i boken är framtagna med hjälp av artificiell intelligens (AI) under författarens redaktionella överseende och utgör en integrerad del av det samlade verket.

Verket tillhandahålls under licensen Creative Commons Erkännande-Ickekommersiell-DelaLika 4.0 Internationell (CC BY-NC-SA 4.0). Detta innebär att du har frihet att dela och anpassa innehållet så länge du anger källan på ett korrekt sätt, inte använder materialet för kommersiella ändamål och sprider eventuella bearbetningar under samma licensvillkor. Fullständiga licensvillkor finns tillgängliga på https://creativecommons.org/licenses/by-nc-sa/4.0/.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Index

Innan du börjar läsa.....4
Terminologi: WLAN vs “Wi-Fi”.....4
1 Vad är ett nätverk?.....4
1.1 Vad är ett nätverk?.....4
1.2 Nätverkskomponenter och topologier.....12
1.3 Protokoll och adressering.....24
1.4 IP: Nätverkets Adressering och Logik.....25
1.5 Kommunikation i nätverk – Ethernet och WLAN.....40
2 Planering och design av nätverk.....55
2.1 Mål och krav.....55
2.2 VLAN och segmentering.....76
2.3 Routing.....93
2.4 DHCP, DNS och NAT.....111
2.5 Vanliga nätverksangrepp och hur vi försvarar oss.....128
3 Nätverksadministration.....148
3.1 Grunder i nätverksadministration.....148
3.2 Verktyg: ping, traceroute, ipconfig, netstat etc.....174
3.3 Dokumentation, etikett, felsökning.....194
3.4 Introduktion till CLI (inkl. Cisco-kommandon).....201
3.5 Windows Server – Domäner och AD.....209
3.6 Ubuntu Server och Linux i nätverk.....221
3.7 MDM och klienthantering.....228
4 Säkerhet och övervakning.....238
4.1 Nätverkssäkerhetens grunder.....238
4.2 Åtkomstkontroll och behörigheter.....247
4.3 Övervakning och loggning.....254
4.4 Incidenthantering och backup.....262
5 Fördjupningsdel.....273
5.1 Virtualisering – plattformar och gemensamma mönster.....273
5.2 Fördjupning: Windows Server och GPO.....284
5.3 Fördjupning: Linux-server och policy-styrning.....295
5.4 MDM i praktiken.....304
5.5 Framtiden för nätverk.....316
Appendix I – Begrepp.....324
Appendix II - CLI-REFERENS.....334

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Innan du börjar läsa

Terminologi: WLAN vs “Wi-Fi”

I den här boken använder vi **WLAN** som samlingsnamn för trådlösa lokala nät enligt **IEEE 802.11** (radiogränssnitt, SSID, autentisering, kanaler, m.m.). Begreppet **”Wi-Fi”** är egentligen ett **certifieringsprogram och varumärke** från **Wi-Fi Alliance** som garanterar kompatibilitet mellan produkter som implementerar 802.11. Många operativsystem och menyer skriver ändå “Wi-Fi” om trådlöst nätverk, men vi använder konsekvent **WLAN** för själva tekniken för att vara tydliga – inte minst för att undvika förväxling med **VLAN** (virtuella LAN), som är något helt annat.

Som orientering: **Wi-Fi 4/5/6/6E/7** motsvarar **802.11n/ac/ax/ax (6 GHz)/be**. Att en enhet är “Wi-Fi-certifierad” betyder att den klarar interoperabilitetstesterna för vissa 802.11-funktioner, inte att “Wi-Fi” är ett annat protokoll än 802.11.

1 Vad är ett nätverk?

1.1 Vad är ett nätverk?

Ett nätverk är när två eller flera enheter kopplas samman för att kunna dela information eller resurser, till exempel en skrivare, en server eller en internetuppkoppling. Det kan handla om något så enkelt som två datorer i ett hem, eller något så avancerat som ett globalt nätverk av servrar som driver tjänster som Google, YouTube eller Microsoft Teams.

I dagens samhälle är nätverk en grundläggande del av vår infrastruktur – de finns i skolor, hem, företag, sjukhus, fabriker och nästan överallt där teknik används. Vi använder nätverk när vi skickar ett mejl, surfar på webben, jobbar i molntjänster eller spelar spel online.

1.1.2 Nätverkstyper

Ett nätverk kan se ut på många olika sätt beroende på var det används, hur stort det är och vad det ska göra. För att kunna planera, förstå och felsöka nätverk behöver vi känna till de vanligaste nätverkstyperna.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

LAN – Local Area Network

Ett LAN är ett lokalt nätverk som kopplar ihop enheter i ett begränsat område, ofta inom en och samma byggnad. Det kan till exempel vara datorer i ett klassrum, skrivare i ett kontor eller en server i skolans teknikrum. LAN är snabba, stabila och har låg fördröjning.

LAN används ofta med switchar och kabeldragning, men kan även använda trådlösa tekniker (WLAN).

> ***Exempel****: I en skola har varje klassrum flera datorer anslutna till ett LAN. Datorerna kommunicerar med en central server som hanterar inloggningar och filer.*

WAN – Wide Area Network

När ett nätverk sträcker sig över stora geografiska avstånd, t.ex. mellan städer, länder eller kontinenter, kallas det ett WAN. Internet är världens största WAN – ett globalt nätverk som kopplar ihop miljoner LAN.

WAN bygger ofta på hyrda fiberförbindelser, satellit, radiolänkar eller mobilnät. Kommunikation över WAN är långsammare och dyrare än inom LAN, men möjliggör fjärrarbete, molntjänster och globala företagssystem.

> ***Exempel***: *En bank har kontor i olika länder som är ihopkopplade via ett WAN. När du loggar in på din internetbank är det detta nätverk som gör att du når din kontoinformation, oavsett var servern står.*

MAN – Metropolitan Area Network

MAN är en typ av nätverk som används i en hel stad eller ett större samhälle. Det är vanligast inom offentlig sektor, t.ex. när skolor, vårdcentraler eller myndigheter kopplas samman via ett stadsnät. MAN byggs ofta ut av kommunala eller kommersiella operatörer och kan fungera som bas för WAN-anslutningar.

> ***Exempel***: *En kommun har ett eget stadsnät där skolor och äldreboenden är ihopkopplade. All trafik går i hög hastighet genom detta MAN innan det går vidare ut till Internet.*

PAN – Personal Area Network

PAN är det minsta nätverket och används av en enskild person – ofta för att koppla samman mobila enheter. Det kan bestå av din mobil, surfplatta,

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

smartwatch och trådlösa hörlurar. Kommunikation sker ofta via Bluetooth eller WLAN i ad-hoc-läge.

> ***Exempel***: När du delar internet från din mobil till en laptop via en personlig hotspot skapar du ett PAN.

1.1.3 Peer-to-peer vs klient–server

Hur enheter är organiserade i ett nätverk har stor betydelse för hur effektivt och säkert det fungerar. De två vanligaste modellerna är peer-to-peer (P2P) och klient–server.

Peer-to-peer (P2P)

I ett P2P-nätverk är alla datorer jämbördiga. Det finns ingen server, utan varje enhet kan både skicka och ta emot filer direkt från andra datorer. Det här är enkelt att sätta upp och kräver ingen särskild serverprogramvara.

Det är vanligt i små hemmiljöer eller för tillfälliga delningar – som när man kopplar ihop datorer via Bluetooth eller delar ut en mapp i Windows.

Men P2P har nackdelar:

- Det blir snabbt rörigt i större nätverk
- Det är svårt att hantera användarrättigheter
- Det är ofta osäkert, eftersom det saknas central styrning

> Tänk dig att varje dator är både chef och anställd – ingen har överblick.

Klient–server

I klient–server-nätverk finns en eller flera servrar som tillhandahåller olika tjänster: inloggning, filhantering, utskrift, e-post, databaser, och så vidare. Alla andra datorer fungerar som klienter – de begär resurser från servern.

Denna modell används i nästan alla professionella nätverk – från skolor och företag till sjukhus och universitet.

Fördelar:

- Central administration (konton, behörigheter, säkerhet)
- Lätt att skapa backup och underhålla systemen

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- Bättre prestanda och skalbarhet

  Tänk dig en server som en bibliotekarie – klienterna frågar efter resurser, och servern bestämmer vem som får vad.

1.1.4 OSI- och TCP/IP-modellen – en introduktion

När du öppnar en webbläsare och surfar till en hemsida sker flera dolda processer i bakgrunden. För att förstå hur dessa fungerar använder vi modeller som OSI-modellen och TCP/IP-modellen. De hjälper oss att se hur data går från en applikation till en annan – genom kablar, routrar och switchar – och hela vägen till mottagaren.

OSI-modellen

OSI-modellen är en teoretisk referensmodell som delar upp kommunikation i sju lager. Varje lager ansvarar för en specifik funktion i dataöverföringen.

Lager	Namn	Funktion
7	Applikation	Programmet du använder (t.ex. webbläsare)
6	Presentation	Kodning, kryptering, formatering
5	Session	Startar och hanterar sessioner
4	Transport	Säkerställer att data kommer fram korrekt
3	Nätverk	Sköter adressering och routing (IP)
2	Datalänk	Hanterar MAC-adresser, paketering inom LAN
1	Fysiskt	Kablar, signaler, elektriska pulser

OSI är särskilt användbar vid felsökning, eftersom den gör det möjligt att isolera problem. Om nätverkskabeln är urkopplad är det lager 1 (fysiskt) som är felkällan. Om DNS inte fungerar är det lager 7 (applikation).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

TCP/IP-modellen

TCP/IP-modellen används praktiskt i dagens Internet och datanätverk. Den är enklare än OSI och innehåller fyra lager:

1. **Applikation** – Appar och protokoll (HTTP, SMTP, DNS)
2. **Transport** – Kontrollerar flöde och felhantering (TCP/UDP)
3. **Internet** – Ansvarar för routing och adresser (IP, ICMP)
4. **Nätverksåtkomst** – All fysisk och elektrisk överföring (Ethernet, WLAN)

TCP/IP är grunden för all modern kommunikation online och är därför extremt viktig att förstå i praktiken.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Vad är ett nätverk?
2. Vad står LAN för, och var används det?
3. Ge ett exempel på när ett PAN används.
4. Vad skiljer peer-to-peer från klient–server?
5. Varför är klient–server säkrare än P2P?
6. Vad heter lagret som hanterar IP-adresser i OSI-modellen?
7. Vad är TCP/IP-modellen främst anpassad för?
8. Nämn två fördelar med att använda servrar.
9. Vilket lager i OSI hanterar fysiska signaler?
10. Vad är den största skillnaden mellan OSI och TCP/IP?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

Cloudflare (The OSI Model Explained) – https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/

GeeksforGeeks (Types of Computer Network) – https://www.geeksforgeeks.org/types-of-computer-network/

Cisco (What is a Network?) – https://www.cisco.com/c/en/us/products/security/what-is-a-network.html

IBM (Networking: A Complete Guide) – https://www.ibm.com/cloud/learn/networking-a-complete-guide

Wikipedia (Internet Protocol Suite / TCP/IP model) – https://en.wikipedia.org/wiki/TCP/IP_model

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

1.2 Nätverkskomponenter och topologier

1.2.1 Routrar, switchar, accesspunkter och brandväggar

För att ett nätverk ska fungera behövs rätt komponenter – alltså fysisk utrustning som kopplar samman enheter och styr hur trafiken rör sig. Det här kapitlet ger dig en överblick över de viktigaste enheterna du kommer att arbeta med i ett nätverk.

Router

En router kopplar samman olika nätverk med varandra. Den används ofta för att ansluta ett LAN till ett WAN, till exempel när ett hemnätverk kopplas till Internet. Routern ansvarar för att hitta den bästa vägen för varje datapaket att ta, baserat på IP-adresser.

En router kan vara både en så kallad hårdvarurouter, som är speciellt anpassad för ändamålet. Det kan likväl vara en så kallad mjukvarurouter, ett operativsystem som du kan installera på vilken dator som helst, som agerar router.

I skol- eller företagsnätverk används ofta mer avancerade routrar eller brandväggssystem (som OPNsense eller Cisco ISR), medan hemnätverk ofta använder enklare kombinerade enheter som innehåller router, switch, accesspunkt och brandvägg i samma låda.

Switch

En switch används för att koppla samman flera enheter inom ett lokalt nätverk (LAN). Den tar emot datapaket och skickar dem vidare till rätt mottagare genom att läsa av varje enhets MAC-adress. Switchar bygger upp en MAC-tabell för att hålla reda på vilka enheter som finns kopplade till vilka portar.

En switch arbetar på lager 2 i OSI-modellen (datalänklagret), men det finns även Layer 3-switchar som har vissa routingfunktioner.

Switchar kan vara:

- **Ohanterade (unmanaged)** – enklare, billigare och används i små nätverk.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Hanterade (managed)** – ger administratören kontroll över VLAN, hastigheter, portövervakning och mer.

I en modern arkitektur används ofta **Layer 3-switchar** som kombinerar denna snabba lager 2-hantering med förmågan att utföra routing mellan de olika virtuella nätverken. Istället för att skicka all trafik till en extern router kan switchen själv flytta data mellan exempelvis VLAN 10 och VLAN 20 direkt i hårdvaran, vilket minimerar latensen och avlastar nätverkets kärna.

Accesspunkt (WLAN AP)

En accesspunkt gör det möjligt för trådlösa enheter att ansluta till ett LAN genom ett WLAN (Wireless LAN). Accesspunkten tar emot data via radio och skickar vidare den till det trådbundna nätverket – eller tvärtom.

I mindre nätverk är accesspunkten ofta inbyggd i routern. I större installationer, som skolor, används istället fristående accesspunkter som är monterade i tak eller väggar och kopplade till switchar via Ethernet.

Det är viktigt att accesspunkter är rätt placerade för att ge jämn täckning, särskilt i miljöer där många klienter rör sig över stora ytor.

Brandvägg

En brandvägg är en komponent – fysisk eller mjukvarubaserad – som filtrerar trafiken till och från nätverket. Den kan blockera eller tillåta trafik baserat på regler som administratören definierar.

Brandväggar kan arbeta med:

- IP-adresser
- Portnummer
- Protokoll (t.ex. TCP, UDP, ICMP)
- Innehåll (t.ex. webbfilter, antivirus, IDS/IPS)

En brandvägg är ett grundläggande skydd och används både på nätverksnivå (gateway-brandvägg) och direkt i operativsystem (lokal brandvägg).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

1.2.2 Nätverkskort, kablar och portar

För att kunna kommunicera i ett nätverk behöver varje enhet ett sätt att ansluta till infrastrukturen – det vill säga ett nätverkskort och en fysisk koppling via kabel eller trådlös förbindelse. Här tittar vi på grunderna.

Nätverkskort (NIC)

Ett nätverkskort (Network Interface Card) är hårdvaran som gör att en dator, server eller annan enhet kan kommunicera över ett nätverk. Nätverkskort finns i två huvudtyper:

- **Ethernetkort** – används för trådbundna anslutningar via RJ-45 och switch.
- **WLAN-kort** – används för trådlös anslutning till en accesspunkt (WLAN).

Varje nätverkskort har en inbyggd **MAC-adress** – en unik identifierare som används i LAN-kommunikation. MAC-adressen är fast programmerad och används av switchar för att avgöra vart data ska skickas.

I servermiljöer används ofta nätverkskort med flera portar eller stöd för 1/10/40/100 Gbit/s och VLAN-tagging.

Kablar

Kabeln utgör det fysiska mediet som bär den elektriska eller optiska signalen mellan nätverkets olika enheter. Valet av kabeltyp direkt påverkar nätverkets prestanda gällande hastighet, räckvidd och motståndskraft mot yttre störningar. De vanligaste kabeltyperna som används i moderna installationer är:

Twisted Pair (TP)

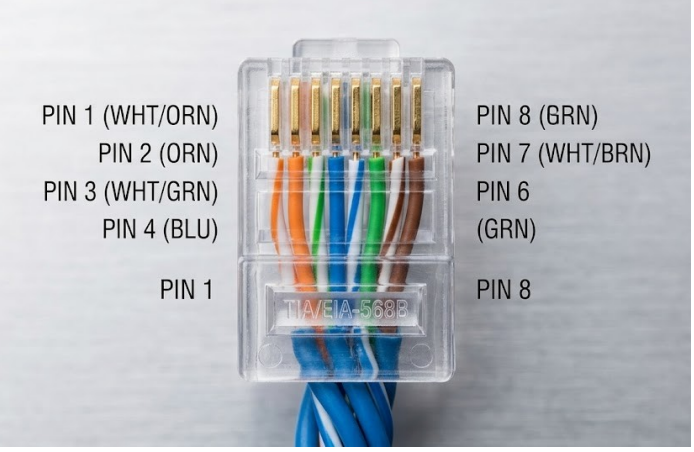
Denna kabeltyp används i nästan alla lokala nätverk (LAN). Den består av åtta ledare organiserade i fyra par, vilka är tvinnade kring varandra för att minimera elektromagnetiska störningar (crosstalk).

Beroende på kraven används kategorier som Cat5e, Cat6 eller Cat6a. För att en TP-kabel ska fungera korrekt krävs en noggrann kontaktering (termination) i en RJ45-kontakt eller i ett uttag. Denna process följer standardiserade färgscheman enligt **TIA/EIA-568A** eller **TIA/EIA-568B**, där den senare är den mest använda standarden i nya installationer.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Vid kontaktering av en kontakt används ett skalverktyg (wire stripper) för att försiktigt avlägsna det yttre höljet utan att skada ledarna, varpå ledarna rätas ut och placeras i rätt ordning:

- Orange-vit
- Orange
- Grön-vit
- Blå
- Blå-vit
- Grön
- Brun-vit
- Brun.



Efter att ledarna klippts till rätt längd och förts in i kontakten används ett pressverktyg (crimping tool) för att fixera ledarna mot kontaktens bleck.

En tekniskt korrekt kontaktering kräver att kabelns yttre hölje kläms fast inne i kontakten för att ge dragavlastning, samt att ledarnas tvinning bibehålls så nära kontaktytan som möjligt för att bibehålla kabelns kategoriklassificering.

Som ett sista steg valideras anslutningen alltid med en kabeltestare (cable tester) för att verifiera att alla ledare har elektrisk kontakt och sitter i rätt ordning.

Kapacitet och räckvidd för TP-kabel

När man väljer kategori på nätverkskabeln är det främst den önskade överföringshastigheten (throughput) och det fysiska avståndet som styr valet. En gyllene regel inom Ethernet-standarderna för kopparkabel är den maximala segmentlängden på **100 meter**. Detta avstånd inkluderar vanligtvis 90 meter fast installation i väggar och totalt 10 meter patchkablar i båda ändar. Om detta

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

avstånd överskrids ökar risken för signalförsvagning (attenuation) och paketförluster, vilket sänker nätverkets prestanda drastiskt.

För att ge en tydlig överblick över de vanligaste kategoriernas förmåga kan följande tabell användas som referens:

Kategori	Max hastighet	Max längd vid max hastighet	Typisk användning
Cat5e	1 Gbps	100 meter	Standard i äldre installationer och hemmanätverk.
Cat6	1 Gbps / 10 Gbps	100 m (1 Gbps) / 37–55 m (10 Gbps)	Klarar 10 Gbps över korta avstånd, bra för mindre kontor.
Cat6a	10 Gbps	100 meter	Standard för moderna professionella installationer och serverrum.
Cat7/8	25–40 Gbps	30 meter	Används främst i datacenter för korta, extremt snabba länk

> Det är värt att notera att även om en Cat6-kabel tekniskt sett kan hantera 10 Gbps, så är den inte optimerad för detta över full längd. För installationer där man planerar för framtida 10 Gbps-hastigheter i hela fastigheten är därför Cat6a det rekommenderade valet, då den har en tätare tvinning och bättre skydd mot överhörning (crosstalk), vilket möjliggör full hastighet upp till den maximala längden på 100 meter.

Fiberoptisk kabel (fiber optics)

Används främst för ryggradsnät (backbone), anslutningar mellan byggnader eller miljöer som kräver extremt hög bandbredd och låg latens. Informationen överförs i form av ljuspulser genom kärnor av ultrarent glas eller plast. Den största tekniska fördelen med fiberoptik är att den är helt immun mot elektromagnetiska störningar (EMI), vilket gör den oumbärlig i industriella miljöer och vid dragning parallellt med högspänningskablar. Dessutom har

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

fiber betydligt lägre dämpning (attenuation) än kopparkabel, vilket möjliggör kommunikation över mycket långa avstånd utan behov av signalförstärkare.

Fiberoptik delas in i två huvudkategorier beroende på hur ljuset färdas i kärnan:

- **Multimode (MMF):** Har en relativt bred kärna som tillåter flera ljusstrålar (moder) att färdas samtidigt. Detta orsakar dock tidsdispersion över längre sträckor, vilket begränsar räckvidden. Multimode är standardvalet för kortare avstånd inom byggnader eller datacenter tack vare billigare sändarhårdvara (LED eller VCSEL).
- **Singlemode (SMF):** Har en extremt tunn kärna där endast en ljusstråle kan färdas. Detta eliminerar dispersion och gör det möjligt att skicka data tiotals mil med mycket hög precision med hjälp av lasrar. Singlemode är standard för stadsnät, operatörsnät och campus-anslutningar mellan geografiskt skilda byggnader.

Till skillnad från kopparkabel, där kategorin (t.ex. Cat6) sätter ett hårt stopp vid 100 meter, styrs fiberoptikens räckvidd av fiberkvaliteten (t.ex. OM-klasser för multimode) och vilken typ av sändtagare (transceiver) som används.

Typ	**Standard**	**Max hastighet**	**Typisk räckvidd**	**Användningsområde**
Multimode	OM3	10 Gbps	300 meter	Äldre LAN och korta länkar.
Multimode	OM4 / OM5	40 / 100 Gbps	100–400 meter	Moderna datacenter och backbone.
Singlemode	OS2	10–400 Gbps	10–40 km+	Stadsnät och långdistanslänkar.

Vid installation av fiber är även valet av kontakt (connector) avgörande för att minimera dämpning. De vanligaste typerna i lokala nätverk är **LC** (Lucent Connector), som är liten och kompakt för hög portdensitet i switchar, samt **SC** (Subscriber Connector), som ofta återfinns i äldre installationer eller vid avlämningspunkter från operatörer.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Koaxialkabel (BNC)

Denna kabeltyp förekommer numera sällan i nya Ethernet-installationer men används fortfarande inom specifika områden som kabel-TV (DOCSIS), äldre övervakningssystem (CCTV) och vissa typer av stadsnät.

En kabels kvalitet och skärmning (shielding) är avgörande faktorer, särskilt vid längre kabeldragningar. Medan oskärmad kabel (UTP) är standard i de flesta kontorsmiljöer, ställer miljöer med hög elektrisk störningsnivå krav på skärmade kablar (STP/FTP) för att garantera signalintegriteten.

Portar

En port är det fysiska gränssnittet där du kopplar in en kabel i en enhet. Vanliga porttyper i nätverk är:

- **RJ-45** – används för TP-kablar. Nästan alla nätverkskort, switchar och routrar har denna port.
- **SFP/SFP+** – moduler för fiberoptiska anslutningar. Vanliga i servrar, switchar och nätverksbackbones.
- **USB-C / Lightning** – används ibland för att dela internetanslutningar eller skapa personliga nätverk (PAN), men inte i traditionella LAN.

Portar kan stödja olika hastigheter, till exempel 10/100/1000 Mbit/s (Gigabit Ethernet), och kan ha stöd för PoE (Power over Ethernet) – vilket gör det möjligt att driva t.ex. accesspunkter via nätverkskabeln.

1.2.3 Vanliga topologier

Topologi beskriver hur nätverksenheterna är organiserade – både fysiskt och logiskt. En topologi kan påverka prestanda, skalbarhet, tillförlitlighet och kostnad. Det finns flera klassiska topologier som är bra att känna till.

Stjärntopologi

I en stjärntopologi är alla enheter anslutna till en central punkt, vanligtvis en switch. Detta är den vanligaste topologin i moderna nätverk, eftersom den är lätt att bygga ut och felsöka.

Om en kabel eller en klient går sönder påverkas inte övriga enheter. Men om switchen går ner slutar hela nätverket att fungera.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

> *Stjärntopologi är grunden för alla moderna Ethernet-baserade LAN.*

Bustopologi

Bustopologi innebär att alla enheter är kopplade längs en enda gemensam kabel (bussen). Denna topologi användes ofta med koaxialkabel i äldre nätverk, där varje enhet behövde en T-koppling och en terminator i vardera ände.

Den är billig, men har många nackdelar: dålig skalbarhet, svår felsökning och känslighet för avbrott.

> *Bustopologi är idag föråldrad och används endast i vissa specialmiljöer.*

Ringtopologi

I en ringtopologi kopplas varje enhet till två andra, vilket bildar en sluten slinga. Data skickas runt i ett bestämt varv tills det når rätt mottagare. I vissa ringnät användes **token passing** för att bestämma vem som fick skicka.

Ringtopologi var vanliga i äldre nätverk (t.ex. Token Ring), men används inte längre i moderna datanät. Dock lever idén vidare i vissa fiber- och transportnät där redundans är viktig.

Mesh-topologi

I en mesh-topologi är varje nod kopplad till flera andra. Detta ger hög redundans och tillgänglighet. Mesh används bland annat i trådlösa mesh-nätverk i hem och företag, i backbone-nät, och i datacenter.

Det finns:

- **Full mesh** – alla noder kopplade till alla andra (dyrt och komplext)
- **Partial mesh** – utvalda noder har flera förbindelser

> Mesh-topologi ger god tolerans mot fel men är dyr att implementera i större skala.

Hybridtopologi

De flesta riktiga nätverk använder en blandning av flera topologier – t.ex. stjärna internt och mesh i överliggande struktur. En hybridtopologi

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

kombinerar fördelar från flera olika modeller och används för att skapa flexibla och skalbara nät.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Vad är skillnaden mellan en switch och en router?
2. Vilken uppgift har en accesspunkt i ett WLAN?
3. Vad är MAC-adress och varför är den viktig?
4. Vilken typ av kabel används mest i moderna LAN?
5. När används fiberoptik istället för koppar?
6. Vad är en brandvägg och vad används den till?
7. Nämn en fördel och en nackdel med stjärntopologi.
8. Varför används inte bustopologi längre i moderna nätverk?
9. Vad är skillnaden mellan full mesh och partial mesh?
10. Vad menas med hybridtopologi?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

1. GeeksforGeeks (Network Devices: Hub, Switch, Router, Gateway, Bridge) – https://www.geeksforgeeks.org/network-devices-hub-switch-router-gateway-bridge/
2. CompTIA (Network Topologies Explained) – https://www.comptia.org/content/articles/network-topologies
3. Diffen (Router vs. Switch: Key Differences) – https://www.diffen.com/difference/Router_vs_Switch
4. Cisco (What is a Network Switch?) – https://www.cisco.com/c/en/us/products/switches/what-is-a-network-switch.html
5. Wikipedia (Network Topology) – https://en.wikipedia.org/wiki/Network_topology

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

1.3 Protokoll och adressering

1.3.1 Varför protokoll?

Protokoll är **regler** som gör att olika system kan förstå varandra. De definierar hur data ska paketeras, adresseras, överföras och kontrolleras för fel. I praktiken arbetar flera protokoll tillsammans i en **protokollstack** (t.ex. TCP/IP), där varje lager har sin uppgift: från fysisk överföring till applikationer som webbläsare och e-post.

Ett vanligt felsökningsgrepp är att "gå neråt i stacken" tills du hittar lägsta lagret där felet uppstår.

Exempel: *En webbplats laddar inte. Du kontrollerar först om applikationen (HTTP) svarar, sedan om transport (TCP/443) etablerar förbindelse, därefter om Internetlagret (IP) ruttnar rätt, och slutligen om nätverksåtkomst (Ethernet/WLAN) har länk.*

1.3.2 TCP och UDP (transportlagret)

TCP (Transmission Control Protocol) är **förbindelseorienterat**: trevägshandskakning (SYN–SYN/ACK–ACK), kvittenser och omsändning vid paketförlust. Det passar där **tillförlitlighet** är viktigt: webben (HTTP/HTTPS), e-post (SMTP/IMAP), filöverföring (SFTP/FTPS).

UDP (User Datagram Protocol) är **förbindelselöst**: inget handskak, inga kvittenser. Fördel: låg latens och mindre overhead. Vanligt för realtidsapplikationer som VoIP, vissa spel och DNS-frågor.

Portnummer identifierar vilken tjänst som ska ta emot datan. "Välkända portar" (0–1023) används av standardtjänster, "registrerade" (1024–49151) av applikationer, och "dynamiska/privata" (49152–65535) väljs oftast som källport av klienter.

> **Exempel:** *Din klient öppnar en HTTPS-anslutning från källport 51234 till serverns port 443. Svaret går tillbaka till 51234.*

1.3.3 Vanliga infrastrukturprotokoll

Se även 2.4 DHCP, DNS och NAT för detaljer och felsökning.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **ARP (Address Resolution Protocol)** – översätter **IP → MAC** inom LAN. Nödvändigt för att skicka Ethernet-ramar till rätt nätverkskort.
- **ICMP (Internet Control Message Protocol)** – skickar kontrollmeddelanden (t.ex. *ping*). Bra för diagnos, inte för applikationsdata.
- **DHCP (Dynamic Host Configuration Protocol)** – tilldelar automatiskt IP-adress, nätmask, gateway och DNS till klienter.
- **DNS (Domain Name System)** – översätter **namn → IP** (t.ex. *example.se → 93.184.216.34*). Kritisk för nästan all applikationstrafik.
- **NTP (Network Time Protocol)** – synkroniserar tid mellan system; viktigt för loggar, Kerberos och certifikat.

Blockeras DNS eller DHCP i fel segment får klienter snabbt "mystiska" problem (ingen adress, eller kan inte nå resurser trots nätverkslänk).

1.4 IP: Nätverkets Adressering och Logik

En IP-adress identifierar ett nätverksinterface så att paket kan levereras rätt, ungefär som gata + postnummer för post. Det finns två versioner i bruk: IPv4 och IPv6. IPv4 har 2^{32} = 4 294 967 296 möjliga adresser; i praktiken färre för vanliga värdar eftersom många intervall är reserverade (t.ex. privatnät, loopback, multicast). Med fler enheter per person, servrar och sensorer tog IPv4-utrymmet i praktiken slut, vilket ledde till tekniker som CIDR (effektivare uppdelning), NAT (många interna delar på en publik adress) och hårdare återanvändning. IPv6 löser bristen med $2^{128} \approx 3{,}4\times10^{38}$ adresser och en hierarkisk modell som skalar – vanligtvis /64 per VLAN i lokala nät.

Vi skiljer också på publika och privata adresser (vilka får "synas" på internet). Publika adresser är globalt routbara och tilldelas via internetleverantörer. Privata IPv4-intervall (10/8, 172.16/12, 192.168/16) används inom organisationer och översätts via NAT när de ska ut på internet. I IPv6 motsvaras detta av Global Unicast (publikt, 2000::/3) och ULA (privat, fc00::/7) som inte ska routas på internet; i IPv6 ersätts NAT normalt av tydliga brandväggspolicys. På lagnivå finns dessutom link-local-adresser för lokal kommunikation: 169.254.0.0/16 i IPv4 (APIPA) och fe80::/10 i IPv6.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

För att lyckas i resten av boken behöver du förstå tre saker:

1. Se att en adress tillhör ett interface, inte “hela datorn”.
2. Kunna läsa prefixlängden (t.ex. /24) och därmed skilja Net ID och Host ID.
3. Förstå publik vs privat och när NAT respektive brandväggspolicy används.

   I IPv6 får förvisso klienterna globala adresser (2001:db8:...), men skyddas av brandväggsregler – ingen NAT krävs.

1.4.1 Vad är en IP-adress? (Post och smeknamn)

En IP-adress identifierar ett **nätverksinterface**, inte nödvändigtvis hela datorn. Om din laptop har både Wi-Fi och en nätverkskabel inkopplad, har den två olika interface och därmed två olika IP-adresser.

För att förstå skillnaden mellan hur vi pratar på internet och i vårt lokala nätverk kan vi använda liknelsen om smeknamn:

- **Privat IP-adress:** Detta är som ett smeknamn. Hemma kanske din familj kallar dig "snuttegubben", men det är inget namn du använder officiellt på banken. På samma sätt används privata adresser (som 192.168.1.15) bara inom ditt lokala nätverk.
- **Publik IP-adress:** Detta är ditt officiella namn och adress som fungerar överallt. Det är den adressen som routern använder när den pratar med omvärlden.

1.4.2 IP-adressen som tekniskt ID-kort (32 bitar)

Även om vi skriver IPv4-adresser som fyra siffergrupper (oktetter) separerade av punkter, t.ex. 192.168.10.15, så ser datorn bara ettor och nollor. En IPv4-adress består av totalt **32 bitar**.

Den binära uppställningen för 192.168.10.15:

Varje oktett består av 8 bitar. Här ser du hur adressen ser ut för nätverkskortet:

- **192** = 11000000
- **168** = 10101000

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **10** = 00001010
- **15** = 00001111

  Hela adressen: 11000000.10101000.00001010.00001111

1.4.3 Speciella adresser du måste känna till

I din vardag som tekniker kommer du stöta på vissa adresser som har specifika regler:

- **Privata adresser (RFC1918):** Dessa får användas fritt internt men kan inte "synas" på internet.
  - 10.0.0.0 till 10.255.255.255
  - 172.16.0.0 till 172.31.255.255
  - 192.168.0.0 till 192.168.255.255
- **Loopback (127.0.0.1):** Datorns egen "spegel". Om du kan pinga 127.0.0.1 vet du att nätverksstacken i ditt operativsystem fungerar.
- **APIPA (169.254.x.x):** En "nöd-adress". Om din dator inte får kontakt med en DHCP-server ger den sig själv en adress i detta intervall. Ser du en 169-adress vet du direkt att det är problem med adressutdelningen.

1.4.4 Net ID och Host ID (Att dra gränser)

Varje IPv4-adress delas logiskt i två delar: **Net ID** (vilken gata/nätverk du tillhör) och **Host ID** (vilket husnummer/enhet du är).

- **Net ID:** Identifierar själva nätverket.
- **Host ID:** Identifierar den specifika enheten i det nätverket.

Inom varje subnät finns två reserverade adresser:

1. **Nätadressen:** Den första adressen (alla hostbitar är 0). Kan inte ges till en dator.
2. **Broadcastadressen:** Den sista adressen (alla hostbitar är 1). Används för att ropa till ALLA enheter i nätet.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel 192.168.1.0/24:

- **Net ID:** 192.168.1.0
- **Broadcast:** 192.168.1.255
- **Användbara adresser:** 192.168.1.1 till 192.168.1.254 (Totalt 254 stycken).

1.4.5 Subnätmasken – Nätverkets mall

Subnätmasken är mallen som bestämmer exakt var gränsen mellan Net ID och Host ID går. Idag använder vi nästan uteslutande **CIDR-notation** (prefix), vilket skrivs som ett snedstreck följt av antalet ettor i masken, t.ex. /24.

Hur masken styr storleken:

- **/24** (255.255.255.0): 24 ettor. Ger plats för 254 enheter.
- **/16** (255.255.0.0): 16 ettor. Ger plats för över 65 000 enheter.
- **/8** (255.0.0.0): 8 ettor. Ger plats för över 16 miljoner enheter.

1.4.6 Klass A, B och C (Historik)

Förr delades nätverk in i fasta klasser. Även om vi idag använder CIDR (klasslös routing) är det bra att känna till "defaultmaskerna":

- **Klass A:** 1.0.0.0 – 126.0.0.0 (Default /8)
- **Klass B:** 128.0.0.0 – 191.255.0.0 (Default /16)
- **Klass C:** 192.0.0.0 – 223.255.255.0 (Default /24)

1.4.7 Räkna ut rätt mask – Steg för steg

Som tekniker "subnätar" du ett nätverk för att öka säkerheten (separera gäster från servrar) och prestandan (minska broadcast-trafik).

Metodik:

1. **Bestäm behov:** Hur många klienter behöver du per klassrum/avdelning?
2. **Hitta bitarna:** Använd formeln $2^h - 2 \geq$ antal hostar (där h är antalet nollor i slutet av masken).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3. **Blockstorlek:** Beräkna avståndet mellan nätverken: $256 - \text{maskvärdet}$.

Exempel: Du har 20 datorer i ett labbrum.

- En /27 ger $2^5 - 2 = 30$ adresser. Det räcker!
- Masken för /27 är 255.255.255.224.
- Blockstorleken är $256 - 224 = 32$.
- Dina subnät hoppar alltså med 32: .0, .32, .64, .96...

1.4.8 CIDR-lathund för /24

Här är en snabbreferens när du delar upp ett vanligt Klass C-nät:

Prefix	Mask	Antal subnät	Användbara hostar
/25	255.255.255.128	2	126
/26	255.255.255.192	4	62
/27	255.255.255.224	8	30
/28	255.255.255.240	16	14
/30	255.255.255.252	64	2 (Perfekt för länk mellan två routrar)

1.4.9 NAT och PAT: Tolken mellan privat och publikt

Som vi lärde oss tidigare finns det speciella IP-adresser (RFC1918) som bara får användas internt. Men hur kan en dator med adressen `192.168.1.10` surfa på webben om den adressen inte får synas på internet? Svaret är **NAT** (Network Address Translation).

NAT – En adressändring i farten

NAT fungerar som en mellanhand. När din dator skickar ett paket till internet, byter din router ut din privata avsändaradress mot routerns egen **publika IP-adress**. När svaret kommer tillbaka från internet, kommer

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

routern ihåg vem som bad om informationen och byter tillbaka adressen så att paketet hittar hem till din dator igen.

PAT – Många delar på en adress (Overload)

I ett vanligt hem eller på en skola finns det hundratals enheter, men ofta bara en enda publik IP-adress. Hur vet routern vilken mobil eller dator som ska ha vilket svar? Här använder vi **PAT** (Port Address Translation), även kallat "NAT Overload".

Tänk dig ett stort företag med ett enda officiellt telefonnummer. För att nå rätt person internt använder man **anknytningar**. PAT fungerar exakt likadant:

1. Din dator skickar en förfrågan från port `5001`.
2. Routern tar paketet, ändrar din privata IP till sin publika, men ger paketet en unik "anknytning" (en port, t.ex. `10001`).
3. När svaret kommer tillbaka till routern på port `10001`, vet den direkt: "Aha, det här ska till datorn som använde port `5001` internt".

Det är denna teknik som gör att hela din familj eller klass kan surfa samtidigt på samma internetuppkoppling utan att paketen hamnar fel.

Port Forwarding (DNAT) – Att öppna dörren utifrån

Normalt sett blockerar en brandvägg all trafik som kommer utifrån om ingen på insidan har bett om den. Men ibland vill du att folk utifrån ska kunna nå en tjänst inne i ditt nätverk, till exempel en webbserver eller en spelserver. Då använder vi **Port Forwarding** (även kallat Destination NAT).

Det innebär att vi instruerar routern: "Om det kommer trafik på port 443 (webbtrafik), skicka den direkt vidare till vår interna webbserver".

Exempel från verkligheten: Tänk dig att du administrerar en **OPNsense**-brandvägg (en vanlig mjukvarubrandvägg).

- Din brandvägg har den publika adressen `203.0.113.10`.
- Du har en webbserver i ett säkert område (DMZ) med den privata adressen `192.168.10.20`.

För att världen ska kunna se din hemsida ställer du in en regel för Port Forwarding:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Inkommande trafik:** Publik IP `203.0.113.10` på port `443` (HTTPS).
- **Mål:** Skickas vidare till den interna adressen `192.168.10.20` på port `443`.

Nu kan vem som helst i världen skriva in din publika adress i sin webbläsare, och din brandvägg kommer automatiskt att slussa in dem till rätt server på insidan.

> **Tips:** När du felsöker nätverk och märker att du kan "pinga" en server men inte surfa till den, är det ofta en Port Forwarding-regel eller en felaktig PAT-inställning som spökar. Det är nätverksteknikerns vardag!

1.4.10 IPv4 vs IPv6: Ett arkitektoniskt skifte

För att förstå varför vi byter protokoll räcker det inte med att säga att adresserna tog slut. Vi måste titta på hur nätverkets själva logik har förändrats. IPv4 skapades i en tid när man trodde att några miljoner adresser skulle räcka för hela världen. IPv6 skapades för en framtid där varje glödlampa, sensor och rymdsond behöver en egen identitet.

1.4.10.1 Den stora skillnaden: Brist mot överflöd

I IPv4 har vi levt i en "bristekonomi". Eftersom det bara finns ca 4,3 miljarder adresser har vi blivit experter på att snåla.

- **IPv4:** Liknar en trångbodd stad där många måste dela på samma officiella postadress (NAT).
- **IPv6:** Liknar att varje individ på planeten får ett eget privat höghus med adresser. Det finns ingen anledning att snåla längre.

1.4.10.2 NAT och PAT – En nödlösning som försvinner

Den största skillnaden i din vardag som tekniker är synen på **NAT (Network Address Translation)**.

- **I IPv4-världen:** NAT är standard. Vi "gömmer" tusentals privata adresser bakom en enda publik adress. Detta skapar säkerhet genom att dölja insidan, men det gör också nätverket komplext. Det bryter den så kallade

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

"End-to-End"-principen (att två enheter ska kunna prata direkt med varandra).

- **I IPv6-världen:** NAT är (i de flesta fall) dött. Varje enhet får en globalt unik adress. Detta återställer internets grundtanke: direkt kommunikation. Säkerheten flyttas från "att vara gömd bakom NAT" till "att vara skyddad av en intelligent brandvägg".

1.4.10.3 Hur enheterna hittar hem: DHCP vs SLAAC

I IPv4 är vi helt beroende av en DHCP-server för att enheter ska få adresser automatiskt. Om servern dör, stannar nätverket. IPv6 introducerar en mycket smartare metod:

- **SLAAC (Stateless Address Autoconfiguration):** En enhet i ett IPv6-nätverk behöver inte be om lov för att få en adress. Den lyssnar på routern för att få veta vilket nätverk (prefix) den tillhör, och sedan bygger den resten av adressen själv (ofta baserat på sitt eget MAC-nummer). Det är äkta plug-and-play.
- **DHCPv6:** Finns fortfarande som ett komplement om man vill ha mer kontroll, men det är inte längre ett absolut krav för att nätverket ska fungera.

1.4.10.4 Effektivitet under huven (Header-design)

När en router ska skicka ett paket i IPv4 måste den läsa igenom ett ganska stökigt "kuvert" (header) med massor av information som ofta är onödig.

IPv6 har en mycket renare design. Trots att adresserna är mycket längre, är själva kuvertet enklare för routrarna att läsa. Det gör att moderna routrar kan skicka IPv6-paket snabbare och med mindre ansträngning än IPv4-paket.

1.4.10.5 Sammanfattning av de tekniska skillnaderna

Funktion	**IPv4**	**IPv6**
Adresslängd	32 bitar (Decimalt)	128 bitar (Hexadecimalt)
Konfiguration	Manuellt eller DHCP	Självkonfigurerande (SLAAC) eller DHCPv6

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Funktion	IPv4	IPv6
Säkerhet	Tillägg (IPsec är valfritt)	Inbyggt stöd för kryptering (IPsec)
NAT	Nödvändigt för att spara adresser	Behövs normalt inte
Broadcast	Ja (skickar till alla i onödan)	Nej (använder effektiv Multicast istället)

> **Teknikerns reflektion:** Den svåraste biten med IPv6 är inte matematiken, det är att sluta tänka att man måste "snåla". I IPv4 försöker vi alltid göra subnäten så små som möjligt för att inte slösa. I IPv6 ger vi bort miljarder adresser till ett enda VLAN utan att blinka. Det är ett mentalt skifte som du måste bemästra.

1.4.11 Hexadecimal logik: Varför bokstäver?

I IPv4 använde vi decimaltal (0–255), men i IPv6 räcker inte det. För att skriva ner 128 bitar på ett effektivt sätt använder vi **hexadecimala tal** (basen 16).

- Ett hexadecimalt tecken kan vara 0–9 eller A–F.
- Varje tecken motsvarar exakt **4 bitar** (en så kallad *nibble*).
- Fyra tecken bildar ett block på 16 bitar (t.ex. `2001`).

Genom att använda hex kan vi trycka ihop massor av information på kort yta. Ett block som `abcd` ser kanske ut som slumpmässiga bokstäver, men för nätverkskortet är det en specifik ström av 16 ettor och nollor.

1.4.11.1 Adressens anatomi: 64/64-delningen

En IPv6-adress är inte bara en lång rad tecken; den är strikt hierarkisk. Den gyllene regeln för oss tekniker är att vi nästan alltid delar adressen på mitten, vid **64-bitarsgränsen**.

1. **Network Prefix (De första 64 bitarna):** Detta är nätverkets identitet. Det talar om för routrarna i världen vilket nätverk paketet ska till. Det delas ofta upp i:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

    - *Global Routing Prefix:* Tilldelas av din internetleverantör (ISP).
    - *Subnet ID:* Den del du som tekniker styr över för att skapa olika VLAN (t.ex. elev, personal, gäst).
2. **Interface ID (De sista 64 bitarna):** Detta är enhetens unika "namn" inom det lokala nätverket.

Varför är Interface ID så stort? Det kan verka som slöseri att ha 64 bitar för enheter i ett enda rum. Men det finns en genial tanke: med 64 bitar är risken för att två enheter väljer samma adress i princip noll. Detta gör att tekniker som **SLAAC** fungerar felfritt – enheten kan generera sitt eget Interface ID utan att fråga någon om lov.

1.4.11.2 Att planera med prefix (Nibble-boundary)

Som nätverkstekniker är din viktigaste uppgift att planera hur adresserna ska fördelas. I IPv6 räknar vi inte enstaka adresser, vi räknar nätverk.

Här är den vanliga hierarkin:

- **/48 (Site Prefix):** Standard för ett helt företag eller ett campus. Ger dig 65 536 stycken /64-nät.
- **/56 (Customer Assignment):** Ofta vad en mindre skola eller ett mindre kontor får. Ger 256 stycken /64-nät.
- **/64 (VLAN Prefix):** Slutstationen. Det minsta prefixet vi använder för ett vanligt lokalt nätverk.

Exempel på planering: Om skolan får prefixet `2001:db8:aaaa::/48`, kan du dela upp det så här:

- `2001:db8:aaaa:0001::/64` – Skolans servrar
- `2001:db8:aaaa:0010::/64` – Elev-WLAN
- `2001:db8:aaaa:0020::/64` – Personal-WLAN

Ser du hur enkelt det är? Du behöver bara ändra siffrorna i det fjärde blocket för att byta nätverk. Ingen binär matte behövs vid köksbordet som i IPv4!

1.4.11.3 Neighbor Discovery: Hej då, ARP!

I IPv4 använde vi ARP (Address Resolution Protocol) för att koppla ihop en IP med en MAC-adress. ARP skickade ut "broadcasts" som störde alla

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

enheter i onödan. IPv6 använder istället **Neighbor Discovery Protocol (NDP)** via ICMPv6.

När din dator vill prata med en annan på samma switch skickar den en **Neighbor Solicitation**. Men istället för att skrika till alla (broadcast), skickar den meddelandet till en mycket specifik **Multicast-adress** som bara den enhet som faktiskt har adressen lyssnar på. Det är som att viska i ett rum istället för att skrika i en megafon – det blir mycket tystare och effektivare i nätverket.

1.4.11.4 SLAAC – Hur enheten skapar sin egen adress

SLAAC är magin i IPv6. När du kopplar in en dator händer följande:

1. Datorn skickar en **Router Solicitation** ("Finns det någon router här?").
2. Routern svarar med en **Router Advertisement** ("Ja, jag finns! Du tillhör nätverket `2001:db8:cafe:1::/64`").
3. Datorn tar nätverksdelen och kombinerar den med ett Interface ID den hittar på själv (ofta en slumpmässig siffra för att skydda din integritet).
4. **Klart!** Enheten har en global adress och kan surfa direkt. Inget krångel med DHCP-pooler som tar slut.

1.4.12 Gateways och grundläggande routing

Default gateway krävs för att nå andra nät. Finns ingen väg (route) till ett nät, kommer trafiken att skickas mot **defaulten**. På routrar kan du skapa **statiska rutter** (manuellt) eller använda **dynamiska protokoll** (t.ex. RIP, OSPF) som lär sig vägar automatiskt.

I små miljöer räcker ofta statisk routing; i större nät blir dynamisk routing en nödvändighet för skalbarhet och redundans.

1.4.13 Vanliga portar och tjänster

- **20/21** FTP (filöverföring – legacy, ersätt ofta med SFTP/FTPS)
- **22** SSH (säker fjärranslutning)
- **23** Telnet (osäkert, lämnas av historiska skäl)

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **25/587/465** SMTP (e-post ut)
- **53** DNS (UDP/TCP)
- **67/68** DHCP (server/klient)
- **80** HTTP (webb)
- **123** NTP (tid)
- **139/445** SMB (fildelning i Windows)
- **389/636** LDAP/LDAPS (katalogtjänster)
- **443** HTTPS (säker webb)
- **3389** RDP (fjärrskrivbord)

I brandväggsdesign öppnas bara nödvändiga portar **inifrån → ut** *och* **utifrån → in***; allt annat blockeras som standard.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Hur många bitar består en IPv4-adress respektive en IPv6-adress av?
2. Du kör kommandot `ip addr` och ser att ditt interface har adressen `169.254.44.12`. Vad har hänt tekniskt och varför kommer du inte ut på internet?
3. I ett IPv4-nätverk med prefixet `/26`, hur många användbara adresser kan du dela ut till datorer?
4. Varför kan du aldrig ge den första och den sista adressen i ett IPv4-subnät till en vanlig dator?
5. Förklara med egna ord varför vi använder PAT (Port Address Translation) i nästan alla hemmanätverk.
6. Förkorta denna adress så mycket som möjligt: `2001:0db8:0000:0000:0000:0000:abc0:0001`.
7. Beskriv kort hur en dator i ett IPv6-nätverk kan få en adress utan att prata med en DHCP-server.
8. Vilket portnummer och vilket protokoll (TCP/UDP) används normalt för DNS-uppslag?
9. Varför anses Telnet (port 23) vara osäkert jämfört med SSH (port 22)?
10. Vad händer med ett paket om en router tar emot det och inte hittar någon specifik rutt till måladressen i sin routningstabell?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

Cloudflare (The OSI Model vs TCP/IP) – https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/

Practical Networking (Subnetting Mastery) – https://www.practicalnetworking.net/index/subnetting/

RIPE NCC (IPv6 for Beginners) – https://www.ripe.net/about-us/press-centre/understanding-ipv6/

Cisco (IPv4 Addressing and Subnetting Guide) – https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13788-3.html

SpeedGuide (Common Ports Reference) – https://www.speedguide.net/ports.php

IANA (IPv4 Special-Purpose Address Registry) – https://www.iana.org/assignments/iana-ipv4-special-registry/

IPv6.com (SLAAC and DHCPv6 Explained) – https://ipv6.com/guides/slaac-vs-dhcpv6/

Wireshark (Understanding the IPv6 Header) – https://wiki.wireshark.org/IPv6

Network World (Why NAT is still alive in IPv6) – https://www.networkworld.com/article/2223363/why-nat-is-still-alive-in-ipv6.html

GeeksforGeeks (Introduction to Routing) – https://www.geeksforgeeks.org/routing-in-computer-networks/

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

1.5 Kommunikation i nätverk – Ethernet och WLAN

Kommunikation i ett lokalt nätverk bygger på gemensamma regler för hur data paketeras och skickas över det fysiska mediet. För att enheter ska kunna förstå varandra används standarder som definierar allt från spänningsnivåer i kablar till hur adresser läses av i nätverkskorten.

1.5.1 Ethernet på datalänk- och fysiskt lager

Ethernet är den dominerande standarden för trådbundna nätverk. Den beskriver hur data delas upp i ramar (frames) för att transporteras på datalänk-lagret (lager 2) i OSI-modellen. En typisk Ethernet-ram är uppbyggd av flera fält som fyller specifika funktioner för att säkerställa att informationen når rätt mottagare och är intakt:

- **Destination och Källa (MAC-adresser):** Varje nätverkskort har en unik fysisk adress (MAC-adress) som används för att identifiera sändare och mottagare inom samma lokala nätverk.
- **EtherType:** Anger vilket protokoll som ligger paketerat i ramen, oftast IPv4 eller IPv6.
- **Payload (nyttodata):** Själva informationen som ska skickas, exempelvis en del av en bild eller ett dokument.
- **FCS/CRC (Frame Check Sequence):** En kontrollsumma som används för felkontroll. Om mottagaren beräknar en annan summa än den som står i ramen, anses datan vara skadad och kastas.

Inom Ethernet används tre huvudsakliga adresseringsmetoder: **Unicast** (till en specifik mottagare), **Multicast** (till en grupp av mottagare) och **Broadcast** (till samtliga enheter i nätverket). En vanlig process där dessa samverkar är när en klient behöver nå en IP-adress men saknar mottagarens fysiska adress. Klienten skickar då en förfrågan (ARP request) som en broadcast till adressen `ff:ff:ff:ff:ff:ff`. Den enhet som äger IP-adressen svarar med sin MAC-adress, varpå den fortsatta kommunikationen kan ske som unicast.

Standardstorleken för en Ethernet-ram, känd som **MTU (Maximum Transmission Unit)**, är 1500 byte. I specialiserade miljöer som datacenter används ibland större ramar, så kallade **Jumbo Frames** på

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

upp till 9000 byte, för att effektivisera transporten av stora mängder data. Detta kräver dock att samtliga enheter i kedjan har stöd för samma MTU-inställning.

Modern Ethernet inkluderar även möjligheten att strömförsörja enheter direkt via nätverkskabeln genom tekniken **PoE (Power over Ethernet)**. Beroende på enhetens behov används olika standarder: **802.3af** (upp till 15,4 W), **802.3at** (PoE+, upp till 30 W) och den kraftfullare **802.3bt** (PoE++, upp till 60–90 W). Vid planering är det kritiskt att kontrollera att switchens totala effektbudget (PoE budget) räcker för samtliga anslutna accesspunkter och kameror.

1.5.2 Mediedelning, duplex och kollisioner

En av de mest fundamentala skillnaderna mellan Ethernet och WLAN är hur de hanterar tillgången till det fysiska mediet. Detta påverkar både nätverkets latens (ping) och dess totala genomströmning.

CSMA/CD i trådbundna nätverk

I nätverkets barndom, när enheter delade samma fysiska kabel via hubbar, användes metoden **CSMA/CD (Carrier Sense Multiple Access with Collision Detection)**. Enheten lyssnar på kabeln (Carrier Sense) och om ingen annan pratar, börjar den skicka data. Om två enheter pratar samtidigt uppstår en kollision (collision). Enheterna upptäcker detta (Collision Detection), avbryter sändningen och väntar en slumpmässig tid innan de försöker igen.

I moderna nätverk med switchar och full duplex (full-duplex) har varje enhet en dedikerad länk till switchen där den kan skicka och ta emot data samtidigt. Detta eliminerar i praktiken behovet av CSMA/CD och tar bort risken för kollisioner, vilket ger en mycket låg och stabil latens.

CSMA/CA i trådlösa nätverk (WLAN)

Trådlös kommunikation fungerar annorlunda eftersom mediet (luften) delas av alla enheter på samma kanal. WLAN är till sin natur **halv duplex (half-duplex)**, vilket innebär att endast en enhet kan prata åt gången på en given kanal. Istället för att upptäcka kollisioner i efterhand, försöker WLAN att undvika dem helt genom **CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)**.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Innan en trådlös enhet skickar data lyssnar den på kanalen. Om den är upptagen väntar enheten. Även om kanalen verkar ledig, väntar enheten en kort slumpmässig tid för att minska risken att krocka med en annan enhet som också väntar. Dessutom kräver nästan varje skickat datapaket en bekräftelse (ACK) från mottagaren. Denna extra väntetid och de administrativa bekräftelserna (overhead) är den främsta anledningen till att svarstider (ping) alltid är högre i ett trådlöst nätverk jämfört med Ethernet. Faktorer som radiostörningar, hinder och att många enheter konkurrerar om samma sändningstid (airtime) förstärker denna effekt ytterligare.

Autonegotiation och felmatchning

För att underlätta drift används **autonegotiation**, där två sammankopplade enheter automatiskt kommer överens om högsta möjliga hastighet och duplex. Om enheterna misslyckas med detta, eller om en tekniker manuellt låser ena änden till en specifik hastighet (t.ex. 100/Full) medan den andra står på auto, uppstår en **duplex-mismatch**. Detta leder till att den enhet som står på halv duplex tror att det sker kollisioner vid normal trafik, vilket orsakar kraftiga paketförluster och prestandaproblem som kan vara svåra att diagnosticera.

1.5.3 VLAN och trunkar i korthet

Detta avsnitt är en översikt. Detaljer som native VLAN, DTP med mera behandlas i avsnitt 2.2.

VLAN (802.1Q) skapar logiska, separata nät på samma fysiska switch. En **accessport** tillhör ett VLAN. En **trunk** bär flera VLAN mellan switchar/routrar med hjälp av 802.1Q-taggar. VLAN-ID 1–4094 (praktiskt: undvik “native VLAN” exponerat på trunkar).

För trafik mellan VLAN krävs **routing** *(t.ex. L3-switch eller “router-on-a-stick”). Segmentering minskar broadcastdomäner och höjer säkerheten.*

1.5.4 WLAN – begrepp och arkitektur

WLAN (IEEE 802.11) ger trådlös åtkomst till LAN. En **accesspunkt (AP)** sänder ett **SSID** (nätverksnamn). Varje AP-radio har en **BSSID** (MAC-adress). En **BSS** är cellen runt en AP; flera BSS som använder samma SSID bildar ett **ESS** (samma logiska nät över större yta).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Band och kanaler

- **2,4 Ghz (802.11/b, 802.11/g, 802.11/n)**: Lång räckvidd, färre icke-överlappande kanaler, mer störningskänsligt.
- **5 GHz(802.11/ac)**: Fler kanaler, högre kapacitet, kortare räckvidd; vissa kanaler kräver DFS.
- **6 GHz (802.11ax i 6 GHz-bandet)**: Många rena kanaler (där det är tillåtet), kortare räckvidd.

I 2,4 GHz används i praktiken 1/6/11 som icke-överlappande kanalval (i vissa länder finns kanal 13, men överlapp måste beaktas).

Kanalbredd (20/40/80/160 MHz) påverkar kapacitet och räckvidd: bredare kanal ⇒ högre topphastighet men större risk för störningar och sämre frekvensåteranvändning.

Exempel: *I en skola med många klassrum ger 5 GHz på 20 eller 40 MHz ofta bäst total kapacitet (mer frekvensåteranvändning) jämfört med några få breda 80/160-kanaler.*

***Viktigt**: 802.11ax finns även i 2,4/5 GHz-bandet; här avses specifikt varianten i 6 GHz.*

1.5.5 Medietillgång och prestanda i WLAN

WLAN använder **CSMA/CA** (collision avoidance): klienter lyssnar innan de sänder. Mekanismer som **RTS/CTS** kan användas för att minska problem med “hidden nodes” (klienter som inte hör varandra).

Moderna WLAN använder **OFDM/OFDMA** och **MIMO/MU-MIMO** för att utnyttja luften effektivt och serva flera klienter samtidigt. Verklig kapacitet styrs av **SNR**, **interferens**, kanalbredd, antal **spatial streams** och hur många klienter som konkurrerar om **airtime**.

Placering av AP, sändareffekt och dämpning (väggar, metall, glas) avgör täckning. För stark effekt kan försämra roaming och skapa mer ko-kanalsstörning.

1.5.6 Säkerhet i WLAN: Från hål till kryptering i världsklass

Att skicka data via radiovågor är i praktiken som att stå och ropa i ett klassrum fullt med folk – vem som helst som befinner sig inom hörhåll kan

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

lyssna på vad du säger. I nätverkssammanhang kallas detta för att etern är ett delat medium. För att förhindra att obehöriga läser vår privata trafik måste vi använda kryptering. Under de senaste 25 åren har vi gått från system som var så svaga att de kunde knäckas på sekunder till moderna protokoll som kräver enorm datorkraft för att ens ha en chans att kompromettera.

WEP – Det första, misslyckade försöket

När Wi-Fi först lanserades 1997 skapades **WEP (Wired Equivalent Privacy)**. Tanken var god: man ville att ett trådlöst nätverk skulle vara lika säkert som ett trådbundet. Men tekniskt sett var det en katastrof. WEP förlitade sig på en algoritm som heter RC4 och en statisk nyckel som aldrig ändrades.

Det stora problemet var den så kallade "initieringsvektorn" (IV). Det är en liten del av krypteringen som ska vara unik för varje paket. I WEP var denna alldeles för kort. När ett nätverk har mycket trafik börjar IV-värdena återanvändas. En angripare som lyssnar på nätverket kan samla in dessa kollisioner och med enkel mjukvara räkna ut själva huvudlösenordet.

- **Hur det fungerar:** Använder en statisk nyckel (ett lösenord) som delas av alla. Data krypteras med en algoritm som heter RC4.
- **Fördelar:** Kräver i princip noll processorkraft. Fungerar på antik hårdvara.
- **Nackdelar:** Extremt osäkert. Den korta startvektorn (IV) gör att krypteringen återanvänds för ofta.
- **Varför det är så dåligt:** En modern laptop kan knäcka ett WEP-lösenord på under en minut genom att bara lyssna på trafiken.

WPA och TKIP – En tillfällig lösning

När säkerhetshålen i WEP blev kända behövde branschen en snabb lösning som inte krävde att alla köpte nya routrar. Resultatet blev **WPA (Wi-Fi Protected Access)** med protokollet **TKIP**. TKIP fungerade som ett "plåster" på RC4-algoritmen genom att byta ut krypteringsnyckeln för varje enskilt paket. Det var säkrare än WEP, men eftersom det fortfarande byggde på samma grundläggande matematik hittade hackare snart nya vägar in. Dessutom blev TKIP en flaskhals för prestandan i takt med att nätverken blev snabbare.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Hur det fungerar:** Behåller RC4-krypteringen men lägger till TKIP (Temporal Key Integrity Protocol) som byter nyckel löpande.
- **Fördelar:** Gick att installera via mjukvaruuppdateringar på gammal WEP-hårdvara.
- **Nackdelar:** TKIP har egna sårbarheter och begränsar hastigheten på moderna nätverk drastiskt (ofta till 54 Mbit/s).

WPA2-PSK och den berömda 4-vägshandskakningen

Med **WPA2** kom den stora förändringen: **AES (Advanced Encryption Standard)**. Detta är samma kryptering som banker och myndigheter använder. För hemmet och mindre kontor använder vi oftast **PSK (Pre-Shared Key)**, där alla använder samma lösenord.

Det som gör WPA2 intressant under huven är hur en enhet faktiskt ansluter till en accesspunkt. Detta sker via en **4-vägshandskakning (4-way handshake)**. Syftet med handskakningen är att bevisa att både klienten och accesspunkten kan lösenordet utan att de faktiskt skickar lösenordet i klartext över luften.

Så här fungerar 4-vägshandskakningen förenklat:

1. **ANonce:** Accesspunkten skickar ett slumpmässigt tal (en "nonce") till klienten.
2. **SNonce + MIC:** Klienten skapar ett eget slumpmässigt tal och kombinerar det med lösenordet för att skapa en unik sessionsnyckel. Sedan skickas klientens "nonce" plus en kontrollsumma (MIC) tillbaka.
3. **GTK:** Accesspunkten skapar nu samma sessionsnyckel på sin sida. Om allt stämmer skickar den ut gruppnyckeln (för broadcast-trafik).
4. **ACK:** Klienten bekräftar att allt är klart och krypteringen startar.

Sårbarheten här är att en hackare kan "spela in" denna handskakning och sedan sitta hemma i lugn och ro och testa miljarder lösenord mot den (en offline-attack).

- **Hur det fungerar:** Introducerade AES-kryptering och 4-vägshandskakning.
- **Fördelar:** Mycket stark kryptering som stödjs av nästan alla moderna enheter.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Nackdelar:** Sårbart för "brute force"-attacker om man använder ett svagt lösenord. Om en person lämnar organisationen måste lösenordet bytas för alla.

WPA2-Enterprise – För den professionella miljön

I skolor och på stora företag räcker det inte med ett gemensamt lösenord. Där använder vi **WPA2-Enterprise**. Skillnaden är att inloggningen sker mot en central databas, oftast via protokollet **802.1X**. Istället för ett lösenord i routern skickas dina inloggningsuppgifter till en **RADIUS-server**.

Detta ger oss som tekniker full kontroll. Om en elev slutar eller tappar bort sin dator kan vi spärra just det kontot direkt utan att påverka någon annan. Det är skillnaden mellan att ha en huvudnyckel till ett helt hus (PSK) och att ha personliga passerkort (Enterprise).

- **Hur det fungerar:** Kräver unik inloggning per användare via en RADIUS-server.
- **Fördelar:** Unika nycklar, hög spårbarhet och enkel administration av många användare.
- **Nackdelar:** Betydligt mer komplext att konfigurera och kräver extra serverresurser.

WPA3 – Framtiden är här

WPA3 lanserades för att täppa till de sista hålen i WPA2. Den största nyheten för privatpersoner är **SAE (Simultaneous Authentication of Equals)**. Det ersätter den gamla handskakningen med en metod som gör det omöjligt för hackare att spela in trafiken och knäcka lösenordet i efterhand. Även om du har ett ganska svagt lösenord är WPA3 extremt svårt att forcera.

För företag innebär **WPA3-Enterprise** ännu starkare krypteringsalgoritmer (192-bitars läge) för att skydda mot framtida hot, inklusive kvantdatorer.

- **Hur det fungerar (SAE):** Använder en metod där lösenordet aldrig kan gissas genom att analysera inspelad trafik.
- **Fördelar:** Skyddar mot de vanligaste attackerna mot WPA2. Tvingar fram skyddade ledningsramar (PMF).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Nackdelar:** Kräver modern hårdvara; många äldre enheter (som skrivare eller gamla mobiler) kan inte ansluta till ett rent WPA3-nätverk.

Hur sätter man upp ett bra WLAN?

Som it-tekniker är din uppgift att balansera säkerhet och användarvänlighet. Här är de gyllene reglerna:

- **Välj rätt säkerhetsnivå:** Använd **WPA3** där det går. För att stödja äldre enheter kan du använda "WPA2/WPA3 Mixed Mode", men aktivera aldrig WEP eller WPA1.
- **Segmentera med VLAN:** Detta är avgörande. Skapa olika trådlösa nätverk (SSID) för olika syften. Gäster ska aldrig vara på samma nätverk som ekonomiavdelningen eller skolans servrar.
    - *Exempel:* **SSID "Personal"** (VLAN 10, WPA2-Enterprise), **SSID "Gäster"** (VLAN 30, WPA2-PSK med isolering).
- **Sluta dölja SSID:** Att dölja nätverksnamnet (Hidden SSID) ger ingen säkerhet alls, eftersom enheterna ändå måste ropa efter nätverket högt i etern. Det gör bara anslutningen instabilare.
- **Använd RADIUS för kontroll:** Så fort du hanterar mer än ett tiotal användare är RADIUS-baserad inloggning (Enterprise) det enda ansvarsfulla valet.

Hänvisning: Vi kommer att titta närmare på hur man sätter upp och konfigurerar en **RADIUS-server** (exempelvis FreeRADIUS eller Windows NPS) i **Kapitel 4: Nätverkstjänster och Autentisering**. Där lär vi oss hur servern pratar med accesspunkterna för att släppa in rätt användare på rätt VLAN.

1.5.7 Roaming och den osynliga upplevelsen

I ett litet hem räcker det ofta med en router, men på en skola eller ett kontor har vi dussintals eller hundratals accesspunkter (AP). När du går genom korridoren med din laptop vill du att den sömlöst ska flytta över från en AP till en annan utan att videosamtalet bryts. Detta kallas för **roaming**.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Det viktigaste att förstå som tekniker är att det nästan alltid är **klienten** (mobilen eller datorn) som bestämmer när det är dags att byta AP. Men nätverket kan hjälpa till på traven genom några viktiga tekniker:

- **802.11r (Fast Transition):** När vi använder WPA2-Enterprise krävs en lång handskakning mot RADIUS-servern varje gång vi byter AP. Det tar tid. 802.11r gör att delar av handskakningen sker i förväg, vilket kapar tiden för bytet från sekunder till millisekunder.
- **802.11k (Neighbor Reports):** Istället för att din mobil ska behöva skanna alla radiokanaler för att hitta nästa AP, skickar nätverket en lista på "grannar" som finns i närheten. Det sparar både tid och batteri.
- **802.11v (BSS Transition):** Här kan nätverket faktiskt ge klienten ett förslag: "Jag ser att du har svag signal hos mig, men AP:n i taket bredvid dig har jättebra signal. Flytta dit!".

Utmaningar för teknikern: Sticky Clients och Airtime

Ett vanligt problem är **"Sticky Clients"** – enheter som envist håller kvar vid en svag AP trots att de står precis under en ny. För att lösa detta kan vi ställa in en **Minimum RSSI**. Det innebär att vi instruerar accesspunkten att sparka ut klienter om deras signalstyrka blir för låg, vilket tvingar dem att leta efter en bättre signal.

En annan fälla är att skapa för många trådlösa nätverk (**SSID**). Varje SSID kräver att accesspunkten skickar ut små "här är jag"-paket (beacons) flera gånger i sekunden. Om du har 10 olika SSID på samma ställe kommer luften vara så full av dessa administrativa paket att det knappt finns plats för riktig data. Håll dig till 2–4 SSID och använd VLAN för att separera trafiken istället.

1.5.8 Ethernet vs WLAN – Varför behöver vi båda?

Som it-tekniker får man ofta frågan: "Varför ska vi dra kablar när Wi-Fi är så snabbt?". Svaret ligger i skillnaden mellan ett **delat medium** och ett **dedikerat medium**.

Ethernet: Den stabila motorvägen

När du kopplar in en nätverkskabel har du en egen "fil" rakt in i switchen. Modern Ethernet körs i **Full Duplex**, vilket betyder att du kan skicka och

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

ta emot data samtidigt med full hastighet utan att krocka med någon annan.

- **Fördelar:** Ingen jitter (variation i fördröjning), maximal säkerhet (du måste fysiskt komma åt kabeln) och stabil kapacitet.
- **När väljer vi det?** Alltid för servrar, stationära datorer, skrivare och förstås som "backbone" – kablarna som matar accesspunkterna med ström (PoE) och data.

WLAN: Friheten med kompromisser

Wi-Fi fungerar i **Half Duplex**. Det är som en walkie-talkie: bara en person kan prata åt gången på samma kanal. Om tio elever laddar ner stora filer samtidigt på samma accesspunkt, måste de dela på den totala hastigheten, och de måste "vänta på sin tur". Dessutom kan mikrovågsugnar, betongväggar och grannens nätverk störa signalen.

- **Fördelar:** Extrem rörlighet och enkel skalbarhet för många enheter utan att behöva borra hål i väggarna.
- **Nackdelar:** Varierande latens (ping) och känslighet för störningar.

Den moderna designen

I ett modernt nätverk bygger vi en hybrid. Vi använder fiber och kopparkabel (Ethernet) i väggarna för att skapa ett stabilt fundament med hög kapacitet (10 Gbps eller mer). Sedan placerar vi ut välplanerade accesspunkter som ger användarna den rörlighet de behöver.

En gyllene regel för oss tekniker är: **"Om det har en fast plats och har ett nätverksuttag – använd kabel"**. Genom att flytta bort tunga laster (som en stationär dator som streamar video eller en server) från det trådlösa nätverket, lämnar vi mer "airtime" ledig för de mobila enheter som verkligen behöver den.

Tänk på: När du designar ett nätverk, tänk på att Wi-Fi bara är så bra som kablarna bakom det. Om du har en modern Wi-Fi 6-accesspunkt men kopplar in den med en gammal sliten Cat5-kabel, kommer kabeln att bli flaskhalsen oavsett hur snabb radiotekniken är.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

1.5.9 802.11-standarder

Nedan är de vanligaste standarderna som nämns i specifikationer för klienter och accesspunkter. (Vi använder **IEEE-namnen** konsekvent.)

Standard	Band	Teknik i korthet	Typiska kanalbredder	Max teoretisk länkhastighet*	Notering
802.11b	2,4 GHz	DSSS/CCK	20 MHz	11 Mbit/s	Legacy, undvik i nya miljöer
802.11g	2,4 GHz	OFDM	20 MHz	54 Mbit/s	Bakåtkompatibel med b
802.11n	2,4 & 5 GHz	OFDM, **MIMO**	20/40 MHz	upp till 600 Mbit/s (4×4)	Vanlig, men äldre; stöd för både band
802.11ac	5 GHz	**MU-MIMO** (DL), högre QAM	20/40/80/160 MHz	>1 Gbit/s (beroende på streams/bredd)	Endast 5 GHz
802.11ax	2,4/5/6 GHz	**OFDMA**, MU-MIMO (UL/DL), TWT	20/40/80/160 MHz	flera Gbit/s (beroende på streams/bredd)	Effektivare airtime, stöd i 6 GHz där tillåtet

*Max “länk-rate” är teoretisk och förutsätter optimala förhållanden; **verklig throughput är lägre**.

Snabba köptips för elever

- Satsa på **802.11ax** för framtidssäkerhet (gärna stöd för både 2,4 och 5 GHz; 6 GHz där det är tillåtet).
- Välj minst **2×2 MIMO** på klienter; fler streams på AP ger bättre kapacitet för många klienter.
- Kontrollera stöd för **WPA3** och regelbundna firmware-uppdateringar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- I skolmiljö: AP med **PoE-stöd**, **DFS** (för fler 5 GHz-kanaler) och möjlig **controller/central hantering**.
- Titta på **kanalbredd** (20/40 MHz räcker ofta bättre i miljöer med många celler än 80/160 MHz).

Exempel: *En elev köper en laptop med 802.11ax (2×2) och får stabil prestanda i 5 GHz-bandet i skolan, även när många är uppkopplade samtidigt.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Vilken funktion fyller FCS (Frame Check Sequence) i en Ethernet-ram?
2. Hur används en MAC-adress på datalänk-lagret (lager 2) för att leverera data till rätt enhet?
3. Beskriv processen när en enhet skickar en ARP-förfrågan (ARP request).
4. Vad är standard-MTU för en Ethernet-ram och vad händer om en enhet i nätverket inte stödjer "Jumbo Frames"?
5. Vilken PoE-standard (802.3) bör användas om en accesspunkt kräver 25 W effekt för att fungera?
6. Förklara skillnaden mellan CSMA/CD och CSMA/CA. Varför behövs olika metoder för trådbunden respektive trådlös kommunikation?
7. Varför uppstår det i praktiken inga kollisioner i ett modernt switchat nätverk som kör full duplex (full-duplex)?
8. Nämn två specifika orsaker till att svarstiden (ping) ofta är högre i ett WLAN jämfört med Ethernet.
9. Vad är en "duplex-mismatch" och vilka symptom kan det ge i nätverket?
10. Vad innebär det att en trådlös kanal är ett halv duplex-medium (half-duplex)?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

IEEE 802.3 Working Group (Officiell webbplats för utveckling av Ethernet-standarder) - https://www.ieee802.org/3/

Wi-Fi Alliance (Information om certifieringar och tekniska framsteg inom Wi-Fi) - https://www.wi-fi.org/

Cisco: Ethernet Technologies (Teknisk djupdykning i Ethernet-ramar och fysiska lager) - https://www.cisco.com/c/en/us/support/docs/lan-switching/ethernet/10561-ethernet-tech.html

PoE Standards Overview (Detaljerad jämförelse mellan 802.3af, at och bt) - https://www.electronics-notes.com/articles/connectivity/ethernet-ieee-802-3/poe-power-over-ethernet-standards.php

Wireshark Wiki: Ethernet (Praktisk genomgång av hur Ethernet-trafik analyseras i paketnivå) - https://wiki.wireshark.org/Ethernet

Cloudflare: What is Network Latency? (Förklaring av latens, ping och faktorer som påverkar hastighet) - https://www.cloudflare.com/learning/network-layer/what-is-latency/

Computer History Museum: The History of Ethernet (Berättelsen om hur Ethernet skapades vid Xerox PARC) - https://www.computerhistory.org/revolution/networking/19/374

GeeksforGeeks: CSMA/CD vs CSMA/CA (Tydlig jämförelse mellan metoderna för mediedelning [medium access]) - https://www.geeksforgeeks.org/difference-between-csma-ca-and-csma-cd/

Practical Networking: How ARP Works (Visuell och pedagogisk guide till Address Resolution Protocol) - https://www.practicalnetworking.net/series/arp/how-arp-works/

Troubleshooting Duplex Issues (Teknisk guide för att identifiera och åtgärda felmatchning av duplex) - https://community.cisco.com/t5/networking-knowledge-base/troubleshooting-ethernet-collisions-and-duplex-issues/ta-p/3116172

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2 Planering och design av nätverk

Innan ett nät byggs, fysiskt, är det alltid av största vikt att ha en plan. Denna plan fyller funktionen av Vad som skall görs, Hur det skall göras och Varför det skall utformas på det sätt. Genom att ha en bra och stabil plan blir det lätt att se fördelar och brister, felsöka nätverket och se vilka svårtigheter som finns vid en eventuell förändring av nätverket. Många nätverk är bra och välstrukturerade inledningsvis, och anpassade efter behovet som finns när de byggs. Men i och med att strukturer förändras, kundflöden varierar, anställda ökar och minskar och mycket annat, så behöver även nätverket vara möjligt att anpass utefter nuvarande och framtida behov, detta genom att behöva göra så små ingrepp i infrastrukturen som möjligt.

2.1 Mål och krav

Designen av ett robust nätverk inleds med en grundlig behovsanalys (requirements analysis) för att fastställa nätverkets syfte och användningsområden. Processen innebär en kartläggning av användarnas beteendemönster, vilket inkluderar vilka applikationer som prioriteras, antalet samtidiga klienter (concurrent clients) och från vilka fysiska platser åtkomst krävs. Denna kravbild utgör fundamentet för tekniska beslut rörande:

Topologi (topology): den fysiska och logiska utformningen av nätverket.

Adressering och segmentering (segmentation): planering av IP-adresser och hur nätverkstrafiken fördelas och isoleras.

Brandväggspolicys (firewall policies): definitioner av vilka tjänster som ska tillåtas och hur åtkomstbegränsningar ska implementeras.

Trådlös kapacitet (wireless capacity): säkerställande av god täckning och metoder för att hantera klientbelastning utan prestandaförlust.

Genom en tydlig kravspecifikation minimeras risken för kostsamma ändringar i ett senare skede, samtidigt som det blir möjligt att verifiera att

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

den slutgiltiga lösningen uppfyller verksamhetens förväntningar. Kraven delas tekniskt in i följande kategorier:

Funktionella krav (functional requirements): omfattar konkreta tjänster såsom röstsamtal via nätverket (VoIP), videoströmning, DHCP och DNS, samt behovet av segmentering via virtuella lokala nätverk (VLAN).

Icke-funktionella krav (non-functional requirements): fokuserar på nätverkets inneboende egenskaper, såsom bandbredd (bandwidth), latens (latency), tillgänglighet (availability), säkerhetsnivå, skalbarhet (scalability) och budgetramar.

Gränssnitt (interfaces): rör externa anslutningar och villkor, vilket inkluderar internetkapacitet, tillgång till publika IP-adresser, behov av krypterade tunnlar mellan platser (site-to-site VPN) samt leverantörsavtal (SLA).

Ett konkret exempel på en kravbild för en gymnasieskola kan innefatta stöd för 600 samtidiga klienter fördelade på strikt separerade segment för elever, personal och gäster. Tekniska mål kan i detta scenario innefatta en latens under 10 ms inom campusområdet, central loggning av trafik för spårbarhet samt en garanterad tillgänglighet på 99,5 % under schemalagd skoltid.

2.1.2 Arkitektur och topologi

Vid utformningen av nätverksarkitekturen eftersträvas en modell som är begriplig, robust och underhållsvänlig. En vedertagen metod för större miljöer, såsom skolor och campusområden, är den hierarkiska modellen (hierarchical network design). Genom att dela upp nätverket i tre logiska lager underlättas både felsökning och framtida expansion. Denna arkitektur styrs av följande huvudkomponenter:

Core (kärnlager): nätverkets ryggrad (backbone) som ansvarar för snabb transport av stora datamängder mellan olika byggnader och zoner. Här prioriteras hastighet och tillgänglighet framför avancerad paketfiltrering.

Distribution (distributionslager): fungerar som en länk mellan kärnlagret och användarna. Här hanteras routing mellan olika virtuella nätverk (VLAN) samt implementering av säkerhetspolicys.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Access (accesslager): den del där klienter, skrivare och accesspunkter ansluts till nätverket. Logiskt används oftast en stjärntopologi (star topology) med redundanta uppkopplingar (uplinks) uppåt i hierarkin.

För att säkerställa driftsäkerhet implementeras ofta redundans genom dubbla länkar eller enheter. Detta medför ett behov av protokoll för loopskydd (loop protection), såsom STP, RSTP eller MSTP, för att förhindra att trafik cirkulerar okontrollerat i redundanta slingor.

Beslutet om var routing (L3) ska ske påverkar nätverkets skalbarhet. Genom att använda L3-switchar i distributionslagret och konfigurera virtuella gränssnitt (SVI - Switched Virtual Interface) kan routing ske lokalt och snabbt. Alternativt kan routing skötas centralt i en brandvägg eller router om all trafik kräver strikt granskning.

Vid designval bör följande principer beaktas:

Hierarki: konsekvent användning av core, distribution och access för att skapa en förutsägbar trafikmiljö.

Redundans: dubblering av kritiska enheter och länkar där avbrott innebär stora konsekvenser för verksamheten.

Routing-placering: val av L3-placering i distribution eller centralt beroende på krav på prestanda och säkerhetskontroll.

Undvikande av kedjekoppling (daisy chaining): långa kedjor av sammankopplade accesswitchar bör undvikas, då ett fel tidigt i kedjan isolerar alla bakomliggande enheter. Trafik bör istället samlas upp direkt i distributionslagret.

Ett praktiskt exempel på denna arkitektur är en fastighet med två distributionsswitchar (konfigurerade som en logisk enhet via stack eller vPC), där accesswitchar på respektive våningsplan ansluts med dubbla fiberlänkar för att eliminera enskilda felkällor (single point of failure).

2.1.3 Hierarkisk design och nätverksnivåer (Tiers)

Den hierarkiska designen delas tekniskt upp i tre nivåer, ofta benämnda som ”tiers”. Varje nivå har specifika uppgifter och ställer olika krav på hårdvarans prestanda och funktionalitet. Genom att separera funktionerna skapas ett nätverk som är enklare att hantera, skala och felsöka. Denna

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

modell är universell och används för att beskriva allt från lokala företagsnätverk till hur de största internetleverantörerna samverkar globalt.

Nivå 3: Accesslagret (Access Tier) Accesslagret utgör nätverkets ytterkant (edge) och är den nivå där slutanvändare och kringutrustning ansluter. Fokus ligger på att erbjuda hög portdensitet och grundläggande säkerhetsfunktioner för att skydda nätverket mot felaktiga anslutningar. Ett större lokalt nätverk (LAN) på ett våningsplan, där hundratals enheter ansluter via vägguttag eller trådlöst, är ett typiskt exempel på en miljö på nivå 3.

- **Typisk hårdvara:** Switchar med 24 eller 48 portar och stöd för strömförsörjning (PoE/PoE+) för att driva enheter utan extra eluttag.
- **Säkerhet:** Implementering av portskydd (port security) för att begränsa antalet tillåtna MAC-adresser per port, samt 802.1X-autentisering för identitetskontroll.
- **Exempel:** En accesswitch i ett klassrum som strömförsörjer trådlösa accesspunkter och ansluter stationära elevdatorer till rätt virtuellt nätverk (VLAN).

Nivå 2: Distributionslagret (Distribution Tier) Distributionslagret fungerar som en länk mellan accesslagret och kärnan. Här aggregeras trafik från ett flertal accesswitchar, och det är på denna nivå nätverkets logiska regler och styrning implementeras. Nivån utgör ofta gränsen för routing (Layer 3 boundary). En skola med en välplanerad arkitektur fungerar som en typisk nivå 2-miljö, där distributionslagret samlar upp trafiken från samtliga klassrum och byggnader för vidare befordran.

- **Trafikstyrning:** Hantering av kommunikation mellan olika virtuella nätverk genom inter-VLAN routing och virtuella gränssnitt (SVI).
- **Policyimplementering:** Användning av åtkomstlistor (ACL - Access Control Lists) för att styra vilken trafik som får flöda mellan segment, samt prioritering av trafik genom tjänstekvalitet (QoS - Quality of Service).
- **Exempel:** En central switch i en byggnad som samlar upp fiberlänkar från alla våningsplan, sköter DHCP-vidarebefordran (DHCP Relay) till centrala servrar och ser till att gästtrafik isoleras från personalnätet.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Nivå 1: Kärnlagret (Core Tier) Kärnlagret utgör nätverkets centrala ryggrad (backbone). Dess enda syfte är att transportera stora datamängder mellan olika nätverksdelar eller distributionspunkter med minsta möjliga fördröjning (latency). Här undviks komplexa processer som kan sänka hastigheten. På den globala nivån representeras nivå 1 av de stora internetleverantörerna (ISP), såsom Telia (Arelion), Tele2 och Telenor, som äger och driver de ryggradsnät som kopplar samman länder och kontinenter.

- **Prestanda:** Användning av höghastighetsanslutningar, ofta via fiberoptik med kapaciteter på 10, 40 eller 100 Gbps.
- **Tillgänglighet:** Extremt hög redundans genom dubblerade enheter och länkar som möjliggör drift även vid hårdvarufel.
- **Exempel:** Två kraftfulla kärnswitchar placerade i ett centralt serverrum som kopplar samman olika skolbyggnader i ett campusnätverk. Kärnlagret ser till att trafik från biblioteket når skolans centrala lagringsserver utan flaskhalsar.

I mindre nätverksmiljöer används ibland en förenklad modell som kallas för en sammanvuxen kärna (collapsed core). I denna design slås kärnlagret och distributionslagret samman till en gemensam enhet för att reducera kostnader och komplexitet, samtidigt som den hierarkiska fördelen mot accesslagret bibehålls.

2.1.4 IP-plan, VLAN och namnstandard

2.1.4 IP-planering, VLAN-struktur och namnstandard

En välstrukturerad plan för IP-adresser och virtuella nätverk (VLAN) utgör fundamentet för en skalbar och lätthanterlig nätverksmiljö. Utan en konsekvent metodik ökar risken för adresskollisioner, säkerhetsbrister och svåröverskådlig dokumentation. Arbetet inleds med att reservera en privat adressrymd (private address space) enligt RFC1918, exempelvis `10.0.0.0/8`, vilken sedan delas upp i logiska enheter baserat på geografisk plats, såsom byggnader, eller specifika funktioner.

Varje virtuellt nätverk (VLAN) fungerar som en isolerad broadcastdomän, och för att möjliggöra kommunikation krävs att varje VLAN knyts till ett unikt subnät (subnet). Genom att använda en konsekvent IP-plan kan

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

tekniker utläsa en enhets funktion eller placering direkt genom dess IP-adress. Följande principer bör ligga till grund för utformningen:

VLAN-konventioner (VLAN conventions): Varje roll i nätverket tilldelas ett unikt VLAN-ID. Det är god praxis att använda ett logiskt numreringsschema som är lätt att komma ihåg och dokumentera. Exempelvis kan VLAN 10 reserveras för personalens administrativa trafik, VLAN 20 för elever, VLAN 30 för gäståtkomst, VLAN 40 för servrar och VLAN 50 för hantering av accesspunkter.

Dimensionering och subnettering (dimensioning): Varje subnät måste dimensioneras utifrån antalet samtidiga klienter med en inbyggd marginal för framtida tillväxt. I miljöer med hög klienttäthet, såsom trådlösa nätverk för elever, krävs ofta större prefix (exempelvis `/22` som ger 1022 adresser) för att säkerställa att adresserna i DHCP-poolen inte tar slut under perioder med hög belastning.

Adresshantering och tjänster (IPAM & services): För varje subnät definieras en fast gateway-adress, vilket vanligtvis är den första eller sista användbara adressen i intervallet. Vidare konfigureras DHCP-inställningar (DHCP scopes) med relevanta optioner. Dessa optioner instruerar klienten om vilken router som ska användas, vilka DNS-servrar som ska sköta namnuppslagning samt vilken NTP-server som används för tidssynkronisering.

Namnstandard (naming convention): En enhetlig namnstandard för nätverksutrustning som switchar, accesspunkter och servrar är avgörande för effektiv övervakning och felsökning. Namnet bör koda information om enhetens fysiska placering, dess roll i den hierarkiska modellen och ett unikt löpnummer. Detta möjliggör snabb identifiering i loggfiler och vid larmhantering utan att tekniker behöver konsultera externa ritningar.

Exempel: *Vid konfigurationen av elevsegmentet på en skola. Här kan VLAN 20 tilldelas subnätet `10.20.0.0/22`. Gateway-adressen sätts till `10.20.0.1` och DHCP-intervallen konfigureras mellan `10.20.0.50` och `10.20.3.200`. DNS-optionerna pekar i detta fall mot nätverkets Active Directory-servrar (AD) på `10.40.0.10` och `10.40.0.11` för att säkerställa korrekt funktionalitet inom domänen. Enligt namnstandarden kan en*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

accesswitch placerad på tredje våningen i hus 1 då ges namnet `HUS1-SW-A3`.

2.1.5 Fysisk infrastruktur

2.1.5.1 Hårdvara

Den fysiska infrastrukturen utgör fundamentet för en tillförlitlig nätverksdrift. Utformningen av serverrum och korskopplingsskåp (wiring closets) påverkar direkt nätverkets livslängd, säkerhet och underhållsvänlighet. En oorganiserad miljö försvårar felsökning och ökar risken för mänskliga fel vid servicearbeten. För att säkerställa en stabil miljö bör följande tekniska aspekter beaktas vid planeringen:

> **Organisering och märkning (patching and labeling):** Racksystem och patchpaneler ska vara logiskt märkta och strukturerade. Genom att separera koppar- och fiberkablage samt använda horisontella och vertikala kabelhanterare (cable managers) skapas en överskådlig miljö där service kan utföras effektivt utan att störa intilliggande utrustning.

Märkningen bör vara hierarkisk och utgå från den fysiska platsen. En vedertagen metod är att använda ett koordinatsystem som identifierar rum, rack, panel och port.

- **Format för uttagsmärkning:** `[Rack-ID] – [Panel-ID] : [Portnummer]`
    - **Exempel:** `A1–P02:24`
    - **Betydelse:** Uttaget leder till rack A1, patchpanel 02, port nummer 24.
- **Märkning i båda ändar:** Varje patchkabel (patch cord) märks i båda ändar med information om var den andra änden är ansluten. Detta eliminerar behovet av att manuellt spåra kablar genom trånga kabelkanaler.
- **Uttag i vägg:** Uttag vid användarens arbetsplats märks med exakt samma beteckning som motsvarande port i korskopplingsskåpet.

Strukturen i ett rack bör planeras för att minimera korsande kablage och optimera luftflödet för kylning.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Interleaving (Varvning): En effektiv metod är att placera en switch mellan två patchpaneler. Detta möjliggör användning av mycket korta patchkablar (ofta 15–20 cm) som går direkt från panelen till switchen, vilket minskar behovet av stora horisontella kabelhanterare och ger en extremt ren installation.

Kabelhanterare (Cable managers):

- **Horisontella:** Används för att leda kablar från en switchport till rackets sidor.
- **Vertikala:** Stora kanaler på sidorna av racket där överskottskabel förvaras och leds uppåt eller nedåt.

Färgkodning (Color coding): Genom att använda olika färger på patchkablarna kan olika tjänster identifieras visuellt på långt håll:

- **Blå:** Standard datatrafik (klienter).
- **Röd:** Kritiska anslutningar (servrar/uplinks).
- **Gul:** Trådlösa accesspunkter (PoE-enheter).
- **Grön:** Övervakningskameror eller säkerhetssystem.

Den fysiska märkningen är endast effektiv om den åtföljs av en uppdaterad dokumentation (rack diagram). Detta inkluderar:

Rackritning: En visuell översikt som visar exakt vilken utrustning som är monterad på vilken höjdenhet (U - Unit).

Patch-schema: En förteckning eller databas som visar förbindelsen mellan varje vägguttag och switchport, inklusive vilket VLAN som är konfigurerat på porten.

Exempel: *En felsökningsprocess i en välmärkt miljö är när en användare rapporterar fel på uttag `C1-P01:12`. Teknikern kan då gå direkt till rack C1, hitta panel 1 och port 12, se på färgen på kabeln att det rör sig om en standardklient, och omedelbart identifiera vilken switchport som är kopplad till uttaget utan att behöva gissa eller dra i kablar.*

2.1.5.2 Kablage

Kablageval (cabling): Valet av kablage styrs av krav på överföringshastighet, fysiska avstånd och den omgivande miljöns elektriska egenskaper.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Access-lagret: I anslutningar till klienter och accesspunkter används vanligtvis partvinnad kopparkabel (twisted pair) av kategorin Cat6 eller Cat6a, vilket möjliggör hastigheter upp till 10 Gbps över begränsade sträckor.

Skärmning (shielding): Utöver kategorivalet ställs krav på hur väl kabeln ska skyddas mot yttre störningar. Här skiljer man på två huvudtyper:

- **UTP (Unshielded Twisted Pair):** Oskärmad kabel som är standard i de flesta kontors- och skolmiljöer. Den är flexibel, kostnadseffektiv och kräver ingen särskild jordning i nätverksutrustningen för att fungera korrekt.
- **STP (Shielded Twisted Pair):** Skärmad kabel utrustad med metallfolie eller fläta för att skydda signalen mot elektromagnetiska störningar (EMI - Electromagnetic Interference). Denna typ är nödvändig i industriella miljöer eller där nätverkskablage dras tätt intill kraftiga elkablar och motorer. Användning av STP kräver att hela infrastrukturen, inklusive kontakter och patchpaneler, är korrekt jordad för att skärmningen inte ska fungera som en antenn för störningar.

Backbone: För anslutningar mellan switchar och byggnader används fiberoptik (fiber optics). Eftersom fiber överför data med ljus istället för elektricitet är den helt immun mot elektromagnetiska störningar, vilket gör den idealisk för ryggradsnätet.

Multimode-fiber (MMF): Använder en bredare kärna och är optimerad för kortare avstånd (upp till ca 500 meter) inom byggnader eller mellan närliggande skåp.

Singlemode-fiber (SMF): Har en mycket tunn kärna och kräver laser som ljuskälla. Den används för långa sträckor (flera mil) eller där extremt hög kapacitet krävs mellan geografiskt skilda noder.

2.1.5.3 Övrigt

Strömförsörjning och PoE-budget: Vid dimensionering av switchar i accesslagret krävs en noggrann beräkning av den totala effektförbrukningen för anslutna enheter (PoE budget). Accesspunkter, övervakningskameror och IP-telefoner belastar switchens nätaggregat olika mycket beroende på deras klassificering

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

(802.3af/at/bt). För kritisk utrustning bör avbrottsfri kraftförsörjning (UPS - Uninterruptible Power Supply) installeras för att skydda mot spänningsspikar och korta strömavbrott.

Driftmiljö och skydd: Serverrum och apparatskåp kräver adekvat kylning (cooling) för att transportera bort den värme som aktiv utrustning genererar. Dessutom ska fysisk låsbarhet och skyddade kabelvägar (conduits/cable trays) planeras för att förhindra obehörig åtkomst eller oavsiktlig skada på infrastrukturen.

Vid nyinstallation är det en ekonomisk och teknisk fördel att planera för en expansionsmarginal på 10–30 %. Att installera extra fiberpar eller lämna tomma platser i patchpaneler vid det initiala arbetet är betydligt mer kostnadseffektivt än att utföra kompletteringar i ett senare skede.

Exempel: *Vid en fysisk planering på ett våningsplan med 40 arbetsplatser och 5 accesspunkter. Här installeras en switch med en PoE-budget på minst 180 W (för att täcka AP-last och framtida behov) i ett låsbart väggskåp. Skåpet utrustas med en mindre UPS och kylfläktar, och kopplas till byggnadens centrala kärnswitch via en redundant multimode-fiberlänk för att säkerställa högsta möjliga tillgänglighet.*

2.1.6 WLAN-design (kanal, täckning, kapacitet)

Vid utformningen av trådlösa lokala nätverk (WLAN) prioriteras kapacitet och användarupplevelse framför enbart täckning. En modern design kräver en djup förståelse för hur radiovågor interagerar med den fysiska miljön och hur tillgänglig bandbredd fördelas mellan klienterna. Målet är att skapa en miljö där anslutningen är stabil även vid hög klienttäthet.

2.1.6.1 Platsundersökning och miljöanalys

Grundbulten i en trådlös design är en noggrann platsundersökning (site survey). Genom denna kartläggs de fysiska lokalerna för att identifiera faktorer som påverkar signalstyrka och kvalitet:

Väggmaterial och dämpning (attenuation): Olika material som betong, glas och metall påverkar radiovågornas förmåga att tränga igenom eller reflekteras.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Störkällor (interference): Identifiering av icke-nätverksrelaterade störningar, exempelvis mikrovågsugnar eller Bluetooth-enheter, samt närliggande trådlösa nätverk.

Användartäthet: Analys av var användare förväntas befinna sig under belastningstoppar, vilket styr placeringen och antalet accesspunkter (AP).

2.1.6.2 Frekvensband och kanalstrategi

Valet av frekvensband och kanaler är avgörande för nätverkets totala prestanda. Genom att använda olika band kan nätverket optimeras för både räckvidd (range) och hastighet:

- **2,4 GHz-bandet (802.11b/g/n):** Erbjuder lång räckvidd och god genomträngningsförmåga men har ett begränsat antal icke-överlappande kanaler (1, 6 och 11). Bandet är ofta hårt belastat och mer känsligt för störningar (interference). Detta band bör främst användas för IoT-enheter (Internet of Things) och inte för ordinarie datatrafik. En realistisk genomströmning i detta band ligger ofta mellan 30–60 Mbit/s.
- **5 GHz-bandet (802.11ac/ax):** Ger betydligt fler kanaler och högre kapacitet, men med kortare räckvidd jämfört med 2,4 GHz. Vissa kanaler inom detta band kräver dynamiskt frekvensval (DFS – Dynamic Frequency Selection) för att undvika störningar med exempelvis väderradar. Detta band är mer anpassat för ordinarie datatrafik, där en realistisk hastighet ligger mellan 300–600 Mbit/s (teoretiskt maximalt 866 Mbit/s eller 1,7 Gbit/s beroende på standard och antal dataströmmar).
- **6 GHz-bandet (802.11ax/Wi-Fi 6E):** Introducerar ett stort antal breda och helt rena kanaler, vilket möjliggör extremt hög kapacitet. Räckvidden är dock kortast av de tre banden, vilket kräver en tätare placering av accesspunkter (access points). Bandet är optimerat för ordinarie datatrafik med en förväntad hastighet omkring 1,2 Gbit/s.
- **Wi-Fi 7 (802.11be):** Denna nya standard på 6 GHz-bandet kan använda ännu bredare kanaler (320 MHz) och nå teoretiska hastigheter på över 5 Gbit/s till en enskild enhet, samt uppåt 40 Gbit/s för en hel accesspunkt genom aggregering av flera band.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kanalbredden (channel width) i miljöer med många accesspunkter, såsom skolor, bör begränsas till 20 eller 40 MHz. Detta möjliggör en mer effektiv frekvensåteranvändning (frequency reuse) och minskar risken för ko-kanalsstörning (co-channel interference), där accesspunkter på samma kanal saktar ner varandra.

2.1.6.3 SSID-hantering och säkerhet

För att minimera administrativ overhead bör antalet trådlösa nätverksnamn (SSID) hållas lågt, vanligtvis mellan två och fyra. Varje SSID skickar ut kontrollpaket (beacons) flera gånger per sekund, vilket vid ett stort antal SSID kan konsumera en betydande del av den tillgängliga sändningstiden (airtime).

Trafiken separeras logiskt genom att mappa varje SSID till ett specifikt virtuellt nätverk (VLAN). En vanlig strategi innefattar separata SSID för personal, elever och gäster. Säkerhetsnivån bör alltid vara den högsta möjliga som enheterna stödjer, företrädesvis WPA3 eller WPA2-Enterprise, medan osäkra protokoll som WEP eller TKIP konsekvent bör undvikas.

2.1.6.4 Roaming och prestandaoptimering

För en sömlös användarupplevelse krävs att klienter kan flytta mellan accesspunkter utan avbrott, en process som kallas roaming. Detta kräver en viss cellöverlappning (ofta 15–20 %) så att klienten kan identifiera en ny AP innan anslutningen till den gamla bryts.

- **Min-RSSI:** Genom att konfigurera en lägsta tillåtna signalstyrka (Minimum RSSI) kan nätverket tvinga klienter att släppa en svag anslutning och istället söka efter en starkare accesspunkt i närheten.
- **Band Steering:** En teknik som aktivt uppmuntrar klienter som stödjer 5 GHz eller 6 GHz att använda dessa band istället för det mer belastade 2,4 GHz-bandet.

Exempel: *En genomtänkt konfiguration är en skola där personalens SSID (VLAN 10) använder certifikatbaserad inloggning, medan elevernas SSID (VLAN 20) prioriteras i 5 GHz-bandet med 40 MHz kanalbredd för att balansera hastighet och stabilitet.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.1.7 Säkerhetsdesign och zoner

Säkerhet bör betraktas som en integrerad del av nätverksdesignen från projektstart (security by design). Genom att dela upp nätverket i logiska zoner begränsas rörelsefriheten för en potentiell angripare och risken för spridning vid en säkerhetsincident minimeras. Detta koncept, ofta kallat segmentering (segmentation), innebär att trafik mellan olika delar av nätverket måste passera en kontrollpunkt, vanligtvis en brandvägg, där strikta regler tillämpas.

2.1.7.1 Zonindelning och brandväggspolicys

En effektiv säkerhetsarkitektur bygger på att definiera zoner baserat på tillit och funktion. För varje zon definieras brandväggspolicys enligt principen om minsta möjliga åtkomst (principle of least privilege). Vanliga zoner inkluderar:

LAN (klientnätverk): Denna zon består ofta av ett eller flera lokala nätverk (LAN/VLAN). Syftet är att gruppera betrodda interna enheter, såsom personalens datorer eller elevernas laptops, i separata segment för att kontrollera trafikflödet mellan dem.

Serverzon: *Ett skyddat område för kritiska resurser som databaser och filservrar. Åtkomst hit begränsas till specifika portar och tjänster (t.ex. TCP 445 för fildelning).*

DMZ (Demilitarized Zone): En zon för tjänster som måste vara nåbara från internet, såsom webbservrar eller externa e-postservrar. Enheterna i DMZ har normalt ingen tillåtelse att initiera anslutningar inåt mot det interna nätverket.

Gästnätverk: Ett helt isolerat segment för externa besökare. Trafik tillåts vanligtvis endast mot internet, medan all kommunikation mot interna zoner (LAN/Servrar) blockeras.

Managementzon (hanteringszon): En dedikerad zon för nätverksutrustningens administrativa gränssnitt (switchar, routrar, brandväggar). Denna zon ska isoleras från vanliga klientnätverk och endast vara nåbar från specifika administrativa subnät eller via krypterad fjärranslutning (VPN).

2.1.7.3 Övervakning och loggning

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

En säkerhetsdesign är inte komplett utan förmågan att upptäcka och analysera avvikelser i realtid.

Central loggning (Syslog): All nätverksutrustning bör konfigureras för att skicka händelseloggar till en central server (Syslog server). Vanliga verktyg för detta är **rsyslog** eller **Graylog**. En loggfil kan exempelvis visa exakt när en switchport gick ner eller när en användare misslyckades med att logga in i administrationsgränssnittet.

- För en djupare genomgång av hur operativsystemet genererar och hanterar interna loggfiler, se boken **Datorteknik**.

SIEM (Security Information and Event Management): I mer avancerade miljöer används system som samlar in och korrelerar data från flera källor. Ett populärt verktyg inom öppen källkod är **Wazuh**, medan **Splunk** ofta används i större företagsmiljöer. En SIEM kan exempelvis upptäcka ett mönster där tio misslyckade inloggningsförsök följs av en lyckad inloggning från en ovanlig IP-adress, och omedelbart larma teknikern om ett pågående intrångsförsök.

2.1.8 Drift, övervakning och livscykel

Ett nätverk bör designas för effektiv drift (operations) från dag ett. En tekniskt avancerad lösning tappar snabbt sitt värde om den är svår att övervaka, underhålla eller felsöka. Genom att implementera fasta rutiner och automatiserade verktyg säkerställs nätverkets tillgänglighet och livslängd.

2.1.8.1 Standardisering och grundkonfiguration

För att minimera komplexiteten i en driftsmiljö krävs en standardiserad grundkonfiguration (base configuration) för all nätverksutrustning. Detta innebär att samtliga enheter, oavsett placering, följer samma regelverk för:

Tidssynkronisering (NTP - Network Time Protocol): Säkerställer att alla loggar har korrekta tidsstämplar, vilket är kritiskt vid felsökning och säkerhetsanalys.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fjärråtkomst och säkerhet: Implementering av säkra protokoll som SSH istället för Telnet, samt central autentisering via AAA (Authentication, Authorization, and Accounting).

Information och banners: Standardiserade meddelanden som visas vid inloggning för att informera om nätverkets ägare och gällande policys.

2.1.8.2 Övervakning och larmhantering

Övervakning (monitoring) handlar om att proaktivt identifiera problem innan de påverkar användarna. Detta sker ofta via protokoll som **SNMP (Simple Network Management Protocol)** eller modernare strömmande telemetri (telemetry). Systemet bör konfigureras för att övervaka kritiska parametrar och generera larm vid överskridna gränsvärden:

Fysisk status: Kontroll av fläktar, temperatur och nätaggregatens hälsa.

Kapacitet: Övervakning av processorns belastning (CPU), minnesanvändning (RAM) och tillgänglig PoE-budget.

Nätverksprestanda: Analys av bandbreddsutnyttjande på upplänkar (uplinks), paketförluster och trådlös sändningstid (airtime).

Exempel på populära programvaror för övervakning är **Zabbix**, **LibreNMS** och **PRTG**. Dessa verktyg kan visualisera nätverkets status i realtid och skicka aviseringar via e-post eller meddelandetjänster vid incidenter.

2.1.8.3 Underhåll, backup och ändringshantering

Livscykelhantering innebär att nätverket kontinuerligt uppdateras och säkras.

Konfigurationsbackup: Automatiserade backuper av enheternas konfigurationsfiler bör ske nattligt. Verktyg som **Oxidized** eller **Rancid** kan användas för att inte bara spara kopior, utan även versionshantera ändringar så att tekniker kan se exakt vad som ändrats mellan två tidpunkter.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Firmware-uppgraderingar: Regelbundna uppdateringar av nätverksutrustningens mjukvara är nödvändiga för att täppa till säkerhetshål och introducera ny funktionalitet.

Ändringshantering (Change Management): Varje större ändring i nätverket bör följa en fastställd process med en tydlig dokumentation av syftet, ett tidsfönster för genomförandet och en plan för att återställa nätverket (rollback plan) om något går fel.

2.1.8.4 Dokumentation och tillgångshantering

En uppdaterad dokumentation är nätverksteknikerns viktigaste verktyg. Detta inkluderar en logisk topologiritning, en aktuell IP- och VLAN-plan samt en matris över brandväggsregler (ACL matrix). För att hålla ordning på hårdvaran används ofta en enkel **CMDB (Configuration Management Database)** eller assetlista som innehåller:

- Fysisk plats och rackenhet.
- Serienummer och inköpsdatum.
- IP-adress och management-VLAN.
- Supportstatus och garantitid (End-of-Life/End-of-Support).

Ett kraftfullt verktyg för att kombinera dokumentation av IP-planering och fysisk infrastruktur är **NetBox**, vilket fungerar som en central källa för sanning (Source of Truth) för hela nätverksmiljön.

För en djupare förståelse av hur serverhårdvara och operativsystemets livscykel hanteras, se boken **Datorteknik**.

2.1.9 Test, validering och acceptans

Innan ett nätverk övergår i full drift krävs en systematisk process för testning och validering. Syftet är att objektivt bevisa att den tekniska lösningen uppfyller de kravspecifikationer (requirements specifications) som fastställdes i nätverkets planeringsfas. Genom att genomföra ett strukturerat acceptanstest (acceptance testing) säkerställs att alla funktioner arbetar korrekt under både normal belastning och vid simulerade felscenarier.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.1.9.1 Funktionstester och policyverifiering

Det första steget i valideringen är att kontrollera de grundläggande nätverkstjänsterna. Detta innefattar verifiering av adressering, namnupplösning och åtkomstkontroll:

Connectivity och VLAN-reachability: Kontroll av att enheter i rätt segment kan nå varandra och att kommunikation mellan olika virtuella nätverk (inter-VLAN routing) fungerar enligt designen.

DHCP och DNS: Test av adressutdelning och kontroll av att korrekta inställningar, såsom gateway och DNS-servrar, levereras till klienterna.

Brandväggspolicys: Verifiering av att trafik som ska blockeras nekas, och att endast tillåten trafik når skyddade zoner. Här används ofta verktyg som **Nmap** för att skanna portar och säkerställa att inga oavsiktliga öppningar finns i brandväggsreglerna.

2.1.9.2 Prestandamätning och kapacitet

När funktionaliteten är bekräftad genomförs mätningar av nätverkets faktiska prestanda (performance measurement). Detta görs för att säkerställa att bandbredd, latens (latency) och jitter (variation i fördröjning) ligger inom fastställda gränsvärden.

Genomströmning (Throughput): Mätning av nätverkets förmåga att transportera data under hög belastning. Programvaran **iPerf3** är en branschstandard för att mäta maximal TCP- och UDP-genomströmning mellan två noder.

Latens och Jitter: Detta är särskilt kritiskt för realtidstjänster som röstsamtal (VoIP) och videokonferenser. Verktyg som **NetBeez** eller inbyggda funktioner i nätverksutrustningen kan användas för att kontinuerligt mäta svarstider och variationer i nätverket.

2.1.9.3 Redundanstester och failover

För att validera nätverkets robusthet måste redundansmekanismerna utsättas för stresstester. Det handlar om att simulera hårdvaru- eller länkfel och mäta hur snabbt nätverket återhämtar sig (convergence time).

STP/RSTP-konvergens: Verifiering av att loopskyddet aktiverar alternativa vägar vid ett kabelbrott utan att orsaka broadcast-stormar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Gateway-redundans: Om protokoll för redundanta gateways som **VRRP** (Virtual Router Redundancy Protocol) eller **HSRP** (Hot Standby Router Protocol) används, testas växling (failover) mellan routrar för att säkerställa att klienterna bibehåller sin internetanslutning vid ett maskinfel.

Uplink-failover: Simulering av fiberbrott mellan distributions- och kärnswitchar för att bekräfta att trafiken automatiskt omdirigeras via sekundära länkar.

2.1.9.4 WLAN-validering under belastning

Trådlösa nätverk kräver specifik validering för att säkerställa täckning och kapacitet i hela den planerade miljön.

Roaming-test: Verifiering av att en rörlig klient kan flytta mellan accesspunkter utan märkbara avbrott i pågående sessioner. Verktyg som **WiFiman** eller **Ekahau** kan användas för att visualisera signalstyrka och roaming-beteende.

Lasttest (Stress test): Genomförande av mätningar under realistisk belastning, exempelvis under en lektion där många klienter streamar video samtidigt, för att observera sändningstid (airtime) och eventuell kanalinterferens.

2.1.9.5 Acceptansprotokoll och dokumentation

Samtliga testresultat sammanställs i ett formellt acceptansprotokoll (acceptance protocol). Detta dokument fungerar som ett kvitto på att installationen uppfyller de funktionella och icke-funktionella kraven. Dokumentationen bör även inkludera instruktioner för återställning (rollback plan) vid framtida uppgraderingar av systemet.

Exempel: *Företaget Roynes Garn & IT simulerar ett fiberbrott mellan en distributionsswitch och kärnnätet. Om loggarna i övervakningssystemet visar att failover skedde på under en sekund och pågående applikationer förblev opåverkade, anses redundanskravet vara uppfyllt.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Vad är skillnaden mellan funktionella och icke-funktionella krav i en behovsanalys (requirements analysis)?
2. Vilka är de tre huvudlagren i den hierarkiska nätverksmodellen (hierarchical network design)?
3. Vilken specifik funktion har distributionslagret (distribution tier) i förhållande till accesslagret?
4. Varför beskrivs kärnlagret (core tier) som nätverkets ryggrad (backbone) och vad prioriteras där?
5. Vad innebär begreppet "collapsed core" och i vilka miljöer är det lämpligt?
6. Varför är en genomtänkt namnstandard (naming convention) avgörande vid felsökning i större nätverk?
7. Vad är den tekniska skillnaden mellan UTP- och STP-kablage gällande störningsskydd och installation?
8. Varför rekommenderas 5 GHz- eller 6 GHz-banden för ordinarie datatrafik snarare än 2,4 GHz-bandet?
9. Vilket syfte tjänar en DMZ (demilitarized zone) i en säkerhetsdesign med avseende på trafikflöden?
10. Vad är målet med att genomföra ett redundanstest (failover test) som en del av ett acceptansprotokoll?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

Cisco Design Guides (Övergripande genomgång av hierarkisk nätverksdesign [hierarchical network design]) - https://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Campus/campover.html

IEEE 802.3 (Standarder och tekniska specifikationer för Ethernet) - https://standards.ieee.org/standard/802_3-2022.html

Wi-Fi Alliance (Viktiga aspekter och nyheter kring Wi-Fi 6 och Wi-Fi 6E) - https://www.wi-fi.org/discover-wi-fi/wi-fi-certified-6

NIST (Guide för nätverkssegmentering [network segmentation] och säkerhet i industriella kontrollsystem) - https://csrc.nist.gov/publications/detail/sp/800-82/rev-2/final

IETF RFC 1918 (Adressallokering för privata nätverk [private address space]) - https://datatracker.ietf.org/doc/html/rfc1918

SNMP (Protokollspecifikationer för nätverkshantering [network management]) - http://www.snmp.com/protocol/

NetBox (Dokumentation för central källa för sanning [source of truth] gällande infrastruktur) - https://docs.netbox.dev/en/stable/

iPerf3 (Användardokumentation och mätning av genomströmning [throughput testing]) - https://iperf.fr/iperf-doc.php

ANSI/TIA-606-B (Standard för administration av infrastruktur för telekommunikation) - https://www.tiaonline.org/

RIPE NCC (Grunderna i IP-planering för IPv6) - https://www.ripe.net/manage-ips-and-asns/ipv6/ipv6-address-planning-basics/

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.2 VLAN och segmentering

Segmentering innebär att ett fysiskt nätverk delas upp i mindre, logiskt åtskilda domäner. Syftet är att begränsa spridningen av nätverkstrafik, tekniska fel och säkerhetshot, vilket resulterar i en infrastruktur som är både effektivare och säkrare. I ett traditionellt, ofragmenterat nätverk – ofta kallat ett platt nätverk (flat network) – ingår samtliga anslutna enheter i en och samma broadcastdomän. Det innebär att varje broadcast-paket som skickas ut, exempelvis en ARP-förfrågan eller DHCP-begäran, måste tas emot och behandlas av precis alla enheter i nätverket. I en miljö med ett stort antal klienter skapas då en betydande mängd onödig trafik som konsumerar både bandbredd och processorkraft, vilket sänker nätverkets totala prestanda.

2.2.1 Varför segmentera nät?

I praktiken genomförs segmentering oftast genom användning av virtuella lokala nätverk (VLAN), vilka kompletteras med brandväggspolicys för att kontrollera trafiken mellan de olika delarna. Denna metodik medför tre huvudsakliga fördelar för nätverksdriften:

Prestandaoptimering genom mindre broadcastdomäner minskar den mängd onödig trafik som når klienternas nätverkskort. Genom att begränsa räckvidden för broadcast- och multicast-trafik säkerställs att nätverksresurserna används mer effektivt, vilket leder till en snabbare och mer stabil användarupplevelse.

Säkerhetsmässiga fördelar är ofta den primära drivkraften bakom segmentering. Genom att isolera olika användargrupper och tjänster i egna segment begränsas möjligheten till lateral förflyttning (lateral movement) för en potentiell angripare eller skadlig kod. Om en infekterad enhet identifieras i ett segment, förhindrar segmenteringen att hotet sprids fritt till kritiska delar av nätverket. Brandväggspolicys mellan VLAN-gränserna gör det möjligt att definiera exakt vilka flöden som är tillåtna, vilket skapar ett starkt skydd för känslig data.

Felsökning och hantering förenklas avsevärt när nätverket har tydliga logiska gränser. Om ett nätverksfel uppstår kan en tekniker snabbare ringa in orsaken genom att observera vilket segment som

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

påverkas. Det gör det också möjligt att utföra underhåll eller ändringar i ett segment utan att riskera oavsiktlig påverkan på hela nätverket.

Exempel: *En skola väljer att segmentera titt nätverk genom att separera trafik för personal, elever, gäster, servrar och administrativ hantering (management) i helt egna VLAN. Genom att endast tillåta specifikt definierade flöden mellan dessa zoner skapas en robust miljö där exempelvis en gäst på skolan inte har någon teknisk möjlighet att nå personalens interna filservrar eller nätverkets administrativa gränssnitt.*

2.2.2 VLAN-grunder: 802.1Q, access och trunk

Ett virtuellt lokalt nätverk (VLAN) är en logisk indelning av ett fysiskt nätverk som gör det möjligt att separera enheter och trafik oberoende av deras fysiska anslutningspunkt. För att en switch ska kunna hantera denna logik delas dess portar in i olika lägen beroende på vilken typ av utrustning som ansluts och hur trafiken ska transporteras genom nätverket.

Den mest grundläggande porttypen är **accessporten (access port)**. En accessport är konfigurerad att tillhöra ett enda specifikt VLAN och används vanligtvis för att ansluta slutanvändarnas utrustning, såsom datorer, IP-telefoner eller skrivare. När trafik lämnar en accessport mot en klient tas all VLAN-information bort; trafiken är då otaggad (untagged). Detta innebär att klienten inte behöver ha någon kännedom om nätverkets virtuella struktur för att kunna kommunicera.

När trafik från flera olika VLAN behöver transporteras över en och samma fysiska länk, exempelvis mellan två switchar eller mellan en switch och en router, används istället en **trunkport (trunk port)**. För att mottagaren ska veta vilket VLAN en specifik dataram tillhör används industristandarden **IEEE 802.1Q**. Genom denna standard läggs en unik identifikation, en tagg (tag), till i Ethernet-ramens header. Denna tagg innehåller ett VLAN-ID, vilket är ett nummer mellan 1 och 4094 som definierar det logiska nätverket. Tidigare fanns tillverkarspecifika protokoll för detta ändamål, såsom Ciscos **ISL (Inter-Switch Link)**, men i modern nätverksdesign betraktas dessa som föråldrade (legacy) då 802.1Q erbjuder universell kompatibilitet mellan olika tillverkare.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Ett centralt begrepp vid konfiguration av trunkar är **Native VLAN**. Detta definierar vilket VLAN som ska hantera trafik som av någon anledning anländer till trunken utan en 802.1Q-tagg. I en säker nätverksdesign är det god praxis att ändra Native VLAN från standardinställningen (ofta VLAN 1) till ett unikt och oanvänt VLAN-ID. Genom att isolera otaggad kontrolltrafik från användarnas datatrafik minskas risken för specifika säkerhetshot, såsom VLAN-hoppning (VLAN hopping).

För att automatisera uppsättningen av trunkar använder vissa tillverkare protokoll som **DTP (Dynamic Trunking Protocol)**. DTP försöker automatiskt förhandla med motparten för att avgöra om en länk ska bli en trunk eller en accessport. Även om detta kan förenkla installationen, medför det risker i form av oförutsägbarhet och potentiella säkerhetshål. En fackmässig och säker design innebär därför att man konsekvent stänger av DTP och istället statiskt konfigurerar portläget som antingen access eller trunk. Detta säkerställer att nätverkets topologi förblir exakt så som teknikern har planerat den.

Exempel: *Två switchar, SW1 och SW2. Dessa kopplas samman med en trunklänk som är konfigurerad att bära trafik för VLAN 10 (Personal), 20 (Elev), 30 (Gäst) och 40 (Servrar). Om en nätverksskrivare ansluts till port 12 på SW1, konfigureras den specifika porten som en accessport i VLAN 20. Skrivaren kan då kommunicera med elevernas datorer i samma VLAN över hela nätverket, trots att den bara ser vanlig, otaggad trafik vid sitt fysiska uttag.*

Switchar i en VLAN-miljö

I sin enklaste form fungerar en switch genom att bygga upp en **MAC-tabell** (MAC address table), även kallad en **CAM-tabell** (Content Addressable Memory). Denna tabell mappar fysiska MAC-adresser till specifika portar på switchen. När en switch tar emot en dataram läser den av avsändarens MAC-adress och sparar informationen om vilken port den anlände på. Nästa gång trafik ska skickas till den adressen vet switchen exakt vilken port den ska använda istället för att skicka ut trafiken på samtliga portar.

När VLAN introduceras förändras switchens interna logik genom att den skapar vad som kan liknas vid flera oberoende switchar i samma fysiska hårdvara. Detta hanteras tekniskt genom att lägga till en kolumn för

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

VLAN-ID i MAC-tabellen. Varje MAC-adress knyts nu inte bara till en port, utan även till ett specifikt virtuellt nätverk.

Denna logiska isolering (logical isolation) innebär att switchen utför en strikt filtrering:

- **Inlärning:** När en dator på en accessport i VLAN 10 skickar data, lär sig switchen dess MAC-adress enbart för VLAN 10.
- **Vidarebefordran:** Om en annan enhet i VLAN 20 försöker skicka data till samma MAC-adress, kommer switchen att neka detta, eftersom den enheten inte existerar i VLAN 20:s logiska tabell.
- **Översvämning (Flooding):** Om destinationsadressen är okänd, eller om det rör sig om en broadcast (ff:ff:ff:ff:ff:ff), skickar switchen endast ut kopior av ramen på de portar som tillhör samma VLAN. Detta är grunden för hur broadcastdomäner begränsas.

Det är här skillnaden mellan en **ohanterad switch** (unmanaged switch) och en **hanterad switch** (managed switch) blir tydlig. En ohanterad switch saknar förmågan att läsa av 802.1Q-taggar eller dela upp sin MAC-tabell; den ser hela nätverket som en enda stor domän. En hanterad switch ger däremot administratören full kontroll över hur dessa logiska gränser dras, vilket möjliggör övervakning, hastighetsbegränsning och avancerad säkerhetsfunktionalitet för varje enskilt VLAN.

2.2.3 Inter-VLAN-routing: L3-switch eller “router-on-a-stick”

Eftersom virtuella nätverk (VLAN) fungerar som helt isolerade broadcastdomäner, kan enheter i olika VLAN som standard inte kommunicera med varandra, även om de är anslutna till samma fysiska switch. För att möjliggöra trafikflöden mellan dessa segment krävs en enhet som arbetar på nätverkslagret (lager 3), en process som kallas **Inter-VLAN-routing**. I modern nätverksdesign löses detta huvudsakligen på två sätt: genom en router eller brandvägg med en trunkanslutning, eller direkt i en lager 3-switch (Layer 3 switch).

Router-on-a-stick

Den traditionella metoden, ofta kallad **router-on-a-stick**, innebär att en router ansluts till switchen via en enda fysisk trunklänk. På routern skapas sedan logiska **subinterfaces** (subinterfaces), där varje subinterface

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

tilldelas en 802.1Q-tagg som motsvarar ett specifikt VLAN. Varje sådant virtuellt gränssnitt ges en unik IP-adress som fungerar som **standard-gateway** (default gateway) för klienterna i det aktuella VLAN-segmentet. Fördelen med denna metod är att den ofta används i kombination med en brandvägg, vilket gör det enkelt att centralisera säkerhetspolicys och granska all trafik som rör sig mellan segmenten. Nackdelen är att all trafik måste färdas upp till routern och tillbaka över samma länk, vilket kan skapa en flaskhals om mängden intern datatrafik är mycket stor.

L3-routing

I större och mer prestandakrävande nätverk används istället en **lager 3-switch**. Här sker routingen direkt i switchens hårdvara med hjälp av specialiserade kretsar (ASIC), vilket ger en extremt hög genomströmning (throughput) och minimal fördröjning. Tekniskt åstadkoms detta genom att konfigurera ett **virtuellt switchgränssnitt** (SVI - Switch Virtual Interface) för varje VLAN. Ett SVI fungerar som en virtuell routerport inuti switchen och agerar gateway för segmentet. Genom att flytta routingen till distributions- eller kärnlagret avlastas nätverkets externa router eller brandvägg, som då kan fokusera helt på trafik mot internet eller andra externa nätverk.

Valet av placering för nätverkets gateway styrs av behovet av skalbarhet och säkerhetskontroll. I en miljö där trafik mellan VLAN kräver djupgående paketinspektion (DPI) eller strikta brandväggsregler är en centraliserad placering i en brandvägg att föredra. Om målet däremot är högsta möjliga hastighet för intern trafik, exempelvis mellan användare och lokala servrar, är en lager 3-switch det mest effektiva valet.

Exempel: *En skola där elevernas VLAN 20 har tilldelats subnätet 10.20.0.0/22. Genom att skapa ett SVI för VLAN 20 med IP-adressen 10.20.0.1 på en central lager 3-switch, får alla elever en väg ut ur sitt eget segment. Switchen kan sedan programmeras med åtkomstlistor (ACL) eller skicka trafiken vidare till en brandvägg som tillämpar specifika policys. I detta scenario kan brandväggen konfigureras för att tillåta trafik från elevernas VLAN mot internet, men samtidigt blockera alla försök att initiera anslutningar mot serversegmentet i VLAN 40, vilket säkerställer att känslig data förblir skyddad trots att routing är aktiverad.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.2.4 Designmönster: Vanliga VLAN och segmenteringsprofiler

En välplanerad segmenteringsstrategi utgår från funktionella zoner där varje zon tilldelas ett eget VLAN-ID och subnät. Följande mönster är vanligt förekommande i moderna nätverksinstallationer:

- **Personal (VLAN 10):** Detta segment är avsett för administrativa medarbetare och lärare. Här tillåts fullständig åtkomst till interna resurser såsom Active Directory (AD), filservrar och administrativa stödsystem. Eftersom enheterna i detta segment hanterar känsliga data krävs ofta högre säkerhetskrav på klientsidan.
- **Elev (VLAN 20):** Ett segment med hög klienttäthet men begränsad behörighet. Elever ges åtkomst till internet och nödvändiga lärplattformar, men nekas konsekvent tillträde till administrativa resurser eller nätverkets infrastrukturkomponenter.
- **Gäst (VLAN 30):** En isolerad zon för externa besökare. Här tillämpas ofta strikt klientisolering (client isolation), vilket innebär att trådlösa klienter i samma VLAN inte kan kommunicera med varandra. Trafiken styrs via hårda utgångsregler (egress rules) i brandväggen som endast tillåter trafik ut mot internet.
- **Servrar (VLAN 40):** En skyddad zon för nätverkets centrala tjänster, inklusive DNS, DHCP, filservrar och domänkontrollanter. Åtkomst till denna zon styrs av strikta åtkomstlistor (ACL) baserat på vilka tjänster (portar) som de olika klientsegmenten behöver nå.

Förutom de användarfokuserade segmenten krävs dedikerade nätverk för nätverkets egen infrastruktur och specialiserade tjänster:

- **Infrastruktur och WLAN-AP (VLAN 50):** Används uteslutande för hantering av accesspunkter och tunnling av trådlös trafik. Genom att separera accesspunkternas egen trafik från klienternas data försvåras försök att manipulera nätverksutrustningen.
- **Voice (VLAN 60):** Dedikerat för IP-telefoni och realtidskommunikation. Eftersom rösttrafik är extremt känslig för fördröjning och jitter, implementeras tjänstekvalitet (QoS - Quality of Service) med hjälp av prioriteringstaggar (DSCP eller 802.1p). I switchen definieras ofta en förtroendegräns (trust boundary) så nära

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

telefonen som möjligt, vilket säkerställer att rösttrafiken prioriteras genom hela nätverket.

- **Management (VLAN 99):** Ett kritiskt segment för nätverksutrustningens administrativa gränssnitt. Detta nätverk bör aldrig vara nåbart direkt från klientsegmenten. Åtkomst tillåts endast från specifika administrativa arbetsstationer eller via krypterad fjärranslutning (VPN).

Genom att konsekvent tillämpa dessa designmönster skapas en förutsägbar miljö där säkerhetspolicys kan appliceras på gruppnivå istället för på enskilda enheter. Ett gästnätverk på en skola kan exempelvis konfigureras så att all trafik mot interna adresser (RFC1918) automatiskt blockeras i brandväggen, medan personalsegmentet ges de öppningar som krävs för det dagliga arbetet.

2.2.5 Lager-2-skydd: “hårda portar” och kontroll av giftig trafik

Grunden för nätverkstopologins stabilitet vilar på **Spanning Tree Protocol (STP)**. För att nätverket ska kunna upptäcka loopar och bestämma vilka redundanta vägar som ska blockeras eller vara aktiva, skickar switchar kontrollmeddelanden till varandra som kallas **BPDU (Bridge Protocol Data Unit)**. En BPDU fungerar som nätverkets ”hjärtslag” och innehåller information om nätverkets struktur och vilken switch som agerar central punkt, så kallad Root Bridge. Genom att förstå och kontrollera dessa meddelanden kan vi implementera avancerade skydd.

En primär säkerhetsåtgärd är att alltid konfigurera portlägen explicit. Genom att stänga av automatisk förhandling (DTP/auto trunk) och låsa accessportar till ett fast läge, förhindras försök att olovligen etablera en trunkanslutning för att få åtkomst till flera VLAN. Vidare används **Port Security** för att begränsa antalet tillåtna fysiska adresser (MAC-adresser) per port. Genom att använda funktionen ”sticky” kan switchen lära in den första enheten som ansluts och därefter blockera alla andra, vilket effektivt stoppar obehöriga från att koppla in egen utrustning i nätverksuttag.

För att skydda nätverkets logiska struktur används funktioner som **BPDU Guard** och **Root Guard**. BPDU Guard konfigureras på accessportar där

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

endast slutanvändare ska finnas; om porten tar emot ett BPDU-meddelande innebär det att någon har anslutit en annan switch eller enhet som försöker påverka nätverkets topologi. Porten stängs då omedelbart av för att skydda nätverket. Root Guard används istället för att säkerställa att ingen annan switch i nätverket kan utse sig själv till Root Bridge, vilket håller nätverkets hierarki intakt och förutsägbar.

Ytterligare lager av skydd hanterar hot relaterade till adressering och identitet:

- **DHCP Snooping:** Denna funktion innebär att switchen endast litar på legitima DHCP-servrar anslutna till specifika, betrodda portar. Det förhindrar att en angripare sätter upp en falsk DHCP-server för att styra användarnas trafik till sig själv (Man-in-the-Middle).
- **Dynamic ARP Inspection (DAI):** Genom att validera ARP-tabellen mot den databas som DHCP Snooping bygger upp, kan switchen stoppa försök till ARP-spoofing, där en enhet försöker utge sig för att vara en annan (exempelvis nätverkets gateway).
- **IP Source Guard:** Denna funktion kontrollerar att den käll-IP som en klient använder faktiskt matchar den adress som tilldelats via DHCP, vilket blockerar försök att använda falska IP-adresser på nätverket.
- **Storm Control:** Fungerar som en säkerhetsventil som stryper mängden broadcast- eller multicast-trafik om den överskrider en viss nivå. Detta skyddar nätverket från att bli helt paralyserat vid hårdvarufel eller loopar som skapar extrem trafikbelastning.

Exempel: *Vikten av dessa skydd visas tydligt i en skolmiljö där en elev ansluter en medtagen ”smart-switch” till ett vägguttag i ett klassrum. Utan skydd skulle denna lilla switch kunna börja skicka ut BPDUs och potentiellt orsaka en omräkning av hela skolans nätverkstopologi, med avbrott som följd. Med BPDU Guard aktiverat på porten identifieras den främmande switchen omedelbart, porten slår ifrån och resten av nätverket förblir opåverkat.*

2.2.6 VLAN i WLAN – SSID, mappning och gästisolering

I ett modernt företagsnätverk fungerar det trådlösa nätverket som en naturlig förlängning av den trådbundna infrastrukturen. För att

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

upprätthålla samma säkerhets- och segmenteringsmodell oavsett hur en användare ansluter, används teknik för att mappa ett **SSID** (Service Set Identifier) direkt till ett specifikt **VLAN**. Detta innebär att när en accesspunkt (access point - AP) tar emot trafik från en trådlös klient, märker den ramen med rätt VLAN-id innan den skickas vidare över nätverkets trunk-anslutningar. På så sätt kan brandväggar och routrar tillämpa samma regler för en elev eller anställd, oavsett om de sitter vid en fast dator eller använder en laptop via Wi-Fi.

En av de viktigaste tillämpningarna av denna teknik är hanteringen av gästnätverk (guest networks). Genom att tilldela gäster ett eget SSID som är kopplat till ett isolerat VLAN (exempelvis VLAN 30), kan man säkerställa att besökare aldrig får tillgång till interna resurser som servrar eller skrivare. I dessa segment är det fackmässig standard att aktivera **klientisolering** (client isolation / client-to-client blocking). Denna funktion förhindrar trådlösa klienter i samma VLAN från att kommunicera med varandra, vilket effektivt stoppar spridning av skadlig kod mellan gästers enheter. För att ytterligare skydda nätverket används ofta strikta **utgångsfilter** (egress filters) i brandväggen som endast tillåter vanlig webbtrafik (TCP/UDP) ut mot internet.

Vid design av trådlösa nätverk bör man sträva efter att hålla antalet SSID på en låg nivå, normalt mellan två och fyra per accesspunkt. Varje extra SSID skapar administrativ overhead i form av kontrollmeddelanden (beacons) som tar upp värdefull sändningstid i luften. För känsliga nätverk där personal eller elever hanterar inloggningsuppgifter används **802.1X/Enterprise-autentisering**, vilket ger varje användare en unik identitet och möjliggör dynamisk tilldelning av VLAN baserat på vem som loggar in. För gästnätverk kan en **captive portal** (inloggningssida) användas om det finns behov av att användare accepterar ordningsregler, men det bör kombineras med hastighetsbegränsning (**rate limit**) för att förhindra att gäster konsumerar all tillgänglig bandbredd.

Förtydligande (sammanfattning):

- **SSID-mappning:** Genom att koppla SSID till VLAN förlängs nätverkets segmentering ut i luften.
- **Optimering:** Begränsa antalet SSID till 2–4 per accesspunkt för att spara bandbredd.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Säkerhet:** Använd 802.1X/Enterprise för personliga nät och klientisolering (client isolation) för gäster.
- **Gästhantering:** Internet-only-åtkomst via strikta filter och eventuellt en captive portal för inloggning.

Exempel: *På en skola mappas det trådlösa nätverket så att SSID "Elev" leder till VLAN 20 och SSID "Personal" till VLAN 10. Ett tredje SSID, "Gäst", är kopplat till VLAN 30. En elev som ansluter till det trådlösa nätverket hamnar därmed i samma logiska segment som de stationära datorerna i datorsalarna. Gäster som ansluter till sitt SSID isoleras helt från varandra på accesspunkten och deras trafik styrs via brandväggen direkt ut till internet, utan att de ens kan se att det finns andra servrar eller enheter i skolans nätverk.*

2.2.7 STP, RSTP och MSTP i redundanta VLAN-miljöer

Spanning Tree Protocol (STP), definierat i standarden IEEE 802.1D, fungerar som nätverkets automatiska loopskydd. Protokollet identifierar redundanta länkar och försätter dem i ett blockerande läge (blocking state) så att endast en aktiv logisk väg existerar mellan två punkter, vilket skapar en trädstruktur i topologin. Om en aktiv länk går ner, upptäcker protokollet detta och öppnar automatiskt en av de blockerade vägarna. Den ursprungliga STP-standarden är dock relativt långsam, med en konvergenstid (convergence time) på upp till 50 sekunder innan trafiken kan flöda igen.

För att möta kraven i moderna nätverk utvecklades **Rapid STP (RSTP)**, standarden 802.1w. RSTP fungerar enligt samma grundprinciper men använder en betydligt snabbare metod för att förhandla fram topologin. Vid ett länkfel kan RSTP återställa anslutningen på bråkdelar av en sekund eller upp till några sekunder, vilket minimerar påverkan på pågående sessioner och applikationer.

När nätverket innehåller ett stort antal virtuella nätverk (VLAN) kan hanteringen av Spanning Tree bli komplex. I en miljö med hundratals VLAN skulle körandet av en separat STP-instans per VLAN (PVST+) konsumera onödigt mycket processorkraft i switcharna. Här introduceras **Multiple STP (MSTP)**, standarden 802.1s. MSTP gör det möjligt att gruppera flera VLAN i ett fåtal logiska instanser (instances). Detta

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

förenklar driften och möjliggör effektiv lastbalansering (load balancing) genom att olika instanser kan ha olika switchar som centralpunkt, så kallad **Root Bridge**.

En kritisk aspekt i designen är just kontrollen över nätverkets Root Bridge. I en oplanerad miljö kan den äldsta eller sämsta switchen i nätverket råka bli Root Bridge baserat på sitt MAC-nummer, vilket tvingar trafiken att ta ineffektiva vägar. En fackmässig konfiguration innebär att man manuellt låser rollen som Root Bridge till nätverkets mest centrala och kraftfulla switchar, vanligtvis i distributions- eller kärnlagret (core). Detta säkerställer en förutsägbar och optimal trafikväg.

För att ytterligare optimera prestandan tillämpas **VLAN-pruning** på trunklänkar. Detta innebär att man begränsar vilka VLAN som tillåts passera en specifik trunk (allowed-list). Genom att endast tillåta de VLAN som faktiskt behövs i motstående switch minskas mängden onödig broadcast-trafik och säkerheten ökar. Det är härvid av yttersta vikt att listorna över tillåtna VLAN matchar i båda ändar av länken för att undvika svårdiagnosticerade anslutningsproblem. Liksom i tidigare kapitel betonas vikten av ett konsekvent konfigurerat **Native VLAN** som hålls separat från produktionstrafik för att minimera risker för otaggad trafik som når fel segment.

Exempel: *En redundant miljö är två kärnswitchar som är sammankopplade med RSTP. Genom att konfigurera den ena som primär Root Bridge för hälften av nätverkets VLAN, och den andra som primär för den resterande hälften, utnyttjas nätverkets totala bandbredd effektivt samtidigt som full redundans bibehålls vid ett maskinfel.*

2.2.8 Dokumentation och felsökning

En nätverksdesign är aldrig starkare än sin dokumentation. I en segmenterad miljö, där logiska gränser korsar den fysiska infrastrukturen, är en uppdaterad dokumentation teknikerens viktigaste resurs för både drift och felsökning. Utan en tydlig karta över VLAN-id, subnät och gateways blir det nästan omöjligt att snabbt identifiera varför en specifik tjänst inte fungerar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Dokumentationsmodeller för olika arkitekturer

Beroende på om nätverket använder en central router eller en distribuerad lager 3-switch ser dokumentationen olika ut. Nedan följer två exempel på hur informationen bör struktureras:

Exempel 1: Router-on-a-Stick (RoaS)

Här ligger fokus på routerns subinterfaces och de 802.1Q-taggar som används för att separera trafiken över trunken.

VLAN-ID	Namn	Subinterface	802.1Q Tag	Gateway (IP/Mask)	Syfte
10	Personal	gi0/0.10	10	10.10.0.1/24	Administrativ trafik
20	Elev	gi0/0.20	20	10.20.0.1/22	Elevdatorer/ Laptops
99	Management	gi0/0.99	99	10.99.0.1/24	Hantering av switchar/AP

Exempel 2: Lager 3-switch (L3)

Här dokumenteras istället de virtuella gränssnitten (SVI) direkt i switchen, samt vilka fysiska portar eller trunkar som bär respektive VLAN.

VLAN-ID	SVI Namn	IP-adress	Nätmask	DHCP-Scope	Primär trunk-rutt
40	Vlan40_Serv	10.40.0.1	255.255.255.0	10.40.0.10–50	Core-SW1 <-> Dist-SW1
60	Vlan60_Voice	10.60.0.1	255.255.255.0	10.60.0.100–250	Samtliga access-trunkar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Strukturerad felsökning (Troubleshooting)

När ett fel rapporteras i ett segmenterat nätverk bör felsökningen ske systematiskt, med start i accesslagret (bottom-up). Genom att verifiera varje lager i ordning undviks onödigt arbete med komplexa routing-inställningar om felet i själva verket är fysiskt.

Felsökningsschema i sex steg:

1. **Fysisk länk och portläge:** Kontrollera att kabeln är hel och att porten är i rätt läge. Är det en accessport till en klient eller en trunk till en annan switch? Är rätt VLAN tilldelat på porten?
2. **Trunk-validering:** Om trafiken passerar mellan switchar, kontrollera att det aktuella VLAN-id:t finns med i listan över tillåtna VLAN (allowed list). Verifiera även att Native VLAN matchar i båda ändar av trunken för att undvika trafikläckage.
3. **Lager 3-status (Gateway):** Kontrollera att det virtuella gränssnittet (SVI) eller subinterfacet är i läge "up/up". Svara gateway på ping? Har klienten rätt IP-adress och nätmask i förhållande till sitt VLAN?
4. **Infrastrukturstjänster:** Verifierar att klienten faktiskt får ett DHCP-lease och att DNS-inställningarna pekar mot rätt servrar. Kan klienten göra namnuppslagningar?
5. **Spanning Tree (STP):** Undersök nätverkstopologin via switchen. Finns det en blockerad länk där du förväntar dig trafik? Är nätverkets Root Bridge placerad på den planerade enheten (Core/Distribution)?
6. **Tabellanalys:** Granska switchens MAC-tabell och routerns ARP-tabell. Ser du klientens fysiska adress i rätt VLAN-segment?

Exempel: *En användare i personalstyrkan rapporterar att de inte når filservern. Teknikern börjar med att kontrollera switchen och upptäcker att uttaget vid användarens plats av misstag har blivit konfigurerat som access i VLAN 20 (Elev) istället för VLAN 10 (Personal). Eftersom brandväggspolicyn blockerar all trafik från elevsegmentet mot serversegmentet, nekades användaren åtkomst. Genom att*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

enkelt flytta porten till rätt VLAN i konfigurationen återställs förbindelsen omedelbart.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Vilka är de tre främsta anledningarna till att man väljer att segmentera ett platt nätverk (flat network)?
2. Förklara den tekniska skillnaden på hur en Ethernet-ram hanteras när den lämnar en accessport (access port) jämfört med en trunkport (trunk port).
3. Vad är syftet med ett Native VLAN och varför anses det vara en säkerhetsrisk att låta det vara kvar på standardinställningen?
4. Beskriv för- och nackdelar med metoden "Router-on-a-stick" jämfört med att använda en lager 3-switch för Inter-VLAN-routing.
5. Hur förändras en switchs MAC-tabell (CAM-tabell) när VLAN introduceras i nätverket?
6. Varför är BPDU Guard en kritisk funktion att aktivera på accessportar i en skolmiljö?
7. Förklara sambandet mellan DHCP Snooping och Dynamic ARP Inspection (DAI). Varför kräver DAI att snooping-funktionen är aktiverad?
8. Vilket problem löser Multiple STP (MSTP) som inte den vanliga Rapid STP (RSTP) kan hantera lika effektivt i miljöer med hundratals VLAN?
9. Vad innebär "VLAN-pruning" och vilken effekt har det på nätverkets prestanda?
10. Vid felsökning enligt "bottom-up"-modellen, vilka är de två första sakerna du bör kontrollera om en klient i ett specifikt VLAN saknar kontakt med sin gateway?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

Cisco Networking Academy (Grundläggande och avancerad genomgång av VLAN-koncept) – https://www.netacad.com/

Practical Networking: Inter-VLAN Routing (Router-on-a-Stick och SVI) – https://www.practicalnetworking.net/featured/inter-vlan-routing/

IEEE Xplore: 802.1Q Standard (Den officiella tekniska specifikationen för VLAN-taggning [tagging]) – https://ieeexplore.ieee.org/document/8403927

PacketLife.net: STP Cheat Sheet (Spanning Tree-protokollen och deras tillstånd) – https://packetlife.net/library/cheat-sheets/

Network World (Artikel om hur man säkrar accesslagret mot vanliga attacker) – https://www.networkworld.com/article/711202/layer-2-security-best-practices.html

Wireshark Wiki: 802.1Q (Hur man analyserar taggad trafik i ett paketanalysverktyg [packet sniffer]) – https://wiki.wireshark.org/VLAN

NIST: Guide to Network Segmentation and Security (Myndighetsrekommendationer för segmentering av kritiska nätverk) – https://csrc.nist.gov/publications/detail/sp/800-125b/final

NetBox Documentation (Guide för hur man dokumenterar VLAN och IP-planer i en central källa för sanning [source of truth]) – https://docs.netbox.dev/

Proxmox VE Documentation: VLAN Network Configuration (Relevant för virtualiserade miljöer och Proxmox-servrar) – https://pve.proxmox.com/wiki/Network_Configuration#_vlan_802_1q

Red Hat: Configuring VLAN Tagging in Linux (Hantera VLAN direkt i moderna Linux-miljöer, exempelvis RHEL 9) – https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_managing_networking/configuring-vlan-tagging_configuring-and-managing-networking

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.3 Routing

För att kunna fatta beslut om vidarebefordran (forwarding) innehåller routingtabellen information om tillgängliga destinationsnät (prefix), nästa hopp (next hop) och vilket utgående gränssnitt (exit interface) som ska användas. När ett paket anländer till routern jämförs dess destinations-IP-adress mot tabellen enligt en strikt prioriteringsordning.

2.3.1 Grunder: hur routrar väljer väg

Den mest fundamentala regeln för vägval är **längsta prefix-matchning** (longest prefix match - LPM). Om en destinationsadress matchar flera rutter i tabellen väljer routern alltid den mest specifika rutten, det vill säga den med längst nätmask (prefixlängd). Ett praktiskt exempel på detta är om ett paket ska skickas till adressen 10.20.1.15. Om routingtabellen innehåller både en standardrutt (default route) på 0.0.0.0/0 och en specifik rutt för 10.20.0.0/22, kommer routern att välja /22-rutten eftersom den är mer exakt och därmed anses vara en bättre matchning.

När information om vägar samlas in från olika källor, såsom statiska rutter (static routes) eller olika dynamiska routingprotokoll som OSPF och RIP, uppstår behovet av att värdera källornas tillförlitlighet. Detta hanteras genom **administrativ distans** (administrative distance - AD). Varje källa tilldelas ett numeriskt värde där ett lägre värde innebär högre prioritet. En statisk rutt (AD 1) anses till exempel mer pålitlig än en rutt inlärd via OSPF (AD 110). Om två olika protokoll föreslår en väg till exakt samma nätverk, installeras den rutt som har lägst AD i den aktiva routingtabellen.

Om flera vägar till samma mål har lärt sig via samma protokoll, används istället en **metrik** (metric) för att jämföra dem. Metriken är en form av kostnadsskala som varierar beroende på protokoll; RIP använder till exempel antal hopp (hop count) medan OSPF använder en kostnad baserad på länkens bandbredd (bandwidth). Den väg som har lägst metrik väljs som den primära vägen. I situationer där två eller flera vägar har exakt samma kostnad kan routern använda **ECMP** (Equal-Cost Multi-Path) för att fördela trafiken över de olika länkarna, vilket ökar den totala kapaciteten och ger en form av lastbalansering (load balancing).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Genom att kombinera dessa mekanismer säkerställer routern att paketen alltid tar den mest logiska vägen genom nätverket, samtidigt som den snabbt kan ställa om ifall en länk går ner och en alternativ väg med högre metrik eller administrativ distans blir aktuell.

2.3.2 Statisk routing – enkelhet och kontroll

För att förstå statisk routing kan vi jämföra det med hur vi navigerar i vardagen. Tänk dig att du ska ge en vän en vägbeskrivning till ett köpcentrum. Du skriver ner exakt vilka gator hen ska svänga på: "Kör rakt fram i två kilometer, sväng höger vid kyrkan, ta sedan andra avfarten i rondellen". Detta är en statisk instruktion. Så länge vägen är fri fungerar det ypperligt och är mycket lätt att följa.

Problemet uppstår om det plötsligt pågår ett vägarbete eller om en bro är avstängd längs den utstakade rutten. En statisk rutt har ingen förmåga att känna av hinder eller anpassa vägbeskrivningen efter rådande trafikläge. Trafiken kommer envist att köra fram till hindret, inse att det inte går att passera, vända tillbaka till utgångspunkten och sedan försöka exakt samma väg igen och igen. Utan en mänsklig administratör som ändrar instruktionen kommer trafiken aldrig fram. Detta skiljer sig markant från moderna GPS-tjänster som Google Maps, vilka automatiskt räknar om rutten vid köer eller olyckor – en funktion som i nätverksvärlden motsvaras av dynamisk routing.

I professionell nätverksutrustning, exempelvis i Ciscos kommandoradsgränssnitt (CLI - Command Line Interface), konfigureras en statisk rutt med ett enkelt kommando. Om vi vill att vår router ska hitta till labbnätverket `10.50.0.0/16` genom att skicka trafiken till en annan router med IP-adressen `192.168.1.2`, skriver vi:

```
Router(config)# ip route 10.50.0.0 255.255.0.0 192.168.1.2
```

Här definierar vi först destinationsnätet, följt av dess nätmask (subnet mask) och slutligen nästa hopp (next hop), vilket är adressen till den router som faktiskt "äger" vägen vidare. Om vi istället vill skapa en standardrutt (default route) – den rutt som tar hand om allt som inte finns i vår lokala tabell, såsom trafik mot internet – använder vi nollor för att representera "vilket nät som helst":

```
Router(config)# ip route 0.0.0.0 0.0.0.0 192.168.1.1
```

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

För den som arbetar med brandväggar som har ett grafiskt gränssnitt (GUI), till exempel **OPNsense**, är funktionen för statisk routing inbyggd direkt i webbgränssnittet. Under menyn för systeminställningar och rutter kan administratören enkelt lägga till rutter genom att fylla i destinationsnät och gateway i tydliga formulärfält. Det bakomliggande konceptet är exakt detsamma som i Ciscos CLI, men gränssnittet gör det mer överskådligt för den som föredrar att klicka fram inställningarna framför att skriva kod.

Trots sin enkelhet kräver statisk routing att man alltid tänker på returvägen (return path). Eftersom routern bara följer den exakta vägbeskrivningen du gett den, kommer svarspaket från destinationsnätet inte att hitta tillbaka om inte även den mottagande routern har en statisk rutt (eller standardrutt) som pekar tillbaka till källan. Utan denna ömsesidiga vägbeskrivning uppstår asymmetrisk routing, vilket i praktiken innebär att kommunikationen bryts.

Fördelar:

- *Enkelt och förutsägbart* – lätt att förstå och felsöka.
- *Ingen protokollchatt* – noll kontrolltrafik och minimal CPU-belastning.
- *Fin kontroll* – du bestämmer exakt vägval.
- *Bra som backup* – “floating static” tar över först vid fel.

Nackdelar:

- *Ingen självläkning* – topologiändringar kräver manuell uppdatering.
- *Skalar dåligt* – många nät = många rader att underhålla.
- *Risk för asymmetri/blackholes* om returvägar missas.

Användningsområden:

- *Små nät / labbar / edge mot internet* (default route).
- *Enkla hub-and-spoke* (spoke pekar mot hub med statik).
- *Backup-vägar* (floating static bakom dynamik).

Exempel: *Router A har en standardrutt (default route) mot internet (0.0.0.0/0) och behöver en manuell statisk rutt (static route) som pekar ut nätet bakom Labbroutern för att trafiken ska hitta dit. Labbroutern måste i sin tur ha en returväg, oftast en egen standardrutt som pekar tillbaka mot*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Router A, för att svarstrafik och internetåtkomst ska fungera. Detta skapar en fast och förutsägbar vägbeskrivning mellan näten som fungerar utmärkt så länge infrastrukturen är stabil, men den saknar helt förmåga att självläka eller hitta omvägar om en länk går ner.

2.3.3 Dynamisk routing – när nätet ska anpassa sig själv

Grunden för dynamisk routing är förmågan att skapa grannrelationer (adjacencies). Routrarna skickar kontinuerligt ut små kontrollmeddelanden, så kallade hello-paket (hello packets), för att bekräfta att deras grannar fortfarande är aktiva och nåbara. Så länge dessa paket tas emot betraktas länken som stabil. Om en router slutar svara eller om en ny länk kopplas in, sprids denna information genom hela nätverket. Processen där alla routrar uppdaterar sina tabeller för att nå en gemensam bild av den nya topologin kallas för konvergens (convergence). Ett nätverk som har konvergerat snabbt minimerar risken för avbrott och säkerställer att datatrafiken alltid tar den bästa vägen.

Dessa protokoll delas grovt in i två huvudfamiljer beroende på hur de beräknar och sprider information om vägar. Den första familjen är distansvektor-protokoll (distance-vector), där RIP (Routing Information Protocol) är ett klassiskt exempel. Dessa fungerar enligt principen ”routing genom rykten”. Varje router skickar sin kompletta routingtabell till sina grannar och berättar i vilken riktning (vector) och hur långt bort (distance) ett nätverk ligger. Den andra familjen är länktillstånds-protokoll (link-state), med OSPF (Open Shortest Path First) som det mest kända exemplet. Istället för att bara sprida rykten delar dessa routrar detaljerad information om statusen på sina egna länkar. Detta gör att varje router kan bygga upp en exakt och komplett karta över hela nätverkets topologi och själv räkna ut den bästa vägen med hjälp av matematiska algoritmer.

Den främsta fördelen med att använda dynamiska protokoll är nätverkets förmåga till självläkning (self-healing). Om en primär länk går ner upptäcker protokollet detta omedelbart och dirigerar om trafiken till en reservväg utan att en tekniker behöver ingripa. Detta gör tekniken extremt skalbar, då nya nätverk och platser kan läggas till utan att dussintals statiska rutter behöver uppdateras manuellt på varje enskild router. Det minskar risken för mänskliga fel och gör driften av stora campusnät eller företagskoncerner möjlig.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Det finns dock en ökad komplexitet som en nätverkstekniker måste ta hänsyn till. Dynamiska protokoll introducerar administrativ trafik (overhead) i form av de meddelanden som skickas mellan routrarna, vilket konsumerar en del av nätverkets bandbredd och routerns processorstyrka (CPU). Dessutom krävs en djupare förståelse för protokollspecifika beteenden såsom timers, mätvärden (metrics) och filtrering. En felkonfiguration av dessa parametrar kan i värsta fall leda till att nätverket väljer ineffektiva vägar eller hamnar i ett instabilt läge där routingtabellerna ständigt ändras. Trots detta är dynamisk routing den absoluta standarden i alla nätverk som kräver hög tillgänglighet och enkel tillväxt.

Fördelar (generellt):

- *Självläkning* – anpassar sig vid fel utan manuell insats.
- *Skalbarhet* – enklare när nät och sajter växer.
- *Mindre manuellt pyssel* – färre statiska rader att underhålla.

Nackdelar (generellt):

- *Ökad komplexitet* – grannskap, timers, filtrering, autentisering.
- *Protokolltrafik* – viss overhead och potentiella angreppsytor.
- *Felkonfiguration kostar* – felaktiga parametrar kan ge sämre vägar.

2.3.4 RIP v2 – enkelt och pedagogiskt

RIP v2 (Routing Information Protocol version 2) är ett klassiskt distansvektor-protokoll (distance-vector protocol) som fungerar som en utmärkt introduktion till dynamisk routing. Det bygger på en enkel logik där varje router delar sin kunskap om tillgängliga nätverk med sina närmaste grannar genom periodiska uppdateringar, vanligtvis var 30:e sekund. Genom att "byta kartor" med varandra bygger routrarna upp en gemensam förståelse för nätverkets struktur. Denna beräkningsmodell bygger på **Bellman-Fords algoritm** (Bellman-Ford algorithm), vilket innebär att varje router fattar sina beslut baserat på den ackumulerade kostnaden – i det här fallet hopp – som rapporteras från dess grannar.

Inom RIP v2 används hoppräkning (hop count) som den enda metriken (metric). Det innebär att den bästa vägen till ett mål alltid anses vara den

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

väg som passerar genom lägst antal routrar. En stor begränsning i protokollet är dock dess skalbarhet; det stöder maximalt 15 hopp. Ett nätverk som ligger 16 hopp bort betraktas som oåtkomligt (unreachable), vilket gör att protokollet inte kan användas i stora eller komplexa nätverksmiljöer.

Till skillnad från den äldre version 1 har RIP v2 stöd för VLSM (Variable Length Subnet Masking) och CIDR (Classless Inter-Domain Routing), vilket innebär att information om nätmasker skickas med i uppdateringarna. För att hantera risker med loopar och felaktig information använder protokollet flera tekniska säkerhetsmekanismer:

Split horizon: En regel som förhindrar en router från att skicka tillbaka information om ett nätverk på samma gränssnitt som den lärde sig informationen ifrån.

Poison reverse: När en rutt går ner markeras den omedelbart med en metrik på 16 (oändlighet) för att informera grannarna om att vägen är bruten.

Hold-down timers: En väntetid som förhindrar att en router för snabbt accepterar en förändring av en rutt som nyss har markerats som osäker, vilket stabiliserar nätverket vid tillfälliga länkfel.

Fördelar och nackdelar med RIP v2

Den främsta fördelen med RIP v2 är dess extrema enkelhet. Det är snabbt att konfigurera och ställa in i mindre miljöer eller labbnätverk, och det kräver minimalt med resurser i form av processorkraft (CPU) och minne. Dessutom har protokollet en mycket bred kompatibilitet och finns tillgängligt på nästan alla typer av nätverksutrustning, oavsett tillverkare.

Nackdelarna är dock betydande i moderna produktionsmiljöer. Eftersom metriken endast baseras på hopp, tar protokollet ingen hänsyn till länkarnas faktiska kvalitet eller hastighet. En router kan därför välja en långsam 10 Mbps-länk över ett hopp istället för en stabil 10 Gbps-länk som kräver två hopp. Vidare lider RIP v2 av långsam konvergens (convergence); vid ett länkfel kan det ta flera minuter innan hela nätverket har uppdaterat sina tabeller, vilket i dagens uppkopplade värld ofta betraktas som ett oacceptabelt avbrott. Det finns även en risk för "count-to-infinity"-problem där felaktiga rutter räknas upp steg för steg innan de når maxgränsen.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Användningsområden och exempel

Idag rekommenderas RIP v2 sällan för skarp produktion där snabb återhämtning och hög prestanda krävs. Istället används det främst inom utbildning för att lära ut grundläggande routingprinciper, eller i mycket små och stabila miljöer där enkelhet prioriteras högre än effektivitet. Det kan även fungera som en temporär lösning i ett "proof-of-concept" där man snabbt behöver få kommunikation att fungera mellan några få enheter utan komplex konfiguration.

- **Fördelar:**
    - Mycket enkelt – snabbt att få igång i labb/mindre nät.
    - Stöd för VLSM/CIDR (till skillnad från v1).
    - Bred kompatibilitet – finns på många plattformar.
- **Nackdelar:**
    - Skalar dåligt – 15 hopp-begränsning.
    - Långsam konvergens – periodiska uppdateringar; “count-to-infinity”-problem.
    - Grovt metric – tar inte hänsyn till bandbredd/kvalitet.
- **Användningsområden:**
    - Utbildning och små labbnät (visa principer).
    - Mycket små, statiska miljöer där enkelhet > prestanda.
    - Temporära proof-of-concept där snabb uppstart är viktig.

Exempel: *Tre routrar kopplade i en triangelform kör RIP v2. Om den direkta länken mellan två av routrarna bryts, kommer de inte omedelbart att hitta omvägen via den tredje routern. De måste först vänta på att deras interna tidtagare (timers) löper ut och att nästa periodiska uppdatering skickas. Efter en stund kommer hoppräkningen att uppdateras och trafiken hittar den alternativa vägen, men användarna i nätverket kommer att uppleva en tydlig fördröjning innan förbindelsen återställs.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.3.5 OSPF – snabbt och skalbart

OSPF (Open Shortest Path First) är ett länktillstånds-protokoll (link-state protocol) som bygger på att varje router i nätverket har en komplett och identisk karta över topologin. Istället för att bara lita på vad grannen säger, annonserar varje enhet sin egen status via **LSA** (Link-State Advertisements). Denna information lagras i en databas (Link-State Database - LSDB), och med hjälp av **Dijkstras algoritm** beräknar varje router den absolut mest effektiva vägen till varje nätverk. Denna metod ger en betydligt snabbare konvergens (convergence) än äldre protokoll, vilket innebär att nätverket kan återhämta sig från ett länkfel på under en sekund.

Metrik och kostnad (Cost)

Till skillnad från RIP, som bara räknar antal hopp, använder OSPF en metrik som kallas **kostnad** (cost). Kostnaden beräknas som standard genom att dela en referensbandbredd (reference bandwidth) med länkens faktiska bandbredd. Detta innebär att en snabb fiberlänk får en mycket låg kostnad, medan en långsammare länk får en hög kostnad. Genom att justera dessa värden kan en tekniker finjustera exakt hur trafiken ska flöda genom nätverket för att undvika flaskhalsar och maximera genomströmningen (throughput).

Hierarkisk design med Areas

För att OSPF ska kunna skala till mycket stora nätverk används ett system med områden (areas). Om alla routrar i ett stort campusnätverk skulle dela all information med varandra, skulle routingtabellerna och de matematiska beräkningarna snabbt konsumera för mycket minne och processorkraft (CPU). Lösningen är att dela upp nätverket i logiska zoner.

Hjärtat i denna hierarki är **Area 0**, även kallad ryggradsområdet (backbone area). Alla andra områden måste vara fysiskt eller logiskt anslutna till Area 0. Genom denna uppdelning begränsas spridningen av detaljerad topologi-information; en router i Area 1 behöver inte känna till varje enskild länkändring i Area 2. Vid gränserna mellan områden används **summering** (summarization) för att aggregera många specifika rutter till ett fåtal mer generella, vilket håller routingtabellerna kompakta och lättlästa. För miljöer med begränsade resurser kan man även använda specialiserade areatyper som **Stub** eller **NSSA** (Not-So-Stubby Area), vilka

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

filtrerar bort onödig extern information för att ytterligare minska nätverkets brus.

DR och BDR i Ethernet-segment

I nätverkstyper där många enheter delar samma medium, såsom ett vanligt Ethernet-segment (multi-access), skulle mängden grannrelationer (adjacencies) snabbt bli ohanterlig om alla routrar pratade direkt med alla. För att optimera detta väljer OSPF en **DR** (Designated Router) och en **BDR** (Backup Designated Router). Istället för ett nät av anslutningar skickar alla routrar sina uppdateringar enbart till DR, som sedan ansvarar för att distribuera informationen till resten av segmentet. Detta minskar mängden kontrolltrafik och säkerställer en stabil och förutsägbar kommunikation.

Säkerhet och prestanda

För att skydda nätverket mot felkonfigurationer eller illasinnade angrepp stödjer OSPF **autentisering** (authentication). Detta innebär att routrar måste bevisa sin identitet med ett lösenord eller en kryptografisk nyckel innan de tillåts utbyta routinginformation. Genom att kombinera detta med avancerad filtrering och summering kan en administratör skapa en miljö som inte bara är snabb och självläkande, utan också mycket robust och säker.

I professionella miljöer där redundans är ett krav är OSPF det självklara valet. Dess förmåga att reagera blixtsnabbt på topologiförändringar, tillsammans med en skalbar arkitektur och ett intelligent vägval baserat på bandbredd, gör det lämpligt för allt från medelstora företag till stora campusnätverk. Den enda egentliga nackdelen är att det kräver en djupare teknisk förståelse för hur LSAs, timers och olika areatyper samverkar för att undvika komplexa felsökningsscenarier.

Fördelar:

- *Snabb konvergens* – reagerar snabbt på fel.
- *Skalbar arkitektur* – areas minskar databas och uppdateringar.
- *Finare metric* – bandbreddsbaserad kostnad ger bättre vägval.
- *Många verktyg* – summering, filtrering, autentisering.

Nackdelar:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- *Mer komplext* – kräver förståelse för areas, LSAs, timers och adjacency-tillstånd.
- *Kräver korrekt design* – fel DR/BDR-placering eller area-gränser kan skapa problem.
- *Lite mer overhead* än statik/RIP.

Användningsområden:

- *Campus/enterprise* (medel–stora till stora nät).
- *Miljöer med redundans* där snabb återhämtning behövs.
- *Nät med många VLAN/subnät* där summering/areas ger ordning.

Exempel: *En skola med distribution i två separata byggnader, som båda tillhör OSPF Area 0 Där konfigureras varje enskilt våningsplan som egna områden (areas). Om en fiberkabel mellan två våningar går av, skickas omedelbart en LSA ut. Samtliga routrar i nätverket uppdaterar sina kartor och räknar om de bästa vägarna. Inom loppet av en bråkdel av en sekund har trafiken styrts om via en sekundär väg, utan att användarna ens märker av avbrottet.*

2.3.6 Första-hopp-redundans – VRRP/HSRP

Tekniken bygger på att två eller flera fysiska routrar samarbetar för att presentera en gemensam **virtuell IP-adress** (VIP) och en **virtuell MAC-adress** för klienterna i ett VLAN. En av enheterna utses till aktiv (Active/Master) och ansvarar för att vidarebefordra all trafik som skickas till den virtuella adressen. Den andra enheten befinner sig i vänteläge (Standby/Backup) och övervakar kontinuerligt den aktiva enhetens status genom kontrollmeddelanden (hellos).

Om den aktiva enheten slutar svara, tar standby-enheten omedelbart över rollen som aktiv. För att informera nätverkets switchar om att den virtuella MAC-adressen nu finns på en annan fysisk port, skickar den nya aktiva enheten ut ett så kallat **gratuitous ARP** (G-ARP). Detta meddelande uppdaterar switcharnas MAC-tabeller utan att klienterna behöver ändra sin konfiguration eller ens märka att ett hårdvarufel har inträffat.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

För att skapa en robust och intelligent redundans används ofta funktionerna **preemption** och **tracking**:

- **Preemption:** Gör det möjligt för en router med högre prioritet att automatiskt återta rollen som aktiv när den kommer online igen efter ett avbrott eller underhåll.
- **Tracking:** Tillåter en gateway att övervaka statusen på sina egna upplänkar (uplinks). Om en kritisk länk mot nätverkets kärna går ner, kan routern automatiskt sänka sin egen prioritet så att en annan router, som har fungerande vägar vidare, kan ta över rollen som aktiv gateway.

Fördelar och nackdelar

Den främsta fördelen med FHRP är att redundansen är helt transparent för slutanvändarna. Eftersom den virtuella gateway-adressen förblir densamma, krävs inga manuella ändringar på klientnivå vid ett haveri. Konceptet är dessutom enkelt att dokumentera och testa, vilket gör det till en hörnsten i professionell nätverksdesign.

Det finns dock vissa begränsningar. Implementeringen innebär ytterligare protokoll att förvalta, där parametrar som prioriteringar, tidtagare (timers) och tracking-objekt måste konfigureras korrekt för att undvika instabilitet. Det är också viktigt att notera att VRRP och HSRP i sitt grundutförande inte erbjuder automatisk lastdelning (load balancing); endast en enhet är aktiv åt gången per virtuell adress. För att utnyttja båda enheternas kapacitet samtidigt krävs mer komplexa uppsättningar, såsom att låta olika enheter vara aktiva för olika VLAN. Slutligen löser FHRP endast redundansen för klienternas första hopp; det skyddar inte mot fel som ligger djupare i nätverkets routing- eller brandväggslogik.

Användningsområden och exempel

FHRP används primärt för VLAN-gateways i nätverkets distributions- eller kärnlager där avbrott i produktionen inte kan accepteras. Det är även ett oumbärligt verktyg vid planerat underhåll, då tekniker kan flytta trafikflöden mellan enheter utan att användarna tappar sina sessioner.

Fördelar:

- *Transparent för klienter* – samma gateway-IP lever vidare vid fel.
- *Enkelt koncept* – lätt att förstå, testa och dokumentera.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- *Flexibelt* – preemption/track ger snabb och rätt failover.

Nackdelar:

- *Ytterligare protokoll att hantera* – prioritet, timers, tracking måste hållas rätt.
- *Ingen automatisk lastdelning* (GLBP kan, men är ovanligt/leverantörsbundet).
- *Löser bara gateway-resiliens* – inte bakomliggande routing- eller brandväggsproblem.

Användningsområden:

- *VLAN-gateways i distribution/core* där hög tillgänglighet krävs.
- *Migrering/underhåll* – byt hårdvara utan klientstopp.

Exempel: *En nätverksdesign för VLAN 10 (Personal) använder den virtuella IP-adressen* `10.10.0.1` *som standard-gateway. I nätverkets distributionslager finns två switchar, Dist-SW-A och Dist-SW-B. Under normal drift är Dist-SW-A konfigurerad som aktiv och hanterar all trafik för personalstyrkan. Vid ett planerat servicearbete på Dist-SW-A, eller vid ett plötsligt strömavbrott, upptäcker Dist-SW-B bortfallet av kontrollmeddelanden och övertar rollen som aktiv gateway på några få sekunder. Genom att skicka ett gratuitous ARP informeras övriga switchar om förändringen, och de anställda kan fortsätta sitt arbete mot den virtuella adressen* `10.10.0.1` *utan märkbara störningar.*

2.3.7 Summering och PBR – kontroll och städning

I takt med att ett nätverk expanderar växer även routingtabellerna, vilket kan leda till ökad resursförbrukning och mer komplex felsökning. För att hantera detta används **prefix-summering** (route summarization / prefix aggregation). Detta innebär att en router annonserar ett enda stort, sammanhållet nätverk (ett prefix) istället för att skicka ut information om varje enskilt litet subnät. Genom att "städa" routingtabellen på detta sätt minskas belastningen på enheternas processorer (CPU) och mängden administrativ trafik som skickas mellan olika geografiska platser eller areor begränsas. En länkändring i ett litet subnät behöver då inte tvinga hela nätverket att räkna om sina kartor, så länge det summerade prefixet fortfarande är nåbart.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Medan summering handlar om att förenkla nätverkskartan, används **Policy-Based Routing (PBR)** för att skapa specifika undantag från den vanliga routinglogiken. Normalt väljer en router väg enbart baserat på paketets destinationsadress, men med PBR kan vi styra trafik baserat på källadress (source address), protokoll eller applikationstyp. Det kan exempelvis handla om att låta vanlig surftrafik gå via en billigare internetlänk, medan kritisk produktionstrafik eller specifika servrar styrs via en mer stabil fiberanslutning eller en specifik proxy. Eftersom PBR åsidosätter den ordinarie "bästa vägen"-logiken krävs noggrann dokumentation och ofta användning av övervakning via **SLA/tracking**. Om den väg som policyn pekar ut slutar fungera, måste routern kunna upptäcka detta för att inte skicka trafiken i ett "svart hål" (black hole).

Användningen av dessa tekniker innebär en avvägning. Summering kräver en välplanerad adressplan där nätverken ligger logiskt efter varandra, men det ger i gengäld ett mer stabilt och skalbart nätverk. PBR ger en granulär kontroll som är ovärderlig i komplexa miljöer för att uppfylla krav på säkerhet eller prestanda, men det ökar också den logiska komplexiteten och kan göra felsökning mer tidskrävande om man inte har full koll på vilka regler som faktiskt styr trafiken.

Fördelar:

- *Summering:* mindre tabeller och mindre brus mellan sajter.
- *PBR:* granular kontroll för särskilda flöden (compliance/prestanda).

Nackdelar:

- *Summering:* kräver planerad adressering; risk att “gömma” fel under en bred annons.
- *PBR:* ökar komplexitet; kan bryta “bästa-väg”-logiken och försvåra felsökning.

Användningsområden:

- *Större nät med OSPF* – summera per site/area-gräns.
- *Speciella trafikfall* – dirigera vissa källor/appar via annan länk/proxy.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.3.8 Sammanfattning, Best Practice och felsökning

När vi ser på routing som helhet handlar det om att hitta balansen mellan enkelhet och automation. Valet av metod beror helt på nätverkets storlek och krav på tillgänglighet.

Metod	Lämplig miljö	Främsta fördel	Främsta nackdel
Statisk routing	Små nätverk, labbar, enskilda länkar	Full kontroll, ingen overhead	Ingen självläkning, svårskalat
RIP v2	Utbildning, mycket små stabila miljöer	Enkel konfiguration	Långsam konvergens, 15 hopp
OSPF	Campus, företag, datacenter	Snabb konvergens, skalbarhet	Högre komplexitet, CPU-last

En fackmässig nätverksdesign (best practice) innebär att man konsekvent använder standardrutter (default routes) mot internet och summerar interna nätverk vid strategiska gränser. Man bör även sträva efter att hålla den administrativa distansen (administrative distance) och metriken (metric) så förutsägbar som möjligt för att undvika asymmetriska flöden.

Vid felsökning av routingproblem bör man arbeta systematiskt lager för lager. Processen inleds med att verifiera länkstatus och den lokala adresseringen (ARP/ND) för att säkerställa att enheten kan prata med sin standard-gateway (default gateway). Därefter granskas routingtabellen för att se om det finns en matchande rutt och att nästa hopp (next hop) är korrekt. Om dynamiska protokoll används kontrolleras status för grannrelationer (adjacencies) och eventuella timers. Först när den logiska vägen är bekräftad bör man kontrollera brandväggspolicys längs vägen och slutligen själva applikationen.

De vanligaste felen som en tekniker möter är ofta relaterade till saknade standardrutter, felaktiga nästa hopp eller överlappande nätverksprefix. Ett effektivt sätt att lokalisera var trafiken stannar är att använda verktyg som **ping** och **traceroute** (tracert i Windows och mtr i Linuxsystem). På professionell

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

utrustning ger kommandon som show ip route och show ip ospf neighbor en direkt inblick i routerns beslutsunderlag.

Summering:

- *Grundverktyg:* `ping`, `traceroute/tracert`, `arp -a`, `ip neigh`, `ip route/route print`.
- *På nätutrustning:* `show ip route`, `show ip ospf neighbor`, `show ip protocols`, `show interfaces`.
- *Vanliga fel:* saknad default, fel nästa hopp, överlappande prefix, asymmetrisk routing, blockerad port i brandvägg.

Exempel: *En klient kan surfa på internet men når inte en server på en annan filial. Genom att köra en `traceroute` upptäcks att trafiken stannar direkt efter den lokala brandväggen. En kontroll av routingtabellen visar att rutten till filialens nätverk (`10.50.0.0/16`) finns där, men brandväggens regelverk (inter-VLAN policy) saknar en tillåtelse för just den trafiken. Efter att regeln har lagts till och validerats fungerar kommunikationen igen.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Förklara begreppet längsta-prefix-matchning (longest prefix match) och ge ett exempel på när det avgör vägvalet.
2. Vad är skillnaden mellan administrativ distans (administrative distance) och metrik (metric)?
3. Vilka är de tre största riskerna med att enbart använda statisk routing i ett växande nätverk?
4. Beskriv liknelsen mellan statisk routing och en papperskarta kontra dynamisk routing och en GPS.
5. Varför anses RIP v2 vara föråldrat i moderna produktionsmiljöer, trots att det är ett dynamiskt protokoll?
6. Vad är syftet med att dela upp ett OSPF-nätverk i olika areor (areas), och vilken roll spelar Area 0?
7. Hur fungerar en Designated Router (DR) i ett OSPF-nätverk och varför behövs den?
8. Förklara hur HSRP eller VRRP kan ge en klient oavbruten internetåtkomst även om en fysisk router går sönder.
9. Vad innebär prefix-summering (route summarization) och vilken teknisk fördel ger det för routerns CPU?
10. Varför är det viktigt att kontrollera returvägen (return path) när man felsöker routingproblem?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

Cisco Networking Academy: Routing Concepts (Omfattande utbildningsmaterial om grundläggande routing) – https://www.netacad.com/

Practical Networking: Routing Table Logic (Tydlig genomgång av hur en router fattar beslut) – https://www.practicalnetworking.net/featured/routing-table-logic/

OSPF Tutorial - Flight-Sustained (Djupdykning i OSPF areas, LSAs och algoritmer) – https://www.flightsustained.com/ospf-tutorial/

RFC 2328: OSPF Version 2 (Den officiella specifikationen för OSPF) – https://datatracker.ietf.org/doc/html/rfc2328

PacketLife.net: Routing Cheat Sheets (Samling av snabbguider för RIP, OSPF och BGP) – https://packetlife.net/library/cheat-sheets/

Cisco: HSRP Configuration Guide (Teknisk vägledning för konfiguration av första-hopp-redundans) – https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst3750/software/release/12-2_55_se/configuration/guide/scg3750/swhsrp.html

OPNsense Documentation: Static Routes (Guide för hur man hanterar rutter i ett grafiskt gränssnitt) – https://docs.opnsense.org/manual/routes.html

Network Lessons: Route Summarization (Pedagogiska exempel på hur man räknar ut summerade prefix) – https://networklessons.com/cisco/ccna-routing-switching/route-summarization-explained

Red Hat: Configuring Static Routes in Linux (Hur man hanterar routingtabellen i en Linux-miljö) – https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_managing_networking/configuring-static-routes_configuring-and-managing-networking

Wireshark Wiki: OSPF (Analysera OSPF-paket och LSAs i realtid) – https://wiki.wireshark.org/OSPF

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.4 DHCP, DNS och NAT

För att ett nätverk ska vara praktiskt användbart i en modern produktionsmiljö räcker det inte med enbart fungerande switchar och routrar. Det krävs även en uppsättning stödtjänster som automatiserar adressering, förenklar navigation och hanterar bristen på publika adresser. Kapitel 2.4 fokuserar på de tre hörnstenarna **DHCP**, **DNS** och **NAT**, vilka tillsammans utgör det logiska ramverket för hur enheter identifieras och kommunicerar både lokalt och över internet.

Dessa tjänster fungerar som nätverkets infrastruktur för användarvänlighet och skalbarhet. Genom att implementera robusta lösningar för namnkonvertering (name resolution) och dynamisk tilldelning av nätverksparametrar (dynamic allocation) minskas det administrativa arbetet avsevärt samtidigt som risken för mänskliga fel, såsom dubblerade IP-adresser, minimeras. Att förstå samspelet mellan dessa protokoll är avgörande för att kunna bygga nätverk som inte bara är tekniskt korrekta, utan också driftsäkra och lätta att underhålla i en professionell miljö.

2.4.1 DHCP – grunder

DHCP (Dynamic Host Configuration Protocol) är den tjänst som automatiserar tilldelningen av nätverksparametrar (network parameters) till klienter. Istället för att en tekniker manuellt behöver konfigurera varje enskild enhet med en statisk adress, ser DHCP till att enheter får all nödvändig information för att kunna kommunicera så snart de ansluter till nätverket. Detta inkluderar inte bara en unik **IP-adress** (IP address), utan även **nätmask** eller **prefix** (subnet mask/prefix), **standard-gateway** (default gateway) och adresser till **DNS-servrar** (DNS servers).

När en klient startar upp och saknar en giltig nätverkskonfiguration inleds en utbyte av meddelanden mellan klienten och servern. Denna sekvens kallas ofta för **DORA-processen** (DORA process), baserat på de fyra stegen i protokollet:

1. **Discover (Upptäckt):** Klienten skickar ut ett broadcast-meddelande (DHCPDISCOVER) på det lokala nätverket för att hitta en tillgänglig DHCP-server. Eftersom klienten ännu inte har en IP-

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

adress, används adressen 0.0.0.0 som avsändare och 255.255.255.255 som mottagare.

2. **Offer (Erbjudande):** En eller flera DHCP-servrar som tar emot förfrågan svarar med ett DHCPOFFER. Detta meddelande innehåller ett förslag på en ledig IP-adress samt övriga nätverksinställningar.
3. **Request (Begäran):** Klienten tar emot erbjudandet och svarar med ett DHCPREQUEST. Genom detta meddelande meddelar klienten formellt att den vill använda den föreslagna adressen och informerar samtidigt eventuella andra servrar om att deras erbjudanden kan dras tillbaka.
4. **Acknowledgement (Bekräftelse):** Servern skickar ett slutgiltigt DHCPACK som bekräftar att klienten nu har rätt att använda adressen. I detta skede registreras även kopplingen mellan klientens fysiska MAC-adress och den tilldelade IP-adressen i serverns databas.

Den tilldelade adressen ägs inte permanent av klienten, utan tilldelas under en begränsad tidsperiod som kallas för en **lease** (lease). Under denna period behåller klienten adressen, men för att undvika att anslutningen bryts påbörjas en automatisk förnyelseprocess (renewal) när hälften av tiden har passerat. Om klienten lämnar nätverket utan att förnya sin lease, blir adressen efter en tid återigen tillgänglig för andra enheter i serverns adresspool (address pool).

Utöver de grundläggande parametrarna kan DHCP-servern skicka med ytterligare instruktioner via så kallade **DHCP-options** (DHCP options). Dessa numeriska koder tillåter administratören att distribuera specifika inställningar som krävs för nätverkets funktion. Vanliga exempel inkluderar **Option 3** för att definiera routern (gateway), **Option 6** för DNS-servrar, **Option 15** för domänsuffix (domain suffix) och **Option 42** för tidsservrar via NTP (Network Time Protocol). För enheter som kräver en fast adress, såsom nätverksskrivare eller servrar, kan en **reservation** (reservation) skapas i servern. Detta innebär att en specifik IP-adress permanent låses till en unik MAC-adress, vilket kombinerar enkelheten hos DHCP med stabiliteten hos statisk adressering.

Exempel: En dator i VLAN 20 startas upp och ropar efter en adress. Eftersom DHCP-servern ofta är centraliserad, används en **relay** (DHCP

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

relay) i nätverkets gateway för att vidarebefordra förfrågan från elevens lokala segment till servern. I detta scenario får klienten exempelvis adressen `10.20.1.57/22`, med `10.20.0.1` som sin gateway och `10.40.0.10–11` som DNS-servrar. Allt detta sker på några millisekunder, vilket gör att användaren kan börja arbeta omedelbart utan manuell handpåläggning från IT-avdelningen.

2.4.2 DHCP-design: scopes, relay och robusthet

När ett nätverk växer och delas upp i flera VLAN (virtual local area networks) blir det opraktiskt och kostsamt att placera en fysisk DHCP-server i varje enskilt segment. Istället används en centraliserad design där en eller två kraftfulla servrar hanterar adressutdelningen för hela organisationen. För att detta ska fungera tekniskt krävs en **DHCP-relay** (DHCP relay), ofta konfigurerad som en **IP helper-address** (IP helper address) på nätverkets gateway. Eftersom DHCP-processen inleds med en broadcast-förfrågan som normalt inte passerar en router, agerar gatewayen som en mellanhand. Den fångar upp klientens förfrågan, paketerar om den till ett unicast-meddelande och skickar den direkt till den centrala servern.

På den centrala servern skapas unika **adressomfång** (scopes) för varje VLAN som ska servas. Ett scope innehåller det specifika subnätets nätverksadress, nätmask och den pool av adresser som är tillgängliga för utdelning. En viktig del av designen är att definiera **undantag** (exclusions) för de adresser som används statiskt i nätverket, såsom adresser till switchar, skrivare, gateways och brandväggar. Genom att exkludera dessa från den dynamiska utdelningen förhindras IP-adresskonflikter (IP address conflicts) som annars skulle kunna sänka kritiska infrastrukturkomponenter.

En strategisk parameter i nätverksdesignen är valet av **giltighetstid** (lease time). Denna bör anpassas efter hur rörlig användargruppen är i det specifika segmentet. I ett **gästnät** (guest network) med hög omsättning av enheter är det klokt att använda en kort giltighetstid, exempelvis 8 timmar. Detta säkerställer att adresser snabbt återgår till poolen när gästerna lämnar byggnaden, vilket förhindrar att adresserna tar slut. För **personalsegment** (staff networks) eller fasta arbetsplatser där samma datorer är anslutna dag efter dag, är en längre tid på 3–7

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

dagar mer lämplig. Det minskar mängden kontrolltrafik i nätverket och ger en stabilare miljö för användarna.

För att säkerställa hög tillgänglighet (high availability) och robusthet implementeras ofta **DHCP-failover** (DHCP failover). Genom att köra två servrar i ett kluster kan de dela på lasten (load share) eller fungera i ett läge där den ena står i beredskap (hot-standby). Om den primära servern går ner tar reserven omedelbart över hanteringen av alla scopes utan att klienterna märker något avbrott. Detta är kritiskt i miljöer där nätverksåtkomst är direkt beroende av att snabbt kunna få en fungerande IP-adress.

Säkerheten kring DHCP hanteras primärt i accesslagret genom **DHCP-snooping** (DHCP snooping). Denna funktion förhindrar att en obehörig eller felkonfigurerad enhet agerar som en falsk DHCP-server (rogue DHCP server). Switchen konfigureras så att endast specifika portar, exempelvis de som leder mot nätverkets kärna och den legitima servern, markeras som **betrodda** (trusted). Alla DHCP-svar som anländer på en obetrodd port blockeras omedelbart, vilket skyddar användarna från att få felaktiga gateways eller DNS-inställningar som skulle kunna användas för avlyssning.

Förtydligande (sammanfattning):

- **DHCP-relay (DHCP relay):** En central server når och servar klienter i många olika VLAN via gatewayens hjälp.
- **Giltighetstid (lease time):** Konfigureras kort i gästnät för att spara adresser och längre i personalnät för stabilitet.
- **Redundans (failover):** Användning av två servrar för att säkerställa hög tillgänglighet (high availability).
- **Säkerhet:** Implementering av DHCP-snooping (DHCP snooping) i accesslagret för att stoppa obehöriga servrar.

Exempel: *I en skolmiljö har VLAN 30 (Gäst) en kort giltighetstid på 8 timmar för att hantera det stora flödet av mobiltelefoner och besökare, medan VLAN 10 (Personal) har en inställning på 5 dagar för att skapa stabilitet för lärarnas datorer. DHCP-snooping är aktivt på samtliga access-switchar, där endast portarna mot nätverkets kärna är betrodda. Om en elev av misstag skulle koppla in en egen router i ett klassrum,*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

kommer dess DHCP-svar att stoppas av switchen innan de hinner störa andra användare på skolan.

2.4.3 DNS – Grundläggande funktion och namnupplösning

DNS (Domain Name System) fungerar som nätverkets och internetets telefonbok. Dess primära uppgift är att översätta mänskligt läsbara namn, som `skolan.se`, till de maskinläsbara IP-adresser som routrar och datorer använder för att hitta varandra. Utan DNS skulle vi tvingas memorera komplexa sifferkombinationer för varje tjänst och webbplats vi vill använda. Processen fungerar även i motsatt riktning genom **PTR-poster** (PTR records), där en IP-adress kan översättas tillbaka till ett namn (reverse DNS), vilket ofta används för loggning och säkerhetsverifiering.

Systemet är uppbyggt i en strikt hierarki som börjar vid **rotnivån** (root), följt av **toppdomäner** (TLD - Top Level Domains) som .se, .com eller .local, och slutligen specifika **zoner** (zones) som ägs av organisationer. När en enhet behöver slå upp ett namn vänder den sig till en **rekursiv resolver** (recursive resolver). Denna fungerar som en förmedlare som letar upp svaret genom att fråga sig fram i hierarkin tills den når den **autoritativa namnservern** (authoritative name server) som faktiskt äger informationen för den specifika domänen.

För att göra nätverket effektivare används **cachelagring** (caching). När en resolver väl har fått ett svar sparar den det lokalt under en viss tid så att nästa klient som frågar efter samma namn kan få ett svar omedelbart. Hur länge informationen får sparas i cachen styrs av ett värde som kallas **TTL** (Time to Live). Ett lågt TTL-värde gör att ändringar sprids snabbt men ökar belastningen på servrarna, medan ett högt värde sparar resurser men gör att gamla adresser lever kvar längre vid en flytt eller förändring.

Inom DNS-zonen finns olika typer av poster för olika ändamål:

- **A-post** (A record) och **AAAA-post** (AAAA record): Kopplar ett namn till en IPv4- respektive IPv6-adress.
- **CNAME-post** (CNAME record): Fungerar som ett alias (alias) där ett namn pekar på ett annat namn.
- **MX-post** (MX record): Anger vilken server som ansvarar för att ta emot e-post (mail) för domänen.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **TXT-post** (TXT record): Innehåller textinformation, ofta använd för säkerhetsinställningar som SPF och DKIM för att verifiera e-postavsändare.
- **SRV-post** (SRV record): Används för att identifiera specifika tjänster (services) i nätverket, såsom LDAP eller Kerberos. I en miljö med Active Directory (AD) är dessa poster helt kritiska för att klienter ska kunna hitta domänkontrollanter (domain controllers).
- **NS-post** (NS record): Pekar ut vilka namnservrar (name servers) som har rätten att svara auktoritativt för zonen.

I en företagsmiljö används ofta en **AD-integrerad DNS** (AD-integrated DNS). Detta innebär att DNS-tjänsten körs direkt på domänkontrollanterna och att zondata replikeras säkert mellan dem. Detta är nödvändigt för att de tidigare nämnda SRV-posterna ska kunna uppdateras dynamiskt när servrar och tjänster startar upp i nätverket.

Exempel: *En klient vill ansluta till servern `files.skolan.local`. Den skickar en förfrågan till nätverkets lokala resolver. Eftersom resolvern är auktoritativ för den interna zonen (tack vare AD-integreringen), hittar den omedelbart A-posten `10.40.0.50` i sin databas. Svaret returneras till klienten, vars webbläsare eller filhanterare då kan kontakta rätt IP-adress direkt utan att användaren någonsin behövt känna till den bakomliggande adresseringen.*

Förtydligande (sammanfattning):

- **Posttyper:** A/AAAA, CNAME, MX, TXT, SRV och NS är de vanligaste och viktigaste posterna i en zon.
- **Rekursiv resolver (recursive resolver):** Den enhet som tar emot klientens fråga, letar upp svaret och sparar det i sin cache (cache).
- **Giltighetstid (TTL):** Ett värde som bestämmer hur länge ett svar får cachelagras innan det måste hämtas på nytt.
- **AD-integrerad DNS:** En förutsättning i moderna Windows-miljöer för att hantera SRV-poster och intern namnlösning (name resolution) på ett säkert och automatiserat sätt.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.4.4 DNS-design: split-horizon, forwarders och redundans

I en professionell nätverksmiljö, särskilt inom skola och förvaltning, räcker det sällan med en enkel standardkonfiguration av DNS. För att balansera behovet av intern säkerhet med snabb åtkomst till internetresurser krävs en genomtänkt design som hanterar olika namnrymder och säkerställer att tjänsten alltid är tillgänglig, även vid hårdvarufel.

Ett av de vanligaste designmönstren i större nätverk är **split-horizon DNS** (split-horizon DNS). Detta innebär att organisationen hanterar två olika versioner av sin DNS-information beroende på varifrån förfrågan kommer. Interna klienter får svar från en lokal namnserver som har kännedom om interna resurser och domäner, exempelvis `*.skolan.local`, vilka aldrig exponeras mot internet. Externa användare som försöker nå skolans publika webbplats på `*.skolan.se` får istället svar från en publik namnserver i en DMZ eller hos en extern leverantör. Denna uppdelning ökar säkerheten dramatiskt genom att dölja nätverkets interna struktur och IP-adresser för omvärlden, samtidigt som interna användare får snabb och korrekt namnupplösning (name resolution) för sina lokala tjänster.

För att optimera hastigheten och kontrollen över externa uppslagningar använder nätverkets rekursiva resolvers ofta **vidarebefordrare** (forwarders). Istället för att den lokala servern själv ska gå hela vägen till rot-servrarna för varje ny förfrågan, skickas frågan vidare till en förvald kraftfull DNS-tjänst, såsom internetoperatörens (ISP) servrar eller en filtrerande säkerhetstjänst (t.ex. Cisco Umbrella eller Cloudflare). Detta snabbar inte bara upp svarstiderna tack vare stora befintliga cacher (caches), utan tillåter även administratören att tillämpa policys och blockera skadliga domäner redan på nätverksnivå innan de når användarens webbläsare.

Redundans (redundancy) är en kritisk faktor i DNS-designen. Eftersom nästan ingen modern nätverkskommunikation fungerar utan fungerande namnupplösning, bör en miljö alltid ha minst två aktiva resolvers. I en virtualiserad miljö är det god praxis att placera dessa på olika fysiska värdar (hosts) och gärna separera dem i olika VLAN eller säkerhetszoner för att minimera risken att båda slås ut samtidigt vid ett nätverksfel. Genom att tilldela båda dessa adresser till klienterna via DHCP, kan

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

klienten automatiskt växla till den sekundära servern om den primära skulle sluta svara.

Förvaltningen av DNS-zonerna kräver också en medveten strategi kring **giltighetstid** (TTL - Time to Live). För stabila resurser som filservrar eller gateways kan ett längre TTL-värde på exempelvis 24 timmar användas för att minska belastningen. Om en tjänst däremot planeras att flyttas eller om man befinner sig i en testfas, bör TTL sänkas till några minuter för att säkerställa att ändringar sprids snabbt i nätverket. Slutligen ger en aktiv loggning (logging) och övervakning av svarstider en ovärderlig insyn i nätverkshälsan. Onormala mönster i DNS-trafiken, såsom en plötslig ökning av uppslagningar mot okända domäner, kan vara ett tidigt tecken på en pågående cyberattack eller infekterade klienter.

Exempel: *En skola använder två AD-integrerade DNS-servrar på adresserna `10.40.0.10` och `10.40.0.11` för att hantera den interna domänen `skolan.local`. För alla externa frågor är dessa konfigurerade att använda operatörens forwarders för maximal prestanda. För att ytterligare stärka säkerheten finns en tredje, läsande resolver placerad i en isolerad DMZ. Denna server loggar all utgående trafik och skickar omedelbart ett larm till IT-avdelningen om den upptäcker att klienter försöker kommunicera med domäner som är kända för att sprida skadlig kod.*

Förtydligande (sammanfattning):

- **Split-horizon:** Separerar interna och externa zoner (zones) för att dölja intern infrastruktur och öka säkerheten.
- **Forwarders:** Delegerar externa uppslagningar till kraftfulla servrar för att öka hastigheten och möjliggöra filtrering.
- **Redundans (redundancy):** Kräver minst två oberoende resolvers för att garantera att namnupplösningen alltid fungerar.
- **Giltighetstid (TTL):** Justeras baserat på hur ofta en post (record) förväntas ändras; kort för rörliga tjänster och lång för stabila.

2.4.5 NAT och PAT – grunder och begrepp

Eftersom antalet tillgängliga publika IPv4-adresser är begränsat, bygger nästan all modern kommunikation mot internet på adressöversättning

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

(address translation). Denna teknik gör det möjligt för ett nätverk att använda privata adresser (private addresses) enligt standarden **RFC1918** internt, men ändå kommunicera med omvärlden genom en eller ett fåtal publika adresser. Den mest grundläggande formen av detta kallas **NAT44** (Network Address Translation för IPv4).

I praktiken är den absolut vanligaste varianten av adressöversättning **PAT** (Port Address Translation), ibland även kallat ”NAT overload”. Här delar ett stort antal interna klienter på en enda publik IP-adress. För att nätverksutrustningen ska veta vilket svarspaket som hör till vilken intern dator används unika portnummer (port numbers) för att skilja de olika flödena åt. När trafiken lämnar nätverket kallas processen för **SNAT** (Source NAT), då källadressen (source address) i paketet skrivs om från en privat adress till den publika adressen på brandväggens utsida.

När vi däremot vill att någon från internet ska kunna nå en specifik tjänst inne i vårt nätverk, exempelvis en webbserver eller ett fjärrskrivbord, används **DNAT** (Destination NAT), mer känt som **portvidarebefordran** (port forwarding). Här skriver brandväggen om destinationsadressen (destination address) i det inkommande paketet så att det styrs till rätt server på det interna nätverket. Det är viktigt att förstå att även om NAT döljer den interna nätverksstrukturen för omvärlden, utgör det i sig inget fullständigt skydd. Som tekniker bör man alltid se NAT som en ren adresseringsteknik som kräver kompletterande brandväggsregler (firewall rules) för att faktiskt kontrollera och filtrera trafiken.

I den modernare standarden **IPv6** har behovet av NAT i princip försvunnit. Eftersom det finns ett nästan oändligt antal adresser tilldelas enheter i stället **globala unicast-adresser** (global unicast addresses) som är unika i hela världen. Säkerheten hanteras här uteslutande genom brandväggspolicys och segmentering (segmentation) i stället för adressöversättning. Även om tekniker som **NPTv6** (Network Prefix Translation) existerar för specifika specialfall, är målet i IPv6-miljöer en rak och obruten kommunikation från källa till mål (end-to-end connectivity).

Exempel: *En skola förfogar över den publika IP-adressen `203.0.113.10` på sin brandväggs yttre gränssnitt. När elever på VLAN 20 (`10.20.0.0/22`) surfar ut på internet används SNAT (PAT) för att översätta deras interna adresser till skolans gemensamma publika adress. För att externa*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

användare ska kunna nå skolans interna webbserver i DMZ-zonen har en DNAT (port forwarding) konfigurerats för TCP-port 443. All trafik som anländer till brandväggens utsida på port 443 skickas då automatiskt vidare till servern på den interna adressen `192.0.2.20`.

Förtydligande (sammanfattning):

- **SNAT/PAT:** Möjliggör att många interna klienter delar på en publik adress för utgående trafik genom att använda unika portnummer.
- **DNAT/Port forwarding:** Dirigerar trafik från en publik port (port) till en specifik intern tjänst eller server.
- **NAT ≠ säkerhet:** Adressöversättning måste alltid kompletteras med en aktiv brandväggspolicy (firewall policy) för att nätverket ska vara skyddat.
- **IPv6:** I normalfallet används ingen NAT; säkerhet och ordning upprätthålls genom brandväggar och logisk segmentering (segmentation).

2.4.6 Vanliga tillämpningar och fallgropar

Adressöversättning (NAT) och namnupplösning (DNS) fungerar ofta sömlöst, men i mer komplexa nätverksmiljöer uppstår specifika utmaningar när dessa tekniker interagerar. En av de vanligaste fallgroparna är behovet av **Hairpin NAT**, även kallat **NAT Loopback**. Detta fenomen uppstår när en intern klient försöker nå en intern server genom att använda serverns publika IP-adress eller domännamn. Eftersom trafik som skickas mot den publika adressen normalt förväntas komma utifrån, kan brandväggen misslyckas med att dirigera om paketet tillbaka in i nätverket utan en specifik regel för detta ändamål.

En annan betydande utmaning är **Double NAT**, vilket ofta uppstår när en egen brandvägg placeras bakom internetleverantörens (ISP) router. Detta skapar två lager av adressöversättning som komplicerar både portvidarebefordran (port forwarding) och uppsättning av VPN-tunnlar. I värsta fall kan internetleverantören själv använda **CGNAT** (Carrier-Grade NAT), vilket innebär att flera av deras kunder delar en enda publik adress redan i operatörens nät. Detta kan i praktiken omöjliggöra inkommande

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

anslutningar till nätverket helt och hållet, då slutanvändaren inte har kontroll över den yttersta adressöversättningen.

Vissa äldre eller mer specialiserade protokoll är dessutom mycket känsliga för adressöversättning. **SIP** (Session Initiation Protocol), som används för IP-telefoni, och äldre **FTP** i aktivt läge (Active FTP) bäddar in IP-adresser direkt i datapaketets innehåll. Detta kräver att brandväggen utför en djupare paketinspektion genom en **ALG** (Application Layer Gateway) för att manuellt skriva om dessa adresser, eller att man går över till modernare och mer NAT-vänliga lägen.

Inom adressering (DHCP) och namnhantering (DNS) är de vanligaste problemen ofta relaterade till grundläggande konfigurationsmissar. Om en klient inte får någon IP-adress alls beror det ofta på att porten tillhör fel VLAN, att en nödvändig **DHCP-relay** (IP helper) saknas på gatewayen, eller att det saknas lediga adresser i rätt adressomfång (scope). För DNS kan felaktiga vidarebefordrare (forwarders) eller missar i **split-horizon**-konfigurationen leda till att interna tjänster inte hittas, medan för långa **TTL-värden** kan orsaka mystiska fel där gamla adresser lever kvar långt efter att en server har flyttats.

Exempel: *Elever på en skola upptäcker att de kan nå intranätet via adressen* `intranet.skolan.se` *när de sitter hemma, men inte när de är uppkopplade i klassrummet. Orsaken är att* ***hairpin NAT*** *saknas i brandväggen; den befintliga DNAT-regeln är endast konfigurerad att fungera för trafik som anländer på brandväggens yttre gränssnitt. För att lösa problemet skapas antingen en specifik loopback-regel i NAT-tabellen, eller så läggs en intern A-post in i den lokala DNS-servern som pekar namnet direkt på serverns interna IP-adress i DMZ-zonen.*

Förtydligande (sammanfattning):

- **Hairpin NAT:** Nödvändigt för att interna klienter ska nå interna tjänster via nätverkets publika IP-adress.
- **Double NAT/CGNAT:** Skapar problem för inkommande trafik, portöppningar och stabilitet i VPN-anslutningar.
- **Protokollkänslighet:** Tjänster som SIP och FTP kan kräva specifik inspektion (ALG) för att fungera genom en brandvägg.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Felsökning (DHCP/DNS):** Vanliga felkällor inkluderar saknade relays, felaktiga VLAN-medlemskap, fel i forwarders eller utdaterad cache på grund av långa TTL-värden.

2.4.7 Felsökning: metodik och verktyg

Vid felsökning av nätverkstjänster är den mest effektiva metoden att arbeta systematiskt från klienten och utåt, lager för lager. Genom att isolera varje komponent i kedjan kan man snabbt avgöra om problemet ligger i den fysiska anslutningen, i adresseringen (DHCP), i namnupplösningen (DNS) eller i själva trafikstyrningen (NAT/brandvägg).

När en klient har problem med att få en IP-adress (DHCP) inleds felsökningen med att kontrollera den fysiska länken (link light) och verifiera att porten i switchen tillhör rätt VLAN. Om nätverket använder säkerhetsfunktioner som **DHCP-snooping** bör loggarna i switchen granskas för att se om förfrågningar har blockerats. På klientsidan kan man tvinga fram en ny adressförfrågan; i Windows görs detta med kommandona `ipconfig /release` följt av `ipconfig /renew`, medan man i Linux-miljöer ofta använder `dhclient -r` för att släppa adressen och `dhclient` för att begära en ny. Om problemet kvarstår bör administratören kontrollera status för **DHCP-relay** (IP-helper) på gatewayen och se efter att det finns lediga adresser kvar i serverns adressomfång (scope).

För problem rörande namnupplösning (DNS) används verktyg som **nslookup** eller **dig** för att testa om klienten kan kommunicera med sin tilldelade namnserver (resolver). Det är viktigt att kontrollera vilket namn som faktiskt efterfrågas och om det finns komplicerade kedjor av alias (**CNAME-poster**). Om en ändring nyligen har gjorts men inte verkar ha slagit igenom, kan det vara nödvändigt att rensa klientens lokala cache (cache) med kommandot `ipconfig /flushdns` i Windows eller motsvarande tjänst i Linux. En viktig del i metodiken är att alltid testa anslutningen direkt mot en IP-adress; om det går att nå en server via IP men inte via namn, har man framgångsfullt isolerat felet till DNS-lagret.

Felsökning av adressöversättning (NAT) kräver ofta insyn i brandväggens eller routerns interna **NAT-tabell** (translation table). Här verifieras att översättningen faktiskt skapas och att trafiken tillåts flöda i rätt riktning enligt brandväggens regelverk. Ett kraftfullt sätt att gå djupare är att

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

utföra **paketfångst** (packet capture/tcpdump) på både det inre och yttre gränssnittet samtidigt. Om paketet anländer på insidan men aldrig lämnar utsidan med en översatt adress, ligger felet i NAT-konfigurationen. I fall där interna tjänster inte nås inifrån nätverket via sitt publika namn, bör man särskilt kontrollera om **hairpin NAT** (loopback) är korrekt implementerat.

Exempel: *En klient kan nå en server via dess IP-adress `10.40.0.50`, men misslyckas med att ansluta via namnet `files.skolan.local`. Vid kontroll med `nslookup` visar det sig att klienten försöker ställa frågan direkt till en publik DNS-server (`8.8.8.8`) istället för nätverkets interna resolver. Detta är en policyavvikelse som ofta beror på manuella inställningar på klienten eller felaktiga parametrar i DHCP. Åtgärden blir att tvinga klienten att använda skolans godkända resolvers genom att justera **DHCP-option 6** och samtidigt lägga till brandväggsregler som blockerar utgående DNS-trafik till andra servrar än de som är godkända.*

Förtydligande (sammanfattning):

- **DHCP:** Verifiera rätt VLAN och fungerande relay; granska lease-status på servern och klientens kommandon (ipconfig/dhclient).
- **DNS:** Använd nslookup eller dig för att testa uppslag; kontrollera TTL och rensa lokal cache (flushdns) vid behov.
- **NAT:** Kontrollera översättningstabellen (translation table), trafikriktning och eventuellt behov av hairpin NAT.
- **Avgränsa:** Testa alltid förbindelse med både IP och namn för att snabbt isolera om felet ligger i namnupplösningen eller i själva anslutningen.

Checklista för felsökning av nätverkstjänster:

1. Fysisk länk och VLAN-tillhörighet?
2. IP-adress erhållen via DHCP? (Kolla `ipconfig` / `ifconfig`)
3. Kan jag pinga min default gateway?
4. Fungerar namnupplösning lokalt? (`nslookup files.skolan.local`)
5. Fungerar namnupplösning externt? (`nslookup google.se`)
6. Finns det en NAT-översättning i brandväggen?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Viktiga portnummer att komma ihåg:

- DNS: Port 53 (UDP/TCP)
- DHCP: Port 67 (Server) och 68 (Klient) (UDP)
- HTTP/HTTPS: Port 80 / 443 (TCP)
- NTP: Port 123 (UDP)

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Beskriv de fyra stegen i DORA-processen (DORA process) och förklara vad som händer i varje steg.
2. Varför är en DHCP-relay (IP helper-address) nödvändig i ett segmenterat nätverk med flera VLAN?
3. Diskutera för- och nackdelar med korta respektive långa giltighetstider (lease times) för olika typer av nätverk.
4. Hur fungerar DHCP-snooping (DHCP snooping) och varför är det viktigt att definiera betrodda portar (trusted ports)?
5. Förklara skillnaden mellan en rekursiv resolver (recursive resolver) och en auktoritativ namnserver (authoritative name server).
6. Vilken roll spelar SRV-poster (SRV records) i en miljö med Active Directory?
7. Vad är syftet med split-horizon DNS och vilken säkerhetsfördel ger det organisationen?
8. Förklara tekniken PAT (Port Address Translation) och hur den gör det möjligt för hundratals enheter att dela på en enda publik IP-adress.
9. Beskriv ett scenario där hairpin NAT (NAT loopback) krävs och vad som händer om funktionen saknas.
10. Om en klient når en server via IP-adress men inte via dess domännamn, vilka tre steg skulle du ta för att felsöka problemet?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

Cisco: Understanding and Troubleshooting DHCP in Switched Networks (En djupdykning i DHCP-processer och relay) – cisco.com

Internet Systems Consortium (ISC): DHCP Failover Design (Teknisk vägledning för redundanta DHCP-lösningar) – isc.org

Cloudflare: What is DNS? (En mycket pedagogisk genomgång av DNS-hierarkin och uppslag) – cloudflare.com/learning/dns/what-is-dns/

Microsoft Learn: AD-Integrated DNS Zones (Hur DNS samverkar med Active Directory i Windows Server) – learn.microsoft.com

Practical Networking: NAT and PAT Explained (Tydliga visualiseringar av hur portar och adresser översätts) – practicalnetworking.net

RFC 1918: Address Allocation for Private Internets (Standarddokumentet för de privata adressrymderna) – datatracker.ietf.org/doc/html/rfc1918

DNS Checker: Propagation Tool (Ett verktyg för att se hur DNS-ändringar sprids över världen) – dnschecker.org

OPNsense Documentation: NAT Port Forwarding (Praktisk guide för DNAT och Hairpin NAT) – docs.opnsense.org

Wireshark Wiki: DHCP Analysis (Hur man läser och analyserar DHCP-paket i en paketfångst) – wiki.wireshark.org/DHCP

Google Public DNS: Security and Privacy (Information om hur publika resolvers hanterar förfrågningar och hot) – developers.google.com/speed/public-dns

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.5 Vanliga nätverksangrepp och hur vi försvarar oss

Många angrepp lyckas inte för att de är särskilt ”smarta” eller tekniskt sofistikerade, utan snarare för att nätverket drivs med osäkra standardinställningar (default settings), saknar tillräcklig segmentering (segmentation) eller lider av bristande övervakning (monitoring). Det är ofta i de små gliporna – en glömd portkonfiguration eller ett obevakat broadcast-domän – som hotaktörer hittar sitt fotfäste. Genom att förstå angreppens anatomi kan en tekniker gå från att vara reaktiv till att bli en proaktiv arkitekt som bygger säkerhet från grunden, istället för att bara lägga till den som ett efterhandsfilter.

Ett robust försvar bygger på principen om försvar på djupet (defense in depth), där varje lager i nätverksstacken bidrar till den totala motståndskraften. Här spelar de mekanismer som tidigare behandlats, såsom VLAN och segmentering (segmentation) i kapitel 2.2 samt DHCP, DNS och NAT i kapitel 2.4, en avgörande roll. Dessa är inte bara funktionella verktyg för att få trafiken att flyta, utan utgör fundamentet för att isolera hot och skydda nätverkets integritet (integrity). I det här avsnittet undersöks sex specifika attacker som är vanligt förekommande i verkliga miljöer, hur de manifesterar sig i trafiken och vilka motåtgärder som krävs för att göra försvaret robust.

2.5.1 DDoS – överbelastning av länk, protokoll eller applikation

En DDoS-attack (**Distributed Denial-of-Service**) är ett koordinerat försök att göra en tjänst, en server eller en hel nätverksinfrastruktur otillgänglig för legitima användare genom att dränka målet i trafik. Till skillnad från en vanlig DoS-attack, som kommer från en enda källa, använder en DDoS-attack tusentals eller miljontals kapade enheter – ofta kallat ett **botnät** (botnet) – för att rikta trafiken mot målet.

Dessa angrepp kan delas in i tre huvudkategorier beroende på vad de försöker slå ut. **Volymetriska attacker** (volumetric attacks) fokuserar på att fylla upp nätverkets bandbredd så att ingen annan trafik kommer fram. **Protokollattacker** (protocol attacks), såsom en **SYN-flood**, utnyttjar svagheter i nätverksprotokoll som TCP för att förbruka serverns resurser genom att lämna tusentals anslutningar halvöppna. Slutligen finns **applikationsattacker** (application layer attacks) som riktar in sig

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

på specifika funktioner, exempelvis genom att skicka en enorm mängd HTTP-förfrågningar som ser legitima ut men som tvingar webbservern att utföra tunga databasberäkningar tills den kraschar.

För att visualisera detta kan man tänka sig att "hela stan" plötsligt försöker kliva på exakt samma buss samtidigt. Bussen har fysiskt inte plats för alla, dörrarna kan inte stängas och bussen kommer ingenstans. Resultatet blir att de vanliga resenärerna, som faktiskt har biljetter och behöver komma till jobbet, blir stående kvar på hållplatsen utan möjlighet att resa.

Försvar och motåtgärder

När en attack väl är igång handlar försvaret om att så snabbt som möjligt identifiera och filtrera bort den skadliga trafiken. Detta görs ofta genom att omdirigera trafiken till ett **skrubbningscenter** (scrubbing center) hos en operatör eller molnleverantör, där avancerade algoritmer rensar bort angreppstrafiken och bara släpper igenom legitima paket. På lokal nivå kan man aktivera **SYN-cookies** för att skydda TCP-stacken mot öppna anslutningsförsök och använda **hastighetsbegränsning** (rate-limiting) för att strypa trafikflöden som avviker från det normala.

För att skydda sig proaktivt krävs en arkitektur som sprider riskerna. Genom att använda **Anycast** eller ett **CDN** (Content Delivery Network) kan man sprida ut belastningen på servrar över hela världen, vilket gör det mycket svårare för en angripare att slå ut hela tjänsten samtidigt. Det är också avgörande att ha kontinuerlig övervakning av nätverksflöden med protokoll som **NetFlow** eller **sFlow**. Genom att sätta upp larm på avvikande mönster kan man upptäcka en attack i sin linda och aktivera skyddsmekanismer innan tjänsten går ner. En välkonfigurerad brandvägg med strikta **åtkomstlistor** (ACL - Access Control Lists) kan också stoppa kända skadliga protokoll och adresser långt ute i nätverkets utkant.

Historiskt exempel och fördjupning

Ett av de mest kända fallen av en storskalig DDoS-attack är botnätet **Mirai** som under 2016 slog mot DNS-leverantören **Dyn**. Angreppet utfördes genom att kapa miljontals osäkrade IoT-enheter (Internet of Things), såsom webbkameror och routrar med standardlösenord. Genom att slå ut Dyn, som agerade namnserver åt många av världens största tjänster, blev sajter som Twitter, Spotify och Netflix oåtkomliga för användare i både USA och Europa under flera timmar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Du kan läsa mer om den fascinerande historien bakom Mirai och de tre unga hackarna som skapade det hos WIRED: The Mirai Botnet: The Untold Story

2.5.2 ARP-förgiftning – Man-in-the-Middle på LAN

ARP-förgiftning (ARP poisoning eller ARP spoofing) utnyttjar en grundläggande svaghet i hur enheter på ett lokalt nätverk (LAN) kopplar samman IP-adresser med fysiska MAC-adresser. Eftersom ARP-protokollet (Address Resolution Protocol) i sitt grundutförande är tillitsbaserat och saknar inbyggd autentisering, litar enheter blint på de svar de får. En angripare kan därför skicka falska ARP-svar (gratuitous ARP) till både en klient och dess nätverks-gateway. Genom att påstå att angriparens egen MAC-adress hör till gatewayens IP-adress luras klienten att skicka all sin trafik till angriparen istället för till routern. Samtidigt luras gatewayen att tro att angriparen är klienten. Resultatet blir en **Man-in-the-Middle (MitM)**-situation där angriparen kan avlyssna, ändra eller helt blockera trafiken utan att användaren märker något.

Detta kan liknas vid att någon sätter upp en falsk namnskylt på din brevlåda. Postbäraren börjar leverera din post till fel person, som kan läsa dina brev innan de eventuellt lämnas vidare till dig. Tekniskt sett kan man upptäcka detta genom att titta i enhetens ARP-tabell (ARP table). Om man ser konstiga bindningar, till exempel att flera olika IP-adresser pekar på exakt samma MAC-adress, eller om man ser dubbla svar på samma fråga, är det ett tydligt tecken på en pågående attack.

Försvar och motåtgärder

När en attack väl är igång handlar det om att snabbt identifiera källan och isolera den. Genom att granska switchens MAC-tabell kan man se vilken fysisk port som skickar ut de falska svaren och stänga ner den. För att skydda sig proaktivt krävs ett aktivt försvar i nätverkets switchar, där det främsta verktyget är **Dynamic ARP Inspection (DAI)**. DAI fungerar som en dörrvakt som kontrollerar varje ARP-paket som passerar genom switchen. Switchen jämför informationen i ARP-paketet med en betrodd databas – oftast den bindningsdatabas som skapas av **DHCP-snooping**. Om IP-adressen och MAC-adressen i ARP-paketet inte matchar det som finns i den betrodda databasen, kastas paketet omedelbart och attacken stoppas innan den hinner göra skada.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

För att göra försvaret ännu mer robust bör DAI kombineras med **IP Source Guard** på accessportarna, vilket förhindrar enheter från att skicka trafik med en förfalskad käll-IP. Det är även rekommenderat att implementera nätverksåtkomstkontroll via **802.1X** för att säkerställa att endast kända och autentiserade enheter överhuvudtaget får ansluta till nätverket. En annan viktig säkerhetsåtgärd är att alltid placera nätverksutrustningens hanteringsgränssnitt i ett eget, isolerat **management-VLAN** som vanliga användare inte har åtkomst till. Genom att se till att nätverket fungerar som en "låst brevlåda med rätt namn" minimeras risken för att interna angripare ska kunna manipulera trafikflödena.

Exempel och fördjupning

Ett typiskt scenario för en utförd attack är i ett oskyddat kontorsnätverk där en angripare ansluter en laptop och kör ett automatiserat verktyg som *Ettercap* eller *Bettercap*. Inom några sekunder börjar trafiken från grannens dator flyta genom angriparens maskin. Angriparen kan då se okrypterade lösenord eller injicera skadlig kod i webbsidor som användaren besöker. Utan DAI i switchen är detta angrepp nästintill osynligt för den vanliga användaren.

Du kan läsa mer om de tekniska detaljerna kring hur man konfigurerar Dynamic ARP Inspection och hur det skyddar nätverket i Ciscos officiella dokumentation: Configuring Dynamic ARP Inspection (Cisco)

2.5.3 Rogue/Evil-Twin-AP – falsk accesspunkt med ditt SSID

En **Rogue AP** (otillåten accesspunkt) eller en **Evil Twin** (falsk tvilling-AP) är ett angrepp där en obehörig person placerar en egen trådlös sändare i eller i närheten av nätverket. Genom att använda exakt samma nätverksnamn (SSID – Service Set Identifier) som det legitima nätverket luras klienternas enheter att ansluta till den falska punkten. Angriparen konfigurerar ofta sin utrustning för att sända med starkare signal eller använder störsändningstekniker för att tvinga bort klienter från det riktiga nätverket. När enheten väl är ansluten till "tvillingen" kan angriparen utföra en Man-in-the-Middle-attack, avlyssna okrypterad trafik eller omdirigera användaren till falska inloggningssidor för att stjäla referenser (credentials).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Detta kan liknas vid att en falsk buss med exakt samma linjenummer som din vanliga buss kör fram till hållplatsen. Du kliver på i god tro, men istället för att komma till din slutstation hamnar du i händerna på någon som kan registrera allt du gör under resan. Fenomenet är särskilt farligt eftersom moderna smartphones och datorer ofta är inställda på att automatiskt ansluta till kända nätverksnamn utan att fråga användaren om lov.

Försvar och motåtgärder

Om en attack upptäcks när den är igång är det primära motdraget att använda ett **WIDS/WIPS** (Wireless Intrusion Detection/Prevention System). Dessa system finns inbyggda i professionella trådlösa lösningar och skannar kontinuerligt etern efter sändare som utger sig för att tillhöra nätverket men som inte är godkända av kontrollenheten (controller). Vid en identifierad attack kan systemet skicka ut avautentiseringsramar (deauthentication frames) som klipper kopplingen mellan klienterna och den falska accesspunkten, vilket effektivt oskadliggör angreppet medan tekniker letar efter den fysiska hårdvaran.

För att skydda sig proaktivt bör man implementera **WPA2/3-Enterprise** (802.1X) istället för att använda ett gemensamt lösenord (PSK – Pre-Shared Key). Med 802.1X krävs att varje användare autentiserar sig individuellt, och viktigast av allt: klienten kan konfigureras att verifiera ett **servercertifikat** (server certificate) från nätverket. Om den falska accesspunkten inte kan visa upp ett giltigt certifikat som är signerat av organisationens betrodda utfärdare, kommer klienten att vägra ansluta. Man bör även aktivera **PMF** (Protected Management Frames / 802.11w) för att förhindra att angripare kan sparka ut klienter från det riktiga nätverket, samt använda **klientisolering** (client isolation) i gästnätverk för att minimera risken att en infekterad enhet kan skada andra användare i samma VLAN.

Exempel och fördjupning

Ett välkänt exempel på detta är angrepp i publika miljöer som flygplatser eller caféer, där angripare sätter upp en sändare med namn som "Free_Airport_WiFi". Användare som ansluter tror att de använder en officiell tjänst, men i själva verket passerar all deras banktrafik och sociala medier-inloggningar genom angriparens dator. I en skolmiljö skulle en

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

elev kunna göra samma sak genom att sända ut skolans officiella SSID från en mobiltelefon eller en liten bärbar router gömd i ett skåp.

Forskningen kring att upptäcka och motverka dessa "tvillingar" är omfattande och använder ofta statistiska metoder för att se skillnad på radioegenskaper mellan äkta och falska sändare. Du kan läsa mer om avancerade metoder för att upptäcka dessa angrepp i denna forskningsartikel från MDPI: Detection of Evil Twin Access Points (MDPI)

2.5.4 Deauth/Disassoc-DoS i WLAN

En **Deauthentication-** eller **Disassociation-attack** är en form av överbelastningsangrepp (DoS) som riktar sig specifikt mot de trådlösa kontrollramarna i ett Wi-Fi-nätverk. I standardutförandet av äldre trådlösa protokoll är hanteringsramar (management frames), såsom kommandon för att koppla ner en klient, okrypterade och saknar autentisering. Detta innebär att en angripare med enkla verktyg kan förfalska en accesspunkts (AP) MAC-adress och skicka ut avautentiseringsramar (deauthentication frames) till en specifik klient eller till alla enheter på ett SSID. Mottagaren kan inte avgöra om kommandot kommer från den legitima accesspunkten eller från angriparen och klipper därför omedelbart anslutningen.

Denna attack kan liknas vid att en person klär ut sig till konduktör och går genom ett tåg och ropar att alla passagerare måste kliva av omedelbart på grund av ett tekniskt fel. Passagerarna, som litar på uniformen, lämnar tåget och kvar står en tom vagn. På samma sätt töms en accesspunkt på sina användare. Angreppet används ofta som ett första steg i en mer avancerad attack, till exempel för att tvinga en klient att koppla ner från det riktiga nätverket och istället ansluta till en **Evil Twin** (falsk tvilling-AP), eller för att fånga upp de första paketen i en handskakning för att knäcka lösenord.

Försvar och motåtgärder

Om en attack pågår i realtid är det primära verktyget för upptäckt ett **WIDS/WIPS** (Wireless Intrusion Detection/Prevention System). Systemet genererar larm när det ser en onormal mängd hanteringsramar som inte kommer från nätverkets egna enheter. Eftersom angriparen måste befinna sig inom radioräckvidd för att utföra attacken, kan tekniker använda signalstyrkemätning för att fysiskt lokalisera och avlägsna störningskällan. Det är dock viktigt att skilja på protokollattacker och ren

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

radiostörning (jamming). Jamming är en fysisk attack där angriparen sänder brus på samma frekvenser som nätverket använder, vilket dränker den legitima trafiken. Detta kan endast lösas genom att fysiskt ta bort sändaren eller genom att byta till andra kanaler och frekvensband.

För att skydda sig proaktivt är den mest effektiva metoden att aktivera **PMF** (Protected Management Frames), även känt som standarden **802.11w**. Med PMF aktiverat signeras hanteringsramarna kryptografiskt, vilket gör att klienten kan verifiera att ett "koppla ner"-kommando faktiskt kommer från den riktiga accesspunkten. Om angriparen försöker skicka en förfalskad ram kommer klienten att ignorera den eftersom signaturen saknas eller är felaktig. I den modernare säkerhetsstandarden **WPA3** är PMF ett krav och alltid aktiverat, men i WPA2-miljöer måste det ofta slås på manuellt i accesspunktens konfiguration. Genom att göra "konduktörens röst" igenkännbar genom kryptering förhindras massutloggningar och nätverket blir betydligt mer stabilt.

Exempel och fördjupning

Ett vanligt exempel på en utförd attack är i en skolmiljö där en elev använder en liten mikrokontroller (exempelvis en ESP8266 med mjukvaran "Wi-Fi Deauther") för att störa ut trådlösa projektorer eller elevdatorer under en lektion. Utan PMF aktiverat kan en sådan billig enhet störa ut hela klassrummets trådlösa kommunikation på några sekunder. Attacken är extremt svår att upptäcka för vanliga användare, som bara upplever att "nätet dog" eller att de ständigt kastas ut.

För en djupare teknisk genomgång av hur man konfigurerar och validerar skyddet för hanteringsramar på professionell utrustning, rekommenderas Ciscos officiella vägledning: Configure 802.11w Management Frame Protection (Cisco)

2.5.5 VLAN-hopping – att smita mellan segment

VLAN-hopping (VLAN hopping) är en attackmetod där en angripare försöker skicka trafik direkt från ett VLAN till ett annat utan att passera de säkerhetskontroller som finns i en router eller brandvägg. I ett välkonfigurerat nätverk ska segmenteringen vara absolut, men genom att utnyttja svagheter i hur switchar hanterar trunking-protokoll (trunking protocols) och taggning kan en angripare "smita" förbi dessa logiska gränser. Det finns främst två sätt detta sker på: genom att lura switchen

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

att förhandla fram en trunk-anslutning (**switch spoofing**) eller genom att manipulera nätverksramarna med dubbla taggar (**double tagging**).

Vid **double tagging** utnyttjar angriparen hur switchar hanterar det så kallade ursprungliga VLAN:et (native VLAN). Angriparen skickar en nätverksram med två 802.1Q-taggar. Den första switchen ser den yttre taggen, som matchar det native VLAN som porten tillhör, och tar bort den (stripping) innan ramen skickas vidare över trunken. Den andra switchen ser då den inre taggen och tror att ramen hör hemma i det målvlan som angriparen vill nå. Detta kan liknas vid felmärkta paket på ett sorteringsband; paketet har en yttre etikett som gör att det passerar första kontrollen, men väl inne på bandet avslöjar en inre etikett att paketet hör hemma på ett helt annat ställe och glider in i fel lastbil.

Försvar och motåtgärder

Att stoppa en pågående VLAN-hopping-attack i realtid är tekniskt svårt då trafiken ofta ser legitim ut för switchens hårdvara. Det handlar därför främst om att genomföra en korrekt och säker grundkonfiguration (hardening) av nätverksutrustningen för att omöjliggöra angreppet innan det startar. Den viktigaste åtgärden är att inaktivera den automatiska förhandlingen av trunkar, känd som **DTP** (Dynamic Trunking Protocol). Genom att hårdkoda (hard-coding) varje port till att antingen vara en ren accessport (access port) eller en statisk trunkport (trunk port) med kommandot `switchport nonegotiate`, förhindras angriparen från att utföra switch spoofing.

För att motverka double tagging är det ett fackmässigt krav att aldrig använda standard-VLAN:et (VLAN 1) som native VLAN. Istället bör man tilldela native VLAN till ett unikt och helt oanvänt ID (till exempel VLAN 999) på alla trunkar. Man bör dessutom konfigurera switchen att uttryckligen tagga även trafiken i native VLAN för att förhindra att otaggade ramar accepteras. Slutligen bör man tillämpa principen om minsta behörighet genom att använda funktionen för tillåtna VLAN (**allowed VLANs**) på alla trunk-anslutningar. Genom att begränsa vilka VLAN som faktiskt får passera mellan switcharna minskas angreppsytan avsevärt. Tillsammans skapar dessa åtgärder en miljö som liknar ett sorteringsband med strikta kontroller och tydlig märkning.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel och fördjupning

Ett klassiskt exempel på en utförd attack är när en angripare ansluten till en accessport som råkar ha kvar standardinställningen "Dynamic Auto" skickar DTP-paket för att lura switchen att porten ska vara en trunk. När trunken väl är etablerad får angriparen tillgång till all trafik som passerar mellan switcharna och kan fritt kommunicera med servrar i känsliga segment som annars skulle ha varit dolda bakom en brandvägg.

Du kan läsa mer om de tekniska detaljerna kring riskerna med VLAN 1 och hur man säkrar sina switchar mot hopping-attacker på Cisco Learning Network: VLAN 1 and VLAN Hopping Attack (Cisco Learning Network)

Det här är två av de mest klassiska attackerna i ett lokalt nätverk som direkt utnyttjar hur DHCP-protokollet är konstruerat för att vara hjälpsamt och snabbt, snarare än säkert. De fungerar ofta bäst i kombination: först rensar man spelplanen (Starvation) och sedan kliver man in som den falska hjälparen (Spoofing).

2.5.6 DHCP-spoofing och Starvation – Attacken mot adressutdelningen

En **DHCP Starvation-attack** går ut på att en angripare försöker tömma nätverkets DHCP-server på alla tillgängliga IP-adresser. Genom att använda automatiserade verktyg skickar angriparen tusentals DHCP-förfrågningar (DHCPDISCOVER) med förfalskade MAC-adresser (MAC spoofing). För DHCP-servern ser det ut som om tusentals nya unika enheter plötsligt har anslutit till nätverket och begärt en adress. Servern delar troget ut sina adresser tills dess **adresspool** (address pool) är helt tom.

Detta kan liknas vid att en person går in på ett apotek och tar samtliga nummerlappar från maskinen i entrén. När de riktiga kunderna kommer in finns det inga nummer kvar att ta, och de kan därför aldrig få hjälp, trots att personalen bakom disken faktiskt är ledig. I nätverket innebär detta en **överbelastning** (Denial-of-Service) där legitima klienter aldrig får den konfiguration de behöver för att komma ut på nätet.

När adresserna väl är slut kliver **DHCP-spoofing** in i bilden. Angriparen startar en egen, falsk DHCP-server (**rogue DHCP server**) på sin dator. Eftersom den riktiga servern är tom på adresser, är angriparens server

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

den enda som svarar när en ny klient ropar efter hjälp. Angriparen delar då ut en fungerande IP-adress, men sätter sin egen dator som **standard-gateway** (default gateway) och en kontrollerad server som **DNS-server**. Detta skapar en perfekt **Man-in-the-Middle**-situation där all klientens trafik passerar angriparen innan den skickas vidare.

Försvar och motåtgärder

Om en attack pågår är det första steget att identifiera den fysiska porten där den onormala mängden DHCP-trafik eller de falska svaren kommer ifrån och stänga ner den. Man kan också rensa DHCP-serverns **bindningsdatabas** (binding database) för att frigöra adresser, men om angriparen fortfarande är aktiv kommer poolen att tömmas igen omedelbart.

För att skydda sig proaktivt är det absoluta standardförsvaret **DHCP-snooping** (DHCP snooping). Som vi gick igenom i kapitel 2.4 innebär detta att switchen delar upp sina portar i **betrodda** (trusted) och **obetrodda** (untrusted). Endast porten som leder till den riktiga DHCP-servern markeras som betrodd. Om switchen ser ett DHCP-svar (DHCPOFFER eller DHCPACK) komma in på en obetrodd port, blockeras det direkt. För att stoppa Starvation-attacker kombineras detta ofta med **port-säkerhet** (port security), där man begränsar hur många unika MAC-adresser som får kommunicera via en enskild accessport. Om en elevdator plötsligt försöker använda 500 olika MAC-adresser stängs porten ner automatiskt.

Exempel och fördjupning

Ett vanligt scenario i en skolmiljö är en elev som kör ett verktyg som *Yersinia* från sin bärbara dator. På bara några sekunder kan verktyget generera tillräckligt med trafik för att tömma DHCP-poolen för ett helt VLAN. Eleverna i klassrummet bredvid tappar då sin anslutning så snart deras **giltighetstid** (lease time) går ut, och angriparen kan börja fiska efter inloggningsuppgifter genom att dirigera om deras trafik till en falsk inloggningsportal.

Du kan läsa mer om hur man sätter upp ett effektivt försvar mot dessa attacker och hur man konfigurerar DHCP-snooping i detalj hos NetworkLessons: DHCP Snooping Configuration and Explained

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2.5.7 CAM-tabells-översvämning (MAC-flood)

En **CAM-tabells-översvämning** (CAM table overflow), mer känd som en **MAC-flood**, riktar in sig på själva "hjärnan" i en nätverksswitch. En switch använder en tabell för innehållsadresserbart minne (CAM – Content Addressable Memory) för att hålla reda på vilken MAC-adress som finns på vilken fysisk port. Detta minne är dock begränsat. En angripare utnyttjar detta genom att skicka tusentals nätverksramar med slumpmässigt genererade käll-MAC-adresser (source MAC addresses) till switchen. Inom några sekunder fylls CAM-tabellen med falsk information och når sin maxkapacitet. När tabellen är full kan switchen inte längre lära sig nya, legitima adresser och går in i ett läge som kallas **fail-open**, där den börjar bete sig som en enkel **hub**. Det innebär att all inkommande trafik skickas ut på samtliga portar i samma VLAN (flooding), vilket tillåter angriparen att avlyssna trafik som egentligen var menad för andra enheter genom en enkel paketfångst (packet capture).

Detta kan liknas vid ett adressregister i en stor postsorteringscentral. Om registret plötsligt fylls med tusentals påhittade namn och adresser kan personalen inte längre slå upp var paketen faktiskt ska ta vägen. För att paketen inte ska bli liggande väljer man att skicka ut en kopia av varje paket till varje enskilt utlämningsställe i hopp om att det hamnar rätt någonstans. På så sätt börjar man broadcasta informationen istället för att sortera den, vilket gör att vem som helst på nätverket kan läsa post som inte är adresserad till dem.

Försvar och motåtgärder

Om en attack pågår är det mest effektiva motdraget att omedelbart stänga ner den port som pumpar in de ogiltiga adresserna och rensa switchens dynamiska MAC-tabell för att återställa normal drift. För att skydda sig proaktivt är standardförsvaret i professionella campus-switchar att aktivera **portsäkerhet** (port security). Genom att begränsat det maximala antalet MAC-adresser (max-MAC) som tillåts på en enskild accessport kan man effektivt stoppa attacken innan tabellen fylls. En vanlig rekommendation är att använda funktionen **sticky MAC**, där switchen lär sig de första adresserna som ansluts och sparar dem i konfigurationen, vilket förhindrar att okända adresser överhuvudtaget får kommunicera på samma port.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Utöver portsäkerhet bör man även implementera **stormkontroll** (storm control) på nätverkets ytterkanter (edge ports). Denna funktion övervakar mängden broadcast-, multicast- och okänd unicast-trafik på porten och stryper anslutningen om trafiken överskrider en fördefinierad tröskel. Det är också god praxis att sätta upp larm (SNMP traps eller syslog) som varnar administratören om en port uppvisar ett onormalt antal MAC-adresser. För den som vill bygga ett mer proaktivt försvar rekommenderas även att kika på avsnitt 3.1 om baslinjer och 3.2 om övervakningsverktyg för att lära sig hur man upptäcker avvikelser i tid. Genom att kombinera dessa tekniker med en tydlig **violation-policy**, där porten exempelvis stängs ner helt (shutdown) vid ett regelbrott, skapar man ett försvar som automatiskt oskadliggör angreppet.

Exempel och fördjupning

Ett typiskt exempel på en utförd attack är en elev som kör ett verktyg som *macof* (en del av dsniff-paketet) från en Linux-dator i klassrummet. Inom några ögonblick kan verktyget generera hundratusentals falska MAC-adresser, vilket räcker för att sänka de flesta enterprise-switchar och tvinga dem att börja läcka trafik till angriparens dator. Utan portsäkerhet konfigurerad är detta ett av de enklaste sätten att snabbt få tillgång till känslig information i ett lokalt nätverk utan att behöva knäcka några lösenord.

Du kan läsa mer om de tekniska detaljerna kring hur man konfigurerar och hanterar portsäkerhet på moderna switchar i Ciscos officiella dokumentation: Configuring Port Security (Cisco)

2.5.8 Broadcast-storm – kaos i nätverkstrafiken

En broadcast-storm uppstår när broadcast-paket (eller multicast/okänd unicast) loopas runt i ett nätverk och dupliceras i en oändlig spiral. Till skillnad från IP-paket (lager 3) som har ett TTL-värde (Time to Live) som gör att de dör efter ett visst antal hopp, saknar Ethernet-ramar (lager 2) en sådan mekanism. Om det finns en cirkulär väg mellan två eller flera switchar kommer en enda broadcast-ram att skickas runt varv efter varv. Varje gång ramen passerar en switch skickas den ut på alla portar, vilket snabbt leder till att antalet ramar i nätverket växer exponentiellt. Inom några sekunder konsumeras all tillgänglig bandbredd och switcharnas

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

processorer (CPU) blir så överbelastade med att hantera dessa ramar att legitim trafik inte längre kan komma fram.

Detta kan liknas vid att två personer i en ekande korridor börjar ropa efter varandra samtidigt. Om ljudet studsar perfekt mellan väggarna och förstärks för varje eko, kommer ljudnivån snabbt att bli så öronbedövande att ingen längre kan höra vad som sägs. I nätverket manifesterar det sig ofta genom att samtliga lampor på en switch börjar blinka febrilt i samma takt (en så kallad "blink-of-death"), och alla anslutna enheter tappar kontakten med sina gateways och servrar.

Försvar och motåtgärder

När en broadcast-storm väl är igång är det mest effektiva motdraget att fysiskt eller logiskt bryta loopen. Detta innebär ofta att en tekniker måste börja dra ur kablar mellan switchar tills stormen lägger sig och nätverket återhämtar sig. Det är en stressig process eftersom man under stormen ofta tappar möjligheten att logga in på switcharna via nätverket (In-band management) för att stänga ner portar mjukvarumässigt. För att skydda sig proaktivt är det absolut viktigaste försvaret att ha **Spanning Tree Protocol (STP)** korrekt konfigurerat. STP upptäcker cirkulära vägar automatiskt och försätter "överflödiga" portar i ett blockerande läge (blocking state) för att förhindra att loopar ens kan bildas.

Som ett extra skyddslager bör man även implementera **stormkontroll** (storm control) på alla accessportar. Denna funktion fungerar som en säkerhetsventil; om mängden broadcast-trafik på en enskild port överstiger en viss procent av länkens totala kapacitet, börjar switchen kasta paketen eller stänger ner porten helt. Det är även rekommenderat att använda funktioner som **BPDU Guard** på portar där användare ansluter sin utrustning. Om någon av misstag (eller avsiktligt) kopplar in en egen switch eller bryggar två uttag med en kabel, känner BPDU Guard av detta och stänger omedelbart av porten innan en loop hinner sänka hela VLAN-segmentet.

Exempel och fördjupning

Ett klassiskt exempel på ett "fel" som blir till en attack mot tillgängligheten är i ett klassrum där en elev hittar en lös nätverkskabel och kopplar in båda ändarna i två olika nätverksuttag i väggen. Om inte STP eller BPDU Guard är aktivt i skolans access-switch, kommer detta att skapa en loop som omedelbart sänker nätverket för hela våningsplanet.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

En angripare kan utnyttja samma princip genom att använda mjukvara för att generera en enorm mängd broadcast-trafik från en enda dator för att se om nätverkets stormkontroll är korrekt inställd.

Du kan läsa mer om hur man diagnostiserar och förebygger broadcast-stormar samt hur man optimerar STP för att undvika dessa scenarier i Ciscos tekniska vägledning: Troubleshooting LAN Switching Environments (Cisco)

2.5.9 DNS-förgiftning – manipulering av namnupplösningen

DNS-förgiftning, ofta kallad **cache-förgiftning** (cache poisoning), går ut på att en angripare introducerar falsk information i en DNS-resolvers minne (cache). Eftersom en resolver sparar svar för att snabba upp framtida förfrågningar, kommer alla klienter som använder den resolvern att få det förfalskade svaret så länge det ligger kvar i cachen. Angreppet kan ske genom att angriparen skickar en mängd förfalskade svar till en resolver innan det riktiga svaret från den auktoritativa servern hinner fram. Om angriparen lyckas gissa rätt transaktions-ID, accepterar resolvern det falska svaret som sanning.

Detta kan liknas vid att någon bryter sig in på ett tryckeri och ändrar i stadens officiella kartor. När folk sedan använder kartan för att hitta till banken, leder vägbeskrivningen dem istället direkt till angriparens gömälle. Eftersom användaren litar på kartan (DNS-servern), ifrågasätter de inte att de hamnat fel förrän det är för sent. Resultatet blir att användare som tror att de besöker sin bank eller myndighet i själva verket hamnar på en exakt kopia av webbplatsen som kontrolleras av angriparen.

Försvar och motåtgärder

Om en attack upptäcks när den pågår är den omedelbara åtgärden att rensa resolverns cache (flush cache) och starta om tjänsten för att tvinga fram nya, korrekta uppslagningar. Man bör även tillfälligt byta till en annan pålitlig **vidarebefordrare** (forwarder) medan man utreder källan till de falska svaren. För att skydda sig proaktivt är den absolut viktigaste tekniken **DNSSEC** (Domain Name System Security Extensions). DNSSEC lägger till digitala signaturer på DNS-posterna, vilket gör att resolvern kan verifiera att svaret faktiskt kommer från rätt källa och inte har manipulerats under vägen.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

En annan viktig försvarsåtgärd är att använda moderna tekniker för krypterad DNS, såsom **DNS over HTTPS (DoH)** eller **DNS over TLS (DoT)**. Dessa protokoll förhindrar att en angripare på det lokala nätverket kan läsa eller ändra i DNS-förfrågningarna. Det är även rekommenderat att använda resolvers som implementerar **source port randomization**, vilket gör det betydligt svårare för en angripare att gissa rätt parametrar för att lyckas med en förgiftning. Genom att kombinera dessa tekniska skydd med en strikt brandväggspolicy (firewall policy) som endast tillåter DNS-trafik till och från kända, betrodda servrar, skapas ett mycket starkt försvar mot manipulering av namnupplösningen.

Exempel och fördjupning

Ett historiskt avgörande ögonblick för DNS-säkerhet var upptäckten av **Kaminsky-sårbarheten** år 2008. Säkerhetsforskaren Dan Kaminsky fann en fundamental svaghet i DNS-protokollet som gjorde det möjligt att förgifta en hel domän på bara några sekunder. Upptäckten var så allvarlig att den ledde till en hemlig, global insats där världens största teknikföretag samarbetade för att täppa till luckan innan den hann utnyttjas i stor skala. Detta blev startskottet för den breda utrullningen av DNSSEC.

Du kan läsa mer om den dramatiska historien bakom Kaminsky-sårbarheten och hur den förändrade internet i den här artikeln från Wired: The Day the Internet Almost Broke (Wired)

2.5.10 Sammanfattning och Best Practice

Att säkra ett nätverk handlar sällan om att hitta en enskild "silverkula", utan om att bygga en arkitektur som är motståndskraftig genom flera samverkande lager. De nio angreppstyper vi har gått igenom – allt från storskaliga **DDoS-attacker** till interna **ARP-förgiftningar** och **VLAN-hopping** – har en gemensam nämnare: de utnyttjar antingen protokoll som är byggda på tillit eller nätverksutrustning som körs med osäkra standardinställningar (default settings). Genom att systematiskt stänga dessa dörrar skapas en miljö där ett enskilt misstag eller en infekterad klient inte tillåts sänka hela infrastrukturen.

Grunden för ett säkert nätverk börjar i accesslagret (access layer), där användare och enheter först ansluter. En av de mest kritiska åtgärderna är att implementera **portsäkerhet** (port security) och **DHCP-snooping**.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Genom att begränsa antalet MAC-adresser per port och endast tillåta DHCP-svar från betrodda källor elimineras risken för både **MAC-flooding** och **DHCP-spoofing** omedelbart. När DHCP-snooping är aktivt skapas dessutom en bindningsdatabas som är en förutsättning för att kunna köra **Dynamic ARP Inspection (DAI)**, vilket i sin tur sätter stopp för **ARP-förgiftning** och Man-in-the-Middle-attacker på det lokala nätverket.

För att hantera logisk segmentering och förhindra att trafik "läcker" mellan olika VLAN via **VLAN-hopping**, krävs en strikt hantering av trunkar. Det är ett fackmässigt krav att inaktivera dynamisk förhandling via **DTP** och istället hårdkoda (hard-code) varje ports roll. Genom att dessutom byta ut det ursprungliga VLAN:et (**native VLAN**) från standardvärdet VLAN 1 till ett oanvänt ID och se till att all trafik taggas, tas de tekniska förutsättningarna för **double-tagging** bort. För att skydda nätverkets tillgänglighet mot både mänskliga fel och medvetna försök att skapa **broadcast-stormar**, måste **Spanning Tree Protocol (STP)** vara korrekt konfigurerat tillsammans med skyddsfunktioner som **BPDU Guard** på alla användarportar.

I den trådlösa miljön är det tydligt att gamla metoder med gemensamma lösenord (PSK) inte längre räcker till i professionella sammanhang. För att motverka **Evil Twin-attacker** och avlyssning är övergången till **WPA2/3-Enterprise** (802.1X) nödvändig, då det tillåter klienten att verifiera nätverkets identitet via servercertifikat. Samtidigt är aktivering av **PMF** (Protected Management Frames / 802.11w) den enda effektiva metoden för att stoppa de störande **Deauth-attackerna** som annars enkelt kan kasta ut användare från nätet. På tjänstenivå måste tilliten till namnupplösningen stärkas genom implementering av **DNSSEC**, vilket förhindrar **DNS-förgiftning** genom att signera svaren kryptografiskt.

Slutligen krävs ett försvar mot de yttre hoten i form av **DDoS-attacker**. Här handlar det om att flytta försvaret så långt ut i kedjan som möjligt, ofta genom att använda molnbaserade skyddstjänster, **CDN-nätverk** och tekniker som **Anycast** för att sprida ut belastningen. Genom att kombinera dessa tekniska spärrar med en kontinuerlig övervakning via **NetFlow** eller **sFlow**, kan administratören etablera en baslinje (baseline) för vad som är normal trafik. Det gör det möjligt att upptäcka och agera på avvikelser innan de hinner utvecklas till fullskaliga avbrott. Att säkra nätverket är en pågående process, men genom att följa dessa

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

branschstandarder (best practices) skapas en miljö som är både professionell, förutsägbar och motståndskraftig.

Kommandon och verktyg för validering:

- Visa ARP-tabellen: `arp -a` (Windows) / `ip neigh` (Linux)
- Visa MAC-tabellen i switch: `show mac address-table`
- Kontrollera DHCP-snooping status: `show ip dhcp snooping binding`
- Testa namnupplösning: `nslookup` / `dig`

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Förklara skillnaden mellan en volymetrisk DDoS-attack (volumetric attack) och en applikationsattack (application layer attack). Vilka olika resurser försöker de överbelasta?
2. Beskriv hur en angripare kan använda ARP-förgiftning (ARP poisoning) för att utföra en Man-in-the-Middle-attack. Varför litar klienterna på de falska svaren?
3. Vilken teknisk funktion i en switch är nödvändig för att Dynamic ARP Inspection (DAI) ska fungera, och varför behövs den kopplingen?
4. Hur kan användandet av WPA2/3-Enterprise (802.1X) och servercertifikat förhindra att en användare ansluter till en Evil Twin-accesspunkt?
5. Varför är hanteringsramar (management frames) sårbara för Deauth-attacker i äldre trådlösa nätverk, och hur löser PMF (802.11w) detta?
6. Beskriv mekanismen bakom "double tagging" vid VLAN-hopping. Vilken roll spelar det ursprungliga VLAN:et (native VLAN) i detta angrepp?
7. Förklara sambandet mellan DHCP Starvation och DHCP Spoofing. Varför utförs de ofta tillsammans?
8. Vad händer rent tekniskt med en switch när dess CAM-tabell (CAM table) blir full under en MAC-flood, och vilken säkerhetsrisk innebär detta för trafiken i VLAN-segmentet?
9. Hur skiljer sig en broadcast-storm från andra attacker när det gäller dess ursprung, och vilka två tekniker är viktigast för att förhindra att den sänker nätverket?
10. På vilket sätt skyddar DNSSEC mot cache-förgiftning (cache poisoning), och vad är den största skillnaden mellan DNSSEC och krypterad DNS (som DoH eller DoT)?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

Cisco: Configuring Dynamic ARP Inspection (Teknisk vägledning för att säkra lager 2-trafik) – cisco.com

Wired: The Mirai Botnet Story (En djupdykning i hur osäkra IoT-enheter skapade ett globalt kaos) – wired.com

Cloudflare Learning: What is a DDoS Attack? (Pedagogiska förklaringar av olika typer av överbelastningsangrepp) – cloudflare.com

MDPI Research: Detection of Evil Twin APs (Vetenskaplig artikel om avancerade metoder för att upptäcka falska accesspunkter) – mdpi.com

NetworkLessons: DHCP Snooping Explained (Praktiska exempel på hur man konfigurerar skydd mot DHCP-attacker) – networklessons.com

Cisco Learning Network: VLAN Hopping (Detaljerad genomgång av switch-spoofing och double tagging) – learningnetwork.cisco.com

Wired: The Kaminsky DNS Bug (Historien om sårbarheten som nästan förstörde internet och ledde till DNSSEC) – wired.com

OWASP: Denial of Service (Information om applikationsspecifika överbelastningsattacker) – owasp.org

Cisco: Port Security Guide (Hur man begränsar MAC-adresser och hanterar violations på accessportar) – cisco.com

SANS Institute: Securing Wireless Networks (Best practice och ramverk för säkra trådlösa implementationer) – sans.org

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3 Nätverksadministration

När de tekniska fundamenten för routing, switching och nätverkstjänster väl är på plats, skiftar fokus från konstruktion till långsiktig förvaltning och nätverksadministration (network administration). Att administrera ett nätverk handlar om betydligt mer än att bara hålla igång trafiken; det är en disciplin som kombinerar teknisk expertis med metodiskt tänkande för att säkerställa hög tillgänglighet (high availability), god prestanda och en hög säkerhetsnivå. En skicklig administratör ser nätverket som en levande organism som ständigt kräver tillsyn, optimering och anpassning efter verksamhetens föränderliga behov.

Detta kapitel fungerar som en brygga mellan de logiska protokollen och den praktiska vardagen i ett serverrum eller ett nätverkscenter (NOC - Network Operations Center). Här utforskas de verktyg och metoder som används för att övervaka systemens hälsa, hantera identiteter i komplexa katalogtjänster (directory services) och konfigurera både servrar och klienter på ett strukturerat sätt. Målet med nätverksadministration är att skapa en miljö som är både förutsägbar och skalbar (scalable), där problem upptäcks och åtgärdas innan de hinner påverka slutanvändarna. Genom att kombinera beprövade administrativa rutiner med moderna verktyg för automatisering läggs grunden för en stabil och professionell IT-infrastruktur.

3.1 Grunder i nätverksadministration

Inom ramen för de administrativa grunderna ligger fokus på det systematiska arbetet och de verktyg som utgör en nätverksteknikers dagliga hantverk. Grundläggande nätverksadministration (fundamentals of network administration) handlar om att etablera fungerande arbetssätt (workflows) som minimerar risker vid förändringar och maximerar effektiviteten vid felsökning. Det handlar om att gå från ett reaktivt läge, där man släcker bränder när de uppstår, till ett proaktivt läge där dokumentation, baslinjer (baselines) och standardiserade konfigurationer skapar stabilitet över tid.

Att behärska dessa grunder innebär att förstå hela kedjan av händelser i ett nätverk, från det att en klient begär en adress via DHCP till dess att en säker fjärranslutning (remote access) etableras för underhåll. Det

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

innefattar även vikten av att hantera ändringar (change management) och säkerhetsuppdateringar (patch management) på ett ansvarsfullt sätt. Genom att tillämpa en tydlig metodik och använda rätt diagnostiska verktyg kan administrativa uppgifter utföras med precision. Detta avsnitt ger den nödvändiga verktygslådan för att hantera driften av moderna nätverksmiljöer, oavsett om det rör sig om små lokala nät eller stora distribuerade system med hundratals noder.

3.1.1 Syfte, roller och arbetssätt

Nätverksadministration (network administration) utgör den disciplin som krävs för att planera, drifta, övervaka (monitoring) och ständigt förbättra nätverksmiljön. Det övergripande syftet är att säkerställa att användare, servrar och tjänster får den tillgänglighet (availability), prestanda (performance) och säkerhet (security) som verksamheten kräver för att fungera effektivt. Arbetet är omfattande och spänner över allt från den fysiska infrastrukturen (physical infrastructure) – såsom kablage, switchar och brandväggar – till det logiska lagret (logical layer) där konfiguration av VLAN, routing, DNS, DHCP och autentisering (authentication) sker. Utöver de tekniska aspekterna innefattar administrationen även de processer som skapar spårbarhet (traceability) och stabilitet, däribland dokumentation, ändringshantering (change management) och etablerade incidentrutiner (incident response routines).

För att upprätthålla en professionell driftnivå underlättar det att tänka i termer av olika roller, även i mindre miljöer där en och samma person ibland förväntas bära alla hattar samtidigt. Genom att särskilja rollen som designar och arkitekterar (network architect) nätet från den roll som sköter den dagliga driften och övervakningen (network operations), skapas en naturlig struktur för hur beslut fattas och genomförs. En viktig del i arbetssättet är att ha en tydlig metodik för vem som godkänner förändringar i nätet, vilket minskar risken för oplanerade avbrott. Att arbeta strukturerat innebär att varje teknisk justering ses som en del av en större livscykel där planering och validering är lika viktiga som själva utförandet.

Förtydligande (sammanfattning):

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Mål:** Att uppnå och bibehålla tillgänglighet (availability), prestanda (performance), säkerhet (security) och fullständig spårbarhet (traceability).
- **Omfång:** Hantering av både den fysiska infrastrukturen (physical infrastructure), de logiska tjänsterna och de administrativa processerna.
- **Roller:** Uppdelning mellan design och arkitektur (design/architecture), löpande drift och övervakning (operations/monitoring) samt hantering av ändringar och incidenter (change/incident management).

Exempel: En elevdator i skolan kan plötsligt inte nå det interna intranätet. Nätverksadministratören agerar då i sin driftsroll och följer en etablerad standardmetodik (se avsnitt 3.1.5) för att isolera felet. Alla observationer och de åtgärder som vidtas dokumenteras löpande, och när felet är avhjälpt sker en formell återkoppling enligt skolans incidentrutin för att säkerställa att liknande problem kan förebyggas i framtiden.

3.1.2 Basverktyg och kommandon

Att behärska ett kärnbibliotek av verktyg för kontroll, mätning och felsökning (troubleshooting) är en grundförutsättning för att kunna arbeta effektivt i moderna nätverksmiljöer. En skicklig administratör växlar sömlöst mellan olika operativsystem och plattformar, vilket gör det nödvändigt att förstå hur samma logiska funktioner uttrycks i exempelvis Cisco IOS kontra Linux eller Windows. I det dagliga arbetet används dessa verktyg för att verifiera konfigurationen på både klientsidan, serversidan och direkt på nätutrustningens kommandoradsgränssnitt (CLI - Command Line Interface). Tabellen nedan sammanställer de vanligaste kommandona för att inspektera nätverkets tillstånd.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Funktion	Cisco IOS	Linux (iproute2/Bash)	Windows (CMD/PowerShell)
Visa IP-adresser	show ip interface brief	ip -brief addr	ipconfig / Get-NetIPAddress
Gränssnittsstatus	show interfaces status	ip link show	ipconfig /all / Get-NetAdapter
Nåbarhet (Ping)	ping [IP/namn]	ping [IP/namn]	ping [IP/namn]
Vägval (Traceroute)	traceroute [IP/namn]	traceroute / mtr [IP/namn]	tracert [IP/namn]
Routingtabell	show ip route	ip route	route print / Get-NetRoute
ARP-tabell	show arp	ip neigh	arp -a / Get-NetNeighbor
Aktiva portar/flöden	show tcp brief	ss -tuln	netstat -ano / Get-NetTCPConnection
DNS-uppslag	*(Begränsat i IOS)*	dig / nslookup	nslookup / Resolve-DnsName

Genom att kombinera dessa verktyg kan en administratör snabbt isolera var i nätverksstacken ett problem uppstått. En systematisk metodik innebär att man jämför resultaten från olika lager för att se var kommunikationen bryts.

Särskilt i Linux-miljöer är `mtr` ett överlägset alternativ vid analys av instabila anslutningar, då den till skillnad från vanlig `traceroute` skickar kontinuerliga paket och visar statistik över latens (latency) och paketförlust (packet loss) för varje hopp över tid. För djupare analys av själva trafikflödena används paketfångst (packet capture); här är **Wireshark** den ledande grafiska lösningen (GUI) för att inspektera protokoll i detalj, medan **tcpdump** är standardverktyget för analys direkt via terminalen. Vid trådlös felsökning används specifika kommandon som **iw** för att granska kanaler och signalstyrka (SNR - Signal-to-Noise Ratio).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Att regelbundet använda dessa verktyg under normal drift är avgörande för att etablera nätverkets **baslinje** (baseline). Genom att känna till vad som är normala svarstider, hur en felfri routingtabell ser ut eller vilka portar som förväntas vara öppna, kan administratören betydligt snabbare identifiera avvikelser som tyder på begynnande hårdvarufel eller nätverksangrepp.

Exempel: *Du utför en* ***ping*** *mot IP-adressen* `10.40.0.50` *och får ett lyckat svar (OK), men när du försöker göra ett uppslag med* ***nslookup*** *mot namnet* `files.skolan.local` *misslyckas det. Slutsatsen blir då att den underliggande IP-anslutningen (IP connectivity) fungerar korrekt och att problemet är isolerat till namnupplösningen (DNS), vilket behandlas mer ingående i avsnitt 3.1.4.*

3.1.3 Standardkonfigurationer och baslinjer

Se även kap. 8 i Datorteknik, för djupare information om loggning och säkerhet i Linux och Windows.

Starka grundinställningar gör nätverket förutsägbart och säkert. Genom att etablera en tydlig **baslinje** (baseline) för alla nätverkskomponenter säkerställs att varje switch, router och brandvägg i infrastrukturen uppträder enhetligt. Detta underlättar inte bara den dagliga driften utan är även en förutsättning för att snabbt kunna rulla ut nya enheter eller återställa system efter ett maskinfel. En väl genomarbetad baslinje adresserar allt från tidssynkronisering och loggning till strikt åtkomstkontroll och säkerhetsinställningar i accesslagret.

Kärnan i en professionell baslinje är tidssynkronisering via **nätverkstidsprotokollet** (NTP - Network Time Protocol) och centraliserad loggning (syslog). För att händelser i nätverket ska kunna korreleras vid en incident krävs att alla enheter har exakt samma tidskälla; utan synkroniserade klockor blir loggfiler från olika servrar och switchar nästintill omöjliga att analysera tillsammans. Loggarna bör skickas till en central loggserver eller en **Event Viewer**-samlare för att skydda dem mot lokal manipulation och för att möjliggöra automatiserade larm vid kritiska händelser.

Säkerheten kring själva administrationen är en annan pelare i baslinjen. Detta innebär att osäkra protokoll som Telnet omedelbart inaktiveras till

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

förmån för **SSH** (Secure Shell) för alla terminalanslutningar. Istället för delade lösenord används unika administratörskonton (unique admin accounts), gärna kopplade till en central autentiseringstjänst (AAA - Authentication, Authorization, and Accounting) med stöd för **flerfaktorsautentisering** (MFA - Multi-Factor Authentication) där det är tekniskt möjligt. All administration bör ske i ett dedikerat och isolerat **administrations-VLAN** (management VLAN) som skyddas av strikta **åtkomstlistor** (ACL - Access Control Lists). Dessa listor säkerställer att endast betrodda administratörsverktyg och kända IP-adresser tillåts kommunicera med nätverksutrustningens hanteringsgränssnitt.

På hårdvarunivå innefattar baslinjen även att stänga ner onödiga och osäkra funktioner som **DTP** (Dynamic Trunking Protocol) och att aktivera skyddsmekanismer som **stormkontroll** (storm control) och **portsäkerhet** (port security). Slutligen är en fungerande rutin för konfigurationsbackup (config backup) och versionshantering (version control) livsviktig. Genom att spara historik över alla ändringar kan administratören snabbt utföra en jämförelse (diff) mellan den nuvarande konfigurationen och en tidigare känd fungerande version, vilket är ovärderligt vid felsökning efter en misslyckad uppdatering.

Förtydligande (sammanfattning):

- **Tid och loggning:** Konfigurera NTP mot en gemensam, pålitlig källa och skicka alla loggar till en central server för analys och arkivering.
- **Åtkomst och identitet:** Använd uteslutande SSH, tillämpa unika administratörskonton och implementera MFA där det är möjligt för att höja säkerheten.
- **Management-design:** Isolera nätverksutrustningen i ett eget management-VLAN och begränsa åtkomsten med brandväggsregler eller ACL:er.
- **Konfigurationshantering:** Etablera en **standardkonfiguration** (golden config), utför regelbundna backuper och använd versionshantering för att kunna följa historik och snabbt återställa tjänster.

Exempel: *Alla switchar i nätverket har konfigurerats med NTP-servern `10.40.0.12` som tidskälla och skickar sin syslog-trafik till `10.40.0.14`. All*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

fjärråtkomst sker via SSH, och administrationsgränssnitten har flyttats till VLAN 99 med en ACL som endast släpper in trafik från IT-avdelningens specifika administratörsverktyg.

3.1.4 DNS, DHCP och adressering i drift

Se även kap. 6.7 i boken Datorteknik för mer information om Active Directory

I vardagen utgör DNS och DHCP själva navet för användarupplevelsen i nätverket. Om dessa tjänster inte fungerar optimalt spelar det ingen roll hur snabba routrarna är; användarna kommer uppleva att "nätet ligger nere" så fort de inte kan få en IP-adress eller slå upp ett domännamn. För en administratör handlar driften om att proaktivt hantera adressomfång (scopes), hantera reservationer (reservations) för fast utrustning och säkerställa att förfrågningar når fram via rätt relay-inställningar. Samtidigt krävs en stabil namnarkitektur där interna zoner, **split-horizon** och effektiva vidarebefordrare (forwarders) samverkar för att ge snabba svarstider.

När nätverket är i produktion är administratörens primära uppgift i DHCP-miljön att övervaka utnyttjandegraden av adresspoolerna. Genom att matcha giltighetstider (lease times) mot användarnas rörlighet kan man förhindra att adresser tar slut; ett gästnätverk (guest network) mår bäst av korta tider för att snabbt frigöra resurser, medan personalens fasta segment kan ha betydligt längre tider för att öka stabiliteten. Det är också här som administrativa reservationer (reservations) görs för att säkerställa att nätverksskrivare och servrar alltid har samma adress utan att behöva konfigureras statiskt på själva enheten. Allt detta kräver en noggrann dokumentation i form av en adressplan (address plan) som tydligt visar kopplingen mellan VLAN, IP-prefix, gateway och tillhörande DHCP-scope.

Inom namnhanteringen (DNS) är det ofta en **Active Directory-integrerad** lösning som utgör basen. Här hanterar administratören de interna zonerna och ser till att klienterna tilldelas minst två oberoende namnservrar (resolvers) via **DHCP-option 6**. Genom att använda split-horizon DNS kan man styra så att interna användare når resurser via privata adresser medan externa besökare ser de publika, vilket ökar både säkerhet och prestanda. En annan viktig

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

driftaspekt är hanteringen av **TTL** (Time to Live) för DNS-poster; ett väl valt värde balanserar behovet av snabb uppdatering vid förändringar mot att minska onödig trafik och belastning på namnservrarna.

För att hålla ordning på infrastrukturen är det fackmässigt att upprätthålla en aktuell tabell över nätverkets adressering. Denna fungerar som administratörens karta och kompass vid felsökning och expansion.

VLAN	Funktion	Prefix / Subnät	Gateway	DHCP Scope
10	Personal	10.10.0.0/24	10.10.0.1	10.10.0.100–250
20	Elever	10.20.0.0/22	10.20.0.1	10.20.0.100–10.20.3.250
30	Gäster	172.16.0.0/24	172.16.0.1	172.16.0.20–250
99	Management	10.99.0.0/24	10.99.0.1	Inget (Statiskt)

Förtydligande (sammanfattning):

- **DHCP:** Säkerställ fungerande relay (IP-helper) och anpassa giltighetstid (lease time) efter målgruppens behov.
- **DNS:** Använd AD-integrerade zoner med redundanta resolvers och optimera TTL för att balansera stabilitet och flexibilitet.
- **Adressplan:** Dokumentera systematiskt förhållandet mellan VLAN, prefix, gateway och adresspooler (scopes).

Exempel: *Gäster i nätverket klagar på att internet upplevs som långsamt. Vid en mätning visar det sig att deras enheter är konfigurerade att gå direkt mot publika namnservrar ute på internet istället för nätverkets lokala resolvers. Detta leder till onödig latens (latency) för varje uppslagning. Lösningen blir att uppdatera policyn i brandväggen för att tvinga all DNS-trafik till skolans egna resolvers, som i sin tur använder snabba forwarders för att hämta externa svar. Detta sänker svarstiderna dramatiskt för gästerna.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.1.5 Felsökningsmetodik (lager för lager)

Vid felsökning av nätverk är en konsekvent och logisk metodik administratörens absolut viktigaste tillgång. Att "skjuta från höften" eller ändra flera parametrar samtidigt leder sällan till en lösning, utan snararare till ökad förvirring och potentiellt nya fel. Genom att arbeta systematiskt, antingen nerifrån och upp (från det fysiska kablaget) eller uppifrån och ner (från applikationen), kan man metodiskt utesluta felkällor lager för lager. Varje steg i processen bör grundas på faktiska mätningar, loggar och observationer för att säkerställa att man bygger sin analys på fakta snarare än antaganden.

Det första steget i all felsökning är att definiera problemets symptom och omfattning (scope). Det är avgörande att veta om felet påverkar en enskild klient, en specifik användargrupp eller en hel byggnad. När omfattningen är känd börjar den tekniska granskningen ofta vid den fysiska länken och lågnivå-inställningarna (link and low-level). Här kontrolleras att länklampor lyser, att kablar är hela och att porten i switchen är konfigurerad med rätt hastighet, duplex och framför allt rätt VLAN. Om den fysiska anslutningen är stabil går man vidare till adressering och gateway (address and gateway). Här verifieras att klienten har fått en korrekt IP-konfiguration via DHCP och att den kan kommunicera med sin standard-gateway (default gateway) genom ett enkelt ping-test.

När den grundläggande IP-anslutningen är bekräftad skiftas fokus mot namnupplösning och rutter (names and routes). Med verktyg som **nslookup**, **dig** och **traceroute** (eller **mtr**) kontrolleras att klienten kan översätta namn till adresser och att trafiken tar rätt väg genom nätverket. Om trafiken stannar halvvägs eller nekas åtkomst, är det dags att granska brandväggar och policys (firewall and policy). Genom att studera brandväggsloggar kan man se om paket kastas på grund av felaktiga regler eller om trafiken blockeras i en viss riktning. Det sista lagret i felsökningen rör själva applikationen (application layer), där man kontrollerar att rätt portar är öppna, att certifikat är giltiga och att autentiseringen (authentication) fungerar som förväntat.

Förtydligande (sammanfattning):

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Symptom och omfattning (scope):** Identifiera exakt vad som inte fungerar och hur många användare som påverkas.
- **Länk och lågnivå:** Kontrollera fysisk anslutning, länkhastighet och VLAN-tillhörighet på portnivå.
- **Adress och gateway:** Verifiera IP-adress (ipconfig/ip addr) och anslutning till gateway.
- **Namn och rutter:** Kontrollera DNS-uppslag och trafikens väg genom nätverket (traceroute/mtr).
- **Brandvägg och policy:** Granska loggar och verifiera att brandväggsregler tillåter trafiken i båda riktningarna.
- **Applikation:** Kontrollera applikationsspecifika inställningar såsom portar, kryptering och autentisering.

Exempel: *Elevernas trådlösa nätverk (VLAN 20) fungerar utan problem i skolans A-hus, men i B-huset kan ingen ansluta. Vid en systematisk kontroll av infrastrukturen upptäcks att trunk-anslutningen (trunk) mellan husens switchar saknar VLAN 20 i sin lista över tillåtna VLAN (allowed-list). Åtgärden blir att lägga till VLAN 20 på trunken, varpå funktionen omedelbart verifieras med en testklient på plats i B-huset.*

3.1.6 Övervakning och loggning

Mer information finns i 11.8 Verktyg, övervakning och loggning i boken Datorteknik

Övervakning (monitoring) och loggning (logging) är två sidor av samma mynt men fyller olika funktioner i nätverksadministrationen. Man brukar säga att övervakningen talar om hur nätet mår i realtid, medan loggningen talar om vad som faktiskt hände vid en viss tidpunkt. Genom att använda protokollet för nätverkshantering (SNMP - Simple Network Management Protocol) eller modern telemetri (telemetry) kan administratören kontinuerligt samla in data om enheternas hälsa. Detta innefattar kritiska parametrar som processorutnyttjande (CPU), minnesanvändning (RAM), länkbelastning, strömmatning via nätverket (PoE - Power over Ethernet) och temperatur.

Loggning fokuserar istället på händelseförlopp och säkerhet. Genom att skicka systemhändelser till en central loggserver (syslog) eller samla in

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

data via Windows **Event Viewer**, får man en samlad bild av brandväggsbeslut (firewall decisions), inloggningar (logins) och felmeddelanden från tjänster. För att denna enorma datamängd ska bli begriplig används visualiseringsverktyg och instrumentpaneler (dashboards) som lyfter fram viktiga nyckeltal (KPIs) såsom upptid (uptime), latens (latency) och antal incidenter. Genom att definiera larmtrösklar (alarm thresholds) kan systemet automatiskt varna när något avviker från det normala, vilket gör att administratören kan agera proaktivt istället för att bara reagera på felanmälningar.

Förtydligande (sammanfattning):

- **SNMP och telemetri:** Används för att övervaka tekniska parametrar på länkar, PoE-budget, resursutnyttjande och belastning i det trådlösa nätverkets lufttid (airtime).
- **Syslog och Event-hantering:** Fokuserar på historik och spårbarhet kring brandväggstrafik, autentisering (authentication) och kritiska systemhändelser.
- **Instrumentpaneler (dashboards):** Sammanställer data till överskådliga nyckeltal som ger en snabb blick över nätverkets status, upptid och latens.

Exempel: *Varje vardag vid lunchtid uppstår en plötslig spik i lufttidsutnyttjandet (airtime spike) på ett specifikt våningsplan, vilket gör nätet instabilt. Genom att granska loggarna ser administratören att gäst-SSID (guest SSID) blir helt överbelastat när elever och personal samlas i lunchrummet. Åtgärden blir att installera en extra accesspunkt (AP), aktivera* ***band steering*** *för att flytta klienter till 5 GHz-bandet samt justera* ***min-RSSI*** *för att tvinga enheter att ansluta till den närmaste sändaren, vilket löser problemet.*

3.1.7 Patchning, ändringshantering och backup

Mer information finns i 11.9 Underhåll och säkerhetskopiering i boken Datorteknik.

För att bibehålla en stabil och säker nätverksmiljö krävs väl fungerande rutiner för underhåll. Det handlar om att balansera behovet av nya funktioner och säkerhetsfixar mot risken för att uppdateringar orsakar oplanerade avbrott. Genom att arbeta med strukturerad patchning

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

(patching), formell ändringshantering (change management) och automatiserade backuper (backups) skapas en miljö där misstag minimeras och återställningstiden vid fel förkortas avsevärt. Dokumentation är här den röda tråden; varje ändring ska kunna spåras bakåt och varje system ska kunna återställas till en känd fungerande version.

Säkerhetsuppdateringar och firmware-uppdateringar (firmware updates) bör hanteras enligt en fastställd patchpolicy (patch policy). Kritiska säkerhetsfixar prioriteras högst, men det är fackmässigt att alltid testa uppdateringen i en mindre skala eller i en labbmiljö innan den rullas ut i full produktion. Vid större ingrepp tillämpas ändringshantering (change management), vilket innebär att man planerar ett servicefönster (maintenance window), dokumenterar syftet med ändringen och identifierar eventuell påverkan på användarna. En central del i planeringen är att ha en tydlig rollback-plan (rollback plan) – en instruktion för hur man snabbt återställer systemet om något går fel under uppdateringen.

När det gäller backuphantering räcker det inte med att ta manuella kopior då och då. Istället bör nätverksutrustningens konfigurationer dumpas automatiskt, till exempel varje natt, och versionshanteras (versioning) så att administratören kan jämföra skillnader (diff) mellan olika datum. För maximal säkerhet bör backuperna sparas på en fysiskt separat plats (off-site backup) och rutinmässigt testas genom återläsningstester (restore tests). Endast genom att faktiskt prova att återställa en konfiguration kan man vara säker på att backupen fungerar den dagen den verkligen behövs. All dokumentation kring ändringar och konfigurationer bör följa minimikrav på tydlighet, inklusive titel, datum, version, ansvarig person och referenser till ärendenummer.

Dokumentationsexempel för storföretag

I en större organisation räcker det inte med lösa anteckningar; dokumentationen måste vara standardiserad för att möjliggöra samarbete mellan olika tekniker och avdelningar. Ett centralt dokument är ändringsansökan (Change Request - CR), som fungerar som ett formellt beslutsunderlag inför ett arbete. En sådan dokumentation innehåller kritiska fält för att minimera osäkerhet och risk.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Ändringsansökan (Change Request)

Fält	Beskrivning / Exempel
Ärende-ID	CR-2026-03-23-001
Ansvarig tekniker	Robin Bräck
Syfte och omfattning	Uppgradering av firmware på kärnswitchar (core switches) för att åtgärda kritisk sårbarhet (CVE-2026-X).
Påverkan (Impact)	Total nätverksnedtid i Hus B under 15 minuter per switch.
Riskbedömning	Medelhög. Risk för konfigurationsfel vid omstart.
Rollback-plan	Vid fel: återställ föregående firmware-image via TFTP och läs in nattlig konfigurationsbackup.
Verifiering	Ping-test mot gateway, kontroll av OSPF-grannskap och verifiering av elev-VLAN.

För att bibehålla ordningen över tid används en ändringslogg (change log) som ger en kronologisk översikt av allt som skett i infrastrukturen. Detta dokument är det första en tekniker läser vid felsökning för att se om ett nyligen utfört arbete kan vara orsaken till ett problem. Loggen bör vara kortfattad men innehålla referenser till djupare dokumentation eller ärenden.

Ändringslogg (Change Log)

Datum	Enhet / System	Version	Ansvarig	Sammanfattning av ändring
2026-03-10	SW-CORE-01	v17.9.2	RB	Patchning av säkerhetshål. Inga avvikelser.
2026-03-12	FW-BORDER	v7.4.1	RB	Ny ACL för att blockera utgående DNS (Policy 3.1.4).
2026-03-20	SW-ACCESS-05	v15.2.4	AL	Utökat DHCP-pool för Gäst-VLAN (VLAN 30).

Slutligen krävs en fysisk och logisk koppling i form av portetikettering (port labeling). Varje port i en switch ska vara dokumenterad så att det är tydligt vad

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

som är anslutet, vilket VLAN som används och var den fysiska slutpunkten finns. Detta underlättar enormt när en tekniker befinner sig fysiskt i ett serverrum och snabbt behöver identifiera en specifik länk.

Portetikettering (Switch Port Map)

Switch	Port	VLAN	Beskrivning / Syfte	Fysisk plats / Uttag
SW-ACC-01	Gi0/1	10	Personal-PC	Rum 204, Uttag A1
SW-ACC-01	Gi0/2	20	Elev-AP	Korridor vån 2, takmontage
SW-ACC-01	Gi0/48	99	Trunk till CORE	Patchpanel 1, Port 12

Förtydligande (sammanfattning):

- **Patchpolicy:** Prioritera kritiska säkerhetsfixar och utför alltid tester i begränsad skala före bred utrullning.
- **Ändringshantering (Change):** Definiera vem som gör vad, när det sker, hur effekten mäts och hur en återgång (rollback) genomförs vid fel.
- **Backup:** Implementera nattliga konfigurationsdumpar med versionshistorik, säkerställ kopior utanför den primära sajten och utför regelbundna återläsningstester.

Exempel: *En planerad uppgradering av skolans centrala brandvägg genomförs klockan 18:00 för att minimera påverkan på undervisningen. Före och efter uppgraderingen genomförs mätningar av latens (latency) och genomströmning för att verifiera resultatet. En fullständig backup av konfigurationen tas automatiskt precis före start, vilket möjliggör en snabb rollback och jämförelse mot nätverkets standardkonfiguration (golden config) om problem skulle uppstå.*

3.1.8 Säker administration och åtkomst

Mer information finns i 11.6 Användarkonton och behörigheter samt 11.10 Säkerhet i boken Datorteknik.

Säker administration och åtkomst utgör fundamentet för att skydda nätverkets kontrollplan (control plane). Om en obehörig får tillgång till

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

administrativa rättigheter spelar övriga säkerhetsinställningar ingen roll, då hela infrastrukturen kan kompromettera inifrån. Grunden för en säker hantering börjar redan vid den första konfigurationen av en enhet genom att skapa ett dedikerat lokalt administratörskonto (local admin account) och omedelbart gå över till krypterad fjärråtkomst via **SSH** (Secure Shell). För att höja säkerheten ytterligare bör lösenordsbaserad inloggning fasas ut till förmån för nyckelautentisering (key authentication/SSH keys), vilket i princip eliminerar risken för brute force-attacker mot administrativa tjänster.

En bärande princip inom nätverksadministration är tillämpandet av minsta behörighet (least privilege). Det innebär att användare och administratörer endast ges de rättigheter som krävs för att utföra specifika uppgifter, och att administrativa konton aldrig används för vardagliga sysslor. I större miljöer separeras administrativ trafik helt från vanlig användartrafik genom ett dedikerat management-VLAN (management VLAN). För att nå detta VLAN från en extern plats eller från det vanliga användarnätet används ofta en **hoppserver** (jump host) eller en bastion (bastion host). Detta är en hårt härdad mellanserver placerad i ett säkert segment; administratören loggar först in på hoppservern, och därifrån etableras anslutningar till nätverksutrustningens hanteringsgränssnitt. Denna arkitektur säkerställer att ingen direktadministration kan ske från osäkra klientnätverk.

Utöver kontroll av åtkomstvägarna krävs stark autentisering (authentication). Detta innefattar unika administratörskonton (unique admin accounts) för varje individ, vilket skapar spårbarhet (traceability) i loggarna, samt implementering av **flerfaktorsautentisering** (MFA - Multi-Factor Authentication) där det är tekniskt möjligt. På nätverkets lägre nivåer, i lager 2 (Layer 2), måste infrastrukturen härdas för att förhindra fysiska angrepp och felkopplingar. Detta görs genom att låsa accessportarna med **portsäkerhet** (port security), aktivera **DHCP-snooping** och **Dynamic ARP Inspection (DAI)** för att säkra adresseringen, samt skydda nätverkets topologi med **BPDU Guard**. Genom att kombinera dessa skyddsmekanismer, som beskrevs mer ingående i kapitel 2.2 och 2.5, skapas en miljö där administrativa åtkomster är strikt kontrollerade och loggade.

Förtydligande (sammanfattning):

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Administrativ åtkomstväg:** Använd säkra anslutningar via VPN eller hoppserver (jump host) för att nå ett isolerat management-VLAN skyddat av brandväggsregler eller åtkomstlistor (ACL).
- **Identitet och behörighet:** Tillämpa unika administratörskonton, använd MFA och tilldela rättigheter baserat på principen om minsta behörighet (least privilege).
- **Lager 2-skydd (L2 hardening):** Säkra fysiska portar med portsäkerhet (port security), förhindra manipulerad adressering med snooping/inspection och skydda nätverkets logik med BPDU Guard och stormkontroll (storm control).

Exempel: *En administratör som arbetar hemifrån kopplar först upp sig via en säker VPN-tunnel till skolans nätverk. Därifrån loggar administratören in på en härdad bastion (jump host) placerad i ett isolerat management-VLAN. Först från denna server tillåts SSH-anslutningar vidare till switchar och brandväggar. Nätverket är konfigurerat så att inga andra IP-adresser än bastion-serverns har rätt att kommunicera med nätverksutrustningens administrativa gränssnitt.*

3.1.9 Fjärråtkomst och drift på distans

Mer information finns i 10 Dataöverföring och Fjärradministration i boken Datorteknik.

Säker fjärrdrift (remote operations) är en förutsättning för modern nätverksadministration, då det sällan är praktiskt eller möjligt att befinna sig fysiskt vid varje enhet. För att detta ska fungera krävs en genomtänkt arkitektur som erbjuder åtkomst på flera olika nivåer, från den underliggande hårdvaran till det grafiska operativsystemet. Valet av verktyg styrs av vilken typ av system som ska administreras och vilket djup av kontroll som krävs för stunden.

Fjärrinloggning sker i huvudsak på tre olika nivåer. På den mest grundläggande nivån, hårdvarunivån, används tekniker för **out-of-band management** (OOB). För servrar innebär detta användning av dedikerade styrkort som **CIMC** (Cisco Integrated Management Controller), **iLO** (Integrated Lights-Out) eller **iDRAC** (integrated Dell Remote Access Controller). Dessa fungerar som en "dator i datorn" med egen nätverksanslutning och strömförsörjning. Det gör det möjligt för en

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

administratör att fjärrstyra servern även om operativsystemet har kraschat, ändra inställningar i BIOS/UEFI eller utföra en hård omstart (hard reset) på distans.

På mjukvarunivå, direkt i operativsystemet eller i nätverksutrustningens konfigurationsläge, är **SSH** (Secure Shell) den absoluta standarden. SSH erbjuder en krypterad textbaserad terminalanslutning som är resurssnål och mycket säker, vilket gör den idealisk för administration av Linux-servrar, switchar och routrar. För system som kräver ett visuellt gränssnitt, framför allt Windows-baserade servrar och klienter, används den grafiska nivån via **RDP** (Remote Desktop Protocol). RDP strömmar skrivbordsmiljön till administratören och tillåter interaktion med mus och tangentbord som om man satt framför skärmen.

Oavsett nivå är säkerheten kring fjärråtkomsten kritisk. Inga administrativa portar (såsom SSH 22 eller RDP 3389) bör någonsin lämnas öppna direkt mot internet. Istället etableras en säker kanal via en **VPN-tunnel** (Virtual Private Network) med stark autentisering (authentication), ofta i kombination med en **hoppserver** (jump host). För kritiska platser, såsom centrala serverrum, bör man även planera för en alternativ väg in om den primära internetförbindelsen skulle gå ner. Detta löses ofta genom att ansluta hårdvarans management-portar till ett separat nätverk som nås via ett **LTE-modem** (4G/5G). På så sätt behåller administratören kontrollen över nätet även under ett omfattande fiberavbrott.

Förtydligande (sammanfattning):

- **Protokoll:** SSH för textbaserad administration och RDP för grafiska miljöer – båda ska alltid vara krypterade.
- **Kanaler:** All fjärråtkomst sker via VPN med stark autentisering (authentication); inga öppna admin-portar direkt mot internet.
- **Hårdvarustyrning (OOB):** Användning av CIMC, iLO eller iDRAC för åtkomst på hårdvarunivå, oberoende av operativsystemets status.
- **Alternativ väg (LTE):** Planera för utomstående kanaler (out-of-band) för att nå kritisk utrustning vid primära nätverksfel.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Felsökning i nivåer – Eskaleringssteg vid anslutningsproblem

När en tjänst eller server slutar svara bör en administratör följa en logisk eskaleringsstege (escalation path) för att identifiera var felet ligger. Genom att börja på den högsta nivån och arbeta sig neråt mot hårdvaran kan man snabbt avgöra om problemet ligger i applikationen, operativsystemet eller i den fysiska utrustningen.

1. **Nivå 1: Applikation och Grafik (RDP/VNC/Webb)**

   Du försöker nå serverns skrivbord eller webbtjänst. Om detta misslyckas men servern fortfarande svarar på **ping**, tyder det på att just den specifika tjänsten har stannat eller att grafikmotorn i operativsystemet är överbelastad.

2. **Nivå 2: Operativsystem och Terminal (SSH/PowerShell)**

   Om den grafiska inloggningen sviker, försöker du ansluta via en textbaserad terminal. SSH kräver betydligt mindre resurser än RDP. Om du kommer in här kan du starta om tjänster eller granska loggar. Om även SSH misslyckas (Connection refused/timeout), tyder det på att hela operativsystemet har hängt sig (kernel panic/blue screen).

3. **Nivå 3: Hårdvara och Out-of-Band (CIMC/iLO/iDRAC)**

   Detta är din "sista livlina". Om operativsystemet är helt dött loggar du in via serverns hanteringskort. Här ser du serverns faktiska skärmutgång (KVM) och kan se felmeddelanden som annars kräver en fysisk skärm i serverrummet.

Exempel på ett felsökningsflöde:

- **Problem:** Anslutningen till filservern går ner.
- **Steg 1:** Du försöker ansluta via **RDP** för att se vad som händer $\rightarrow$ **Misslyckas** (Timeout).
- **Steg 2:** Du försöker ansluta via **SSH** för att starta om tjänsten $\rightarrow$ **Misslyckas** (Inget svar).
- **Analys:** Serverns operativsystem har sannolikt hängt sig totalt.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Lösning:** Du loggar in via **CIMC**, ser en felkod på den virtuella skärmen och utför en hård omstart (Power Cycle).
- **Verifiering:** Du följer uppstarten via CIMC, testar att **SSH** svarar när OS:et laddat klart, och verifierar slutligen att **RDP** och filtjänsten fungerar igen.

Exempel: *Den primära fiberanslutningen till skolan går plötsligt ner. Tack vare ett installerat OOB-LTE-modem i serverrummet kan administratören ändå logga in via VPN till brandväggen. Väl inne kan administratören använda CIMC för att kontrollera serverstatus och omdirigera trafiken till en reservlänk utan att behöva åka till skolan fysiskt.*

3.1.10 Automatisering och skriptning

Mer information finns i 7.7.3 Bash-scriptning och 8.2.7 Linux automatisera säkerhetsuppgifter (cron, script, loggning) i boken Datorteknik.

I takt med att nätverk växer i storlek och komplexitet blir det omöjligt att hantera varje enskild enhet manuellt. Automatisering (automation) och skriptning (scripting) är verktygen som gör det möjligt för en administratör att hantera hundratals enheter lika enkelt som en enda. Genom att använda Bash-skript eller schemalagda uppgifter via **cron** i Linux kan repetitiva manuella uppgifter automatiseras, vilket minskar risken för den mänskliga faktorn (human error). Detta arbetssätt kallas ofta för **Infrastructure as Code** (IaC), där nätverkets tillstånd definieras i filer som sedan rullas ut automatiskt.

Grunden för automatisering ligger i att behärska skriptspråk som **Python**, **Bash** eller **PowerShell**. Utöver rena skript används ofta automatiseringsverktyg som **Ansible**, **Terraform** eller **Puppet**. Dessa verktyg tillåter administratören att skapa "spelböcker" (playbooks) som beskriver hur nätverket ska se ut. Istället för att logga in på 50 switchar för att skapa ett nytt VLAN, körs ett skript som pushar ut ändringen till alla berörda enheter samtidigt.

Förtydligande (sammanfattning):

- **Syfte:** Öka effektiviteten, minska konfigurationsfel och möjliggöra storskalig drift genom att gå från manuell till kodstyrd hantering.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Verktyg:** Python och Bash för generell automatisering; PowerShell för Windows-miljöer; Ansible eller Terraform för orkestrering (orchestration) av infrastruktur.
- **Metodik:** Infrastructure as Code (IaC) – lagra nätverkets konfiguration i läsbara filer som enkelt kan versionshanteras och återrullas.

Exempel: En skola ska rulla ut en ny säkerhetsinställning (t.ex. BPDU Guard) på 200 switchar. Istället för att logga in på varje enskild enhet, skriver administratören ett kort skript. Skriptet körs en gång, loggar in på samtliga switchar, applicerar inställningen och genererar en rapport över att allt lyckats. Det som tidigare tog dagar att genomföra manuellt är nu klart på några minuter.

3.1.11 Praktiska exempel på automatisering

En stor fördel med att använda verktyg som Ansible är konceptet **idempotens** (idempotency). Det innebär att om du kör skriptet en gång så genomförs ändringen, men om du kör det igen och inställningen redan finns, så gör skriptet ingenting. Detta gör automatiseringen mycket säkrare än enkla rad-för-rad-skript (scripting) som bara skickar kommandon blint.

Här är ett exempel på en enkel spelbok (playbook) som säkerställer att VLAN 10 (Personal) skapas på en grupp switchar.

YAML

```
---
- name: Konfigurera VLAN på skolans switchar
  hosts: access_switchar
  gather_facts: no
  tasks:
    - name: Skapa VLAN 10 med namnet Personal
      cisco.ios.ios_vlans:
        config:
          - name: Personal
            vlan_id: 10
        state: merged
```

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Inventariefiler och distribution

För att kunna distribuera (deploy) konfigurationen krävs även en **inventariefil** (inventory file). Detta är en lista som talar om för verktyget vilka enheter som tillhör gruppen `access_switchar`. I listan definieras IP-adresser och ofta vilka inloggningsuppgifter som ska användas för att ansluta via **SSH** (Secure Shell).

Ini, TOML

```
[access_switchar]
10.99.0.10
10.99.0.11
10.99.0.12
```

När filerna är klara distribueras konfigurationen från administratörens dator eller en **hoppserver** (jump host) genom ett enda kommando i terminalen:

```
ansible-playbook -i inventory.ini configure_vlan.yml
```

När kommandot körs etablerar Ansible en krypterad anslutning till varje switch i listan, kontrollerar om VLAN 10 redan finns och skapar det om det saknas. Administratören får sedan en tydlig rapport i terminalen som visar vilka enheter som ändrades (changed) och vilka som redan var korrekt konfigurerade (ok). Detta arbetssätt gör att nätverket blir homogent och att risken för att en switch glöms bort vid en stor förändring elimineras helt.

Bash-skript och SSH-loopar för enkel hantering

Medan **Ansible** är det professionella valet för storskalig drift, är **Bash-skript** (Bash scripting) utmärkt för att lära sig grunderna i hur man automatiserar enklare uppgifter. Här handlar det om att använda en loop (loop) för att utföra samma kommando på flera enheter efter varandra.

Ett vanligt scenario för en nätverkstekniker är att snabbt vilja hämta konfigurationen från flera switchar för att göra en backup, eller att skicka ut en enkel ändring till alla enheter i en viss korridor. Istället för att logga in på varje switch manuellt, skapar vi en textfil med alla IP-adresser och ett skript som läser filen rad för rad.

Först skapas en fil, till exempel `switchar.txt`, med de adresser som ska hanteras:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Plaintext

```
10.99.0.10
10.99.0.11
10.99.0.12
```

Därefter skriver vi skriptet `backup_switches.sh`. Skriptet använder en `for`-loop för att gå igenom listan och utföra ett kommando via **SSH**.

Bash

```
#!/bin/bash
# Enkelt skript för att hämta backup från switchar

# Definiera listan med IP-adresser
SWITCH_LIST="switchar.txt"

# Starta loopen som läser filen rad för rad
for IP in $(cat $SWITCH_LIST); do
    echo "--- Påbörjar backup av $IP ---"

    # Använd SSH för att köra kommandot och spara utdatan till en fil
    # 'admin' är användarnamnet på switchen
    ssh admin@$IP "show running-config" > backup_$IP.txt

    echo "Klart! Backup sparad som backup_$IP.txt"
done

echo "Samtliga backuper är genomförda."
```

Distribution och körning (Deployment)

Att distribuera (deploy) ett sådant här skript är enkelt eftersom det körs lokalt från din administratörsdator (exempelvis din Linux Mint-laptop) eller från en **hoppserver** (jump host). Innan skriptet körs behöver du se till att det har rättigheter att köras som ett program med kommandot `chmod +x backup_switches.sh`.

När du sedan kör skriptet med `./backup_switches.sh`, kommer din dator att försöka ansluta till varje switch i listan i tur och ordning. Om du har konfigurerat **SSH-nycklar** (SSH keys) mellan din dator och switcharna, kommer skriptet att köra helt automatiskt utan att fråga efter lösenord. Detta är en mycket kraftfull metod som visar hur grundläggande Linux-kunskaper (från kapitel 7 och 8 i din bok) direkt kan användas för att göra nätverksadministrationen mer effektiv och mindre felbenägen.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Förtydligande (sammanfattning):

- **Logik:** Använd en textfil för att lagra IP-adresser och en `for`-loop i Bash för att iterera genom dem.
- **Kommando:** Använd SSH för att skicka kommandon till switcharna och omdirigera utdatan (>) till filer på din egen dator.
- **Förutsättning:** Nyckelbaserad autentisering (key-based authentication) är nästan ett krav för att automatiseringen ska flyta smidigt utan manuella avbrott.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Förklara skillnaden mellan rollerna nätverksarkitekt (network architect) och nätverksdrift (network operations). Varför är det viktigt att skilja på dessa även i mindre organisationer?
2. Du misstänker att en Windows-klient har problem med sin gateway. Vilket kommando använder du i Windows för att se routingtabellen, och vad heter motsvarande kommando i Linux?
3. Varför är NTP (Network Time Protocol) en så kritisk del av en nätverksbaslinje (baseline) när det kommer till logganalys?
4. Beskriv begreppet split-horizon DNS. I vilket scenario är det fördelaktigt att använda denna teknik?
5. Vid felsökning enligt lager-för-lager-metodiken: Vilka steg kontrollerar du efter att den fysiska länken är bekräftad men innan du börjar titta på brandväggsregler?
6. Vad är den principiella skillnaden mellan vad SNMP talar om för dig jämfört med vad syslog berättar?
7. Inom ändringshantering (change management) talar man om en rollback-plan. Vad bör en sådan plan innehålla och när ska den upprättas?
8. Beskriv arkitekturen för en hoppserver (jump host). Vilket säkerhetsproblem löser den i samband med fjärradministration?
9. Du kan inte nå en server via RDP, men du kan logga in via CIMC/iLO. Vad drar du för slutsats om serverns tillstånd och vilka åtgärder kan du vidta via CIMC?
10. Vad innebär det att ett automatiseringsverktyg som Ansible är idempotent, och varför är det säkrare än att köra ett vanligt Bash-skript mot 100 switchar?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

Cisco: Guide to Network Management Design (Strategier för att bygga en stabil driftorganisation) – https://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Campus/campover.html

Ansible for Network Automation (Komma igång med att hantera nätverk som kod) – https://www.ansible.com/use-cases/network-automation

Microsoft Learn: Troubleshoot Network Connectivity (Systematisk felsökning i Windows-miljöer) – https://learn.microsoft.com/en-us/windows-server/networking/technologies/network-subsystem/net-sub-ts-guide

Red Hat: Introduction to mtr (Varför mtr är bättre än traceroute för nätverksdiagnostik) – https://www.redhat.com/sysadmin/mtr-network-diagnostics

NTP.org: Why time is important (Teknisk bakgrund till tidssynkroniseringens betydelse) – http://support.ntp.org/bin/view/Support/WhyIsTimeImportant

Cisco: Integrated Management Controller (CIMC) Guide (Fjärrstyrning på hårdvarunivå för M4/M5-servrar) – https://www.cisco.com/c/en/us/td/docs/unified_computing/ucs/c/sw/gui/config/guide/2-0/b_Cisco_UCS_C-series_GUI_Configuration_Guide_201.html

SANS Institute: Secure Jump Stack Architecture (Hur man bygger säkra bastion-lösningar) – https://www.sans.org/white-papers/36552/

Wireshark Foundation: Troubleshooting with Wireshark (Video-tutorials för djupgående paketanalys) – https://www.wireshark.org/docs/

ITIL Foundation: Change Management Process (Branschstandard för hur man hanterar ändringar i IT-miljöer) – https://www.axelos.com/certifications/itil-service-management

ISC2: The Principle of Least Privilege (Djupdykning i behörighetskontroll och säker administration) – https://www.isc2.org/Insights/2021/05/The-Principle-of-Least-Privilege

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.2 Verktyg: ping, traceroute, ipconfig, netstat etc.

Mer information om grundläggande kommandon och diagnostik finns i 7.7.2 Kommandon samt 11.8 Verktyg, övervakning och loggning i boken Datorteknik.

Att behärska diagnostikverktyg handlar om betydligt mer än att bara kunna skriva in rätt kommandon; det handlar om förmågan att tolka den information som strömmar tillbaka på skärmen och använda den för att metodiskt utesluta felkällor. Varje verktyg i en administratörs arsenal ger en unik pusselbit i nätverkets aktuella status, från den lägsta fysiska nivån till applikationslagret. Genom att kombinera resultaten från olika verktyg kan en tekniker snabbt verifiera hypoteser under felsökning: Är det kabeln som är trasig, routern som tappat vägen, eller är det en tjänst i operativsystemet som slutat svara?

I detta avsnitt fördjupar vi oss i hur dessa verktyg fungerar under ytan och hur du som administratör läser av de subtila ledtrådar som finns i deras utdata (output). Vi kommer att titta på allt från klassisk nåbarhetskontroll (reachability) och vägvalsanalys till granskning av lokala nätverksstackar och portar. Målet är att utveckla en intuitiv förståelse för nätverkets beteende så att du snabbare kan gå från symptom till lösning.

3.2.1 Varför dessa verktyg?

När en användare rapporterar att något ”inte fungerar” gäller det för administratören att snabbt och systematiskt avgöra exakt var i kedjan felet ligger. Det kan röra sig om allt från en fysisk nätlänk (network link) eller felaktig adressering (addressing) till problem med namnupplösning (name resolution), routing (routing) eller själva tjänsten och dess specifika port (port). De klassiska verktygen – såsom **ping**, **traceroute**/**tracert**, **ipconfig** (Windows) eller **ip** (Linux), **netstat**/**ss**/**nmap**, samt **nslookup**/**dig**, **arp**/**route** och vid behov **Wireshark**/**tcpdump** – låter dig testa kommunikationskedjan steg för steg. Grundtanken är att bekräfta de delar som fungerar för att kunna ringa in det exakta lager där det slutar fungera.

Genom att använda dessa verktyg i en logisk ordning undviker administratören att slösa tid på gissningar. Om man exempelvis kan verifiera att den lokala IP-konfigurationen är korrekt och att kontakten

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

med nätverkets första hopp (hop) fungerar, kan felsökningen snabbt flyttas vidare till högre lager som namnhantering eller applikationsspecifika inställningar. För mer komplexa problem krävs en djupanalys (deep analysis) av trafiken, vilket möjliggörs genom paketfångst (packet capture) där varje enskilt protokoll kan granskas i detalj för att hitta avvikelser.

Förtydligande (sammanfattning):

- **Länk och IP:** Kontrollera adressering med **ipconfig** eller **ip** och testa anslutningen till gateway med **ping**.
- **Namnupplösning:** Verifiera DNS-funktionen med **nslookup** eller **dig**.
- **Vägval (Routing):** Följ paketens väg genom nätverket med **tracert**, **traceroute** eller **mtr**.
- **Tjänst och port:** Granska anslutningar lokalt med **netstat** eller **ss**, och testa tillgänglighet för specifika tjänster med **Test-NetConnection** (Windows) eller **nc** (Linux).
- **Djupanalys:** Utför detaljerad granskning av trafikflöden med **Wireshark** eller **tcpdump**.

3.2.2 ping – når vi fram överhuvudtaget?

Kommandot **ping** är nätverksadministratörens mest använda verktyg för att snabbt verifiera grundläggande nåbarhet (reachability). Det bygger på kontrollprotokollet **ICMP** (Internet Control Message Protocol) och fungerar genom att skicka en förfrågan, en så kallad **echo request**, till en målenhet som i sin tur förväntas svara med en **echo reply**. Genom denna enkla mekanism kan administratören mäta tre kritiska faktorer: om målet är uppe, hur lång tid kommunikationen tar (latency/latens) och om några paket försvinner på vägen (packet loss/paketförlust). Vid en systematisk felsökning är det fackmässigt att börja "nära" genom att pinga den egna IP-adressen (för att kontrollera den lokala nätverksstacken), därefter sin standard-gateway (för att kontrollera den lokala länken) och slutligen en extern adress på internet.

Det är dock viktigt att komma ihåg att **ping** inte är ett ofelbart bevis på att en tjänst fungerar. Många moderna brandväggar och servrar är

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

konfigurerade att blockera eller ignorera ICMP-trafik av säkerhetsskäl, vilket innebär att en enhet kan framstå som "död" i ett ping-test trots att applikationer som webb eller e-post fungerar utmärkt. Ping bekräftar alltså att vägen är öppen för ICMP, men 100 % paketförlust behöver inte betyda att nätet är nere – det kan helt enkelt vara en säkerhetspolicy som stoppar just dessa kontrollpaket. Vid tolkning av resultaten tyder hög latens eller stora variationer i svarstid (**jitter**) oftast på överbelastning eller störningar i infrastrukturen, medan enstaka paketförluster kan indikera hårdvarufel eller dålig signalstyrka i trådlösa nät.

Förtydligande (sammanfattning):

> **Windows:** Använd `ping [IP/namn]` för fyra paket, `ping -t` för en kontinuerlig ström som hjälper vid felsökning av glappkontakt, eller `ping -4/-6` för att tvinga fram ett specifikt protokoll.
>
> **Linux:** Använd `ping -c [antal]` för att begränsa antalet paket (annars fortsätter det i oändlighet) eller `ping -I [gränssnitt]` för att testa nåbarhet från ett specifikt nätverkskort.
>
> **Tolkning:** Hög latens eller jitter tyder på nätproblem. Om 100 % av paketen går förlorade men tjänsterna i övrigt fungerar, är det sannolikt en brandvägg som blockerar ICMP.

Exempel: *Du utför ett test där* ***ping 10.20.0.1*** *(gateway) fungerar felfritt, men när du kör* ***ping 8.8.8.8*** *börjar du tappa paket. Slutsatsen av detta test är att felet sannolikt ligger "utåt", någonstans mellan din lokala gateway och internetleverantören, snarare än i det interna nätverket i byggnaden.*

Metodisk felsökning med ping – stegen utåt

Det första steget är att verifiera att den lokala nätverksstacken (network stack) i operativsystemet mår bra. Detta görs genom att pinga **loopback-adressen** `127.0.0.1`. Om detta misslyckas är felet mjukvarubaserat; det tyder på att drivrutiner eller nätverksprotokoll i själva operativsystemet är korrupta eller felkonfigurerade, och ingen trafik kommer kunna lämna datorn oavsett hur kablarna ser ut. När mjukvaran är bekräftad går man vidare till att pinga sin **egen IP-adress**. Detta testar kommunikationen med det fysiska nätverkskortet (NIC - Network Interface Card) och bekräftar att hårdvaran är vaken.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Efter att den egna hårdvaran bekräftats är nästa naturliga steg att testa kontakten med en **granne i samma VLAN**. Det innebär att du pingar en annan känd enhet, till exempel en kollegas dator eller en skrivare, som har en IP-adress inom samma subnät. Detta steg är kritiskt eftersom det helt utesluter routern ur ekvationen. Om du kan pinga en granne men inte din gateway, vet du med säkerhet att din **Lager 2-anslutning** (Layer 2 connectivity) fungerar och att felet specifikt rör kommunikationen mot routern eller en felaktig konfiguration av gateway-adressen. Om du däremot inte når din granne tyder det på ett lokalt problem i switchen, såsom en felaktig VLAN-taggning (VLAN tagging), problem med adresstjänsten (ARP - Address Resolution Protocol) eller att **portsäkerhet** (port security) har blockerat din anslutning.

När den lokala kommunikationen är säkerställd går man vidare till att pinga sin **standard-gateway** (default gateway). Detta testar den fysiska och logiska förbindelsen till nätverkets router. Därefter pingas en **publik IP-adress** ute på internet, till exempel `8.8.8.8`, för att bekräfta att din router lyckas skicka paket vidare genom brandväggen och ut till internetleverantören (ISP). Det sista steget i kedjan är att pinga ett **domännamn**, exempelvis `google.com`, vilket bekräftar att namnupplösningen (DNS - Domain Name System) fungerar korrekt.

Sammanfattning av felsökningsstegen:

- **127.0.0.1 (Loopback):** Kontrollerar nätverksprotokollen i operativsystemet.
- **Egen IP-adress:** Kontrollerar att nätverkskortet (NIC) svarar och är aktivt.
- **Granne i samma LAN/VLAN:** Kontrollerar lokal Lager 2-kommunikation och LAN/VLAN-tillhörighet utan inblandning av routern.
- **Standard-gateway:** Kontrollerar den logiska vägen ut ur det lokala delnätet.
- **Publik IP (t.ex. 8.8.8.8):** Kontrollerar internetanslutningen och routing ut ur huset.
- **Domännamn (t.ex. google.com):** Kontrollerar att namnupplösningen (DNS) fungerar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel: *En elev kan inte nå skolans lärplattform. Du pingar elevens gateway (OK), men när du testar att pinga elevens bänkgranne (som sitter i samma VLAN) misslyckas det. Genom detta enkla test har du ringat in att felet inte ligger hos internetleverantören eller i skolans centrala router, utan sannolikt i switchens konfiguration eller i den fysiska kabeldragningen mellan de två datorerna.*

3.2.3 traceroute / tracert – var tar trafiken vägen?

När vi med hjälp av ping har konstaterat att en destination inte går att nå, eller att svarstiderna är ovanligt höga, behöver vi veta exakt var på vägen problemet uppstår. Här använder vi verktyg för vägvalsanalys (path analysis). I Windows heter kommandot **tracert** och i Linux används **traceroute**. Dessa verktyg fungerar genom att skicka ut en serie paket med ett medvetet lågt värde på fältet **TTL** (Time to Live). Varje router (hop) som paketet passerar minskar TTL-värdet med ett. När värdet når noll kastas paketet, och routern skickar tillbaka ett kontrollmeddelande (ICMP Time Exceeded) till avsändaren. Genom att successivt öka TTL från 1 och uppåt kan administratören kartlägga varje enskilt hopp längs vägen till målet.

Det är viktigt att känna till de tekniska skillnaderna mellan plattformarna för att kunna tolka resultaten korrekt. Windows-versionen (**tracert**) använder som standard ICMP-paket för hela mätningen. Linux-versionen (**traceroute**) använder däremot ofta UDP-paket mot höga portnummer som standard. Detta kan leda till att resultaten skiljer sig åt om det finns brandväggar längs vägen som tillåter en typ av trafik men blockerar den andra. I Linux kan man vid behov tvinga verktyget att använda ICMP genom växeln `-I`. Om en rad i utdatan endast visar asterisker (`* * *`), betyder det att den specifika routern inte svarade i tid, vilket ofta beror på att administratören för den enheten har valt att prioritera ner eller helt blockera kontrolltrafik av säkerhetsskäl.

mtr – nätverksteknikerns realtidsvy

För en mer djupgående analys i Linux-miljöer är **mtr** (My Traceroute) det överlägsna valet. Till skillnad från den klassiska traceroute-funktionen, som endast ger en ögonblicksbild av vägen, kombinerar **mtr** funktionaliteten från både ping och traceroute i en levande realtidsvy. Verktyget fortsätter att skicka paket och samla in statistik så länge det

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

körs, vilket gör det möjligt att se trender över tid. Detta är ovärderligt för att identifiera periodiska paketförluster (intermittent packet loss) eller variationer i svarstid (jitter) på specifika hopp. Medan en vanlig traceroute kan se "ren" ut vid ett enstaka test, kan **mtr** avslöja att en router i mitten av kedjan faktiskt tappar 5 % av trafiken under belastning.

Förtydligande (sammanfattning):

- **Windows:** Använd `tracert -d [IP/namn]`. Växeln `-d` förhindrar att verktyget försöker göra DNS-uppslag på varje hopp, vilket gör att mätningen går betydligt snabbare.
- **Linux:** Använd `traceroute -I [IP/namn]` för att använda ICMP, eller kör **mtr** för att få en interaktiv vy med statistik över latens och paketförlust för varje hopp.
- **Tolkning:** Om privata IP-adresser (t.ex. 10.x.x.x) dyker upp mitt i en sökning mot internet tyder det på att trafiken passerar genom lokala nätverk, NAT-lösningar eller operatörers interna infrastruktur. Ett stopp mellan hopp N och N+1 indikerar en blockering eller ett fel på just den sträckan.

Exempel: *Du kör kommandot `tracert intranet.skolan.se` och ser att trafiken flyter fint genom de första tre hoppen, men stannar helt vid det fjärde hoppet som identifieras som skolans interna brandvägg. Slutsatsen blir att anslutningen fungerar fram till brandväggen, men att antingen en brandväggsregel blockerar trafiken eller att det saknas en korrekt rutt (route) vidare mot det skyddade nätverket (DMZ).*

3.2.4 ipconfig (Windows) / ip (Linux) – grunder och snabbrengöring

Mer information om nätverkskonfiguration i operativsystem finns i 7.7.2 Kommandon samt 11.2 Grundläggande arkitekturprinciper i boken Datorteknik.

För att förstå hur en specifik enhet ser på nätverket används verktyg som inspekterar det lokala gränssnittet (interface). Här ser administratören avgörande detaljer som aktuell IP-adressering, standard-gateway (default gateway), DNS-servrar, anslutningsspecifika suffix, DHCP-status och det fysiska länkläget (link state). I Windows-miljöer är det klassiska

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

kommandot **ipconfig** standard, medan Linux-system har gått över till det mer kraftfulla verktyget **ip** (från paketet iproute2), vilket ersätter det numera föråldrade `ifconfig`.

När en klient har problem med anslutningen är det första steget att kontrollera om gränssnittet överhuvudtaget har tilldelats en giltig IP-adress. Om en Windows-klient visar en adress i spannet `169.254.x.x` har den tilldelats en **APIPA**-adress (Automatic Private IP Addressing). Detta är ett tydligt tecken på att enheten inte har fått kontakt med en DHCP-server, trots att nätverkskortet och kabeln fungerar. I sådana lägen kan administratören utföra en "snabbrengöring" genom att tvinga fram en ny adressförfrågan eller rensa lokala cacher som kan innehålla gammal eller felaktig information.

I moderna nätverksmiljöer, särskilt sådana som använder systemd, hanteras namnupplösningen ofta av specifika tjänster snarare än bara en statisk fil. I Linux används då verktyget **resolvectl** för att granska och styra vilken DNS-server som faktiskt används för ett specifikt gränssnitt. Att kunna rensa DNS-cachens innehåll (**flush DNS**) är också en viktig del av felsökningsprocessen, då klienten annars kan fortsätta försöka ansluta till en gammal IP-adress trots att DNS-posten har uppdaterats centralt.

Syntax-exempel för Windows (PowerShell/CMD)

I Windows är `ipconfig` huvudverktyget för att interagera med nätverksstacken (network stack). För att se den fullständiga konfigurationen, inklusive fysiska adresser (MAC addresses) och DNS-servrar, används växeln för alla detaljer:

```
ipconfig /all
```

När en klient har fått en felaktig adress, eller om du nyligen har ändrat ett DHCP-scope i servern, kan du tvinga klienten att släppa sin nuvarande adress och begära en ny genom att köra dessa två kommandon i följd:

```
ipconfig /release
ipconfig /renew
```

För att hantera problem med namnupplösning (name resolution) där klienten minns gamla eller felaktiga IP-adresser, kan du först granska innehållet i den lokala cachen (cache) och sedan rensa den helt:

```
ipconfig /displaydns
```

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

```
ipconfig /flushdns
```

Syntax-exempel för Linux (Bash)

I Linux-miljöer används det moderna `ip`-kommandot för att hantera nästan all nätverkskonfiguration. Det är modulärt uppbyggt där du anger vilket objekt du vill inspektera, såsom adresser (addr), länkar (link) eller rutter (route). För att se alla IP-adresser på systemets gränssnitt (interfaces) används:

`ip addr` (eller kortversionen `ip a`)

Om du enbart vill se om de fysiska eller virtuella kablarna är anslutna, det vill säga länkhastighet (link speed) och status, används link-modulen:

```
ip link
```

För att se hur datorn skickar trafik vidare, alltså dess routingtabell (routing table) och vilken enhet som är standard-gateway (default gateway), skrivs följande:

```
ip route
```

I moderna distributioner som kör **systemd** (vilket är standard i exempelvis Ubuntu och Linux Mint) hanteras DNS-inställningarna ofta av en specifik tjänst. För att se vilka namnservrar (resolvers) som faktiskt används för varje specifikt nätverkskort används kommandot för upplösningskontroll:

```
resolvectl status
```

Genom att använda dessa kommandon direkt i din terminal kan du snabbt verifiera att de logiska inställningarna stämmer överens med din adressplan (address plan).

Förtydligande (sammanfattning):

- **Windows:** Använd `ipconfig /all` för att se all detaljerad information. Med `/release` följt av `/renew` begärs en ny adress från DHCP-servern. Kommandot `/flushdns` tömmer den lokala namn-cachen, medan `/displaydns` visar dess nuvarande innehåll.
- **Linux:** Använd `ip addr` för adresser, `ip link` för fysisk länkstatus och `ip route` för att se routingtabellen. För systemd-baserade

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

system används `resolvectl status` för att verifiera DNS-inställningar.

- **Snabbkoll:** Administratören kontrollerar systematiskt: Har gränssnittet (interface) en IP? Är det rätt VLAN? Stämmer gateway och DNS med nätverkets adressplan?

Exempel: *En klient i nätverket visar adressen `169.254.12.45` (APIPA). Detta bevisar att nätverkskortet är aktivt men att DHCP-förfrågan inte når fram eller inte besvaras. Som administratör börjar du då kontrollera om* ***DHCP relay*** *(ip-helper) är korrekt konfigurerad i routern och att switchens port tillhör rätt VLAN, så att klientens förfrågan faktiskt kan nå DHCP-servern.*

3.2.5 netstat / ss – lyssnar tjänsten, och vem pratar den med?

När du har bekräftat att IP-anslutningen och namnupplösningen fungerar, skiftas fokus till själva applikationslagret (application layer). Frågan är då: Har tjänsten startat korrekt och "lyssnar" den på rätt port? För att ta reda på detta används **netstat** i Windows och det modernare **ss** (Socket Statistics) i Linux. Dessa verktyg ger administratören en direkt insyn i nätverksstackens aktuella anslutningar och visar vilka portar som är öppna för inkommande trafik. Det är här du kan avgöra om ett problem beror på att en tjänst har kraschat lokalt eller om trafiken blockeras av en brandvägg (firewall) någonstans på vägen.

I Windows ger kommandot en överblick över alla aktiva anslutningar och lyssnande portar (listening ports). Genom att använda specifika växlar kan administratören se vilket process-ID (PID) som äger en viss anslutning, vilket gör det möjligt att spåra en misstänkt nätverksaktivitet till ett specifikt program. I Linux-världen har `ss` ersatt det gamla `netstat` eftersom det är avsevärt snabbare och kan visa mer detaljerad statistik om nätverksuttag (sockets). Om du ser att en tjänst har status **LISTEN** på servern, men en klient fortfarande inte kan ansluta, är det ett starkt bevis på att problemet sannolikt ligger i en brandvägg eller felaktig routing (routing) mellan enheterna, snarare än i själva applikationen.

Nmap – Nätverksteknikerns blick utifrån

Som ett komplement till de lokala verktygen används **nmap** (Network Mapper) för att skanna nätverket från utsidan. Medan `netstat` och `ss`

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

talar om vad som händer inuti servern, visar `nmap` vad en angripare eller en klient faktiskt kan se över nätverket. Det är ett oumbärligt verktyg för att verifiera att brandväggsregler och **NAT** (Network Address Translation) fungerar som tänkt. För administratörer som föredrar ett grafiskt gränssnitt (GUI) finns **Zenmap**, vilket är den officiella grafiska fronten för nmap och fungerar utmärkt på både Windows och Linux. Med dessa verktyg kan du utföra allt från enkla portskanningar till avancerad tjänsteidentifiering (service discovery) och operativsystemdetektering (OS detection).

Förtydligande (sammanfattning):

- **Windows:** Använd `netstat -ano` för att se alla portar och deras tillhörande PID. Med `netstat -anob` (kräver administratörsrättigheter) kan du även se namnet på den körbara filen direkt. För att matcha ett PID med ett programnamn används `tasklist /FI "PID eq [nummer]"`.
- **Linux:** Använd `ss -tulpen` för att se TCP/UDP-anslutningar, lyssnande portar, processnamn och användar-ID på ett mycket detaljerat sätt.
- **Nmap/Zenmap:** Används för att skanna enheter utifrån för att se vilka portar som faktiskt är nåbara över nätverket. Det bekräftar att brandväggar och portöppningar är korrekt konfigurerade.
- **Tolkning:** Om status är **LISTEN** lokalt på servern men anslutningen misslyckas utifrån, ligger felet i infrastrukturen (brandvägg/routing) mellan klienten och servern.

Exempel: *En webbserver svarar inte på anrop från internet trots att servern är igång. Du kör `netstat -ano` lokalt på servern och ser att tjänsten står som* ***LISTENING*** *på port 443. Eftersom tjänsten lyssnar korrekt drar du slutsatsen att felet troligen ligger i nätverkets brandvägg eller att en felaktig* ***DNAT*** *(Destination NAT) pekar trafiken mot fel intern IP-adress. Du verifierar detta genom att köra en `nmap`-skanning från en extern klient och ser att porten visas som* ***filtered*** *eller* ***closed****.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.2.6 nslookup / dig – namnen som allt hänger på

Mer information om felsökning av nätverkstjänster finns i 11.8 Verktyg, övervakning och loggning i boken Datorteknik.

Domännamnssystemet (DNS - Domain Name System) är den ryggrad som nästan alla moderna nätverksapplikationer vilar på. Utan fungerande namnupplösning (name resolution) stannar kommunikationen av, trots att den underliggande IP-anslutningen kan vara felfri. För en administratör räcker det inte med att veta *att* namnupplösningen fungerar; man måste kunna kontrollera *vilket* svar en specifik server ger och för *vilken* typ av resurs. Verktygen **nslookup** (tillgängligt i både Windows och Linux) och **dig** (Domain Information Groper, standard i Linux) är oumbärliga för att ställa direkta frågor till namnservrar och verifiera att zondata är korrekt publicerad.

En av de största fördelarna med dessa verktyg är möjligheten att frångå datorns standardinställningar och istället fråga en specifik namnserver (resolver) direkt. Detta är avgörande vid felsökning av interna zoner eller när man precis har migrerat en tjänst och vill se om ändringen har propagerat (propagated) till publika servrar som Google (8.8.8.8) eller Cloudflare (1.1.1.1). Administratören kan också specificera posttyper (record types) för att hämta mer än bara vanliga IPv4-adresser (**A-poster**). Det kan handla om IPv6-adresser (**AAAA**), e-postservrar (**MX**) eller de kritiska **SRV-posterna** som används av Active Directory och andra tjänster för att lokalisera domänkontrollanter och specifika nätverksresurser.

För djupare analys i Linux-miljöer erbjuder `dig` funktionen `+trace`, vilket gör att verktyget följer hela kedjan från rotzonen (root zone) ner till den auktoritativa namnservern (authoritative name server) för den specifika domänen. Detta avslöjar exakt var i hierarkin ett fel uppstår. Ett vanligt symptom vid drift av komplexa nätverk är att ett namn fungerar utmärkt internt men inte externt, eller tvärtom. Detta tyder ofta på en felaktig konfiguration av **split-horizon DNS** eller problem med vidarebefordrare (forwarders), där interna förfrågningar inte hittar rätt väg ut till det publika internet.

Förtydligande (sammanfattning):

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Snabbtest:** Verifiera ett internt namn mot en specifik server med `nslookup files.skolan.local 10.40.0.10` eller ett publikt namn i Linux med `dig A example.com @1.1.1.1`.
- **Fördjupning:** Kontrollera tjänsteposter i AD-miljö med `nslookup -type=SRV _ldap._tcp.skolan.local` eller följ hela uppslagsresan med `dig +trace example.com`.
- **Symptom:** Om namnupplösningen fungerar internt men misslyckas för externa domäner krävs en granskning av DNS-serverns forwarders eller brandväggens regler för port 53 (UDP/TCP).

Exempel: *Vid ett test körs `nslookup intranet.skolan.se` och svaret blir en publik IP-adress. Problemet är att användare som befinner sig inne på skolan behöver få den interna DMZ-adressen för att komma åt tjänsten. Denna diskrepans löses antingen genom att skapa en motsvarande intern A-post i skolans lokala DNS-zon (split-brain) eller genom att implementera* ***hairpin NAT*** *i brandväggen (se avsnitt 2.4), vilket tillåter interna klienter att nå den publika adressen via brandväggens insida.*

3.2.7 arp / route – lager-2-cache och standardvägar

Mer information om lagren i OSI-modellen finns i 9.2 Arkitektur – Lagren och PDU samt praktiska kommandon i 7.7.2 Kommandon i boken Datorteknik.

För att kommunikation ska fungera hela vägen från en applikation till den fysiska kabeln måste datorn kunna översätta logiska adresser till fysiska adresser och veta vilken väg paketen ska ta. Det är här **ARP** (Address Resolution Protocol) och **routingtabellen** (routing table) kommer in i bilden. Medan de verktyg vi tidigare gått igenom (ping, traceroute) testar anslutningen på avstånd, visar dessa verktyg hur datorns interna logik ser ut i detalj.

Inom IPv4 används **ARP** för att mappa en IP-adress till en fysisk **MAC-adress** (Media Access Control address) på det lokala nätverket. Utan ett fungerande ARP-uppslag kan datorn inte kapsla in sina paket i ramar (frames) på lager 2, vilket innebär att ingen trafik kan skickas även om IP-inställningarna är korrekta. I Windows används kommandot `arp -a` för att se den lokala cachen, medan Linux-administratören använder det moderna `ip neigh` (neighbor). För den moderna efterföljaren IPv6

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

används istället **Neighbor Discovery** (ND), vilket fyller samma funktion men är integrerat i ICMPv6.

När trafiken ska lämna det lokala delnätet (subnet) konsulterar datorn sin **routingtabell**. Denna tabell fungerar som en karta som talar om för operativsystemet vilket gränssnitt (interface) och vilken **standard-gateway** (default gateway) som ska användas för olika mål. Genom att granska tabellen med `route print` i Windows eller `ip route` i Linux kan en administratör upptäcka felaktiga rutter som skär av trafiken eller skapar onödiga omvägar. En särskilt viktig detalj är att kontrollera att det inte finns flera motstridiga standardvägar, vilket ofta händer om en laptop är ansluten till både trådburet nätverk (Ethernet) och trådlöst nätverk (WLAN) samtidigt.

Förtydligande (sammanfattning):

- **ARP/ND:** Om uppslag till gatewayens MAC-adress saknas i cachen tyder det på ett fel i det fysiska nätverket eller felaktig VLAN-konfiguration på switchen (Lager 2-fel).
- **Default-rutter:** Om tabellen visar dubbla default-rutter kan operativsystemet skicka trafik slumpmässigt mellan olika nätverkskort, vilket skapar asymmetrisk trafik (asymmetric routing) och instabila anslutningar.
- **Specifika rutter:** En felaktig statisk rutt med en hög noggrannhet (t.ex. en /32-mask) kan "stjäla" trafik ämnad för ett specifikt mål och skicka den till helt fel gateway.

Exempel: *Kommandot `route print` visar att en klient har två aktiva default gateways, en via WLAN och en via Ethernet. På grund av detta skickar klienten ibland trafik via det långsammare trådlösa nätet trots att kabeln är ansluten. Lösningen är att antingen stänga av det ena gränssnittet eller justera gränssnittets **metric** (viktning) så att det trådbundna nätverket alltid prioriteras av operativsystemet.*

3.2.8 Wireshark / tcpdump – när du behöver se paketen

Mer information om paketanalys och diagnostik finns i 11.8 Verktyg, övervakning och loggning samt i 10.2.2 HTTPS – Den krypterade efterföljaren i boken Datorteknik.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

När de grundläggande verktygen för nåbarhet och namnupplösning inte räcker till för att identifiera ett problem, krävs en djupare insyn i nätverkstrafiken. Detta görs genom paketfångst (packet capture/sniffing), där administratören spelar in och granskar de faktiska datapaketen som skickas över nätverkskortet. För detta ändamål är **Wireshark** den ledande lösningen med sitt kraftfulla grafiska gränssnitt (GUI), vilket gör det enkelt att visualisera protokollhierarkier och följa trafikflöden. I miljöer där ett grafiskt gränssnitt saknas, såsom på servrar, brandväggar eller vid fjärradministration via SSH, används standardverktyget **tcpdump**. Det är ett textbaserat terminalverktyg (CLI) som är extremt resurssnålt och effektivt för att snabbt fånga trafik direkt vid källan.

För att undvika att drunkna i informationsmängden är det avgörande att fånga trafik så nära felet som möjligt och använda snäva filter (filtering). Genom att definiera specifika kriterier för värddator (host), port (port) och protokoll (protocol) kan administratören filtrera bort bakgrundsbrus (noise) och fokusera på den relevanta kommunikationen. Vid analys av nätverkstrafik är det fackmässigt att först kontrollera att den grundläggande transporten fungerar korrekt genom att studera trevägshandskakningen (three-way handshake). Om analysen visar ett utgående **SYN**-paket som aldrig besvaras med ett **SYN/ACK**, tyder det på att trafiken antingen blockeras av en brandvägg eller att det inte finns någon aktiv tjänst som lyssnar på målporten. Om handskakningen däremot fullbordas men applikationen ändå felar, ligger problemet högre upp i applikationslagret (application layer).

Vid arbete med paketfångst är det av yttersta vikt att beakta sekretess och personlig integritet (privacy/confidentiality). Eftersom dessa verktyg kan fånga känslig information i klartext, såsom användarnamn eller innehåll i okrypterade protokoll, bör de endast användas i felsökningssyfte och på ett sätt som följer verksamhetens säkerhetspolicy. I moderna krypterade miljöer (HTTPS/TLS) kan innehållet i paketen vara dolt, men administratören kan fortfarande utläsa värdefull metadata om certifikat, krypteringsalgoritmer (ciphers) och anslutningsförsök som ofta är nyckeln till att lösa komplexa anslutningsproblem.

Förtydligande (sammanfattning):

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **tcpdump:** Används i terminalen med syntax som `tcpdump -i eth0 host 10.40.0.50 and port 443` för att fånga trafik på ett specifikt gränssnitt.
- **Wireshark:** Används för grafisk analys med filteruttryck som `ip.addr == 10.40.0.50 && tcp.port == 443`.
- **Tolkning:** En SYN utan efterföljande SYN/ACK indikerar blockering eller saknad lyssnare. En komplett handskakning följt av fel tyder på problem i applikationslagret eller med krypteringen.

Exempel: *En klient kan nå en servers port 443, men vid inloggningsförsök stannar processen och "snurrar" oavbrutet. En paketfångst genomförs och visar att trevägshandskakningen lyckas, men att anslutningen därefter bryts på grund av ett* ***TLS-fel*** *(Handshake Failure). Vidare analys visar att klienten försöker använda en för gammal eller osäker kryptering (cipher) som serverns härdade säkerhetspolicy inte längre tillåter. Åtgärden blir att uppdatera klientens mjukvara eller justera serverns TLS-policy för att återigen tillåta säkra anslutningar mellan parterna.*

3.2.9 Ett snabbt felsökningsrecept (kedjan från klient till tjänst)

Mer information om metodik finns i 3.1.5 Felsökningsmetodik (lager för lager) samt i 11.8 Verktyg, övervakning och loggning i boken Datorteknik.

Felsökning i ett nätverk handlar om att eliminera felkällor metodiskt genom att röra sig från det lokala och kända utåt mot det fjärranslutna och okända. En erfaren administratör ser felsökningen som en kedja av beroenden; om den första länken brister spelar det ingen roll hur väl resten är konfigurerat. Genom att följa ett fastställt ”recept” kan du snabbt ringa in om problemet ligger i klientens mjukvara, nätverkets infrastruktur eller i den slutgiltiga tjänsten. Att bygga en vana att dokumentera varje steg – vilket kommando som kördes och vad resultatet blev – gör dessutom återkopplingen till användare och kollegor rak och professionell.

Felsökningskedjan: Från länk till applikation

Den logiska ordningen för att härleda ett fel bör alltid följa denna hierarki:

1. **Länk & IP:** Har vi en fysisk eller trådlös anslutning och en giltig adress?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2. **Gateway:** Når vi nätverkets utgång och därmed vår lokala infrastruktur?
3. **Extern IP:** Har vi en väg ut till internet (utan att blanda in namnupplösning)?
4. **Namn (DNS):** Kan vi översätta mänskliga namn till IP-adresser?
5. **Väg (Routing):** Tar paketen rätt väg genom nätverket utan att fastna?
6. **Port & Tjänst:** Svarar applikationen på den specifika porten?
7. **Applikation:** Fungerar logiken inuti tjänsten (t.ex. inloggning)?

Genomförande i Windows-miljö

I en Windows-miljö börjar du med att öppna en terminal (PowerShell eller CMD). Du kontrollerar först din adressering med ipconfig /all för att se att du inte har en APIPA-adress ($169.254.x.x$). Därefter bekräftar du kontakten med din router genom ping [gateway-ip]. För att utesluta DNS-problem pingar du en publik adress som 8.8.8.8. Om det fungerar men ping google.com misslyckas, använder du nslookup för att granska DNS-svaren. För att kontrollera om en specifik tjänst, till exempel en webbserver, är nåbar på port 443 använder du det kraftfulla PowerShell-kommandot Test-NetConnection -ComputerName [server] -Port 443. Detta bekräftar att brandväggar tillåter trafiken och att tjänsten lyssnar.

Genomförande i Linux-miljö

I Linux använder du motsvarande verktyg men med en något annorlunda syntax. Du inleder med ip addr för att verifiera ditt gränssnitt (interface) och IP-adress. Kontroll av gateway och internet sker med ping, precis som i Windows. För att granska namnupplösningen är dig det föredragna verktyget eftersom det ger en mer detaljerad teknisk utdata än nslookup. Om du misstänker problem längs vägen kör du mtr [mål-ip] för en levande vy över varje hopp. För att testa om en port är öppen på en fjärrserver kan du använda nc -zv [server] [port] (netcat), vilket snabbt bekräftar om anslutningen (connection) kan etableras. Om allt annat ser bra ut men tjänsten ändå krånglar, kan curl -v [url] användas för att se exakt vad som händer i HTTP-kommunikationen.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Förtydligande (sammanfattning):

- **ipconfig/ip:** Kontrollera om vi har en IP-adress samt rätt VLAN, gateway och DNS-inställningar.
- **ping gateway:** Verifiera lokal nåbarhet och att kabeln/WLAN fungerar.
- **ping extern IP:** Testa internetanslutningen utan beroende av DNS.
- **nslookup/dig:** Verifiera att namnupplösningen fungerar korrekt.
- **tracert/traceroute/mtr:** Identifiera var i nätverket trafiken stannar eller går omvägar.
- **netstat/ss:** Kontrollera lokalt på servern om tjänsten lyssnar på rätt portar.
- **Wireshark/tcpdump:** Används vid behov för djupanalys av protokoll och felaktiga handskakningar.

Exempel: *En elev kan inte nå filservern files.skolan.local. Du följer receptet:*

1. *ipconfig visar att eleven har en korrekt IP-adress (**OK**).*
2. *ping mot gateway ger svar (**OK**).*
3. *ping 10.40.0.50 (filserverns IP) ger svar (**OK**).*
4. *nslookup files.skolan.local returnerar en helt annan IP-adress än den förväntade.*

   Slutsats: *Problemet är varken nätet eller servern, utan en felaktig post i den interna DNS-servern. Genom att uppdatera DNS-posten löser du problemet direkt utan att behöva ”röra nätet” eller starta om någon hårdvara.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kontrollfrågor

1. Du utför en ping mot en server som du vet är igång och som kör en fungerande webbtjänst, men du får 100 % paketförlust (packet loss). Vad är den mest troliga orsaken till detta beteende?
2. Förklara hur fältet TTL (Time to Live) används tekniskt av verktyg som tracert och traceroute för att identifiera varje hopp (hop) längs en nätverksväg.
3. Vilken specifik fördel erbjuder verktyget mtr jämfört med en vanlig traceroute när man felsöker instabila nätverksförbindelser (intermittent connectivity)?
4. Om kommandot ipconfig visar adressen 169.254.10.12, vad berättar det för dig om klientens kontakt med nätverkets DHCP-server?
5. Varför är det ofta nödvändigt att köra kommandot ipconfig /flushdns efter att man har ändrat en A-post i en DNS-server?
6. Du misstänker att en webbserver inte har startat korrekt. Vilket kommando och vilka växlar använder du i Linux för att se om servern faktiskt lyssnar (listening) på port 443?
7. Beskriv skillnaden i perspektiv mellan att använda netstat lokalt på en server och att använda nmap från en extern klient.
8. Hur använder du nslookup eller dig för att kontrollera en specifik DNS-post mot en extern namnserver (resolver), till exempel 1.1.1.1, istället för att använda datorns standardinställningar?
9. Om en klients ARP-tabell (arp -a / ip neigh) saknar en post för standard-gatewayen (default gateway), vilka slutsatser kan du dra om anslutningen på Lager 2 (Layer 2)?
10. Vid analys i Wireshark ser du att en klient skickar ett SYN-paket men aldrig får något SYN/ACK tillbaka från servern. Ge två exempel på vad som kan orsaka detta avbrott i trevägshandskakningen (three-way handshake).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Fördjupningslänkar

Cloudflare: What is ICMP? (Genomgång av protokollet som driver ping och traceroute) – https://www.cloudflare.com/learning/network-layer/what-is-icmp/

PacketLife: Traceroute Internals (Detaljerad visualisering av hur TTL-manipulering fungerar) – https://packetlife.net/blog/2008/dec/22/how-traceroute-works/

BitWizard: mtr manual (Officiell dokumentation och avancerade växlar för mtr) – https://www.bitwizard.nl/mtr/

Microsoft Learn: ipconfig command reference (Samtliga växlar och funktioner för Windows nätverkskonfiguration) – https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/ipconfig

Red Hat: Guide to the ip command (Modern nätverkshantering i Linux med iproute2) – https://www.redhat.com/sysadmin/linux-ip-command

Nmap.org: Nmap Network Scanning (Den officiella boken och guiden till nmap och portskanning) – https://nmap.org/book/man.html

DigitalOcean: Troubleshooting DNS with Dig (Praktiska exempel på hur man diagnostiserar DNS-problem i terminalen) – https://www.digitalocean.com/community/tutorials/how-to-use-dig-to-query-dns-servers-on-ubuntu-20-04

Wireshark: Filter Reference (Komplett lista över filteruttryck för att hitta specifik trafik) – https://www.wireshark.org/docs/dfref/

IETF: RFC 792 Internet Control Message Protocol (Den tekniska specifikationen för ICMP) – https://datatracker.ietf.org/doc/html/rfc792

Cisco: Troubleshooting TCP/IP (Omfattande guide för metodisk felsökning av nätverksstacken) – https://www.cisco.com/c/en/us/support/docs/ip/transmission-control-protocol-tcp/10044-8.html

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Egna anteckningar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.3 Dokumentation, etikett, felsökning

3.3.1 Varför dokumentation?

Dokumentation gör nätet **förutsägbart, överlämningsbart och felsökningsbart**. Utan uppdaterade kartor, IP-/VLAN-planer, portlistor och ändringsloggar blir varje fel en gissningslek och varje uppgradering en chansning. Bra dokumentation sänker MTTR (Mean Time To Repair), minskar dubbelarbete och gör att nya i teamet snabbt blir produktiva. Den skapar också **spårbarhet** – du kan visa vad som ändrades, när och varför, vilket är ovärderligt vid incidenter och revisioner.

Exempel: *En skrivare slutar fungera efter ett switchbyte. Tack vare aktuell portmatris ser du direkt att skrivaren satt i VLAN 20 men nya porten är default VLAN 1. En rad ändring, klart.*

3.3.2 Vad ska dokumenteras?

Minsta gemensamma nämnare är: **topologidiagram**, **IP-/VLAN-plan**, **enhetsinventarie**, **konfigbackup**, **portmatriser**, **brandväggspolicys** och **ändringslogg**. Lägg till **runbooks** (steg-för-steg för vanliga åtgärder) och **driftjournal** (vad som hände, när, varför). Tänk “**en sanning**”: en central plats där allt hålls uppdaterat, med tydlig ägare per dokument och versionsmärkning så att historiken går att följa.

Förtydligande (sammanfattning):

- *Topologi:* core/distribution/access, uplinks, trunkar, redundans.
- *IP/VLAN:* prefix, gateways, DHCP-scopes, DNS/NTP.
- *Enheter:* modell, serienummer, plats, management-IP, version.
- *Portmatris:* vägguttag ↔ patchpanel ↔ switchport ↔ VLAN.
- *Policys:* översikt över tillåtna flöden mellan zoner.
- *Ändringslogg:* datum, syfte, risk, utförare, rollback, resultat.

Exempel: *En enkel runbook “Lägga till nytt VLAN” innehåller: skapa VLAN, SVI + DHCP-scope, tillåta på relevanta trunkar, uppdatera brandvägg, dokumentera i IP-plan och portmatris, validera med testklient.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.3.3 Namnstandard, märkning och port-etikett

Konsekventa namn gör att du “läser” nätet som en karta. Namnge enheter med **plats–roll–löpnummer** (t.ex. *HUS1-DIST-A*, *HUS1-ACC-03*). Märk **vägguttag**, **patchpanel**, **switchport** och **fiber** på båda sidor. Etikett i nätet handlar också om hur du lämnar efter dig: **återställ patchning**, **ta bort tillfälliga labbgrejer**, och **spara konfig**. En tydlig namngivning sparar tid i alla led – från beställning och felsökning till städning efter projekt.

Förtydligande (sammanfattning):

- *Namn:* kort, meningsfullt, konsekvent.
- *Portdisciplin:* dokumentera innan du drar om; inga “dolda” hopkopplingar.
- *Märkning:* tydliga etiketter + matchande i dokumentationen.

Exempel: *Port A3 i HUS2-ACC-04 märks “Klassrum 214 – skrivare” och loggas i portmatrisen. Vid fel ser du portens VLAN på två sekunder.*

3.3.4 Ärenden och kommunikation – etikett i drift

Felsökning sker inte i vakuum. Ha en **ärendeprocess**: ta emot felanmälan, bekräfta mottagning, sätt prioritet, återkoppla vid milstolpar och stäng ärendet med **kort RCA** (root cause analysis). Var **transparent**: skriv vad du mätt och ändrat. Var **blameless** i analys – fokusera på system och process, inte personer. God kommunikation reducerar frustration, skapar förtroende och gör att alla vet vad som händer – även när lösningen dröjer.

Förtydligande (sammanfattning):

- *Prioritet:* påverkan + brådska → P1–P4.
- *Status:* uppdatera när något ändras – även “vi undersöker” hjälper.
- *RCA light:* orsak, åtgärd, hur vi undviker återkomst.

Exempel: *“P2: Elevnät i B-huset påverkat. Åtgärd: trunk bar inte VLAN 20. Fix: tillåta VLAN 20 på uplink. Förebygg: mall för trunkar + checklista vid switchbyte.”*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.3.5 Change management – planera, testa, backa

Ändringar ska vara **förutsägbara**. Använd **underhållsfönster**, skriv en kort **risk- och effektbedömning**, och ha en **rollback-plan**. Testa i labb eller på en **pilot** först. Efter ändringen: mät och dokumentera utfallet. En enkel checklista minskar risken för "små missar" som ger stora effekter, och gör att samma kvalitet kan upprepas gång på gång.

Förtydligande (sammanfattning):

- *Plan:* syfte, omfattning, risk, testplan, rollback.
- *Tid:* skoltid vs icke-skoltid – minimera påverkan.
- *Efterkontroll:* mät latens/throughput, kontrollera loggar/alarms.

Exempel: *Firmware på distributionsswitchar: pilot i HUS3, sedan rullas samma version ut övriga hus med samma checklista och rollback.*

3.3.6 Felsökningsmetodik – från symptom till rotorsak

Börja med **scope** (en enhet eller många? ett VLAN eller alla?), definiera **symptom** och **förväntat beteende**, och jobba **lager för lager** (se 3.1 och 3.2). Verifiera med **mätbara tester** i varje steg och **ändra inte flera saker samtidigt**. Skriv ned *vad du testade och vad resultatet blev* – det blir dokumentation i realtid och hjälper nästa person som tar över ärendet.

Förtydligande (sammanfattning):

- *Scope → Hypotes → Test → Resultat → Nästa steg.*
- *Isolera:* byt variabel (kabel, port, VLAN) en i taget.
- *Bevis:* spara kommandon/utdata i ärendet.

Exempel: *"Ingen IP i sal 215." Test:* `show int` *OK, port i VLAN 20 OK, men klienter får APIPA.* `show ip dhcp snooping` *visar block – uplink ej trusted. Åtgärd: trust uplink, verifiera* `ipconfig /renew`*.*

3.3.7 Säkerhetsetikett i vardagen

På delade nät (skola!) gäller "**gör ingen skada**". Starta inte egna **DHCP/DNS** på klientnät, scanna inte nät blint, och kör inte osignerade

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

verktyg på produktionsservrar. Paketfångst kan innehålla känsliga data – **minimera, filtrera, radera**. Admin-gränssnitt nås via **jump host/bastion** i **management-VLAN**, inte direkt från klientnät. *(En* **jump host/bastion** *är en hårt härdad mellanserver i ett separat, säkert nät; du loggar in där först och därifrån når du management-IP — ingen direktadministration från användarnätet.)* Etiketten handlar också om att lämna spår: skriv vad du gjorde och varför, så att nästa person kan förstå förändringen.

Förtydligande (sammanfattning):

- *Rogue-tjänster:* förbjudet på elev-/personalnät.
- *Fångst:* minst möjligt, lagra säkert, rensa efteråt.
- *Åtkomst:* VPN/jump host, MFA, minst behörighet.

Exempel: *Ett "mystiskt" DHCP-scope dök upp. Med DHCP Snooping spårar du källporten och hittar en elevs hemmarouter – porten stängs och incidenten loggas.*

3.3.8 Mallar och checklistor

Standardisera vanliga jobb med **mallar** (VLAN-skapande, SSID-sättning, ny switch i access, nytt subnät, ny server-VM). Lägg **checklistor** i samma dokumentkällor som konfigmallarna så att de hålls uppdaterade samtidigt. När checklistan samtidigt är en **valideringslista** (mätpunkt per steg) blir kvaliteten jämn och lätt att revidera.

Förtydligande (sammanfattning):

- *En källa:* wiki/git för mallar + checklistor.
- *Versioner:* märk med datum/version och ägare.
- *"Definition of done"*: vad ska vara sant när du är klar (mät/validera).

Exempel: *Checklistan "Ny accesswitch" inkluderar: namn, mgmt-IP, NTP/syslog, SSH/AAA, portprofiler, trunk-VLAN, STP-root-prioritet, dokumentation, validering.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.3.9 Mätetal för driftkvalitet

Välj enkla **SLO:er** (service-nivåmål) som går att mäta: **uptime** per zon, **latens** mellan hus, **WLAN-airtime** vid toppar, **DHCP-svarstid**, **DNS-hit-rate**, antal **P1-incidenter** per termin och genomsnittlig **MTTR**. Visualisera i en dashboard så att trender upptäcks tidigt och att förbättringar kan följas upp med fakta – inte magkänsla.

Förtydligande (sammanfattning):

- *Få, relevanta tal:* hellre 5 som används än 30 som ignoreras.
- *Visualisera:* dashboard – grönt/gult/rött.
- *Lärdomar:* varje P1 → en förbättring i design, process eller dokumentation.

Exempel: *Efter återkommande P2 p.g.a. “saknar IP” införs krav: DHCP-relay + Snooping-kontroll i checklista och auto-test i övervakning.*

3.3.10 Ett mini-bibliotek (vad och var)

Ha en **enkelsida** som länkar till allt: topologi, IP/VLAN, portmatriser, policies, runbooks, mallar, ändringslogg och kontaktlista. Då vet alla **var** man börjar – även när det brinner. En “start-här”-yta minskar startsträcka för nya i teamet och sparar tid i akuta lägen.

En bra startsida är värd sin vikt i guld under incidenter.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Nämn tre konkreta sätt som dokumentation sänker MTTR.
2. Vad ska minst ingå i en uppdaterad IP-/VLAN-plan och varför?
3. Ge ett exempel på en bra namngivningskonvention för nätverksenheter.
4. Hur definierar du prioritet (P1–P4) i ärendehantering och varför är det viktigt?
5. Vad bör en change-plan innehålla utöver själva åtgärden?
6. Beskriv en lager-för-lager-metod för att hitta rotorsaken till “ingen IP i sal 215”.
7. Vilka etiska regler bör gälla för paketfångst i skolmiljö och varför?
8. Varför bör admin-åtkomst gå via jump host/bastion i management-VLAN?
9. Ge tre mätetal (SLO:er) som ger tidig varning om kvalitetsproblem i nätet.
10. Vad ska finnas på en “start-här”-sida för nätverksdrift?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- Google SRE – Incident Management & Postmortems: https://sre.google/sre-book/table-of-contents/
- NIST SP 800-61 r2 – Computer Security Incident Handling Guide: https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final
- Cisco – Network Troubleshooting Best Practices (översiktssidor): https://www.cisco.com/c/en/us/support/docs.html
- NetBox (IPAM/DCIM) – dokumentation & inventarie: https://docs.netbox.dev/
- diagrams.net (draw.io) – topologidiagram: https://www.diagrams.net/
- Grafana – dashboards & larm: https://grafana.com/docs/
- Wireshark – User Guide: https://www.wireshark.org/docs/wsug_html_chunked/
- The Practice of System and Network Administration (bokresurser): https://www.sage.org/ (resurser/vidare läsning)

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.4 Introduktion till CLI (inkl. Cisco-kommandon)

3.4.1 Vad är CLI och varför använda det?

CLI (Command Line Interface) är textbaserad administration där du skriver kommandon istället för att klicka i ett GUI. För nätverk ger CLI **precision**, **hastighet** och **reproducerbarhet**: du ser exakt vad som konfigureras, kan scripta uppgifter, och får samma resultat varje gång. I felsökning är CLI överlägset eftersom du kommer åt detaljer (tabeller, räkneverk, debug) som ofta saknas i ett GUI.

CLI är också "språket" som förenar olika fabrikat: koncepten liknar varandra även om kommandona skiljer sig.

Exempel: *Du hittar en duplex-mismatch genom att köra* `show interfaces` *och läsa felräknare; i GUI:n syntes bara "länk up".*

3.4.2 Åtkomst till nätutrustning: console, SSH och sessionshygien

Du når switchar/routrar via **console** (seriell kabel lokalt) eller **SSH** (krypterad fjärranslutning). **Telnet** undviks (okrypterat). I skolmiljö används ofta en terminalklient (t.ex. Windows Terminal, PuTTY) och säkra **jump host/bastion** för att nå management-nätet först.

Förtydligande:

- *Console:* räddar dig när nätet ligger nere; kräver fysisk närvaro.
- *SSH v2:* standard för fjärr; stäng av telnet (`transport input ssh`).
- *Sessionshygien:* sätt `exec-timeout`, `logging synchronous`, och jobba från management-VLAN.

Exempel: *En felkonfigurerad trunk släpper inte in dig via nätet. Du kopplar konsolkabel, rättar VLAN-listan och återfår fjärråtkomst via SSH.*

3.4.3 Cisco IOS – lägen och hjälp

Cisco-CLI har tydliga **lägen**:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **user EXEC** (>): läs-kommandon.
- **privileged EXEC** (#): `enable` för fler kommandon.
- **global config** (`(config)#`): `configure terminal`.
- **under-lägen:** t.ex. `interface`, `line`, `router`, `vlan`.

Navigering och hjälp:

- `?` visar tillgängliga kommandon i kontext.
- `tab` auto-kompletterar; delord + `?` visar alternativ.
- Prefixet `no` negaterar (tar bort) en inställning.
- `do <show-kommando>` funkar inne i config-läge.

Exempel: *Du skriver* `spanning-tree bpdug?` *och ser* `bpduguard` *som val – snabb påminnelse utan att lämna läget.*

3.4.4 Visa, spara och jämföra konfiguration

Det viktigaste "visa-kommandot" är `show running-config`. Spara ändringar med `copy running-config startup-config` (eller `write memory`). Använd filter för att hitta rätt rader snabbt.

Förtydligande:

- *Visa:* `show run | section interface`, `| include`, `| exclude`, `| begin`.
- *Spara:* `copy run start`.
- *Jämför/återställ:* `show archive` (om aktiverat), `reload in 5` som "fallskärm" inför riskfyllda ändringar.

Exempel: *Du sätter* `reload in 10` *innan du ändrar en fjärrtrunk. Om du tappar sessionen rullar enheten automatiskt tillbaka om du inte* `reload cancel`*:ar.*

3.4.5 "Show" du använder varje dag (snabbdiagnos)

De här kommandona ringar in fel på minuter:

Förtydligande:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- *Länk & portar:* `show interfaces status`, `show interfaces` (felräknare, duplex/hastighet), `show power inline` (PoE).
- *Lager 2:* `show vlan brief`, `show mac address-table`, `show spanning-tree`, `show storm-control`.
- *Grannar:* `show cdp neighbors detail` och/eller `show lldp neighbors detail`.
- *L3 & ARP:* `show ip interface brief`, `show ip route`, `show arp`.
- *WLAN/AP via controller:* kanaler, SNR, klienter (vendor-specifikt).

Exempel: *Klienten når inte DHCP.* `show vlan brief` *visar att porten ligger i VLAN 1 istället för VLAN 20. Justera portprofilen och testa igen.*

3.4.6 Grundläggande lager-2-konfiguration (switch)

Det här är kärnan för access- och uplink-portar.

Förtydligande:

- *Beskriv & profil:*
  ```
  interface gi1/0/12
  description Klassrum214-Skrivare
  switchport mode access
  switchport access vlan 20
  spanning-tree portfast
  spanning-tree bpduguard enable
  ```
- *Trunk:*
  ```
  interface gi1/0/48
  switchport mode trunk
  switchport trunk allowed vlan 10,20,30,40
  switchport trunk native vlan 999
  ```
  (oanvänt)
- *VLAN:* `vlan 20` → `name ELEV`

Exempel: *En elev kopplar in en "smart-switch" som skickar BPDUs. Med* `bpduguard enable` *err-disablas porten direkt och nätet skyddas.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.4.7 Grundläggande lager-3-konfiguration (SVI och routing)

På L3-switchar skapar du SVI:n som gateways och slår på routing. På routrar konfigurerar du adresser per gränssnitt.

Förtydligande:

- *SVI & routing:*
  ```
  interface vlan 20
  ip address 10.20.0.1 255.255.252.0
  no shutdown
  ip routing
  ```
- *Statik:* `ip route 0.0.0.0 0.0.0.0 10.0.0.1`
- *OSPF exempel:*
  ```
  router ospf 1
  network 10.20.0.0 0.0.3.255 area 0
  ```

Exempel: *Efter att ha lagt till SVI för VLAN 40 kan elever nå servrar först när* `ip routing` *aktiveras. Utan den stannar trafiken på L2.*

3.4.8 Härda åtkomst och management

Säkra accessen innan du släpper lös nätet.

Förtydligande:

- *SSH v2 & användare:*
  ```
  ip domain-name skolan.local
  crypto key generate rsa modulus 2048
  username admin secret <långt-lösenord>
  line vty 0 4 → transport input ssh → login local → exec-timeout 10
  ```
- *Banners:* `banner login` för juridisk varning.
- *Syslog/NTP:* `service timestamps log datetime msec` + skicka loggar; ställ NTP.
- *AAA/RADIUS (översikt):* central auth (beskrivs mer i senare kapitel).

Exempel: *Efter att ha stängt Telnet med* `transport input ssh` *och lagt* `login local` *syns inloggningar i syslog med korrekta tidsstämplar.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.4.9 Effektivitet i CLI (små knep som sparar timmar)

Små vanor gör stor skillnad.

Förtydligande:

- *Historik & redigering:* piltangenter, `ctrl-a/e` (början/slut), `ctrl-u` (rensa).
- *Rörledning:* `show run | section interface` → kopiera bara det du behöver.
- *Interface-range:* `interface range gi1/0/1-24` för massändringar.
- *Alias:* `alias exec svi show ip interface brief | include Vlan` (snabbvy).
- *"Do show" i config:* slipp hoppa mellan lägen.

Exempel: *Du behöver slå på* `portfast` *på 24 portar. Med* `interface range` *gör du det på 10 sekunder istället för 10 minuter.*

3.4.10 Vanliga fallgropar och hur du undviker dem

Många incidenter beror på små missar.

Förtydligande:

- *Glömd "write"*: ändringar överlever inte om du inte sparar.
- *Fel VLAN på accessport:* ser "länk up" men klienten hamnar i fel nät.
- *Ofullständig trunk:* VLAN saknas i `allowed vlan`.
- *Looprisk:* portfast saknas på klientportar → lång konvergens.
- *Telnet kvar:* okrypterade lösenord på nätet.

Exempel: *"Allt funkar – tills omstart." Klassikern:* `copy run start` *glömdes. Inför rutin: spara och verifiera varje change.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. När är Console nödvändigt och när räcker SSH?
2. Vilka lägen finns i Cisco-CLI och vad gör `no`-prefixet?
3. Hur använder du `?`, `tab` och `do` för att arbeta snabbare och säkrare?
4. Vad gör `show interfaces` respektive `show vlan brief` och när använder du dem?
5. Ge tre steg för att felsöka en port som “ser upp” men där klienten inte får IP.
6. Skriv de fyra raderna som gör en port till säker accessport för klient: VLAN, portfast, bpduguard, description.
7. Hur aktiverar du L3-routing på en switch och konfigurerar en SVI-gateway?
8. Vilka inställningar krävs för att säkra fjärråtkomst (SSH v2, login, timeout, timestamps)?
9. När använder du `reload in` och varför är det smart vid fjärrändringar?
10. Nämn tre vanliga fallgropar i trunk-konfigurationer och hur du upptäcker dem.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- Cisco – Introduction to the Cisco IOS CLI (guide): https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/fundamentals/configuration/15-s/fund-15-s-book/cf_cli.html
- Cisco – Basic Switch Configuration: https://www.cisco.com/c/en/us/support/docs/lan-switching/catalyst-2960-x-series-switches/118763-config-2960x-00.html
- Cisco – Using the `show` and filtering pipelines: https://www.cisco.com/c/en/us/support/docs/ios-nx-os-software/ios-operating-system-ios/7133-logging.html
- OpenSSH – Secure Shell best practices: https://www.openssh.com/manual.html
- Microsoft – Windows Terminal & OpenSSH Client: https://learn.microsoft.com/windows/terminal/
- man7 – Linux `ip` och `ss` referens (iproute2): https://man7.org/linux/man-pages/man8/ip.8.html
- Wireshark – User Guide (för djup felsökning): https://www.wireshark.org/docs/wsug_html_chunked/

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.5 Windows Server – Domäner och AD

3.5.1 Överblick: vad är en domän och Active Directory (AD DS)?

En **domän** är en överenskommelse om identiteter och regler: vem du är, vad du får, och hur det bevisas på ett sätt som alla datorer i domänen litar på. **Active Directory Domain Services (AD DS)** är katalogen som håller ihop detta – ett “telefonregister med hjärna” som lagrar konton, grupper, datorer, skrivare och relationer mellan dem. När en användare loggar in händer tre saker i snabb följd: klienten **hittar** en domänkontrollant (via DNS), **bevisar** vem användaren är (Kerberos/NTLM), och **hämtar** regler och resurser som hör till identiteten (GPO, gruppmedlemskap, rättigheter). I större miljöer delas katalogen in i flera **domäner** under samma **skog** (forest), där skogens gemensamma språk – **schemat** – gör att alla förstår samma objekttyper. För en skola betyder detta att inloggning och datorupplevelse kan vara lika i alla salar, samtidigt som administration kan **delegeras** lokalt utan att ge bort “superkrafter”.

Förtydligande (sammanfattning):

- *Domän:* en gemensam säkerhetsgräns och policyyta.
- *Skog:* en eller flera domäner som delar schema och kan lita på varandra.
- *Domänkontrollant (DC):* server som autentiserar och replikerar katalogen.

Exempel: En elev får samma skrivare och samma hemkatalog oavsett dator. Det är inte “turen” som avgör, utan domänens identitets- och policylogik.

3.5.2 Grundkomponenter: DC, FSMO-roller, tid och platser (Sites)

En domän behöver **flera domänkontrollanter** av samma skäl som en stad behöver fler än ett sjukhus: driftstopp ska inte bli en kris. Vissa uppgifter är “ensamrättsroller” – **FSMO** – för att undvika kollisioner (t.ex. schemaändringar, tilldelning av SID-delar). Den mest synliga i vardagen är

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

PDC Emulatorn, som fungerar som tidsreferens och där många lösenordsrelaterade uppgifter prioriteras. **Tid** låter trivialt, men i Kerberos är den en säkerhetsparameter: ligger klockorna fel upphör biljetterna att gälla. I ett nät som täcker flera byggnader hjälper **Sites and Services** AD att förstå geografin: klienter i Hus A ska i första hand använda “sin” DC, och replikering mellan hus kan ske med rimliga intervall.

Förtydligande (sammanfattning):

- *Minst två DC:* för robusthet och underhåll.
- *FSMO:* särskilda ansvar som inte mår bra av att vara “flera samtidigt”.
- *Tidssynk:* en kedja från PDC Emulator → övriga DC → klienter.
- *Sites:* kopplar subnät till “närmaste” DC och styr replikering.

Exempel: När en ny elev byter lösenord på morgonen ska samma lösenord gälla i alla hus efter minuter, inte timmar. Det är replikeringstopologin som gör det möjligt.

3.5.3 DNS-integrering – AD:s nervsystem

AD lutar sig mot **DNS** i nästan varje steg. När en klient vill logga in måste den hitta en DC genom att slå upp särskilda **SRV-poster** i den interna zonen. Därför lagras zonen ofta **AD-integrerat**: den följer med katalogens replikering och accepterar **säkra, dynamiska** uppdateringar från domänanslutna maskiner. Det gör att en dator automatiskt kan registrera sig med rätt namn och att klienter spontant hittar rätt väg även när adresser ändras. Fel i den här kopplingen maskerar sig gärna som “AD-problem”, men roten är ofta att någon dator råkar fråga **fel DNS** (t.ex. en publik resolver som inte känner till interna SRV-poster).

Förtydligande (sammanfattning):

- *SRV-poster:* “här finns LDAP/Kerberos för den här domänen”.
- *AD-integrerad zon:* säker dynamik + samma replikering som katalogen.
- *Klientupplevelse:* rätt DNS på klienten är en förutsättning för “AD funkar”.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel: En dator kan surfa men "hittar inte domänen". Förklaringen är ofta att DNS pekar på internet istället för skolans egna resolvers – då syns aldrig AD:s SRV-poster.

3.5.4 OU-design och delegation

OU:er (Organizational Units) är katalogens ordning och reda. De fungerar både som **mappar** för att gruppera liknande objekt och som **gränser** där man kan fästa policyer och ansvar. En bra design speglar **driften**: datorer för elever, personal och servrar i skilda grenar; underindelning per hus eller sal när det hjälper arbetet; och tydliga platser där **delegering** sker utan att släppa in någon på fel nivå. Principen **LSDOU** (Local → Site → Domain → OU) berättar i vilken ordning policyer tolkas; den sista som passar vinner – med undantag för några specialflaggor. Poängen är att kunna uttrycka "hur ska det vara här?" utan att behöva uppfinna regler på varje enskild dator.

Förtydligande (sammanfattning):

- *Struktur efter drift:* vem äger vad, vem behöver ändra vad, var gäller vilka regler?
- *Placera objekt rätt:* lämna standardcontainrar, lägg i OU.
- *Delegation:* ge "lagom" makt nära där jobbet sker.

Exempel: En datasal kan få hårdare skrivbordspolicyer än lärarrummet, även om samma elev loggar in på båda. Det avgörs av datorns OU och hur policyerna är kopplade.

3.5.5 GPO – hur policyn "blir verklighet"

Group Policy är verktygslådan där **datorinställningar** och **användarinställningar** bor. När en användare loggar in "läggs" deras egna regler ovanpå datorns regler enligt **LSDOU**. Ibland vill man tvärtom: att **datorns plats** bestämmer även användarupplevelsen – då används **loopback processing** (t.ex. i datasalar). **Security Filtering** och ibland **WMI-filter** gör att en policy träffar rätt grupp eller rätt sorts dator, och **SYSVOL** bär runt själva innehållet så alla DC:er kan dela på jobbet. Allt detta är osynligt när det fungerar, men tydligt när det inte gör det: GPO är den tysta kraften bakom "samma upplevelse varje dag".

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Förtydligande (sammanfattning):

- *Två sidor:* dator- vs användarpolicys.
- *Ordning:* Local → Site → Domain → OU (senast relevant “vinner”).
- *Riktning:* filtrera på grupp eller egenskap; loopback när rummet ska styra.

Exempel: Om alla i datasalen ska se samma startmeny oavsett vem de är, låter man datorernas OU tala högst via loopback. Då följer upplevelsen rummet, inte personen.

3.5.6 Konton, grupper och rättigheter (AGDLP, UPN, lösenpolicy, gMSA)

Identitet i AD blir begriplig först när **konton** och **grupper** hålls isär i tanken. Kontot beskriver *en person eller en tjänst*; gruppen beskriver *en roll*. Med **UPN**-formatet (namn@domän) känns inloggning igen från andra tjänster. **AGDLP** är bara ett sätt att säga att vi **först** samlar människor i passande **globala grupper**, **sedan** kopplar rollerna till **resurser** via **Domain Local-grupper** som bär behörigheten. För tjänster är **gMSA** en tryggare variant än lokala konton med “hemliga” lösenord – domänen sköter rotationen. Tillsammans ger detta ett språk där förändringar i verkligheten (en elev byter klass, en lärare byter roll) speglas av **gruppmedlemskap**, inte av specialfall på resurserna.

Förtydligande (sammanfattning):

- *Kontot = individ/tjänst, gruppen = roll.*
- *AGDLP:* skär isär “vem” och “var” så att behörigheter blir underhållbara.
- *gMSA:* minskar riskerna med tjänstkonton och stillastående lösenord.

Exempel: När en elev byter program byts inte “filservern” – bara gruppen hen tillhör. Därför följer rättigheter och skrivare med på köpet.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.5.7 Användarrättigheter i OS vs rättigheter till filer och mappar

Det är lätt att blanda ihop två olika sorters rättigheter: **User Rights Assignment** i operativsystemet handlar om *vad ett konto får göra mot datorn* (logga in, köra som tjänst, ta ägarskap), medan **NTFS- och share-rättigheter** handlar om *vad kontot får göra mot data*. Den första påverkar inloggning och tjänster, den andra påverkar mappar och filer. I praktiken leder tydlig **gruppbaserad** hantering till färre överraskningar: datorn vet vilka som får logga in; filservern vet vilka som får läsa och skriva – och de två världarna korsar bara varandra där de måste.

Förtydligande (sammanfattning):

- *User Rights:* OS-beteenden (t.ex. “Log on as a service”).
- *NTFS/Share:* dataåtkomst; effektiv rätt = snittet av båda.
- *Gruppbaserat:* stabilare än individuella undantag.

Exempel: En tjänst vägrar starta “trots admin”. Felet är ofta att kontot saknar själva **rätten** *att köras som tjänst, vilket inte är samma sak som filrättigheter.*

3.5.8 DHCP i domänmiljö (autorisering, redundans, namn)

DHCP i en domän är inte bara “adressutdelning”, utan också **tillit** och **namn**. Servern behöver vara **auktoriserad** i katalogen för att få dela ut adresser – ett skydd mot oväntade källor. Två servrar kan arbeta tillsammans så att utdelningen fortsätter även under patchar och fel. Och eftersom namnet betyder mycket i en AD-miljö är det vanligt att låta DHCP hjälpa till med **dynamiska DNS-uppdateringar**, så att “vem bor var?”-frågan alltid har ett rimligt svar. Allt detta är små detaljer var för sig, men tillsammans gör de att klassrum “bara fungerar” när många maskiner kommer och går varje dag.

Förtydligande (sammanfattning):

- *Autorisering:* domänen godkänner DHCP som “talar för den”.
- *Redundans:* två servrar delar ansvar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- *DNS-uppdatering:* adresser får namn och namn får adresser utan handpåläggning.

Exempel: I elevnätet byts adresser ofta. När DHCP och DNS pratar med varandra slipper du spökinlägg – namnen pekar dit datorerna faktiskt finns just nu.

3.5.9 Fildelning – struktur, NTFS vs share, ABE, VSS och GPP

En filserver är inte bara en hårddisk på nätet; den är en **scenografi** för hur människor arbetar. Genom att skilja på **share-rättigheter** (grov grind vid ingången) och **NTFS-rättigheter** (finmaskig styrning inne i huset) går det att uttrycka “alla som kommer in i den här korridoren får läsa – men bara vissa får öppna klassrumsdörrarna”. **Access-Based Enumeration (ABE)** döljer dörrar du ändå inte får öppna, vilket gör upplevelsen lugnare och säkrare. **Skuggkopior (VSS)** låter användare backa tiden själva när någonting råkade bli fel, och **Group Policy Preferences** gör att rätt diskar dyker upp där de behövs, kopplade till roller istället för datorbilder. Med den här kompositionen blir filservern begriplig: du ser det du ska se, och det du gör går att ångra.

Förtydligande (sammanfattning):

- *Share = grov nivå, NTFS = fin nivå.*
- *ABE minskar “kan inte öppna”-support.*
- *VSS och GPP ger mindre friktion i vardagen.*

Exempel: En elev ser bara sin klassmapp och sin hemkatalog. Läraren ser dessutom ämnesmappar. Blir något fel i gårdagens dokument finns “Tidigare versioner” ett högerklick bort.

3.5.10 Standardmönster för behörighet: AGDLP i praktiken

Det är viktigt att tänka på att **AGDLP** används konsekvent eftersom det separerar *vem* som behöver åtkomst från *var* åtkomsten faktiskt ges. I praktiken innebär det att individuella **konton** samlas i **globala grupper** (per roll, klass eller funktion) för att visa *vilka* personer som hör ihop. Dessa globala grupper läggs i sin tur in i **Domain Local**-grupper som är knutna till *resurserna* (t.ex. en fildelning eller en applikation) och som **bär själva behörigheten**. Poängen är att förändringar i personal, klasser

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

eller roller kan göras genom att bara flytta medlemmar mellan **globala grupper** – utan att behöva röra de känsliga behörighetslistorna på resurserna. På det sättet minskar risken för misstag, och helpdesk kan lösa de flesta åtkomstärenden snabbt och spårbart.

Namnstandarden blir också en del av läsbarheten. Om **Domain Local**-gruppen berättar *vilken resurs och nivå* den representerar (t.ex. `DL_Share_Personal_Modify`) och den **globala** gruppen berättar *vilken roll* den samlar (t.ex. `G_Larare` eller `G_23TE`) blir behörighetstabellen nästan självdokumenterande. Att dessutom ange en **ägare** för varje DL-grupp gör att det alltid är tydligt vem som får besluta om medlemskap – något som är viktigt i en skolmiljö där klasser byts och personal tillkommer över tid. Om miljön växer till flera domäner i samma skog kan **Universal**-grupper användas som ett mellansteg, men grundidén är densamma: separera människorna från resurserna och låt grupperna vara "kopplingsdosor" mellan dem.

Exempel: När en ny lärare börjar läggs kontot i `G_Larare`*. Den gruppen är redan medlem i* `DL_Share_Personal_Modify`*, som i sin tur har NTFS-rättigheten* **Modify** *på* `\\srv-fil-01\Personal`*. Läraren får åtkomst utan att någon behöver öppna NTFS-dialogen — och när läraren slutar räcker det att ta bort kontot ur* `G_Larare`*.*

3.5.11 Felsökning: var börjar jag, och varför just där?

När något "AD-relaterat" strular är **namn** och **tid** de två första frågorna. Utan rätt DNS hittar klienten inte sin DC, och utan rimlig tid accepterar Kerberos inte biljetten. Därifrån följer två spår: **replikering** (delar DC:erna samma bild av världen?) och **policytillämpning** (träffar policyn rätt objekt, på rätt plats?). Genom att hålla sig till den ordningen blir felsökningen mindre magi och mer metod: vi slår fast "vad fungerar", ringar in "var slutar det fungera", och låter detaljerna leda oss vidare.

Förtydligande (sammanfattning):

- *DNS först, tid sen:* de är fundamenten.
- *Replikering:* en sanning i katalogen, inte många halvsanningar.
- *Policy:* rätt plats, rätt riktning, rätt villkor – annars händer "inget".

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel: En dator i datasalen får inte “datasalsupplevelsen”. Felet visar sig vara att den flyttats till fel OU – policyn missar helt enkelt målet.

3.5.12 Rekommenderade bas-GPO:er (varför just de?)

Det är bra att etablera ett litet **grundgolv av policyer** som alltid finns, eftersom det ger förutsägbarhet i undervisningen och minskar antalet incidenter. **Windows Update** med en enkel “ring”-modell (test → pilot → brett) hjälper till att få ut säkerhetsfixar utan att riskera att en hel datasal tappar funktion samma dag; några maskiner får först, resten när det visat sig stabilt. **Windows-brandväggen** med korrekta profiler (domän/privat/offentlig) säkerställer att samma dator beter sig olika i skolnät och hemma — öppet där det måste vara öppet, stängt i övrigt. **Tid/NTP** styrs centralt för att Kerberos ska fungera; en liten tidsdrift räcker för att inloggningar och biljetter ska börja fallera.

Windows LAPS motiveras av att lokala administratörslösenord inte ska återanvändas mellan datorer; varje maskin får ett unikt, roterande lösenord som går att läsa vid behov, vilket minskar risken för lateral förflyttning. Att **stänga Telnet och andra legacy-protokoll** handlar om att ta bort onödiga angreppsytor – moderna, krypterade protokoll räcker. **RDP** bör vara tillåtet **endast** från management-nät eller via bastion, för att inte exponera administrativa gränssnitt i elev- och personalnät. **Audit-policyer** (inloggningar, policyändringar, åtkomstförsök) gör det möjligt att efterhand förstå “vad som hände” när något går fel eller när en incident måste följas upp.

Till det kommer **Group Policy Preferences** för skrivare och enhetskopplingar (P:, S: osv.). Poängen är att kopplingar då följer **grupper** snarare än enskilda datorbilder – när en elev byter klass följer skrivare och mappar med utan manuell handpåläggning. En **standardiserad startmeny** i datasalar gör att lektioner börjar likadant oavsett vilken dator man sätter sig vid; både elever och lärare sparar tid när viktiga program ligger på samma plats. Slutligen är det klokt att **versionera** policyer och **dokumentera** var de är länkade, för att korta vägen från “något ändrades” till “nu vet vi vad”.

Exempel: En ny datasal får samma upplevelse som de andra eftersom “Datasal-Golv” redan innehåller brandväggsprofiler, LAPS, RDP-

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

begränsning, audit och standardmeny. Skrivare och nätverksenheter kopplas in via GPP baserat på grupper, så lärarna slipper jaga genvägar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Vad betyder det att en domän är en *säkerhetsgräns*, och hur märks det i vardagen?
2. Varför är **tid** en säkerhetsfråga i en AD-miljö, och vilken roll spelar PDC Emulatorn?
3. Hur hänger **DNS** ihop med inloggningen, och vilka symptom får du när klienten frågar “fel” resolver?
4. På vilket sätt hjälper **OU:er** både policys och delegation, och varför bör standardcontainrar undvikas?
5. Förklara **LSDOU** och ge ett exempel där **loopback** är nyckeln till rätt upplevelse.
6. Vad vinner du på att tänka **konton = individer/tjänster** och **grupper = roller**?
7. Varför är **AGDLP** lättare att underhålla än individuella rättigheter direkt på resurser?
8. Vad skiljer **User Rights** i OS från **NTFS-/share-rättigheter**, och varför är den skillnaden viktig?
9. Hur bidrar **DHCP-autorisation** och **dynamiska DNS-uppdateringar** till att domäninloggning “bara funkar”?
10. Vad gör **ABE**, **VSS** och **GPP** för skillnad i en filservermiljö för skola?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- Microsoft Learn – Active Directory Domain Services (översikt): https://learn.microsoft.com/windows-server/identity/ad-ds/get-started/active-directory-domain-services-overview
- DNS i AD (SRV, AD-integrerade zoner, dynamiska uppdateringar): https://learn.microsoft.com/windows-server/identity/ad-ds/plan/active-directory-integrated-dns
- Group Policy – grunder, loopback, filtrering: https://learn.microsoft.com/windows/security/threat-protection/security-policy-settings/
- DHCP Server – auktorisering, failover, DDNS: https://learn.microsoft.com/windows-server/networking/technologies/dhcp/dhcp-top
- File and Storage Services – NTFS, SMB, ABE, FSRM, VSS: https://learn.microsoft.com/windows-server/storage/file-server/file-server
- Windows LAPS: https://learn.microsoft.com/windows-server/identity/laps/laps-overview
- AD-replikering och DC-hälsa (repadmin, dcdiag): https://learn.microsoft.com/troubleshoot/windows-server/identity/active-directory-replication-troubleshooting

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.6 Ubuntu Server och Linux i nätverk

3.6.1 Grundläggande nätverkskonfiguration i Linux

På Linux finns nätverket i flera “lager” samtidigt: **kärnan** håller reda på interface, adresser och tabeller; **användarverktyg** (som `ip`) pratar med kärnan i realtid; **tjänster** (som DNS-resolvern och tidssynk) påverkar hur namn och klockor upplevs; och **beständig konfiguration** (på Ubuntu via **Netplan**) beskriver hur allt ska se ut efter en omstart. Det låter abstrakt, men poängen är enkel: man skiljer på *tillstånd just nu* och *tillstånd över tid*. I undervisning och drift blir det därför naturligt att “läsa” nätet med verktyg som visar nuläget, samtidigt som man förstår var den beständiga sanningen faktiskt bor.

På **Ubuntu Server** beskriver **Netplan** nätet i YAML-filer (vanligen `/etc/netplan/*.yaml`) och låter sedan en “motor” – ofta **systemd-networkd** på servrar – göra jobbet vid boot. Det gör att sådant som **statisk IP**, **gateway**, **namnservrar**, **VLAN**, **bonding** och **bryggor** (bridges) lever i samma modell, och att alla delar kan uttryckas lika tydligt för IPv4 och IPv6. Nuläget speglas av moderna kommandon: `ip address` och `ip route` visar adresser och rutter, `resolvectl` visar hur namn faktiskt slås upp, och `networkctl` ger en översikt över kända länkar och deras status. **VLAN** på Linux är inte magiskt: ett taggat underinterface (t.ex. `enp3s0.20`) *är* bara samma fysiska port med en tydlig etikett för nätet; **bonding** binder flera fysiska portar till en logisk (för redundans/kapacitet), och **bryggor** kopplar virtuella gäster (container/VM) till “riktiga” nät. I praktiken liknar det hur du tänker i en switch: *namn* på saker betyder mindre än *hur de kopplas*.

DNS-funktionen på moderna Ubuntu sköts ofta av **systemd-resolved**, som både cache:ar och vet vilken namnserver som gäller på vilket interface. Det är här **split-horizon** och “rätt resolver på rätt plats” blir konkret: interna namn ska fråga interna resolvers, och externa namn får gå utåt. Samma logik gäller **tidssynk** – **systemd-timesyncd** eller NTP-klient – där en liten avvikelse får stora följder om servern ingår i en domän som använder **Kerberos**. I servermiljö är det dessutom vanligt att separera **management-trafik** (administration) från **tjänstetrafik** (det som servrarna levererar) till olika VLAN och att styra åtkomst med

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

brandvägg (Ubuntu har **UFW** som ett vänligt gränssnitt till underliggande paketfilter). Det gör inte servern “paranoid”; det gör den **förutsägbar**.

Förtydligande (sammanfattning):

- *Två perspektiv:* **nuläge** (kärna + verktyg) och **beständighet** (Netplan).
- *Byggstenar:* VLAN för etiketter, **bond** för redundans/kapacitet, **bridge** för att koppla gäster till rätt nät.
- *Tjänster:* **resolved** för namn, **timesyncd/NTP** för tid – båda styr “allt annat”.
- *Säker drift:* separera **management** från **tjänstetrafik**, låt brandvägg och riktade regler göra sitt.

Exempel: En Ubuntu-filserver svarar långsamt på domäninloggningar men “internet” är snabbt. Förklaringen är sällan CPU — det är oftare att servern råkar fråga **fel DNS** *(extern resolver utan SRV-poster) eller har en liten* **tidsdrift***. När namn och tid blir rätt, blir allt annat “magiskt” rätt också.*

3.6.2 SFTP, SMB, SSH, SCP

I Linuxvärlden gör man en poäng av att **välja rätt verktyg för rätt typ av överföring och åtkomst**. **SSH** är grunden: ett säkert skal som ger fjärrterminal, **tunnel** (port forward) och **subsystem** som **SFTP**. Det viktiga för elever att förstå är att **SFTP inte är “FTP med S”**, utan en logisk filkanal *inuti* SSH – vilket betyder att *exakt samma identitet och kryptering* som skyddar terminalen också skyddar filen. **SCP** är historiskt det snabbaste sättet att “bara få över en fil”, men i dag föredrar många **SFTP** (för robusthet och interoperabilitet) eller **rsync över SSH** (för diff/återupptagning och rättigheter). Poängen är inte att minnas alla flaggor, utan att kunna resonera: *är detta en engångskopia, en synk som ska köras nattligen, eller ett incheckat arbetsflöde?*

SMB (Samba) adresserar en annan situation: **samarbete med Windowsvärlden**. När en Linuxserver ska dela mappar till Windows-klienter är Samba språktolken som gör att **NTFS-liknande rättigheter** (inklusive **ACL**) och **Windows-inloggning** känns hemma. I en skolmiljö är det vanligt att låta Samba **läsa identiteter** från en domän – oavsett om

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

servern formellt “går med i” domänen eller bara konsumerar gruppinformation. Det som ser enkelt ut (“mappar till elever och lärare”) blir stabilt först när identiteter, grupper och rättigheter **hänger ihop** över protokollgränsen. Därför är det bra att elever kopplar **SMB-upplevelsen** till **gruppbaserad behörighet** (se 3.5): på Linuxsidan **ägare/grupper/ACL**, på Windows-sidan **roller** – och Samba översätter.

Tillbaka till **SSH**: utöver terminal och SFTP är SSH också ett **transportlager** för andra behov. Med en **local forward** (t.ex. `localhost:8443` → “serverns 443”) går det att administrera webbtjänster utan att öppna dem externt, och med **ProxyJump** blir en **jump host/bastion** första halt på varje resa – vilket ger färre öppna portar och ett tydligt spår av vem som gjorde vad. Nyckelbaserad inloggning är inte bara “bekvämt”: den *byta ut* det som annars vore ett lösenord på nätet mot **kryptografiska bevis**. I skolmiljö försvinner mycket friktion när man *förklarar varför* det är värt att hålla ordning på **nycklar** (och deras passfraser), snarare än att lägga energi på att komma ihåg “det senaste superlösenordet”.

Förtydligande (sammanfattning):

- *SSH:* fjärrskal, tunnlar, **SFTP-subsystem**; **nycklar** ger både säkerhet och smidighet.
- *SFTP/SCP/rsync:* tänk **fall** – engång vs synk vs rättighetsbevarande.
- *SMB (Samba):* bro mellan Linuxfilsystem och Windowsklienter; identitet + ACL gör att allt “känns likadant” för användaren.
- *Bastion:* en obligatorisk mellanlandning gör adminspår tydligt och ytan mindre.

Exempel: En ämnesansvarig lärare vill lägga upp stora kursfiler på filservern från hemmet. Med **rsync över SSH** *kopieras bara det som ändrats, rättigheterna bevaras, och överföringen kan återupptas. När läraren är på skolan monteras samma mapp som* **SMB** *på lärardatorn – två vägar, samma källa, ingen dubbeladministration.*

3.6.3 Användarskapande och rättigheter via CLI

Linux håller identitet enkelt och synligt: **användare** har **UID**, **grupper** har **GID**, och filsystemet bryr sig om **ägare**, **grupp** och “alla andra” när

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

det avgör vad som är tillåtet. I praktiken betyder det att **ägarskap** och **gruppmedlemskap** är den vardagliga valutan – och att **rättigheter** (r, w, x) läses lite olika för filer och kataloger. Elever som snabbt lär sig "**x på en katalog betyder 'få gå igenom'**" får *plötsligt* superkrafter i felsökning: de förstår varför en mapp "som alla har rätt till" ändå inte går att nå (för att en föräldramapp saknar x), och de ser varför en **sticky bit** på en gemensam "inlämningskorg" gör att alla kan lägga till, men bara ägaren kan ta bort.

I en skolmiljö är det dessutom värt att se **grupp** som "rollen" i vardagen. En **lärargrupp** och en **klassgrupp** går att använda både i **Linux-ACL** (för finmaskiga rättigheter och **default-ACL** i träd) och i **Samba** (för att matcha Windowsvärldens förväntningar). Samtidigt håller **sudo** ordning på *vem* som får eskalera till administratör – inte för att försvåra, utan för att ge ett **spårbart ja** till en begränsad uppgift i stället för ett tyst "allt". När eleverna ser hur **ägarskap → grupp → ACL** hänger ihop blir filservern *begriplig* och uppgiften "ge *bara* lärare rätt att ändra i ämnesmappen" slutar vara ett mysterium.

Förtydligande (sammanfattning):

- *Identitet:* **UID/GID** och **primär/sekundär grupp** driver mycket av vardagen.
- *Rättigheter:* **r/w/x** tolkas olika för fil vs katalog; **sticky**, **setgid** på kataloger ger förutsägbarhet i delade ytor.
- *ACL:* kompletterar klassiska rättigheter; **default-ACL** är "ärva framåt" i nyckelkataloger.
- *Säker admin:* **sudo** ger smala, spårbara rättigheter; **grupper** representerar roller och mappar snyggt till Samba/SMB.

Exempel: En gemensam "inlämna-hit"-mapp får **skrivrätt** *för elever och* **sticky bit** *så att ingen kan ta bort någon annans filer. Ämnesmappen under samma träd har* **setgid** *på katalogen så att nya filer alltid får gruppen "Larare", och en* **default-ACL** *som ger lärargruppen* **write** *utan att någon behöver "komma ihåg" att ändra.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Varför skiljer Linux mellan **nuläge** (kärnans bild) och **beständig konfiguration** (Netplan), och hur märks det i felsökning?
2. Hur relaterar **VLAN**, **bonding** och **bridges** till vardagsproblemet “koppla rätt gäster till rätt nät”?
3. Varför påverkar **DNS-resolvern** och **tidssynk** ofta “allt annat”, och vilka symptom brukar du se när de är fel?
4. Förklara varför **SFTP** inte är “FTP + kryptering” utan ett **SSH-subsystem**, och vad det betyder för identitet och säkerhet.
5. När är **SCP** tillräckligt och när bör du hellre välja **rsync över SSH**?
6. Vilken roll spelar **Samba** i en blandad miljö, och hur kopplar den samman **Linux-rättigheter** med **Windows-roller**?
7. Vad betyder **x på katalog** jämfört med **x på fil**, och varför stoppar avsaknad av x ibland åtkomst trots “read”?
8. Hur hjälper **setgid** på en katalog och **default-ACL** till att hålla en delad ämnesmapp konsekvent?
9. Varför är **sudo** att föredra framför “logga in som root”, och vad vinner du på gruppbaserad delegering?
10. Beskriv ett scenario där en server upplevs “seg mot domänen” fast internet är snabbt — vilka två saker kontrollerar du först, och varför?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- **Ubuntu Server Guide** (nätverk, lagring, tjänster): https://ubuntu.com/server/docs
- **Netplan** – översikt och exempel: https://netplan.io/
- **systemd-networkd / networkctl** – nät på systemd: https://www.freedesktop.org/software/systemd/man/latest/systemd.network.html
- **systemd-resolved / resolvectl** – DNS i systemd: https://www.freedesktop.org/software/systemd/man/latest/systemd-resolved.service.html
- **OpenSSH** – klient/server och nycklar: https://www.openssh.com/manual.html
- **Samba** – dokumentation och wiki: https://www.samba.org/samba/docs/
- **rsync** – manual och best practices: https://download.samba.org/pub/rsync/rsync.html
- **POSIX ACL** – `getfacl`/`setfacl`: https://man7.org/linux/man-pages/man5/acl.5.html
- **Filrättigheter och** `chmod`: https://man7.org/linux/man-pages/man1/chmod.1.html
- **Användare och grupper (**`useradd`**,** `groupadd`**)**: https://man7.org/linux/man-pages/man8/useradd.8.html

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.7 MDM och klienthantering

3.7.1 Vad är MDM – och varför spelar det roll?

MDM (Mobile/Modern Device Management) är idén att styra **klienternas beteende och säkerhet centralt**, oavsett var de befinner sig. Där **AD + GPO** utgår från att datorn befinner sig "på plats" i domänen, utgår MDM från att enheten kan stå **var som helst** – hemma, i klassrummet, på ett café – och ändå få rätt policy, rätt appar och rätt nätprofiler över internet. Tekniskt sker detta via en **push-kanal** (Apple/Google/Microsofts notistjänster) som väcker klienten och säger "hämta din nästa policy", medan en **MDM-agent** eller inbyggt stöd i OS ser till att inställningarna faktiskt sätts.

Det viktiga för administratörer att förstå är **skillnaden i filosofi**: MDM modellerar datorn som ett **hanterat objekt** med mätbara tillstånd (kryptering? brandvägg? låsskärm?) och **efterlevnad** (compliance). Uppfyller enheten kraven får den tillgång (t.ex. till e-post eller lärarportaler); annars blockeras eller begränsas den tills kraven är uppfyllda. På så sätt blir **säkerhet en egenskap av klientens tillstånd**, inte bara av vilket nät man sitter på.

Förtydligande (sammanfattning):

- *MDM → internetförst:* funkar oavsett plats och VLAN.
- *Policy som tillstånd:* "är BitLocker/FileVault på?", "är OS uppdaterat?".
- *Efterlevnad → åtkomst:* klarar du kraven får du tjänsten (annars inte).

Exempel: En lärardator som är helkrypterad, har skärmlås och aktuell patchnivå får automatiskt full åtkomst till skolans e-post och molnlagring. En dator som halkat efter får bara läsa kursplaner tills den har uppdaterats.

3.7.2 Ekosystemet: Intune, Jamf – och öppna alternativ

I Windowsvärlden är **Microsoft Intune** den vanligaste MDM-plattformen. Den talar alla de språk Windows behöver (Windows-MDM-CSP, Autopilot,

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Entra-ID-integration) och hanterar även **iOS/iPadOS/macOS** och **Android** i samma verktyg, vilket är praktiskt i skola. På Apple-sidan är **Jamf** branschstandard för djup styrning av **macOS** och iOS/iPadOS – särskilt när man vill in i Apples "alla detaljer" (Configuration Profiles, appdistribution, "supervised" läge, kontrollerad återställning). För Android är **Android Enterprise** själva modellen: "Work profile" för BYOD och "Fully managed" för skolägda enheter, ofta orkestrerat via Intune eller en Android-fokuserad MDM.

Det finns **öppna och öppet licensierade** alternativ, men de är oftast mer **plattformsspecifika** och kräver mer **hands-on**: t.ex. **MicroMDM** för macOS (ren Apple-MDM), eller Android-fokuserade community-varianter. De kan vara utmärkta i labb eller nischade behov, men ersätter sällan en **heterogen** skolmiljös krav på rapporter, efterlevnad och integrationer (identitet, appar, certifikat).

Förtydligande (sammanfattning):

- *Intune:* bred täckning (Win/macOS/iOS/Android), bra på Windows-livscykel.
- *Jamf:* djup Apple-styrning och Apple-flöden (ADE, VPP).
- *Android Enterprise:* Work profile (BYOD), Fully managed (skolägda).
- *Öppna projekt:* lärorika och vassa, men oftare smalare.

Exempel: En kommun kör Intune för Windows-parken och låter Jamf sköta Mac-datorer i estetiska programmet. Båda plattformarna rapporterar efterlevnad till samma identitetstjänst, så åtkomstregler blir konsekventa.

3.7.3 Centralt styrda policyer och säkerhetsinställningar

MDM uttrycker säkerhet som **mätbara regler**. Istället för "har vi rullat ut brandväggen?" frågar du "**är brandväggen på och blockerar den inkommande trafik?**". Därifrån följer ett antal byggstenar som återkommer oavsett OS:

- **Enhetskonfiguration:** skärmlås, lösenordspolicy, brandvägg, kryptering (BitLocker/FileVault), USB-policy, WLAN-profiler (SSID, 802.1X-certifikat), VPN-profiler, webbfiltret.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Efterlevnad (compliance):** minsta OS-version, kryptering på, ingen jailbreak/root, aktiva skydd.
- **Apphantering:** paketera, distribuera, uppdatera och – vid behov – ta bort appar; i Applevärlden via **VPP** (licenser) och i Android via **Managed Google Play**.
- **Villkorad åtkomst (Conditional Access):** låt **identiteten** kontrollera **tjänsten** baserat på **enhetens status**.
- **Certifikatlivscykel:** utfärda, förnya och dra tillbaka klient-certifikat för 802.1X, VPN och appar.

Det här låter stort, men effekten i vardagen är just **enkelhet**: du behöver inte längre fråga "var står datorn?", utan "**uppfyller den kraven?**".

Exempel: En elevdator ansluter till skolans **WLAN** *utan att någon rör en knapp: MDM har redan lagt* **WLAN-profilen** *med 802.1X-certifikatet, och brandväggen vet att bara skolans resolvers får svara på DNS.*

3.7.4 Provisionering och livscykel: från kartong till återbruk

Det moderna idealet är **zero-touch**: att en ny enhet kan öppnas av användaren och "hitta hem" utan att IT-personal rör den. För Windows kallas detta **Autopilot**, för Apple **Automated Device Enrollment (ADE)** via Apple School/Business Manager, och för Android **Zero-touch** (eller QR/NFC-inrullning). Poängen är inte magi, utan **kedjan**: enheten känner igen sig som skolägd, ansluter till **MDM**, får **policyer, appar, profiler** – och presenterar login på rätt sätt. Lika viktigt är **offboarding**: att kunna **återtaga licenser**, **rensa data** (helt eller selektivt vid BYOD), och **återanvända** enheten till nästa elev.

Förtydligande (sammanfattning):

- *Ägda enheter:* full styrning, full wipe; kan låsas till skolans ägarskap.
- *BYOD:* **selektiv rensning** (ta bort skoldata, lämna privatdata).
- *Livscykel:* inrullning → drift/uppdatering → support → återlämning/återbruk.

Exempel: Årskurs 1 får nya laptops. Autopilot visar skolans välkomstbild, kopplar upp mot MDM, lägger in WLAN-profil och Office-appar – lektionen kan börja direkt utan imaging eller manuell patchning.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3.7.5 Mobilitet, BYOD och externa enheter

BYOD ("ta med egen enhet") och **externa minnen** (USB, mobil lagring) rör inte bara teknik, utan **juridik och förtroende**. Bra MDM-upplägg håller isär **personlig integritet** och **skolans data**. På Android innebär det **Work profile**: skolans appar och data bor i en skyddad "bubbla" som IT kan styra, medan privatdelen förblir privat. På iOS/iPadOS finns **User Enrollment** för liknande separation, och på macOS kan policyer riktas utan att ge IT "insyn" i allt. För USB är poängen inte förbud i sig, utan **riskreducering**: vissa roller kanske behöver fullt stöd (t.ex. media-elever), andra bara läsning eller helt avstängt.

Förtydligande (sammanfattning):

- *BYOD:* separera **skoldata** från **privatdata**; var tydlig med vad som loggas.
- *USB och externa enheter:* differentiera per program/roll; logga beslut.
- *Rese-scenarier:* nätprofiler/VPN gör att "skolan" följer med även utanför skolans väggar.

Exempel: En vikarie loggar in på sin privata iPad och får **bara** *access till lärarportalen via en skyddad container. När uppdraget är slut kan skolan rensa container-delen utan att röra privata bilder och appar.*

3.7.6 Varför Windows Server ofta är navet – och var Linux glänser

I många svenska miljöer inom offentlig sektor och utbildning fungerar **Windows Server med Active Directory (AD DS)** *som ett* **nav för identitet, policy och infrastruktur**. *Det beror både på klientflottans sammansättning (ofta Windows) och på att AD samlar* **autentisering (Kerberos/NTLM)**, **grupp- och rättighetsmodeller**, **DNS/DHCP** *och* **gruppolicy (GPO)** *i en sammanhängande helhet. Alltifrån skrivare och filresurser till inloggningsupplevelse kan därför kopplas till samma identitetskälla. Under senare år har många organisationer gått mot* **hybrida upplägg** *där ett lokalt AD samverkar med* **molnidentitet och MDM** *(t.ex. Entra ID/Intune), så att traditionella GPO:er och moderna, internetförsta policyer kan samexistera.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Linuxservrar fyller samtidigt en **stark och bred tjänsteroll**. De används ofta för **filer och utskrifter** (Samba/CUPS), **webb och applikationer** (NGINX/Apache, applikationsramverk), **infrastrukturtjänster** (reverse proxy, cache/proxy, databaser, RADIUS, syslog), **automatisering** (t.ex. Ansible) och **containrar** (Docker/Kubernetes). Linux kan även bära **identitetstjänster** (t.ex. Samba-AD-DC eller FreeIPA/IdM) när arkitekturen kräver det, men där Windows-klienter dominerar är AD ofta fortsatt den **primära identitetskällan**, medan Linux fungerar som **stabila tjänstenoder** runt navet. Valet handlar mindre om "bäst i test" och mer om **arkitektur**: *AD där sammanhållen klientidentitet och policystyrning är centralt; Linux där öppna, skalbara servertjänster och plattformar behövs* – med **interoperabilitet** via LDAP/Kerberos/SMB som gör att helheten sitter ihop.

Exempel: Ett verksamhetsnät använder AD för konton, grupper, DNS/DHCP och skrivare. Runtom levererar Linux en filresurs för elevmaterial, en webbtjänst för schema och en containerplattform för interna appar. Klienterna hanteras med MDM så att policy och åtkomst är konsekvent även utanför kontoret.

3.7.7 Vanliga arketyper i organisationer

I praktiken framträder några återkommande mönster för hur klienter och tjänster hanteras. De skiljer sig främst i var identiteten bor, hur policy distribueras och var data/appar levereras.

Klassisk on-prem
Identitet och policy ligger i lokalt **AD** med **GPO** för Windows-klienter; **Configuration Manager** eller likvärdigt används för paket och imaging; **Jamf** används ofta för macOS/iPadOS; fil- och utskriftstjänster körs lokalt. Fördelen är förutsägbarhet och låg latens på plats. Nackdelen är ett starkt beroende av skolans nät och schemalagda uppkopplingar.

Hybrid
Ett lokalt AD samverkar med molnidentitet och **MDM** (t.ex. Entra ID/Intune för Windows, Jamf för Apple). GPO hanterar miljöer som mår bra av platsnära styrning (t.ex. datasalar), medan bärbara enheter och plattor

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

följer **internetförst-policy** via MDM. Filresurser rör sig delvis mot moln, och åtkomst styrs villkorat utifrån enhetens efterlevnad. Fördelen är att både platsnära och rörliga behov får stöd. Nackdelen är ökad komplexitet i gränssnittet mellan system.

Cloud-first
Identitet och policy lever primärt i molnet; **Intune** hanterar Windows, **Jamf/Android Enterprise** används där det passar; lokalt AD minimeras eller avvecklas; applikationer och filer levereras som molntjänster. Fördelen är att hanteringen i hög grad blir oberoende av plats och VLAN. Nackdelen är beroendet av externa tjänster och behovet av väl definierade processer för efterlevnad och dataskydd.

Exempel: I en kommun med flera skolor drivs salar med platsnära policy via GPO för stabil uppstart och låg latens, medan lärarbärbara och plattor följer MDM-policy över internet. Identiteten är gemensam, men styrningen sker via två kompletterande spår.

3.7.8 Vanliga fallgropar och hållbara tumregler

De problem som uppstår i MDM-miljöer beror oftare på **otydliga målbilder** än på teknikfel. Om önskat **enhetstillstånd** inte är definierat blir policyer motsägelsefulla. Om **efterlevnadskriterier** är vaga blir åtkomstregler svårförutsägbara. Och om **insyn och loggning** inte kommuniceras tydligt riskerar förtroendet att urholkas. Stabilitet uppnås när grundkraven är mätbara, när skolägda och privata enheter hanteras separat och när nätanslutning görs friktionsfri genom förkonfigurerade profiler.

Förtydligande (sammanfattning):

- *Mätbara grundkrav:* kryptering, patchnivå, skärmlås, brandvägg.
- *Tydlig segmentering:* skolägda enheter vs **BYOD** med olika policy och rensningsrättigheter.
- *Förkonfigurerad anslutning:* **WLAN**-/VPN-profiler och certifikatlivscykel som del av MDM.
- *Transparens:* vad som samlas in/loggas, varför det görs och vad som uttryckligen inte övervakas.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel: Införandet av BYOD möter initial skepsis. När informationen förtydligar att MDM endast hanterar arbetsprofilens appar, OS-version och krypteringsstatus—och inte privata foton, SMS eller webbhistorik—ökar acceptansen och supportärenden minskar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Vad är den konceptuella skillnaden mellan **GPO** och **MDM**?
2. Hur omsätter MDM “säkerhet” till **mätbara tillstånd**, och varför är det viktigt för åtkomst?
3. När passar **Intune** bäst, när passar **Jamf** bäst, och vad vinner du på att kombinera dem?
4. Vad innebär **Android Work Profile** jämfört med **Fully Managed**, och hur påverkar det BYOD?
5. Vilka fem saker vill du nästan alltid ha som **MDM-policy** på skolägda datorer?
6. Hur fungerar **zero-touch** i Windows/Apple/Android i stora drag, och varför förenklar det support?
7. Vad menas med **selektiv wipe** och när är det avgörande?
8. Varför är **certifikatlivscykel** central i WLAN/VPN-profiler, och hur hjälper MDM?
9. I vilken roll är **Windows Server/AD** oftast navet, och när lyfter du hellre in **Linuxservrar**?
10. Beskriv en typisk **hybrid**-arkitektur i skola och hur MDM knyter ihop den.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- **Microsoft Intune** – översikter och scenarier: https://learn.microsoft.com/mem/intune/
- **Windows Autopilot** – koncept: https://learn.microsoft.com/windows/deployment/windows-autopilot/
- **Conditional Access & device compliance**: https://learn.microsoft.com/entra/identity/conditional-access/
- **Jamf Pro** – dokumentation: https://docs.jamf.com/
- **Apple School/Business Manager & Automated Device Enrollment**: https://support.apple.com/guide/apple-business-manager/welcome/web
- **Managed Apple IDs och User Enrollment**: https://support.apple.com/guide/apple-business-manager/intro-to-user-enrollment/web
- **Android Enterprise** – översikt: https://developers.google.com/android/work
- **Samba (SMB på Linux)** – docs: https://www.samba.org/samba/docs/

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

4 Säkerhet och övervakning

4.1 Nätverkssäkerhetens grunder

4.1.1 Grundidéer: lager, rättigheter och synlighet

Nätverkssäkerhet handlar i grunden om att **minska konsekvensen** när något går fel, inte om att garantera att inget någonsin händer. Två idéer ramar in allt annat: **försvar i flera lager (defense in depth)** och **minsta möjliga rättighet (least privilege)**. Lager-tänket gör att ett enskilt fel – en sårbar klient, ett läckt lösenord, en missad uppdatering – inte automatiskt blir ett stort driftstopp. Rättighetsprincipen gör att ett kompromettterat konto eller en felkonfigurerad tjänst inte får mer makt än vad den faktiskt behöver. Allt detta står och faller med **synlighet**: utan loggar, larm och mätningar vet man varken vad som hände eller om åtgärden fungerade.

I praktiken börjar säkerhetsarbetet med tre frågor: **Vad skyddas? Mot vad? Hur märks det?** Tillgångar (elevdata, lärarmaterial, administration), hot (misstag, skadlig kod, obehörig åtkomst) och **telemetri** (loggar, larm, mätvärden). När det är definierat blir prioriteringen enklare: **segmentering** (VLAN/zoner), **uppdateringar**, **stark identitet** (MFA, gruppbaserad åtkomst), **baspolicys** (klient/servrar med brandvägg på) och **loggning som faktiskt läses** ger ofta mest effekt.

Förtydligande (sammanfattning):

- *Lager:* små fel blir inte stora.
- *Rättigheter:* konton/tjänster har bara det de behöver.
- *Synlighet:* utan logg/larm finns ingen säkerhet i praktiken.
- *Prioritering:* segmentering, patchning, stark identitet ger hög “träff per timme”.

Exempel: Elevklienter och serversystem ligger i olika VLAN. Även om en elevdator infekteras kan skadlig kod inte prata direkt med servern –

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

brandvägg och routingregler står i vägen. Händelsen syns i loggarna och kan hanteras innan den sprider sig.

4.1.2 CIA-triaden

CIA-triaden beskriver tre mål: **konfidentialitet** (bara rätt mottagare kan läsa), **integritet** (data är oförändrad) och **tillgänglighet** (tjänsten fungerar när den behövs). Varje kontroll bör kunna motiveras med vilket C, I eller A den stöttar. Kryptering och behörighet skyddar **C**, signering och checksummer skyddar **I**, redundans och kapacitetsplanering skyddar **A**.

4.1.3 Brandväggar: klassisk → NGFW (Next-Generation Firewall)

Brandväggar uttrycker vad som **är tillåtet mellan zoner**; allt annat nekas. En **klassisk, stateful brandvägg** arbetar främst på lager 3/4: käll-/mål-IP, portar och protokoll, och håller reda på anslutningars tillstånd (t.ex. TCP SYN/ACK). Det räcker långt för kända flöden (“server X får prata TCP/443 ut”), men blir trubbigt när många applikationer **delar samma portar**.

En **NGFW (Next-Generation Firewall)** lägger till **lager-7-förståelse**. Den kan identifiera **applikationer** oavsett portar, koppla regler till **användaridentitet** (via t.ex. AD/Entra), och utföra **innehållsinspektion**. I praktiken innebär det moduler som **IPS (Intrusion Prevention System)**, **URL-/webbfiltrering**, möjlighet till **TLS-inspektion** (dekryptera, kontrollera, kryptera om), **sandboxning** av misstänkta filer samt flöde av **threat intelligence**. Poängen är inte fler “knappar”, utan att **policyn kan uttryckas närmare verksamheten**: tillåt rätt **tjänst** för rätt **grupp**, blockera osäkra mönster.

Förtydligande (sammanfattning):

- *Stateful brandvägg:* IP/port/protokoll + tillstånd; enkel och snabb.
- *NGFW:* app-igenkänning (lager 7), identitetsmedvetna regler, IPS, URL-filter, ev. TLS-inspektion, sandboxning, hotflöden.
- *Att tänka på:* prestanda (särskilt TLS-inspektion), integritet/juridik, kompatibilitet (certifikat-pinning) och tydliga undantag.

Exempel: I en klassisk modell blir policyn “tillåt TCP/443 ut”, vilket i praktiken öppnar för “allt som talar HTTPS”. Med NGFW kan policyn i

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

stället säga “tillåt Microsoft Update och ett fåtal samarbetsverktyg för Personal, men blockera okända TLS-appar och fildelning för Elever” – även om allt använder 443.

4.1.4 IDS/IPS (Intrusion Detection/Prevention System) – och samspelet med NGFW

IDS/IPS kompletterar brandväggar genom att fokusera på *beteenden och mönster* i trafiken snarare än bara källor och portar. En **NIDS (Network Intrusion Detection System)** observerar trafik i nätet – ofta via **SPAN (Switched Port Analyzer)** eller en **TAP (Test Access Point)** – och ser flöden **mellan** system (“north-south” mot internet och “east-west” mellan zoner). En **HIDS (Host Intrusion Detection System)** tittar inne i värden (processer, loggar, filändringar) och fångar sådant som inte alltid syns på nätet.

Detekteringsmetoderna är två: **signaturbaserad** detektering jämför mot kända mönster (snabb och exakt för det kända), medan **anomalibas** letar avvikelser från “normal” trafik (bättre chans på okända hot, men kräver inlärning och kan ge fler falsklarm). Körs motorn **inline** blir den **IPS**: samma analys, men med möjlighet att **blockera** eller **reseta** misstänkta förbindelser i realtid; körs den vid sidan om (out-of-band) är rollen att **larma**.

Placering och signal. Vid gränsen mot internet fångas angrepp utifrån, men mycket verksamhetslogik passerar **internt** mellan klient- och serverzoner; där ger en NIDS-sensor på rätt länk hög signal. **Kryptering** (t.ex. TLS) begränsar innehållsinsyn; då blir **metadata** (tidsmönster, anslutningsfrekvens, kända domäner/IP:er) och **HIDS-telemetri** extra värdefullt. Ofta används **flera sensorer** för att skapa **sammanhang**: NIDS ser oväntade anslutningar, HIDS ser onormala processer, och loggkällor (AD, brandvägg, proxy) fyller i tidslinjen.

NGFW och IDS/IPS i samspel. Många NGFW innehåller en **IPS-motor**. Det ger fördelen att stoppa välkända mönster **i brandväggens policyflöde**. Samtidigt behåller **friliggande NIDS/HIDS** värde: de kan placeras på **andra länkar** (t.ex. mellan klient- och serverzon) och fånga **east-west-rörelser** som inte passerar brandväggens kant, eller ge **second opinion** utan att riskera falskpositiv blockering. En vanlig väg är

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

att börja i **detekteringsläge** och **blockera** först när regler har hög tillit – särskilt på systemkritiska länkar.

Förtydligande (sammanfattning):

- *NIDS/HIDS:* nät- respektive värd-perspektiv; kompletterar varandra.
- *Signatur vs anomali:* känt-snabbt kontra okänt-upptäckbart.
- *Inline vs out-of-band:* blockera vs larma.
- *Samspelet:* NGFW-IPS stoppar vid kanten; NIDS/HIDS ger djup och kontext internt.

Exempel: En serverzon visar plötsliga utgående anslutningar mot ovanliga mål. NIDS (via TAP) larmar för "rare outbound" och HIDS på samma server rapporterar nyskapade processer i ett ovanligt katalogträd. Brandväggen sätter en tillfällig block på utgående trafik från den servern (hög-tillitsregel i IPS-läget), och korrelation med AD-loggar visar ett läckt tjänstkonto som spärras och roteras.

4.1.5 Honeypots och honeytokens

Honeypots och **deception** skapar medvetna "mål" som ska vara tysta i normal drift. Poängen är inte att "locka angripare för sport", utan att skapa **tidig varning** och **telemetri** i zoner där man i övrigt vill se så lite trafik som möjligt. En enkel variant är en **honeytoken** (falsk uppgift/fil) som larmar när den läses.

Förtydligande (sammanfattning):

- *Syfte:* blixtlampa – inte produktion.
- *Placering:* i zoner där legitim kontakt är sällsynt.
- *Åtgärd:* tydlig ägare och process när det blinkar.

Exempel: En "tyst" filserver-alias annonseras internt. All aktivitet dit larmar – signal som annars hade drunknat bland normaltrafik.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

4.1.6 Säkerhet i VLAN-design

VLAN är inte bara ordning och adressplan; det är ett **säkerhetsverktyg**. När zoner och flöden uttrycks i VLAN och routing/ACL minskar risken att "tillfälliga lösningar" skapar genvägar.

Återkommande byggstenar:

- **Minimera trunkar** och **pruna** (tillåt) bara relevanta VLAN per trunk; **native VLAN** sätts till ett **oanvänt** (t.ex. 999).
- **BPDU Guard** och **PortFast** på klientportar; **Root Guard** nära roten; **storm-control** för att dämpa toppar.
- **DHCP Snooping** (avgör vilka portar som får vara DHCP), **Dynamic ARP Inspection (DAI)**, **IP Source Guard** i accesslagret.
- **ACL på SVI** (gateway): "default deny – öppna minsta möjliga".
- **Private VLAN** där klienter i samma VLAN inte ska kunna prata direkt.
- **802.1X** i accesslagret för att knyta **identitet till port** (MAB-fallback för skrivare/IoT).
- Separera **managementytor** (SSH/HTTPS/SNMP) i eget VLAN och nå dem via bastion/jump host.

Förtydligande (sammanfattning):

- *Zoner:* klient, server, gäst, management – och uttryckta flöden.
- *Accessskydd:* skydda mot felkopplade switchar och kringutrustning.
- *Förgiftning:* Snooping/DAI/Source Guard hindrar "falska sanningskällor".

Exempel: Ett "mystiskt" DHCP-scope dyker upp – klienter får fel adresser. Med **DHCP Snooping** *blockeras den privata routern, och Snooping-tabellen pekar ut exakt switchport. Port Security begränsar MAC-antal och dämpar effekten av medtagen utrustning.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

4.1.7 Protokollanalys med Wireshark

Protokollanalys svarar på frågan **vad som faktiskt hände**. Wireshark blir då ett mikroskop som visar ramar, flaggor och tidslinjer. En bra metod undviker datatsjöar: börja med ett **smalt syfte** (“klienter får inte IP”, “långsam inloggning”), fånga **så nära källan som möjligt** (klient, port-SPAN, server), och **filtrera** både vid fångst och visning. Etik och lag spelar också in: **minimera** (bara det som behövs), **begränsa innehåll** (undvik att spara payload i onödan), och **rensa** material när analysen är klar.

Några vyer ger mycket värde: **tidslinje**, **Follow Stream**, **I/O Graphs** och **Expert Information**. Välkända filter: `dhcp || bootp` för adressdelning, `dns` för namn, `arp` för tabeller, `icmp` för nåbarhet, `tcp.flags.syn==1 && tcp.flags.ack==0` för handskakningar, `tcp.analysis.retransmission` för störningar, `tls.handshake` för att se *att* TLS sker, `ip.addr==X` för att isolera en värd. På **WLAN** krävs ibland **monitor-läge** för att se råa 802.11-ramar – annars syns mest det som passerar via AP/controller.

Förtydligande (sammanfattning):

- *Fråga först:* vad vill tidslinjen bevisa eller motbevisa?
- *Smala fångster:* nära källan, kort tid, rätt filter.
- *Hjälpytor:* Follow Stream, Expert Info, I/O Graphs.
- *Etik:* fånga minsta möjliga, skydda, rensa.

Exempel: Klienter “saknar IP” i ett VLAN. Fångst visar DISCOVER men inga OFFER. På uplinken saknas **trusted-flagga** *i DHCP Snooping, så relätrafiken stoppas. När uplinken sätts som* **trusted** *syns OFFER/REQUEST/ACK och klienterna får adresser direkt.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Vad menas med **defense in depth** och **least privilege**, och hur samverkar de?
2. Koppla tre kontroller till **C**, **I** respektive **A** i CIA-triaden.
3. Förklara skillnaden mellan **stateful brandvägg** och **NGFW (Next-Generation Firewall)**.
4. Vad gör en **NIDS (Network Intrusion Detection System)** respektive **HIDS (Host Intrusion Detection System)** – och när behövs båda?
5. Hur skiljer sig **IDS** från **IPS (Intrusion Prevention System)**, och vad innebär **inline** kontra **out-of-band**?
6. Varför kan **NIDS/HIDS** vara värdefulla även när NGFW redan har **IPS**?
7. Nämn fem byggstenar som stärker **VLAN-säkerhet** i accesslagret och motivera dem kort.
8. Vad gör **DHCP Snooping**, **DAI** och **IP Source Guard**, och hur hänger de ihop?
9. Beskriv en metod för att felsöka “långsam inloggning” med Wireshark utan att exponera onödig data.
10. Ge tre visningsfilter i Wireshark som snabbt ringar in problem med DHCP, DNS och TCP-anslutningar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- NIST – Computer Security Incident Handling (översikt): https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final
- NIST – Zero Trust (ramverkstänk): https://csrc.nist.gov/publications/detail/sp/800-207/final
- RFC 2131 – DHCP: https://www.rfc-editor.org/rfc/rfc2131
- RFC 1034/1035 – DNS grunder: https://www.rfc-editor.org/rfc/rfc1034
- RFC 826 – ARP: https://www.rfc-editor.org/rfc/rfc826
- RFC 8446 – TLS 1.3: https://www.rfc-editor.org/rfc/rfc8446
- Wireshark – User Guide: https://www.wireshark.org/docs/wsug_html_chunked/
- Cisco – Campus LAN Security (designidéer): https://www.cisco.com/c/en/us/solutions/enterprise-networks/index.html

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

4.2 Åtkomstkontroll och behörigheter

4.2.1 AAA-modellen och identitet som “valuta”

AAA (Authentication, Authorization, Accounting) är tre perspektiv på åtkomst:

- **Authentication** – *autentisering*: att **bevisa** vem som försöker komma in (lösenord, certifikat, nyckel, passkey).
- **Authorization** – *behörighet*: att **besluta** vad den identifierade får göra (roller, grupper, regler).
- **Accounting** – *spårbarhet/loggföring*: att **registrera** vad som faktiskt hände (vem gjorde vad, när, från var).

I praktiken bärs identiteten runt i en **session** (t.ex. Kerberos-biljett eller webb-token). Behörighet uttrycks ofta som **RBAC (Role-Based Access Control)** – “rollen avgör” – ibland kompletterat med **ABAC (Attribute-Based Access Control)** där attribut som *plats, tid, enhetens status* påverkar beslutet. AAA binder ihop teknik och verksamhet: man bevisar *vem*, beslutar *vad*, och **bevisar i efterhand** via loggar *att* rätt sak hände.

Förtydligande (sammanfattning):

- Autentisering = **bevis**; behörighet = **beslut**; accounting = **bevis i efterhand**.
- RBAC modellerar **roller** (grupper); ABAC lägger på **villkor** (t.ex. “från hanterad enhet”).
- Sessioner bär identiteten; loggar och tidsstämplar gör den **revisionsbar**.

Exempel: En användare loggar in i domänen (autentisering), hamnar i grupperna **G_Larare** *och* **G_Mentor** *(behörighet), och allt deras konto gör i filservern loggas med tid, IP och filstig (accounting).*

4.2.2 Multifaktorautentisering (MFA) och motstånd mot phishing

MFA (Multifaktorautentisering) kombinerar **minst två** av: *något du vet* (lösenord/Pin), *något du har* (telefon, säkerhetsnyckel, certifikat),

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

något du är (biometri). Syftet är att **bryta kedjan** där ett läckt lösenord annars räcker. Alla “två steg” är inte lika: **phishing-resistenta** metoder som **FIDO2/WebAuthn** (säkerhetsnyckel/passkey) eller **klientcertifikat** är robustare än SMS-koder och enkla push-notiser. Riskbaserade signaler – plats, ovanlig tid, okänd enhet – kan **förstärka** kravet (ABAC), men ersätter inte MFA.

Förtydligande (sammanfattning):

- Faktorer: **vet/har/är** – blanda minst två.
- Starkast: **FIDO2/WebAuthn** och **certifikat** (motstånd mot phishing).
- Praktik: minimera **push-trötthet**; använd villkor (ABAC) för att be om starkare faktor vid risk.

Exempel: E-post öppnas på en skolhanterad laptop utan extra steg (enheten är känd och compliant). Samma konto på en privat platta kräver **passkey** *innan åtkomst beviljas.*

4.2.3 Gruppolicyer och lösenordspolicyer (Windows-miljö)

I Windowsmiljö uttrycks mycket av **authorization** via **grupper** och **GPO (Group Policy Objects)**. GPO styr både **User Rights Assignment** (vad ett konto *får göra* i OS: logga in lokalt, via RDP, köra som tjänst) och miljöpolicys som påverkar **säker nivå** (låsning, brandvägg, enhetsregler). Lösenord, låsning och relaterade regler definieras i en **domänpolicy** och kan förfinas med **FGPP (Fine-Grained Password Policies)** för särskilda grupper.

Det finns ett skifte i rekommendationer: **långa lösenfraser** och **MFA** ger bättre effekt än täta tvångsbyten och komplexitetsdikt. Tvingad rotation utan indikator på läcka skapar ofta **svagare** lösenord och mer återanvändning. För lokala administratörer ligger säkerheten i **Windows LAPS** (unika, roterade lösenord per klient/server) – inte i “ett gemensamt hemligt lösenord”.

Förtydligande (sammanfattning):

- GPO: **User Rights**, **säkerhetslinjer** (brandvägg, lås), **policyuttryck** per OU.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- Lösenord: **längd + MFA** slår “komplexitet + frekvent byte”.
- Lokala admin: **Windows LAPS** för unika hemligheter.

Exempel: RDP tillåts endast för gruppen **G_Admin** *via User Rights Assignment. Lösenordspolicyn kräver minst 15 tecken; gruppen* **G_HögRisk** *får dessutom FGPP med lågnivå-tröskel och obligatorisk MFA för alla molnappar.*

4.2.4 Behörigheter och arv i Linux och Windows

Båda världarna behöver svara på samma fråga: *vem får vad, och hur ärvs det vidare?* Svaren ser olika ut i verktyg – men logiken rimmar.

Linux – DAC, grupper och ACL
Linux bygger på **DAC (Discretionary Access Control)**: **ägare**, **grupp**, **andra** med rättigheterna **r/w/x**. För kataloger betyder **x** “få gå igenom/angöra”, **w** “skapa/ta bort i katalogen”. **setgid** på kataloger gör att **nya filer** får **katalogens grupp** (bra för gemensamma träd). **sticky bit** på gemensamma “inlämningskorgar” gör att man inte kan ta bort andras filer. **POSIX-ACL** kompletterar med finare rättigheter och **default-ACL** som ärvs nedåt när nya objekt skapas. **sudo** ger spårbar eskalering till administrativa kommandon; **polkit** och **Linux capabilities** kan ge mer granulär kontroll per tjänst.

Windows – NTFS-ACL och arv
NTFS använder **ACL (Access Control List)** med **ACE**-poster (Allow/Deny) för användare och grupper. Rättigheter **ärvs** nedåt i träd om inte arv bryts. **Ägarskap** styr rätten att ta över/ändra rättigheter. Effektiv rättighet blir summan av tilldelade **Allow** minus eventuella **Deny** (som vinner). Skillnad mot Linux är att arvet är **explicit synligt** och att **ärvda** poster skiljs från **direkta** poster. I nätutdelningar “multipliceras” **share-rättigheter** med **NTFS** – den mest begränsande gäller.

Gemensam nämnare – grupper som språk
I båda miljöer blir **grupper** det praktiska stället där **RBAC** uttrycks. På Linux kopplas grupper till **ägarskap/ACL**; i Windows kopplas grupper till **NTFS-ACL** (och i domänen till **GPO** och **User Rights**). Arv gör att rättigheten följer **strukturen**, inte manuella undantag.

Förtydligande (sammanfattning):

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- Linux: **ägare/grupp/andra**, **setgid/sticky**, **default-ACL**; **sudo** för spårbar eskalering.
- Windows: **NTFS-ACL** med arv och **ägarskap**; **share × NTFS** ger effektiv rätt.
- Båda: **grupper** uttrycker roll; låt **arv** göra det tunga lyftet.

Exempel: Ett ämnesträd på filservern har **setgid** *och* **default-ACL** *som ger "Larare"* **write***. I NTFS-världen motsvaras detta av en ärvd* **Allow-Modify** *för gruppen* **G_Larare** *på rotmappen. I båda fall räcker det att lägga en ny lärare i rätt* **grupp** *– inget handpill i djupt liggande mappar.*

4.2.5 Vanliga mönster och missförstånd

Ett par mönster återkommer oavsett plattform: **minsta möjliga rättighet** (ge bara det som krävs), **separation of duties** (dela ansvar så att ingen ensam kan skapa och godkänna sin egen åtkomst), och **förutsägbar arvsväg** (rättigheter definieras högt upp och ärvs ned). Missförstånd uppstår ofta när **individuella undantag** landar på objekt långt ner i ett träd – de fungerar idag, men bryter ihop imorgon. Ett annat är att blanda ihop **OS-rättigheter** (vem får logga in) med **data-rättigheter** (vad får läsas/ändras); de lever i olika lager.

Förtydligande (sammanfattning):

- Satsa på **gruppbaserad** behörighet och **arv**; undvik manuella "lappar och lag".
- Separera **logon-rättigheter** från **filrättigheter** i tanke och praktik.
- Låt **accounting** (loggar) bevisa efterlevnad och underlätta felsökning.

Exempel: Ett enskilt "Deny" på en djupt liggande mapp "låser ute" en lärare oförutsett. Efter städning tillbaka till rent arv (bara **Allow** *via grupp) försvinner sidoeffekterna och felsökning blir begriplig.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Beskriv **AAA (Authentication, Authorization, Accounting)** och ge ett konkret exempel på varje del.
2. Vad skiljer **RBAC** från **ABAC**, och när är ABAC särskilt användbart?
3. Vilka MF-metoder räknas som **phishing-resistenta**, och varför?
4. Vad är **User Rights Assignment** och hur skiljer det sig från filrättigheter?
5. När passar **FGPP (Fine-Grained Password Policies)**, och varför föredras **långa lösenfraser + MFA** framför täta tvångsbyten?
6. Förklara **setgid** och **sticky bit** på kataloger i Linux och hur de hjälper i gemensamma mappar.
7. Hur fungerar **arv** i **NTFS-ACL**, och varför bör **Deny** användas sparsamt?
8. Varför leder **individuella undantag** ofta till oförutsägbart beteende över tid?
9. Ge ett scenario där **ABAC** (t.ex. enhets-compliance) påverkar åtkomstbeslutet.
10. Vilka loggkällor behövs för **Accounting** när man vill kunna spåra “vem gjorde vad, när, från var”?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- NIST SP 800-63-3 – Digital Identity Guidelines: https://pages.nist.gov/800-63-3/
- FIDO Alliance – Overview (passkeys/säkerhetsnycklar): https://fidoalliance.org/overview/
- W3C – Web Authentication (WebAuthn) Level 2: https://www.w3.org/TR/webauthn-2/
- Microsoft Learn – User Rights Assignment (Group Policy): https://learn.microsoft.com/windows/security/threat-protection/security-policy-settings/user-rights-assignment
- Microsoft Learn – Fine-Grained Password Policies (FGPP): https://learn.microsoft.com/windows-server/identity/ad-ds/manage/component-updates/fine-grained-password-policies
- Microsoft Learn – Windows LAPS (översikt): https://learn.microsoft.com/windows-server/identity/laps/laps-overview
- man7 – POSIX ACL (`acl(5)`): https://man7.org/linux/man-pages/man5/acl.5.html
- man7 – `chmod(1)` (filrättigheter): https://man7.org/linux/man-pages/man1/chmod.1.html
- sudoers manual (kontrollerad eskalering): https://www.sudo.ws/docs/man/sudoers.man/
- polkit – PolicyKit-dokumentation: https://www.freedesktop.org/software/polkit/docs/latest/

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

4.3 Övervakning och loggning

4.3.1 Varför loggar och övervakar man?

Övervakning och loggning handlar om **synlighet**: att kunna svara på *vad hände, när, var, och av vem* – och att upptäcka när något **avviker** innan det blir ett driftstopp eller en incident. Loggar är dessutom bevismaterial när något måste utredas i efterhand. Två praktiska förutsättningar bär hela området: **tidssynk** (utan gemensam klocka går det inte att korrelera händelser) och **centralisering** (att samla loggar på en plats där de kan sökas, korreleras och larma).

I ett modernt nät är loggkällorna många: klienter och servrar, nätverksutrustning (switchar, routrar, brandväggar, **WLAN**-kontroller), katalogtjänster (AD/LDAP), DNS/DHCP, applikationer och identitetstjänster. Poängen är inte att “logga allt”, utan att logga **rätt** saker med **lagom detaljer** – och ha en tydlig **ägare** som faktiskt tittar på signalerna.

Förtydligande (sammanfattning):

- *Synlighet:* loggar = svar på “vad, när, var, vem”.
- *Grundkrav:* **tidssynk** och **centralisering**.
- *Urval:* logga det som behövs för att se avvikelser och felsöka – inte allt.

Exempel: En kort driftstörning i inloggning spåras på minuter tack vare gemensamma tidsstämplar: DNS-logg visar misslyckat uppslag, brandväggslogg visar blockerat flöde mot extern resolver, AD-logg visar ökade autentiseringsfel – kedjan blir tydlig och åtgärdas.

4.3.2 Logghantering: Event Viewer (Windows), syslog (Unix/Linux/nät)

Windows loggar till **Event Viewer** (Händelsevisaren) i kanaler som *System*, *Application*, *Security* och applikationsspecifika loggar. Händelser har **nivåer** (Information/Warning/Error/Critical), **källor** och ofta **händelse-ID** som gör dem sökbara. Säkerhetsloggar (t.ex. inloggning,

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

gruppändringar, policy) blir väldigt användbara när de **korreleras** med nät- och applikationsloggar.

Unix/Linux och nätutrustning använder oftast **syslog** – ett standardprotokoll och ett **format** för loggar. Syslog har **facility** (vilken del av systemet) och **severity** (emerg, alert, crit, err, warning, notice, info, debug). På Linux skrivs loggar numera ofta till **systemd-journald** (den binära journalen), och skickas därifrån vidare till en syslog-tjänst (t.ex. **rsyslog**/**syslog-ng**) för central insamling. Nätutrustning skickar syslog direkt till en **central loggserver**. UDP/514 är klassiskt, men **TCP/TLS** bör användas när det är möjligt för att undvika tapp och skydda innehåll.

För att loggar ska vara **användbara** behövs också: **namnstandarder** (värdnamn som går att tolka), **loggrotation** och **retention** (hur länge sparas vad), **åtkomstkontroll** (vem får läsa), samt **normalisering** (göra olika källor jämförbara) i den centrala plattformen.

Förtydligande (sammanfattning):

- Windows: **Event Viewer** med kanaler, nivåer, event-ID.
- Unix/Linux & nät: **syslog** (facility/severity) + ofta **journald** på Linux.
- Praktik: **TLS/TCP** när det går, tydliga namn, genomtänkt retention och rättigheter.

Exempel: En switchport “flappar” (går upp/ner). Syslog från switchen visar portens statusändringar, Event Viewer på filservern visar samtidigt tappade nätverksresurser. Tidslinjen kopplar ihop händelserna och gör felet lätt att lokalisera.

4.3.3 Verktyg: Wazuh, SNMP, SIEM

Wazuh är en **öppen plattform** för säkerhetsövervakning som kombinerar logginsamling med funktioner som **FIM (File Integrity Monitoring)**, sårbarhets-indikering och regelbaserade larm. Den använder agenter (på klient/servrar) men kan också ta emot loggar via syslog. Vanligt är att lagra/söka i en **Elastic/OpenSearch-stack** med färdiga instrumentpaneler. Poängen är att få **sammanhang**: “denna fil ändrades, på denna värd, direkt efter denna inloggning”.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

SNMP (Simple Network Management Protocol) är nätutrustningens telemetri. Med **GET** kan man läsa mätvärden (interfacestatistik, CPU, temperatur), med **TRAP/INFORM** skickar enheten **händelser** proaktivt (länk ner/upp, fläktfel, överhettning). **MIB/OID** beskriver vad som kan läsas. Version **SNMPv3** ger **autentisering och kryptering** (till skillnad från v1/v2c som använder community-strängar). I övervakning utgör SNMP "hälsopulsen" som kompletterar loggar.

SIEM (Security Information and Event Management) är den centrala plattformen som **samlar, normaliserar, korrelerar** och **larmar** på tvärs av källor: serverloggar, nätflöden, AD-händelser, brandvägg, DNS, proxy. Exempel på funktioner: **korrelationsregler** ("många misslyckade inloggningar + lyckad från ovanlig klient"), **dashboard** med indikatorer, **larmhantering** och **playbooks** för åtgärd. SIEM är inte lika med "dyra licenser" – principen kan uppnås med både öppna och kommersiella byggstenar – men **tidsstämplar, normalisering och ägarskap** är alltid avgörande.

Förtydligande (sammanfattning):

- **Wazuh:** agent + regler + dashboards; binder ihop fil/host/nät-händelser.
- **SNMP:** mätvärden och traps från nät- och infrastruktur; **SNMPv3** för säkerhet.
- **SIEM:** central hjärna för insamling, korrelation och larm.

Exempel: SNMP visar ökande felräknare på ett switchinterface; Wazuh larmar samtidigt om ovanligt många TCP-återförsök på en server i samma VLAN. SIEM visar korrelationen på en tidslinje – felet ligger på länken mellan dem, inte i applikationen.

4.3.4 Att tolka loggar och upptäcka avvikelser

Loggar blir meningsfulla när de sätts i **sammanhang**. Ett fungerande arbetssätt börjar med en **fråga** ("varför är inloggning långsam?", "vem skapade denna delning?"), väljer **källor** (AD/brandvägg/klient), och binder ihop händelser via **tidsstämplar** och **identiteter** (användare, värd, IP, process). Två vägar används parallellt:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Regel- och tröskelbaserat:** “>10 misslyckade inloggningar (Windows Event ID) från samma IP på 5 min” → bruteforce-indikator; “ny medlem i lokal admins” → larma.
- **Avvikelsebaserat:** “denna klient gör plötsligt tusentals DNS-frågor mot externa domäner” eller “den här servern pratar ut på en port den aldrig använt”.

Båda kräver **baslinjer**: hur “ser normalt ut”? Utan baslinje blir allt antingen larm eller brus. Därför finjusteras regler (trösklar, **suppressionsfönster**, **deduplicering**) tills larmen är **få men viktiga**. Och varje “rött larm” behöver en **runbook**: vem äger det, vilka steg tas, när eskaleras?

Förtydligande (sammanfattning):

- *Frågedriven metod:* fråga → källor → korrelation → slutsats.
- *Två blickar:* regler/trösklar **och** avvikelse/baslinje.
- *Tuning:* trösklar, dedup, suppression; **runbooks** för åtgärd.

Exempel: SIEM larmar: “Många misslyckade inloggningar följt av en lyckad (samma konto), därefter ny RDP-session från ovanlig klient”. Korrelation med AD-logg visar att kontot lades till i en lokal admin-grupp — en tydlig kedja som stoppas genom att spärra kontot och rulla lösenord/nycklar.

4.3.5 Larm, brus och ansvar (vem gör vad när det blinkar?)

Ett övervakningssystem utan **ägare** blir snabbt en “blinkande tavla”. Definiera **allvarsnivåer**, **SLA** för hantering (t.ex. kritiskt = 15 minuter till åtgärd), och **vägar** för eskalering. Arbeta bort **falskpositiv** genom att justera regler, lägga **undantag** för kända mönster och tidsbegränsa undertryckning. Testa larmkedjan med **table-top-övningar** och simulera verkliga händelser (t.ex. många misslyckade logins) för att se att kedjan håller.

Förtydligande (sammanfattning):

- *Sätt nivåer och tider:* så vet alla vad som förväntas.
- *Minska brus:* justera, vitlista kända mönster, följ upp.
- *Öva:* simuleringar visar var kedjan brister.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel: Ett “stormlarm” om misslyckade inloggningar visar sig bero på lösenordsbyte i en klass. En suppression-regel införs (1 h för just den källan), men ett separat larm ligger kvar för inloggningar **från internet** *– så att verkliga försök inte tystas.*

4.3.6 Dataskydd och etik

Loggar kan innehålla **personuppgifter** (användarnamn, IP, tid, ibland filnamn). Det gör **dataminimering**, **behörighetsstyrning**, **retention** och **syftesbeskrivning** viktiga. Loggar ska skyddas lika omsorgsfullt som andra system och bara vara tillgängliga för dem som behöver dem i sitt arbete.

Förtydligande (sammanfattning):

- *Minimera:* samla det som behövs – inte mer.
- *Skydda:* åtkomstkontroll, kryptering, revision.
- *Rensa:* retention kopplad till syfte och lagkrav.

Exempel: Säkerhetsloggar sparas 12 månader för incidentutredning och revisionskrav; applikationsloggar med personnära data rensas efter 30 dagar. Åtkomst till SIEM är gruppstyrd och loggas.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Varför är **tidssynk** och **centralisering** grundläggande för användbara loggar?
2. Vilka är de viktigaste skillnaderna mellan **Event Viewer** och **syslog**?
3. Vad gör **SNMP (Simple Network Management Protocol)** i övervakning, och varför föredras **SNMPv3**?
4. Beskriv hur **Wazuh** kan ge sammanhang mellan loggar, filändringar och inloggningar.
5. Vad betyder **SIEM (Security Information and Event Management)** i praktiken – vilka tre saker gör ett SIEM alltid?
6. Ge exempel på en **regelbaserad** respektive en **avvikelsebaserad** detektion.
7. Varför behövs **baslinjer** och hur påverkar de antalet falsklarm?
8. Vad är en **runbook**, och varför är den central när ett larm går?
9. Nämn tre beslut man måste ta kring **retention** och **åtkomst** till loggar.
10. Vad bör loggas för att kunna utreda “plötsligt långsam inloggning” utan att samla onödigt innehåll?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- Microsoft – Event Viewer (översikt): https://learn.microsoft.com/windows/client-management/troubleshoot-event-viewer
- systemd-journald (man-sida): https://www.freedesktop.org/software/systemd/man/latest/systemd-journald.service.html
- rsyslog – dokumentation: https://www.rsyslog.com/doc/
- IETF – Syslog Protocol (RFC 5424): https://www.rfc-editor.org/rfc/rfc5424
- SNMPv3 – översikt (Cisco): https://www.cisco.com/c/en/us/tech/snmp/
- Wazuh – dokumentation: https://documentation.wazuh.com/
- Elastic Security (SIEM-koncept, översikt): https://www.elastic.co/security/siem
- Microsoft Sentinel (SIEM/SOAR-koncept, översikt): https://learn.microsoft.com/azure/sentinel/overview
- FIRST – Common Event Expression & Logging best practices: https://www.first.org/cvss/user-guide & https://www.first.org/resources/papers/logging
- NIST – Guide to Computer Security Log Management (SP 800-92): https://csrc.nist.gov/publications/detail/sp/800-92/final

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

4.4 Incidenthantering och backup

4.4.1 Rutiner, roller, dokumentation

Incidenthantering handlar om att begränsa påverkan, återställa drift och lära av händelsen så att den inte händer igen. För att lyckas krävs tydliga rutiner, definierade roller och dokumentation som alla faktiskt hittar och använder. En enkel men effektiv process följer NIST-modellen: **Förberedelse → Identifiera → Inneslut → Åtgärda → Återställ → Lärdomar**. I skolmiljö där många användare delar nät är snabb kommunikation och spårbarhet minst lika viktiga som teknik.

Roller (förtydligande):

- **Incident Manager:** leder arbetet, sätter prioritet/severity, ser till att logg förs och att beslut dokumenteras.
- **Teknikansvarig (t.ex. Nät):** leder felsökning/åtgärd inom sitt område.
- **Skribent (Scribe):** för **tidslinje**, kommandon, observationer och beslut.
- **Kommunikation:** informerar ledning/personal/elever enligt mallar.
- **Ägare efteråt:** äger uppföljning och permanenta förbättringar.

Dokumentation som bör finnas nära till hands:

- **Kontaktlista & eskalering:** jour, leverantörer, ISP, beslutsfattare.
- **Topologi/IP/VLAN:** aktuella diagram, adresstabeller, brandväggsflöden.
- **Runbooks & mallar:** vanliga fel (DHCP, DNS, WLAN), **incidentrapport-mall**, ändringsmall.
- **Backuppolicy:** RPO/RTO per system, lagringsplatser, retention, testplan.

Exempel: En lärare anmäler “ingen kan logga in i A-huset”. Incident Manager skapar ärende (P2), Teknikansvarig (Nät) leder felsökning, Skribent loggar alla steg. Kommunikatören postar status i personalgruppen var 20:e minut tills felet är löst.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Mall: Incidentrapport (tom)

- **Ärende-ID:**
- **Datum/tid (start–slut):**
- **Severity (P1–P4):**
- **Påverkan:** *vilka, var, hur mycket*
- **Symptom:** *vad märktes*
- **Tidslinje:**
  - *[klockslag] → åtgärd/observation*
  - *[klockslag] → åtgärd/observation*
- **Rotorsak:**
- **Akuta åtgärder:**
- **Återställning (RTO/RPO utfall):**
- **Korrigerande/förebyggande åtgärder (CAPA):** *vad ändras, vem äger, när klart*
- **Bilagor:** *loggar, kommandon, skärmdumpar*
- **Godkänd av / datum:**

Exempel: Incidentrapport (ifylld, kort)

- **Ärende-ID:** 2025-08-14-B-DHCP-VLAN20
- **Datum/tid:** 08:12–09:03
- **Severity:** P2 (elevnät påverkat i ett hus)
- **Påverkan:** ~18 klassrum i B-huset, VLAN 20 (elev), “ingen IP”
- **Symptom:** Klienter fick 169.254.x.x (APIPA)
- **Tidslinje:**
  - 08:12 – anmälan: “ingen internet i B-huset”
  - 08:15 – `show ip dhcp snooping` → uplink ej *trusted*
  - 08:18 – `show interfaces trunk` → VLAN 20 saknas i allowed-lista
  - 08:22 – Åtgärd: trust uplink + tillåt VLAN 20 på trunk

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

    - 08:26 – `ipconfig /renew` OK på testklient
    - 08:40 – Lägesuppdatering till personal: “åtgärd pågår, förbättring syns”
    - 09:03 – Stängning, alla testpunkter gröna
- **Rotorsak:** Ny accesswitch med ofullständig trunk-mall
- **Akuta åtgärder:** Uppdaterad trunk + trust uplink
- **Återställning:** RTO 51 min, RPO ej relevant
- **CAPA:** Uppdatera mallen “Ny accesswitch” (ägare: Nät, klart 2025-08-20). Lägg auto-validering i övervakning: larma på DHCP-svarstid > X ms.
- **Bilagor:** Kommando-logg, skärmdumpar från övervakning
- **Godkänd:** IT-chef 2025-08-14

4.4.2 Backupmetoder: fullständig, inkrementell och differentiell

Tre klassiska metoder balanserar **backupfönster**, **lagringsutrymme** och **återställningstid**. Valet styrs av dina RPO/RTO-mål och hur mycket data som förändras dagligen.

Fullständig (full) backup
En **komplett kopia** av allt valt innehåll varje gång.
Fördel: enklast att återställa (du behöver **en** backupuppsättning).
Nackdel: längst backupfönster, mest lagring.
Passar: små/medelstora datamängder, månatliga “baseline”-kopior, system där **enkel restore** väger tyngst.

Inkrementell backup
Efter en initial **full** lagras **endast förändringar sedan senaste backup** (full **eller** inkrementell).
Fördel: snabb daglig backup, minst lagring.
Nackdel: restore kräver **kedja**: full + **alla** inkrementella till önskad tidpunkt.
Passar: stora datamängder, smala backupfönster, när RTO kan vara längre vid full återställning.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Differentiell backup
Efter en initial **full** lagras **alla förändringar sedan senaste full** (inte sedan senaste diff).
Fördel: restore kräver **två** uppsättningar (senaste full + **senaste diff**).
Nackdel: diff-filer **växer** dag för dag tills nästa full.
Passar: miljöer där **snabb restore** prioriteras över minimalt lagringsutrymme.

Förtydligande (sammanfattning):

- *Full:* **enkel restore**, **dyrt** i tid/lagring.
- *Inkrementell:* **billig** i tid/lagring, **dyrare restore** (kedja).
- *Differentiell:* **mittemellan** – större dag för dag, men **snabb restore** (full + diff).

Exempel: Datamängd 1 TB, daglig förändring ~5 % (= ~50 GB).

- *Inkrementell vecka:* Sön **1 TB full**, mån–lör **~50 GB/dag** ⇒ total vecka ~1 TB + 6×50 GB ≈ **1,3 TB**. Restore till torsdag kräver **1 full + 3 inkrementella**.
- *Differentiell vecka:* Sön **1 TB full**, mån **50 GB**, tis **100 GB**, ons **150 GB** … ⇒ restore till torsdag kräver **1 full + 1 diff (150–200 GB)** – snabbare än inkrementell kedja men större dagliga backups sent i veckan.

Exempel: Skolans filserver (Tier 0): **Söndag full**, **mån–lör inkrementell**, off-site med **immutabel** retention 30 dagar. RTO säkras via **regelbundna restore-tester** och möjlighet till **filnivå-restore** utan att spela upp hela kedjan.

Exempel: Administrativa dokument (Tier 1) där snabb återställning efter "misstag" är viktig: **Söndag full**, **mån–lör differentiell**. Restore = **full + senaste diff** (färre steg i skarpt läge).

Exempel: Klassrumsklienter: "Golden image" hanteras separat; använd **inkrementell** för hemkataloger och kör **månatlig full** för att korta kedjorna.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

4.4.3 Backupstrategier och återställning

Backup är din **sista försvarslinje** mot misstag, hårdvarufel och ransomware. En bra strategi börjar med två mål per system: **RPO** (*Recovery Point Objective* – hur mycket data får vi förlora?) och **RTO** (*Recovery Time Objective* – hur snabbt måste vi vara uppe igen?). Välj sedan teknik som möter målen: full/inkrementell/differentiell, **immutabla/air-gappade** kopior *(immutabla = kan inte ändras/raderas under en vald retention, "WORM"; air-gappade = kopior som är fysiskt eller logiskt isolerade från produktionsnätet, utan ständig åtkomst)* och off-site lagring. Kom ihåg: **snapshots ≠ backup** – snapshots är fantastiska före en ändring, men skyddar inte mot korruption som replikerats eller mot attacker som krypterar primär och replika.

Bra principer (förtydligande):

- **3-2-1(-1-0):** 3 kopior, 2 olika medier, 1 off-site, *(+1* **immutabel** – *skrivskyddad/WORM-kopia;* **air-gap** *innebär isolerad kopia)*, **0** fel i testad återställning.
- **Klassificera system:** Tier 0 (kritiska), Tier 1 (viktiga), Tier 2 (övriga).
- **Kryptering & nycklar:** kryptera off-site; skydda nycklar separat (MFA/HSM).
- **App-konsistens:** VSS för Windows-servrar; "quiesce" databaser.
- **Retention:** GFS (daglig/veckovis/månatlig), juridik/arkivkrav.

Exempel: Filserver (Tier 0) på Proxmox: nattlig inkrementell, veckovis full, månatlig off-site till S3-kompatibelt objektlager med immutabilitet 30 dagar. Teståterläsning 1×/vecka (filnivå) och 1×/månad (hel VM till isolerat VLAN). RPO 24 h, RTO 4 h.

Mall: Backupprofil per system (tom)

- **System/ägar-OU:**
- **Klass (Tier):**
- **RPO / RTO:**
- **Datakälla & metod:** *agent/VSS, konsistens*
- **Schema:** *full/incr/diff – när?*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Retention:** *dag/vecka/månad/år*
- **Lagring:** *on-site, off-site, immutabel, media*
- **Kryptering/nyckelhantering:**
- **Återställningstester:** *frekvens, senast utfört, resultat*
- **Kontakt/ansvarig:**
- **Relaterade runbooks:** *steg-för-steg för restore*

Exempel: Backupprofil (ifylld, kort)

- **System:** SRV-FIL-01 (OU Skolan\Servrar\File)
- **Tier:** 0
- **RPO/RTO:** 24 h / 4 h
- **Metod:** Agentlös VM-backup + VSS inuti gästen (app-konsistens)
- **Schema:** Full sön 02:00, inkrementell mån–lör 02:00
- **Retention:** Daglig 14 d, veckovis 8 v, månadsvis 12 m
- **Lagring:** On-site NFS (primär), Off-site S3 immutabel 30 d
- **Kryptering:** AES-256; nyckel i separat valv med MFA
- **Test:** Fil-restore varje fredag; hel VM-restore månad 1:a tisdag (senast OK 2025-08-05)
- **Ansvarig:** Nät/Server-admin
- **Runbook:** “Restore SRV-FIL-01 till isolerat VLAN”

4.4.4 Test av redundans och disaster recovery

Redundans som **inte testas** är **inte** redundans. Planera återkommande **spärrtester** där ni medvetet bryter komponenter: primär internetlänk, aktiv gateway (VRRP/HSRP = *förstahoppsredundans*: flera routrar delar en **virtuell gateway-IP**; en är aktiv **master**, övriga **standby**; vid fel tar standby över IP/MAC och klienterna märker bara ett kort avbrott), en DNS-resolver, en DHCP-server, en Proxmox-nod, en lagringslänk. Dokumentera förväntat utfall, mät faktiska tider och uppdatera konfiguration/mallar där gap hittas. På strategisk nivå behövs en **DR-plan (Disaster Recovery)**:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

hur ni startar upp det viktigaste efter ett större avbrott (brand, ransomware, långvarigt strömavbrott).

Att testa (förtydligande):

- **Gateway-failover:** byt aktiv VRRP/HSRP (VRRP = **Virtual Router Redundancy Protocol** – öppen standard. HSRP = **Hot Standby Router Protocol** – Cisco. Båda ger en virtuell gateway-IP med automatisk failover mellan noder.); verifiera att ping/trafik fortsätter (mät "hit").
- **DNS-redundans:** stoppa ena resolvern; mät namnupplösning och TTL-effekt.
- **DHCP-failover:** stäng en server; bevisa utdelning/lease-förnyelse.
- **WLAN-kontroller/AP:** simulera kontroller-fel; klienter ska återansluta.
- **Proxmox/lagring:** dra ner en nod/länk; VM/IO ska fortsätta enligt design.
- **Internet dual-homing:** BGP/SD-WAN/backup-länk tar över; verifiera NAT/policy.

Exempel: Planerat test "Gateway-failover Q3": kl 18:00 sänks prioritet på Dist-A (VRRP). Dist-B tar över inom 1 s. Övervakning visar 1 förlorad ping på elev-VLAN, 0 på personal (SLA uppfyllt). Slutsats: höj "track" på internetporten för snabbare detektion.

DR-plan (kärnpunkter):

- **Utlösare & roller:** vem deklarerar DR-läge, vem leder.
- **Prioritering:** Tier 0 → Tier 1 → Tier 2 (lista system i ordning).
- **Miljö:** isolerat återställnings-VLAN, temporära lösenord, säkra kanaler.
- **Data:** var finns senaste *grön-testade* backupen (hash/rapport).
- **Checklista:** återställ AD/inloggning (om tillämpligt), DNS/DHCP, fil/resurs, sedan övriga.
- **Exit-kriterier:** när lämnar vi DR-läge; post-mortem inom 5 dagar.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel: Ransomware-övning: filserver låtsas krypterad. DR-teamet isolerar servern, återställer gårdagens backup till ny VM i karantän-VLAN, kör antivirus, verifierar fil-hashar, svänger om DFS-pekare. Tid från "start DR" till "tjänst åter" = **2 h 40 min** *(inom RTO 4 h).*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Beskriv de sex stegen i en enkel incidentprocess och syftet med varje steg.
2. Vilka roller vill du alltid bemanna i en P1/P2-incident och varför?
3. Vad ska en **incidentrapport** minst innehålla för att vara användbar i efterhand?
4. Förklara RPO och RTO och ge ett konkret exempel från skolmiljö.
5. Varför räcker inte snapshots som backup, och när är de ändå värdefulla?
6. Redogör för 3-2-1(-1-0)-regeln och vad “-1-0” betyder i praktiken.
7. Vad innebär app-konsistenta backuper (t.ex. VSS) och när krävs de?
8. Lista fem redundanstester du bör köra varje termin och hur du verifierar utfallet.
9. Vad är skillnaden mellan **incidenthantering** och **disaster recovery**?
10. Nämn tre vanliga förbättringar som brukar komma ur en bra post-mortem.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- NIST SP 800-61r2 – Computer Security Incident Handling Guide: https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final
- ISO/IEC 27035 – Information security incident management (översikt): https://www.iso.org/standard/44379.html
- CIS Controls – Incident Response & Recovery: https://www.cisecurity.org/controls
- CISA – Ransomware Guide (beredskap & återställning): https://www.cisa.gov/stopransomware
- 3-2-1 Backup Strategy (översikter, leverantörs-agnostiskt): https://www.backupstrategy.com/3-2-1
- Microsoft – Backup best practices (Windows/Server, VSS): https://learn.microsoft.com/windows-server/storage/backup
- Proxmox Backup Server – docs: https://pbs.proxmox.com/docs/
- Veeam – SureBackup / testade återställningar (koncept): https://helpcenter.veeam.com/docs/backup

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

5 Fördjupningsdel

5.1 Virtualisering – plattformar och gemensamma mönster

5.1.1 Varför virtualisera? (problembakgrund och mål)

Virtualisering blev standard därför att den **löser flera problem samtidigt**: hårdvaran utnyttjas bättre (fler servrar per fysisk värd), **isolation** mellan tjänster ökar driftsäkerheten, **snabb återställning** blir möjlig med snapshots/kloner/backup, och **automation** (skript, mallar, "infrastruktur som kod") gör att du kan bygga och återskapa miljöer på minuter istället för dagar. För skola och SMB betyder det att labbar och skarp drift kan leva sida vid sida, att uppgraderingar testas **innan** de rullas ut, och att enskilda fel inte behöver ta ner hela miljön. Virtualisering förenklar också **disaster recovery** – när servrar är filer/diskavbilder kan de flyttas och startas på annan hårdvara.

Vanliga drivkrafter: konsolidering och kostnad, snabb provisionering, enklare DR/backup, bättre säkerhetszonering, samt möjligheten att **skala och ändra** utan att byta fysisk hårdvara varje gång.

Exempel (Exempel:*) *Inför nationella prov körs en separat provtjänst i egen VM. Efter provperioden tas en snapshot för arkiv, och resurserna (CPU/RAM) återgår till ordinarie drift.*

5.1.2 Gemensam arkitektur (oavsett plattform)

Alla moderna plattformar delar samma byggstenar: **hypervisor** (kärnan som kör VM:er), **virtuella switchar/bridges** som kopplar VM:er mot nätet, **lagring** (lokal eller delad), och **hantering** (webb-UI/CLI/API). Du arbetar med **mallar** (golden images), **nätprofiler** (VLAN/portgrupper), samt **policies** för resurser och säkerhet. Två viktiga principer:

1. **Snapshots ≠ backup** (snapshot är kortlivad säkerhetslina; backup är separata, helst immutabla/air-gappade kopior).
2. **Segmentera** trafik: management, lagring, backup och produktion i egna VLAN/subnät och med tydliga brandväggsregler.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel (Exempel:*) *Management-VLAN nås bara via bastion/jump host; backupflöden går i separat VLAN så att en överbelastad applikationslänk inte stör backupfönstret.*

5.1.3 Plattformerna (bakgrund, styrkor och begränsningar)

Proxmox VE (KVM/QEMU + LXC, Debianbaserad)
Proxmox bygger på **Debian Linux** och använder KVM/QEMU för fullvirtualisering (bra för Windows Server) samt **LXC** för lätta Linux-containrar. Styrkorna är **öppen källkod**, snabb utveckling, **VLAN-aware Linux bridges**, **ZFS**-stöd, klusterhantering och **Proxmox Backup Server** (dedupe, verifiering, immutability-policy). Passar skolor/labbar/SMB som vill ha kraftfullt och kostnadseffektivt utan licenslåsning. Begränsningar: kräver Linux-vana, färre “färdigpaketerade” enterprise-ekosystem än vSphere.

VMware vSphere/ESXi (proprietär standard i många företag)
vSphere består av **ESXi** (hypervisor) och **vCenter** (central hantering). Styrkorna är **mogen drift**, enormt ekosystem (backup, övervakning), **vMotion/DRS** (liveflytt/lastbalansering) och **Distributed vSwitch**. Lämpligt när du behöver beprövad, stor skala och certifierat stöd från många leverantörer. Begränsningar: licenskostnad, beroende av VMware-ekosystemet; avancerade funktioner kräver rätt edition.

Microsoft Hyper-V (Windowsintegrerad hypervisor)
Hyper-V är integrerad i **Windows Server** (roll) och har **virtual switch**, **VHDX**, **Checkpoints** och bra **PowerShell**-stöd. Styrkor: tajt integration med Windows, AD/GPO, SMB3/Storage Spaces, och gott stöd i många backupverktyg. Lämpligt om ni redan är “Windows-tunga” och vill hålla verktygen enhetliga. Begränsningar: mindre ekosystem för Linux-containers, vissa funktioner kräver Datacenter-licens; Microsofts fristående Hyper-V Server är utfasad (fokuset ligger på Windows Server-rollen).

XCP-ng (Xen-baserad, öppen)
XCP-ng bygger på **Xen** och erbjuder enterprise-känsla i open-source-tappning, med **Xen Orchestra** för hantering/backup (GUI/SaaS eller self-hosted). Styrkor: bra balans mellan enkelhet och kapacitet, stabil hypervisor, vettiga snapshot/backupflöden, och aktiv community.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Begränsningar: färre “plug-and-play”-integrationer än VMware, viss inlärningskurva kring Xen-terminologi.

libvirt/KVM “ren” (t.ex. virt-manager/terraform-libvirt)
Det “rena” KVM/libvirt-spåret ger maximal kontroll (nära Linux), med **bridge/OVS**, **cloud-init**, och automation via Ansible/Terraform. Styrkor: **full transparens**, inga licenser, perfekt för djup Linuxlabb.
Begränsningar: kräver mest **Linux-kompetens**; färre bekvämligheter out-of-the-box för större team.

Exempel (Exempel:*) *En kommunal IT-avdelning med Windows-tyngd kör Hyper-V för skarp drift (AD, fil, print) men använder Proxmox för kurslabb och snabb prototypning. En partner kör vSphere för “tunga” produktionstjänster med höga SLA, och XCP-ng på filialkontor.*

5.1.4 Nätmönster: bridge, trunk, VLAN och NAT

Oavsett plattform möter du tre mönster:
Bridge till produktion (VLAN-aware): uplink som **trunk** från switchen; skapa portgrupp/bridge som *bär flera VLAN* och tilldela **VLAN-ID** per VM-interface.
Isolerat NAT-nät för labb: en virtuell switch/bridge med NAT mot värden eller särskild edge-VM; perfekt för att låta elever labba utan att röra produktion.
Dedikerad management/backup/storage-väg: separata nät för admin, backup och lagring (t.ex. iSCSI/NFS) för prestanda/säkerhet.

Exempel (Exempel:*) *En webb-VM i DMZ får vNIC på VLAN 70; brandvägg släpper endast 80/443 in och 443 till intern API-server. En andra vNIC på VLAN 99 används enbart för backupflöden till backupservern.*

5.1.5 Lagring, snapshots och backup

Snapshots fryser VM-disk (och ev. minne) på primär lagring; de är perfekta inför ändringar men ska **rensas** efter test. **Backup** skapar en **separerad kopia** (helst **immutabel** och/eller **air-gappad**) som inte påverkas om primär lagring skadas eller krypteras.
Proxmox + **Proxmox Backup Server** ger dedupe/komprimering/verifiering. vSphere har stort stöd i verktyg som

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Veeam. Hyper-V fungerar med VSS och Windows-/tredjepartsbackup. XCP-ng har XO/XOA-backup. Libvirt/KVM kan använda borg/restic/rsync-strategier och “agent-i-gäst” för app-konsistens.

Exempel (Exempel:*) *Inför patch: snapshot → patch → test → rensa snapshot; ordinarie nattbackup (full + inkrementell) fortsätter och* **restore-test** *körs varje månad i isolerat VLAN.*

5.1.6 Hög tillgänglighet, kluster och DR

Kluster låter dig hantera flera noder som en enhet (gemensam vy, policyer). **Live migration** flyttar VM utan avbrott vid underhåll. **HA** startar automatiskt om VM:er på annan nod vid fel. För **DR** (större avbrott) behövs replikerad backup, dokumenterad återstartsordning (Tier 0→1→2) och regelbundna **spärrtester** (se kapitel 4.4).

Exempel: *Kvällsunderhåll: vMotion/Live Migration flyttar VM bort från noden som ska patchas. Under driftfel startar HA upp kritiska VM på frisk nod; användarna märker en kort paus.*

5.1.7 Resurser och licenser (CPU, RAM, disk, Windows Server)

Allokera resurser **konservativt** först och mät. **Overcommit** kan fungera för CPU men var försiktig med RAM och I/O-tunga diskar. Planera **nätprofiler** (vNIC-antal/roller) och håll dig till namngivning/mallar för spårbarhet.
Tänk även på **licenser**: Windows Server **Datacenter** ger rätt att köra obegränsat antal Windows-VM:er på licensierad hårdvara; **Standard** ger rätt för ett mindre antal VM:er per licens. Detta kan styra var du placerar Windows-tunga arbetslaster.

Exempel: *Filserver-VM: 2 vCPU, 8–12 GB RAM (6–8 GB garanterat), virtio-SCSI + iothread, två vNIC (produktion/backup). Justera efter mätvärden i övervakningen.*

5.1.8 Säkerhet och driftetikett

Separera **management** från **produktion**, använd **MFA** och **bastion/jump host** för adminåtkomst, och logga alla förändringar. Håll **brandväggspolicy** mellan VLAN/zoner även i labb. Vid WLAN-anslutna

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

klienter som når VM-miljön: säkerställ rollbaserade brandväggsregler och övervaka ovanliga flöden.

Exempel: *Admin når hypervisorer via VPN → bastion i management-VLAN → SSH/RDP vidare till värdar/VM. Inga öppna adminportar mot internet.*

5.1.9 Jämförelsetabell (styrkor, när använda, begränsningar)

Proxmox (Debian)

- **Nätmodell (VLAN/NAT):** Linux bridge (VLAN-aware), trunk på uplink, NAT-bridge för labb, bond/LACP.
- **HA/Live migration:** Kluster med Proxmox HA; live migration (delad lagring eller replikering).
- **Backup-ekosystem:** Proxmox Backup Server (dedupe, verifiering, immutability-policy) + 3:e-partsverktyg via standardprotokoll.
- **Styrkor:** Öppen källkod, kostnadseffektiv, LXC + KVM, ZFS-stöd, starkt API/community.
- **Begränsningar:** Kräver mer Linuxvana; färre “stora” certifierade integrationer än vSphere.
- **Lämplighet:** Skola/SMB/labb; mixad Win/Linux; där budget, flexibilitet och transparens väger tungt.

VMware vSphere/ESXi

- **Nätmodell (VLAN/NAT):** vSwitch/Distributed vSwitch, port groups (VLAN-ID), NAT via Edge-/gateway-VM eller NSX.
- **HA/Live migration:** Mycket moget: vMotion/Storage vMotion/DRS/HA.
- **Backup-ekosystem:** Stort (VADP-stöd); Veeam m.fl. med bred funktionalitet.
- **Styrkor:** Mognad, skala, brett ekosystem, leverantörsstöd/certifieringar.
- **Begränsningar:** Licenskostnader och editioner; viss inlåsning i ekosystemet.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Lämplighet:** Större/kritiska miljöer med hårda SLA och krav på certifierade integrationer.

Microsoft Hyper-V (Windows)

- **Nätmodell (VLAN/NAT):** Virtual Switch med VLAN-ID, NIC Teaming; NAT via RRAS/Windows NAT eller gateway-VM.
- **HA/Live migration:** Failover Clustering, Live/Storage Migration.
- **Backup-ekosystem:** VSS-integration; Windows Server Backup, Veeam m.fl.
- **Styrkor:** Tajt Windows/AD-integration, PowerShell/automation, låg tröskel i MS-miljö.
- **Begränsningar:** Vissa funktioner kräver Datacenter-licens; mindre fokus på Linux/containers; fristående Hyper-V Server utfasad.
- **Lämplighet:** Windows-tung drift (AD, fil/print, RDS); skolor/organisationer med MS-stack.

XCP-ng (Xen)

- **Nätmodell (VLAN/NAT):** Networks med VLAN-taggning; NAT via gateway-VM (t.ex. OPNsense).
- **HA/Live migration:** Live migration/HA med delad lagring; orkestreras via Xen Orchestra (XO/XOA).
- **Backup-ekosystem:** Inbyggt i XO/XOA (delta, replikering) + 3:e-partsverktyg.
- **Styrkor:** Open source med “enterprise-känsla”, enkel drift, bra GUI (XO).
- **Begränsningar:** Mindre ekosystem än VMware; vissa features beroende av XO.
- **Lämplighet:** Filial/SMB; behov av HA/backup utan stora licenskostnader.

libvirt/KVM “ren”

- **Nätmodell (VLAN/NAT):** Linux bridge/OVS, trunking; NAT via `virbr0`/dnsmasq eller egen brandvägg.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **HA/Live migration:** Möjligt (Pacemaker/Corosync/virt-tools), men mer “bygg själv”.
- **Backup-ekosystem:** DIY: borg/restic/rsync + `qemu-img`/virsh; app-konsistens via agenter/skript.
- **Styrkor:** Maximal kontroll, inga licenser, ideal för automation/DevOps/labb.
- **Begränsningar:** Mest handpåläggning och Linuxkompetens; färre bekvämligheter för större team.
- **Lämplighet:** Labb/utbildning, CI/CD, speciallösningar där full kontroll prioriteras.

5.1.10 Snabba “recept” per plattform

Proxmox (Debian-bas):
VM på rätt VLAN: Trunka uplink, sätt `vmbr0` **VLAN-aware**, ge vNIC **VLAN-tagg**.
NAT-labb: Skapa `vmbrN` med NAT; lägg kurs-VM:er där.
Snapshot + rollback: Snapshot → ändra → testa → rensa.
Enklast backup: PBS-target, daglig inkrementell + månatlig verifiering.

VMware vSphere/ESXi:
VM på rätt VLAN: Skapa **port group** med VLAN-ID, koppla vNIC dit.
NAT-labb: Använd Edge-VM (pfSense/OPNsense) eller NSX-Edge för NAT.
Snapshot + rollback: Snapshot via vCenter; rensa efter test.
Enklast backup: Veeam-jobb mot vCenter, GFS-retention, regelbunden restore-test.

Hyper-V (Windows):
VM på rätt VLAN: **Virtual Switch** + VLAN-ID på vNIC.
NAT-labb: Intern switch + Routing/NAT-roll i en liten gateway-VM.
Checkpoint + rollback: Checkpoint före ändring; “Apply”/Delete när klar.
Enklast backup: Windows Server Backup eller Veeam med VSS-konsistens.

XCP-ng:
VM på rätt VLAN: Skapa network med VLAN-tagg, koppla vNIC.
NAT-labb: Gateway-VM för NAT (t.ex. OPNsense).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Snapshot + rollback: Via XO/XOA, rensa när klart.
Enklast backup: XO-backup (delta), regelbunden verifiering.

libvirt/KVM:
VM på rätt VLAN: Linux bridge/OVS + trunk uthamnad; vNIC med VLAN-tagg.
NAT-labb: `virbr0` (default NAT) eller egen `dnsmasq`/firewalld-NAT.
Snapshot + rollback: `qemu-img`/virsh snapshot.
Enklast backup: stoppa-konsistens + rsync/borg/restic; för app-konsistens använd agent/scripting.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Ge tre skäl till att virtualisera som direkt påverkar tillgänglighet och arbetstakt.
2. Förklara skillnaden mellan **snapshot** och **backup** och när respektive används.
3. Varför bör management, lagring, backup och produktion separeras nätmässigt?
4. När väljer du Proxmox framför vSphere – och tvärtom?
5. Vad talar för Hyper-V i en Windows-tung skolmiljö?
6. Hur sätter du en VM på rätt VLAN i vSphere, Hyper-V respektive Proxmox (kort steg för steg)?
7. Vad innebär **overcommit** och vilka risker finns för RAM och I/O?
8. Nämn två sätt att skapa ett **isolerat NAT-labb** utan att påverka produktion.
9. Vilka komponenter behövs för **HA + live migration** i praktiken?
10. Hur skulle du licensiera Windows-tunga VM-värdar kostnadseffektivt (Standard vs Datacenter)?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- Proxmox VE docs: https://pve.proxmox.com/pve-docs/
- VMware vSphere docs: https://docs.vmware.com/
- Microsoft Hyper-V docs: https://learn.microsoft.com/windows-server/virtualization/hyper-v/
- XCP-ng & XO: https://xcp-ng.org/ och https://xen-orchestra.com/
- libvirt/KVM: https://libvirt.org/ och https://qemu.readthedocs.io/
- Linux bridge/OVS grunder: https://wiki.linuxfoundation.org/networking/bridge

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

5.2 Fördjupning: Windows Server och GPO

5.2.1 Vad är GPO – och var börjar man?

Tänk **Group Policy (GPO)** som en central **regibok** för Windows-klienter och -servrar. Du beskriver hur datorer och användarsessioner ska se ut och bete sig, och **domänen** ser till att det händer — automatiskt och konsekvent. Det kan vara allt från skrivbordsdetaljer och genvägar till säkerhet, kryptering, brandvägg, programvaror och uppdateringar. Vitsen är **förutsägbarhet**, **spårbarhet** och **fart**: samma in, samma ut, utan handpåläggning på varje maskin.

Du arbetar i **Group Policy Management Console (GPMC)**. Där ser du skog → domän → **Group Policy Objects** (själva "dokumenten") och **OU:er** (behållare där du **länkar** dem). Öppnar du en GPO möts du av två huvuddelar:

- **Computer Configuration** (gäller datorn oavsett vem som loggar in).
- **User Configuration** (gäller användarsessionen på den datorn).

En bra start är att skapa en enkel GPO (t.ex. standardbakgrund), **länka** den till en test-OU och se hur inställningen rullar ut. För att alla admins ska se samma policyspråk och versioner lägger man **administrativa mallar** (ADMX/ADML) i en **Central Store** i SYSVOL. Då blir det lättare att samarbeta och undvika "det såg inte ut så på min dator".

Exempel: *Skapa "GPO-Desktop-Standard". Sätt ett neutralt skrivbordsunderlägg och dölja vissa kontrollpaneler. Länka till en test-OU med några datorer. Vid nästa uppdatering får användarna samma skrivbordsmiljö — inga manuella klick.*

5.2.2 Hierarki och utvärdering – vem "vinner" när policies krockar?

Windows tillämpar policy i ordningen **Lokal → Site → Domain → OU** (L-S-D-O-U). Varje steg kan lägga till eller skriva över inställningar. I en och samma behållare styr även **Link Order**: länken som ligger **sist** kör **sist** — och vinner vid krock.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Två “brytare” påverkar arvet:

- **Enforced (No override):** tvingar igenom en länkad GPO nedåt även om någon längre ner försöker skriva över den.
- **Block Inheritance:** stoppar arv från ovan — men **Enforced** går igenom ändå.

Särfallet **Loopback Processing** gör att **användarinställningar** kan tas från **datorns OU**. Det är ovärderligt för delade maskiner, kiosker och VDI.

- **Merge:** användarpolicyn kombineras; datorns OU kan överstyra detaljer.
- **Replace:** användarpolicyn kommer enbart från datorns OU.

Exempel: *En domänlänkad GPO “Säkerhetsbas” sätter standardregler. I en viss server-OU finns “RDP-Tilldelning”. Om de krockar vinner OU-länken — om inte “Säkerhetsbas” är* **Enforced***.*

Bakgrundsnot: Klienter uppdaterar i bakgrunden med jämna mellanrum. Viss funktionalitet (t.ex. programvaruinstallation) sker dock vid **start/inloggning**. `gpupdate /force` triggar en omedelbar uppdatering, men kräver ibland omstart för att allt ska slå igenom.

5.2.3 Från lätt till tungt – vad kan du styra med GPO?

Börja med **det synliga** — det ger snabb bekräftelse på att kopplingar och arv fungerar — och bygg sedan upp mot säkerhet och drift.

- **Administrative Templates:** färdiga policyer som i grunden skriver välkända registernycklar. Exempel: skrivbordsunderlägg, kontrollpaneler, webbläsarpolicyer, Onedrive/Office-inställningar.
- **Group Policy Preferences (GPP):** lägger till filer, mappar, genvägar, drivmappar och skrivare. Med **Item-Level Targeting** kan du villkora per **grupp**, **OU**, **OS-version**, **plats**, **IP-område**, **datornamn** m.m. Resultatet blir samma precision som skript — men mer robust.
- **Security Filtering:** låt bara en viss **säkerhetsgrupp** påverkas av GPO:t.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **WMI-filter:** träffa t.ex. bara klient-OS, eller bara bärbara (har batteri). Använd sparsamt — många eller tunga filter ökar inloggningstid.

Exempel: *En GPP-drivmapp syns bara för gruppen "Ekonomi"* **och** *endast på datorer i "OU=Ekonomi". Ingen annan påverkas, även om GPO:t är länkat högre upp.*

5.2.4 Skript via GPO – när är Startup/Logon rätt, och när är Schemalagd uppgift bättre?

Startup/Shutdown körs i **dator-kontekst** (före/efter inloggning) som **Local System**. Perfekt för maskinnära saker: registry, tjänster, protokoll, filkopior som kräver admin.
Logon/Logoff körs i **användarsession** och passar profilnära uppgifter: genvägar, små städjobb, användar-specifika registerjusteringar.

Vill du ha **retry, fördröjning, villkor, tidsgränser** och bättre loggning använder du **Scheduled Tasks via GPP**. Kör som **SYSTEM** eller **tjänstkonto**, trigga "At startup", "At logon" eller vid schema. Lägg skript i **SYSVOL** (gemensamt, versionsstyrt) och styr **Execution Policy**/kodsignering i GPO. Tänk på att nätverksresurser kanske inte finns tillgängliga precis i uppstart — vill du garantera det aktiverar du policyn *Always wait for the network at computer startup and logon* i en bas-GPO.

Exempel: *Ett Startup-skript stänger av SMBv1 och sätter NTP-källa. En Schemalagd uppgift (SYSTEM) kör varje natt och rensar temporära mappar; resultatet loggas i Event Viewer.*

Litet skript-exempel (Startup, körs som SYSTEM)
Exempel: *Lägg i* `\\<domän>\SYSVOL\<domän>\Scripts\Startup\Hardening.ps1` *och kör via* **Computer Configuration → Policies → Windows Settings → Scripts (Startup/Shutdown)** *eller som* **Scheduled Task (At startup)**.

```
# Hardening.ps1 - körs som Local System vid startup

# 1) Stäng av SMBv1 (klientkomponent)
Try { Disable-WindowsOptionalFeature -Online -FeatureName SMB1Protocol -
NoRestart -ErrorAction Stop } Catch { }

# 2) Stäng av SMBv1 (serversidan)
```

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

```
Try { Set-SmbServerConfiguration -EnableSMB1Protocol $false -Force } Catch
{ }

# 3) Sätt NTP-källa (ersätt med er tidskälla)
$ntp = 'time.example.org,0x8'
w32tm /config /syncfromflags:manual /manualpeerlist:$ntp /update | Out-Null
w32tm /resync /force | Out-Null

# 4) Logga enkel status
$msg = "Hardening klar: SMBv1 avstängd, NTP=$ntp"
Write-EventLog -LogName Application -Source "Windows PowerShell" -EntryType
Information -EventId 1001 -Message $msg
```

5.2.5 Programvarudistribution – strategier, val och fallgropar

Utmaningen: Program kommer som **MSI**, **EXE** eller **MSIX**. De har olika tysta switchar, kräver olika **detektion** (hur vet vi att appen redan finns?), har beroenden (drivrutiner, VC++-omfördelningsbara), och vissa kräver **omstart**. En bra strategi ger: spårbarhet (loggar/status), **piloter** (testgrupper), **ringar** (stegvis utrullning) och **rollback**.

A) GPO + MSI (enkelt och stabilt)
Med **Software Installation** i GPO kan du **Assign to Computer** ett **MSI**. Installationen sker vid start, innan inloggning, vilket är robust. Lägg källan på **UNC-share** med läsrätt för datorer. Anpassa via **MST** (transform). Begränsningar: främst MSI, svagt för EXE, enkel rapportering.

B) GPO + skript/GPP (för EXE och specialfall)
Kör EXE med tyst flagga (`/quiet`, `/S`, `/VERYSILENT`) via **Startup-skript** eller **Scheduled Task** (SYSTEM). Bygg in **detektion** (kontrollera fil/reg-nyckel, kör inte igen om det redan finns) och **logga** exit-koder. Bra för mindre/medel miljö — men standardisera loggformat, uninstall och rollback.

C) Intune (MDM) – ringar, självbetjäning och internetnoder
Paketering till **.intunewin**, definiera **install/uninstall**-kommandon, **detektionsregler** och **beroenden**. Kör mot användare eller enheter, gör **Required** (tvingad) eller **Available** i portal. Styrka: ringar, självbetjäning, funkar utanför LAN.

D) Configuration Manager (SCCM/MECM) – djup kontroll on-prem
Full kontroll över **applikationer**, **samlingar**, **phased deployments**,

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

detaljerad status och rapporter. Passar större on-prem-miljöer med höga krav på precision och uppföljning.

Gemensamma råd:

- Ha en **paketstandard** (katalognamn, logg, detektion, uninstall).
- Skilj **installationskonto** från **detektionslogik**; detection ska vara snabb och pålitlig.
- Undvik "eviga" installationsförsök genom att **bryta** på tydliga exit-koder.
- Planera **omstarter** (t.ex. via deadlines eller nattfönster).
- Kör **pilot** först; utvärdera innan du breddar.

Hands-on: MSI via GPO (Assigned to Computer)

1. Lägg `App.msi` (ev. `App.mst`) på `\\fileserver\software\App\` och ge **Read** för *Domain Computers*.
2. Skapa GPO "GPO-Apps-App".
3. Öppna GPO → **Computer Configuration → Policies → Software Settings → Software Installation** → *New* → *Package...* → peka på **UNC** → **Assigned**.
4. (Valfritt) Lägg `App.mst` under *Modifications*.
5. **Länka** GPO:t till mål-OU (datorer).
6. Sätt *Always wait for the network at computer startup and logon* = **Enabled** i bas-GPO.
7. Testa: `gpupdate /force`, **omstart**, verifiera i loggar/Program och funktioner.
8. Uppgradera via *Upgrades*-fliken, eller ta bort med **Remove** (ev. *Immediately uninstall*).

Exempel: *"GPO-Apps-Reader" installerar Adobe Reader tyst från* `\\fileserver\software\Reader\AcroRead.msi`*, med* `AcroRead.mst` *för språk och avstängd auto-update.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

5.2.6 Centrala uppdateringar – WUfB, WSUS och ringar

Målet är att få en trygg rytm: små säkra steg ofta, och stora steg när du är redo — med **ringar** som fångar fel tidigt.

- **Windows Update for Business (WUfB):** Klienter hämtar från Microsoft Update, men du styr **deferral** (hur länge de väntar), **deadlines** (när de måste installera/boota) och **ringar** (Pilot → Bred → Sen). Fungerar utmärkt för enheter som inte alltid är på LAN.
- **WSUS:** Du **godkänner** uppdateringar centralt och styr vilka **datorgrupper** som får vad — när. Bra när du behöver lokal cache, strikt kontroll eller avskilda nät.

Tänk i två flöden:

- **Quality Updates** (månatliga säkerhetsfixar): rulla snabbt genom ringar med kort deferral.
- **Feature Updates** (större OS-lyft): längre deferral och tydligare pilot.

Glöm inte **Delivery Optimization** så klienter delar paket i LAN; det sparar internetkapacitet. Bestäm hur du hanterar **drivrutiner**, **.NET** och **Microsoft 365 Apps** (de kan ha egna kanaler).

Hands-on: WUfB ring-setup (Pilot → Bred → Sen)
A. Grupper och GPO:n

1. Skapa tre **säkerhetsgrupper**: `WUfB-Pilot`, `WUfB-Bred`, `WUfB-Sen`.
2. Skapa tre **GPO:n**: “WUfB-Pilot”, “WUfB-Bred”, “WUfB-Sen”. Länka dem högt upp, men låt **Security Filtering** styra att rätt grupp får rätt GPO.

B. Policys (Computer Configuration → Administrative Templates → Windows Update)

1. **Select when Feature Updates are received** = **Enabled** → *Deferral days*: Pilot **0**, Bred **7–14**, Sen **21–28**.
2. **Select when Quality Updates are received** = **Enabled** → *Deferral days*: Pilot **0**, Bred **2–7**, Sen **10–14**.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

3. **Specify deadlines for automatic updates and restarts** = **Enabled** (sätt rimliga deadlines och grace).
4. **Configure Automatic Updates** = **Enabled** (auto-nedladdning och schemalagd install).
5. **Delivery Optimization**: sätt läge för LAN-delning.
6. Har ni haft **WSUS** tidigare? Säkerställ att intranät-WSUS **inte** längre pekas ut i dessa GPO:n (WUfB och WSUS samtidigt skapar konflikt).

C. Rulla

1. `gpupdate /force` på Pilotenheter. Kontrollera att inställningarna syns på Windows Update-sidan.
2. Följ upp status med dina verktyg (oavsett om det är Intune, SCCM eller egen inventering).
3. När piloten är grön: flytta fler enheter in i **Bred**. Upprepa mot **Sen**.

Exempel: *Pilot har 0 dagars deferral och 7 dagars deadline. Bred har 14/14. Sen har 28/21. Om något fallerar i Piloten pausar du ringen eller höjer deferral i Bred/Sen tills fix finns.*

5.2.7 Felsökning – se vad som faktiskt gäller

Börja alltid med **vad klienten tror är sant**:

- `gpresult /h rapport.html` visar **vilka GPO:er** som tillämpats (för dator och användare), vilka som filtrerats bort (säkerhet/WMI) och varför.
- **RSOP.msc** ger en snabb översikt av “Resultant Set of Policy”.
- **Event Viewer → Microsoft → Windows → GroupPolicy/Operational** avslöjar timing och fel.
- I **GPMC** kan du **modellera** förväntat utfall (vilka GPO som *bör* gälla) och jämföra med **Results** från en faktisk dator.

Vanliga bovar:

- **Fel OU/ingen länk** → klienten nås inte av GPO:t.
- **Security Filtering** träffar inte datorn/användaren.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **WMI-filter** för snävt eller långsamt.
- **Nätverk inte klart vid boot** → programvaruinstall missar (aktivera "Always wait for the network…").
- **AD-replikering** släpar → testa på DC där klienten hämtar policy.
- **SYSVOL-behörigheter**/delningsproblem → MSI-källa når inte fram.

Exempel: *En skrivare mappas inte.* `gpresult` *visar att GPO:t filtrerats bort av ett WMI-filter för "Bärbar". Datorn är stationär. Lösning: flytta villkoret till* **Item-Level Targeting** *(OU/grupp) eller justera WMI-villkoret.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Vad vinner du på att styra Windows via GPO istället för att manuellt konfigurera varje maskin?
2. Förklara **L-S-D-O-U** och hur **Enforced** respektive **Block Inheritance** påverkar utfallet.
3. När väljer du **Loopback (Merge/Replace)** och hur påverkar det användarpolicyn?
4. Ge ett exempel där du kombinerar **Administrative Templates**, **GPP** och **Item-Level Targeting**.
5. När är **Startup/Logon-skript** rätt val, och när är **Scheduled Task via GPP** bättre?
6. Beskriv ett komplett **MSI via GPO**-flöde och hur du planerar **uppgradering/rollback**.
7. Hur hanterar du **EXE-appar** centralt med detektion och loggning — utan tredjepartsverktyg?
8. Vad skiljer **WUfB** från **WSUS**, och när väljer du det ena framför det andra?
9. Hur designar du **ringar** (Pilot → Bred → Sen) för både **Quality** och **Feature** Updates?
10. Bygg en enkel felsökningskedja när ett GPO inte tillämpas (vilka verktyg, i vilken ordning, och varför).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

Group Policy – översikt: https://learn.microsoft.com/windows-server/identity/ad-ds/manage/group-policy/group-policy-overview

Group Policy processing (LSDOU, Enforced/Block): https://learn.microsoft.com/windows-server/identity/ad-ds/manage/group-policy/group-policy-processing

Loopback processing: https://learn.microsoft.com/troubleshoot/windows-server/group-policy/loopback-processing-of-group-policy

Group Policy Preferences & Item-Level Targeting: https://learn.microsoft.com/windows-server/identity/ad-ds/manage/group-policy/group-policy-preferences

WMI-filter – skapa och använda: https://learn.microsoft.com/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/jj717288(v=ws.11)

Installera program via GPO (MSI): https://learn.microsoft.com/troubleshoot/windows-server/group-policy/use-group-policy-to-install-software

Intune – Win32 app management: https://learn.microsoft.com/intune/intune-service/apps/apps-win32-app-management

Windows Update for Business – konfigurera: https://learn.microsoft.com/windows/deployment/update/waas-configure-wufb

WSUS – kom igång: https://learn.microsoft.com/windows-server/administration/windows-server-update-services/get-started/windows-server-update-services-wsus

gpresult – visa effektiv policy: https://learn.microsoft.com/windows-server/administration/windows-commands/gpresult

Central Store för ADMX/ADML: https://learn.microsoft.com/troubleshoot/windows-client/group-policy/create-and-manage-central-store

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

5.3 Fördjupning: Linux-server och policy-styrning

5.3.1 Varför Linux på server?

Om man vill bygga stabila, resurssnåla och flexibla serversystem är Linux ofta ett naturligt val. Man får stor kontroll över vilka tjänster som körs, kan automatisera nästan allt och skala från en liten VM till ett helt kluster. Licensmodellen (öppen källkod eller abonnemang) gör också att ekonomi och support kan anpassas efter verksamheten. Tanken är att beskriva hur systemet **ska** se ut och uppföra sig, och sedan låta verktygen upprepa det — varje gång, på varje server.

Exempel: En gemensam "bas" med samma SSH-policy, brandvägg och loggning används för flera servrar. Varje maskin får sedan sin roll (webb, fil, proxy), men grundbeteendet är likadant och enkelt att återskapa.

5.3.2 Grunden: distribution, paket, konton och baspolicy

Om man är ny och ovan vid att sätta upp Linuxservrar är det lämpligt att välja en **LTS-distribution** (t.ex. Ubuntu Server LTS eller Debian Stable; Rocky/Alma om man vill ha RHEL-kompatibilitet). LTS brukar innebära längre säkerhetsstöd och lugnare förändringstakt. Paket hanteras typiskt via **APT** (Debian/Ubuntu) eller **DNF** (RHEL-familjen); många upplever att det blir tryggast att i första hand hålla sig till distributionens egna arkiv.

När det gäller **konton och rättigheter** börjar många med vanliga användarkonton och låter administratörsrätt gå via **sudo**. Det kan kännas tryggt att logga förhöjda kommandon, så att man i efterhand kan se vad som körts och av vem. Åtkomsten sker ofta via **SSH**; då väljer många att använda **nycklar** i stället för lösenord och, där riskbilden motiverar det, komplettera med tvåfaktor via PAM.

Baspolicy i praktiken: tidsynk (chrony eller systemd-timesyncd), en enkel brandvägg (ufw eller firewalld), ett MAC-skydd (AppArmor eller SELinux), central loggning (rsyslog) och förutsägbar loggrotation. Det är inte komplicerat — men gör stor skillnad över tid.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel: En nysatt server får ett administratörskonto med sudo, SSH-nyckelinloggning, brandvägg som bara öppnar 22/80/443, tidsynk mot intern NTP, och loggar som skickas till en central loggtjänst.

5.3.3 Nätverk på Linux: IP, VLAN, bond och brygga

När du ska konfigurera nätverket i en Linuxserver är det viktigt att man håller sig till **ett** konfigurationssätt (netplan/systemd-networkd, NetworkManager eller distributionsspecifika filer) och inte blandar. Det underlättar felsökning och minskar risken för konflikter där flera komponenter försöker göra samma sak. I serverdrift handlar nätet ofta om några återkommande byggstenar: **statisk IP** med gateway/DNS, **VLAN** för segmentering, **bonding/LACP** för redundans/kapacitet, och ibland en **bridge** om värden ska husera KVM-VM:er eller containrar.

Exempel: En webb-VM i DMZ (VLAN 70) pratar bara med reverse-proxyn. Backuptrafik går på ett andra interface i VLAN 99 till backupservern — skilt från produktionstrafiken.

Så här kan en minimal netplan-definition se ut (statisk IP + VLAN):

```
# /etc/netplan/01-server.yaml
network:
  version: 2
  ethernets:
    eno1:
      dhcp4: no
  vlans:
    vlan40:
      id: 40
      link: eno1
      addresses: [192.0.2.10/24]
      gateway4: 192.0.2.1
      nameservers:
        addresses: [192.0.2.53, 192.0.2.54]
```

5.3.4 Vanliga tjänster: SSH/SFTP, SMB/NFS, webb och certifikat

I många miljöer börjar man med **SSH/SFTP** för administration och filöverföring. Det är vanligt att stänga av lösenordsinloggning, styra vilka användare eller grupper som får logga in och låta åtkomst gå via en bastionvärd i stället för direkt från internet. För filresurser väljer många

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Samba/SMB när Windows-klienter ska ansluta och **NFSv4** när det är Linux-till-Linux — båda kan använda POSIX-ACL för finmaskiga rättigheter. För webb är **Nginx** ett vanligt val, ofta som **reverse proxy** framför applikationer, med automatiserad hantering av TLS-certifikat (t.ex. via ACME/certbot) så att förnyelser sköts utan handpåläggning.

Exempel: En publik webbapp kör bakom Nginx. Endast 80/443 är öppna i brandväggen. SSH är åtkomligt enbart från bastion, inte från internet i stort.

5.3.5 Central identitet och “policy”: AD/SSSD, FreeIPA och polkit

Om man vill slippa separata lokala konton på varje server går det fint att koppla Linux till **central identitet**. Med **realmd/SSSD** kan system autentisera mot **AD** och tilldela sudo-rättigheter till vissa grupper. I Linuxvärlden finns också **FreeIPA**, som kombinerar LDAP, Kerberos, certifikat och host-baserad åtkomstkontroll. Lokalt på varje maskin handlar det ofta om **sudoers** och **polkit** (vad en användare får göra när en tjänst ber om höjda privilegier), samt **PAM** för hur inloggningar ska gå till.

Exempel: Gruppen “ops-admin” i AD får sudo på alla Linux-servrar (styrt via SSSD och en enkel fil i `/etc/sudoers.d/`). Gruppen “ops-ro” får bara läsa journalen och köra ett fåtal diagnostikkommandon.

5.3.6 “Policy som kod”: Ansible som Linux-motsvarighet till GPO

När man vill att servrar ska se likadana ut över tid, oavsett vem som satt dem, brukar **Ansible** vara ett lättillgängligt verktyg. Idén är att beskriva önskat tillstånd i **YAML**; körningen är **idempotent**, så bara det som faktiskt behöver ändras ändras. Många börjar med att standardisera paket, tjänster, sshd-inställningar, brandvägg, användare och filrättigheter — och lägger allt i ett versionshanterat repo.

Så här kan en liten bas-playbook se ut (illustration):

```
- hosts: linuxservrar
  become: true
  tasks:
    - name: Installera baspaket
      package:
        name: [vim, curl, htop, fail2ban]
```

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

```
        state: present

    - name: Hårda sshd (stäng lösenord)
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^PasswordAuthentication'
        line: 'PasswordAuthentication no'
      notify: restart ssh

    - name: Enkelt brandväggsläge
      ufw:
        state: enabled
        rule: allow
        port: "{{ item }}"
      loop: [22, 80, 443]

  handlers:
    - name: restart ssh
      service:
        name: ssh
        state: restarted
```

Exempel: En nattlig körning applicerar “Bas-Linux” på alla servrar. Om någon råkat ändra en konfigfil för hand, återställs den till överenskommet läge.

5.3.7 Programvarudistribution och uppdateringar på Linux

För själva programvaran upplever många att det blir smidigast att låta distributionens pakethanterare (APT eller DNF) göra jobbet. Det kan vara hjälpsamt att tänka i **ringar** även här: en liten pilotgrupp först, därefter bredare utrullning, och sist de mest känsliga systemen. Säkerhetsuppdateringar tenderar att rullas snabbare, medan funktionsuppdateringar ofta får passera en testperiod. På Debian/Ubuntu kan **unattended-upgrades** sköta säkerhetsfixar automatiskt; på RHEL-familjen finns **dnf-automatic**. Kärnuppdateringar kan kräva omstart, så planerade fönster gör livet enklare — i vissa miljöer använder man livepatch för att minska antalet omstarter.

Så här kan ett första steg mot automatiska säkerhetsfixar se ut (illustration):

```
# Debian/Ubuntu
sudo apt install unattended-upgrades
```

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

```
sudo dpkg-reconfigure unattended-upgrades

# RHEL/Rocky/Alma
sudo dnf install dnf-automatic
sudo systemctl enable --now dnf-automatic.timer
```

Exempel: Tre inventariegrupper (pilot → bred → sen). Säkerhetsfixar går direkt till alla; större förändringar provar man en vecka i pilot innan de släpps vidare.

5.3.8 Lokala paketförråd (repos) – försörjningskedjans säkerhet i praktiken

När många servrar ska uppdateras regelbundet vill man ofta ha **en egen, kontrollerad källa**. Ett lokalt repo — som spegel, caching-proxy eller **kuraterad** källa — gör att man kan granska och signera innehåll, frysa en viss versionsmängd och "promota" den från **dev → test → prod**. Det minskar exponeringen mot internet, ger förutsägbara byggen och gör rollback enklare.

Exempel: Produktion pekar bara mot …/`prod`/ och kräver signerat index. Test pekar mot …/`test`/ och får nya paket en vecka tidigare. Upptäcks en regression i test, stannar promotion — produktion påverkas inte.

Så här kan det se ut i Debian/Ubuntu-spår (översikt):
— repo-servern har en egen GPG-nyckel; indexfiler signeras.
— klienterna installerar **din** keyring och pekas mot **din** URL (med `signed-by=`).
— pinning kan låsa så att inget annat arkiv används av misstag.

Och i RHEL-spår:
— metadata genereras (t.ex. med `createrepo_c`), signeras och exponeras via HTTPS,
— klienterna pekar på ditt `baseurl` och kräver signatur/verifiering,
— externa `.repo`-filer avaktiveras och egress kan stängas i brandvägg.

5.3.9 Loggning och övervakning

Om man vill kunna förstå vad som hänt i efterhand — eller upptäcka avvikelser i tid — blir central loggning och egna mätvärden värdefulla. I Linux börjar det i **systemd-journald**; sedan skickas loggar ofta vidare till

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

t.ex. rsyslog och en central loggtjänst. För mätvärden använder många en liten agent (t.ex. node_exporter) och ett system som kan larma när värden avviker från det normala.

Exempel: SSH-inloggningar, sudo-försök och tjänstestarter skickas till en central vy. Ett enkelt larm reagerar om flera misslyckade inloggningar sker på kort tid.

5.3.10 Backup och återställning

När man pratar backup handlar det i grunden om att bestämma **RPO** (hur mycket data man får förlora) och **RTO** (hur snabbt det måste gå att komma tillbaka). På Linux är verktyg som **restic** eller **borg** vanliga för filnivåbackup (deduplicering och kryptering), medan hypervisorn ofta tar **VM-backup**. Det brukar bli mest robust att kombinera båda: en bild av hela VM:n **och** en applikationskonsistent backup av själva datan i gästen. Snapshots (t.ex. ZFS/LVM) kan hjälpa vid förändringar — men ses inte som backup i sig.

Exempel: Varje natt kör restic till ett S3-kompatibelt objektlager (med immutabilitet). En gång i veckan återställs en verklig mapp till en "karantän-server" och jämförs med checksumma.

5.3.11 Säkerhetsbaslinje – hur "bra" brukar se ut

Många utgår från några återkommande byggstenar: **SSH med nycklar** (och gärna avstängd lösenordsinloggning), **default-deny** i brandväggen, **fail2ban** eller motsvarande för att dämpa bruteforce, ett **MAC-skydd** (AppArmor/SELinux) som faktiskt är påslaget, och enkel **filintegritetskontroll** på känsliga system. Principen "minsta privilegium" gäller i sudo/polkit — tillräckligt för att lösa uppgiften, men inte mer.

Exempel: En bastion får samma bas som övriga, men med extra hårdning: enbart nyckel-SSH, tvåfaktor via PAM, stram polkit och realtidsloggning till SIEM.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Vilka fördelar får man av att beskriva Linux-servrar “som kod”, och hur påverkar det felsökning och återställning?
2. När kan en LTS-distribution vara ett klokt val, och vad vinner man på att främst använda standardarkiv?
3. Hur skulle du motivera nyckelinloggning i SSH och loggning av förhöjda kommandon för en ny admin?
4. Varför blir det lättare att felsöka om man håller sig till **ett** nätkonfigurationssätt, och när kommer VLAN/bond/bridge in i bilden?
5. I vilka lägen känns Samba/SMB respektive NFS mest naturligt, och hur påverkar valen rättighetshanteringen?
6. Vad kan AD/SSSD eller FreeIPA lösa i central identitet, och vad behöver fortfarande styras lokalt (sudo/polkit)?
7. Hur hjälper Ansible dig att hålla en baspolicy intakt, och vad betyder idempotens i vardagen?
8. Hur kan man lägga upp uppdateringar i ringar för Linux, och varför brukar säkerhetsfixar gå snabbare?
9. Vad vinner man på lokala repos (spegel, proxy eller kuraterat), och hur hänger signering och promotion ihop med rollback?
10. Hur kombinerar du VM-backup och gäst-backup för att nå RPO/RTO, och hur visar du att återställning faktiskt fungerar?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- Ubuntu Server Guide: `https://ubuntu.com/server/docs`
- Debian Admin Handbook: `https://debian-handbook.info/`
- Red Hat Enterprise Linux – Security & Hardening: `https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/`
- Ansible Documentation: `https://docs.ansible.com/`
- Netplan Examples: `https://netplan.io/examples`
- OpenSSH manpages: `https://man.openbsd.org/ssh_config`
- Samba Documentation: `https://www.samba.org/samba/docs/`
- FreeIPA Documentation: `https://www.freeipa.org/page/Documentation`
- restic (backup): `https://restic.net/`
- Prometheus node_exporter: `https://prometheus.io/docs/guides/node-exporter/`

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

5.4 MDM i praktiken

5.4.1 Vad är MDM, EMM och UDM/UEM — likheter och skillnader

Om man vill styra datorer och mobila enheter centralt möter man tre begrepp: **MDM**, **EMM** och **UDM/UEM**. De bygger på samma idé men har olika räckvidd.

- **MDM (Mobile Device Management)** fokuserar på *enheten*: registrera den, tilldela profiler (WLAN, VPN, certifikat), slå på kryptering/brandvägg, rulla ut appar och vid behov fjärr-låsa eller rensa.
- **EMM (Enterprise Mobility Management)** bygger vidare med app-hantering och dataskydd i mobila appar (t.ex. separera jobbdata från privat).
- **UEM/UDM (Unified Endpoint Management/Device Management)** samlar *alla slutpunkter* (Windows, macOS, iOS/iPadOS, Android — och ofta även Linux/särskilda enheter) i ett och samma system och knyter ihop **identitet**, **compliance**, **villkorad åtkomst**, **patch** och ibland **endpoint-skydd**.

I vardagen används orden ibland blandat; viktigast är vad plattformen faktiskt klarar och hur djupt den når i varje operativsystem.

Exempel (Exempel:) *En verksamhet börjar med MDM för att få kryptering och WLAN-profiler på bärbara. När behoven växer (app-livscykel, villkorad åtkomst, patch) landar man i en UEM-plattform som knyts till identitetstjänsten.*

5.4.2 Varför MDM/UEM behövs — automatisering, förutsägbarhet och säkerhet

Om man ska hantera många enheter över tid fungerar inte manuellt pill. MDM/UEM gör konfiguration **upprepad och spårbar**: nya enheter går från kartong till drift på minuter, förändringar rullas ut kontrollerat och säkerhetskrav kan **bevisas** (t.ex. “alla bärbara är krypterade, brandväggen är på, och OS är patchat”). Utan MDM/UEM uppstår ofta

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

driftsspridning, bortglömda uppdateringar, delade WLAN-lösenord och osäker onboarding/offboarding.

Exempel (Exempel:) *En borttappad laptop med MDM är krypterad, spärras omedelbart och kan fjärr-rensas. Utan MDM vet ingen om den var krypterad eller vad den fortfarande når.*

5.4.3 Enrollment — manuell, automatisk och “zero-touch”

Enrollment kopplar enheten till plattformen så att policyer och appar börjar gälla.

- **Manuell enrollment** passar BYOD: användaren installerar “företagsportalen”, loggar in och får en arbetsprofil (Android) eller hanteras via enhetens hanteringsvy (iOS/macOS).
- **Automatisk/zero-touch** passar företagsägda enheter: serienummer/hårdvaru-ID registreras i förväg hos återförsäljaren. När enheten startar vet den vem som äger den och låser sig till rätt MDM.
    - **Windows:** Autopilot (användardrivet, self-deploying eller pre-provisioned).
    - **Apple:** Automated Device Enrollment via Apple Business Manager (MDM-tvingning).
    - **Android:** Android Enterprise med zero-touch/QR/KNOX Mobile Enrollment. **Knox** är **Samsungägt/styrt** och ger extra djup på Samsung-enheter.
    - **Chromebook:** enrollment via Google Admin.
    - **Linux:** ingen universell MDM-kanal; man använder agent-/policyverktyg i en UEM-lösning (compliance, skript, patch).

Exempel (Exempel:) *En företagsägd Android startas, känner igen sin ägare via zero-touch och skapar en arbetsprofil. Policyer och företagsappar landar utan att IT rört enheten.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

5.4.4 Microsoft Intune — arkitektur, förutsättningar och koppling till Windows AD

Om man vill ha en modern, molnburen plattform som kan styra många olika operativsystem är **Microsoft Intune** *ett naturligt val. I praktiken fungerar Intune som ett* **styrplan***: tjänsten pratar den inbyggda* **MDM-kanalen** *mot varje klient (Windows, iOS/iPadOS, macOS, Android och i viss mån Linux) och ser till att profiler, policyer, appar och uppdateringsregler landar där de ska. Intune krokar djupt i* **Microsoft Entra** *(identitet och grupper), i* **villkorad åtkomst** *(bara "gröna" enheter får nå känsliga appar) och samspelar ofta med* **Microsoft Defender** *för att koppla säkerhetsstatus till åtkomstbeslut. I kontrast till äldre, lokala lösningar (som enbart* **GPO** *eller en fristående* **SCCM/MECM***) är Intune designad för att fungera var enheten än befinner sig — på kontoret, i hemmet eller på resa — så länge den når internet.*

För att Intune ska kunna arbeta behöver några grundbultar vara på plats. Själva **tenanten** måste ha licenser som ger Intune/UEM-funktionalitet, och man sätter **MDM-auktoriteten** till Intune så att plattformen verkligen är "ägaren" av MDM-kanalen. Därefter kopplar man varje plattform till sitt respektive ekosystem: Apple-enheter kräver ett **APNs-certifikat** och blir starkast när de hanteras via **Apple Business Manager** (där både **ADE** för automatisk enhetsregistrering och **VPP** för app-licenser finns). Android-sidan kopplas via **Android Enterprise-bindning**, och i företagsägda scenarier använder man gärna **zero-touch**; på Samsung-enheter finns dessutom **Knox-stödet** som ger extra djup. Windows-datorer registreras smidigast via **Autopilot**, där enhetens hårdvaru-ID (hash) gör att rätt profil och rätt appar rullar in redan vid första uppstarten.

Så fort man vill använda **certifikat** för saker som **WLAN (EAP-TLS)** eller **VPN** behöver man också en **certifikatkedja**. Det löses vanligtvis med en **Intune Certificate Connector** på en server **on-prem** som pratar med organisationens **AD CS**. För Windows-klienter kan man antingen dela ut cert direkt (**PKCS**) eller via **SCEP/NDES** för mer skalbarhet och automation. Poängen är att Intune bestämmer *vem* som ska få vilket certifikat och *när*, medan AD CS faktiskt **utfärdar** det; klienterna kan därefter ansluta till rätt **WLAN** (operativsystemet kallar det ofta "Wi-Fi",

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

men i boken använder vi begreppet **WLAN**) och rätt VPN utan delade lösenord.

Hur Intune hänger ihop med ett befintligt **Windows AD** beror på vilken väg man väljer. I ett **moln-först-upplägg** blir enheten **Entra-joinad** (d.v.s. ansluten till molnets katalog) och styrs helt via Intune; GPO minimeras och identitet/åtkomst villkoras i molnet. Det här brukar vara enklast att förstå för nya klienter eftersom all styrning kommer från samma håll, och enheten fungerar lika bra på kontoret som utanför. I ett **hybrid-join-upplägg** lever enheten samtidigt i **on-prem AD** och i **Entra**. Då kan vissa historiska GPO:er finnas kvar medan Intune tar över det mesta av klientstyrningen (MDM-policy, appar, uppdateringar). Om man dessutom vill köra **Autopilot Hybrid Join** behövs en liten komponent — **Intune Connector for Active Directory** — som kan skapa en “offline domain join”-blob mot en DC under första uppstarten. Slutligen finns **co-management**, där en miljö med **SCCM/MECM** gradvis delar på ansvaret med Intune; till exempel kan **Windows Update for Business** och compliance flyttas till Intune medan vissa appar fortsatt levereras via SCCM. Oavsett väg är grundidén densamma: låt Intune vara **policy-motorn**, låt **Entra** vara **identitetslagret**, och använd **AD CS** som **tillitsfabrik** för certifikat.

När allt detta sitter på plats blir flödena begripliga. En ny bärbar dator som är registrerad i **Autopilot** startar upp, känner igen sin ägare, **enrollerar** sig själv i Intune och hämtar **basprofiler**: kryptering (BitLocker), brandvägg, uppdateringsring, certifikat och **WLAN-profil**. **Compliance** sätter ribban för vad som är “grönt”, och **villkorad åtkomst** använder den statusen när användaren försöker nå exempelvis e-post eller affärssystem. Om datorn istället ärver några kvarvarande **GPO** (i ett hybridläge) gäller det att hålla ansvarsfördelningen tydlig så man undviker policykrockar: det som ligger i molnet bör i första hand också styras från molnet, medan servernära eller speciella arv kan få leva kvar i GPO under en övergångsperiod.

Exempel: *En organisation med AD on-prem synkar användare och grupper till Entra för att slippa dubbla kataloger. Nya Windows-klienter körs som* **moln-först** *med Entra-join och Intune för all klientstyrning. Certifikat delas ut via SCEP-connectorn, och klienterna ansluter till skolans* **WLAN** *med* **EAP-TLS***. Några särskilda labbmiljöer ligger kvar i en* **hybrid-OU** *där ett fåtal GPO:er används — men allt som rör klienternas*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

säkerhetsbas, appar och uppdateringar ligger i Intune. Resultatet blir snabb onboarding, jämn säkerhetsnivå och tydlig spårbarhet.

5.4.5 Intune på djupet — bygg en bra grund (varför, hur, och vad som händer om man låter bli)

Gruppdesign (dynamiska grupper)
Varför: policyer/ appar ska landa rätt automatiskt (t.ex. “alla Windows-laptops”).
Hur: använd attribut (plattform, enhetstyp/ägande, taggar). Håll grupperna få och begripliga.
Om man låter bli: manuell målning → felträffar, glapp i säkerhet, långsam utrullning.

Basprofiler för nät & certifikat (WLAN/VPN + PKCS/SCEP)
Varför: EAP-TLS ersätter delade lösenord, minskar läckage och support.
Hur: sätt upp certifikatconnector + AD CS; skapa cert- och WLAN-profiler för aktuella SSID.
Om man låter bli: delade PSK sprids, byten blir stökiga, obehöriga kan “läcka in”.

Compliance + villkorad åtkomst
Varför: endast “gröna” enheter (kryptering, brandvägg, OS-nivå) får nå känsliga appar.
Hur: definiera compliance per plattform; lås kritiska appar bakom villkorad åtkomst.
Om man låter bli: stulna/eftersatta klienter når data; svårt att bevisa efterlevnad.

Uppdateringsringar (WUfB)
Varför: kvalitetsuppdateringar snabbt; funktionsuppgraderingar i ringar (Pilot → Bred → Sen).
Hur: skapa ringar med rimliga deferrals/deadlines; följ upp regelefterlevnad.
Om man låter bli: ojämn patch-nivå, fler incidenter och “stora hopp” som skapar driftstopp.

App-livscykel (Win32-paket, detektion, beroenden)
Varför: rätt version till rätt grupp; repeterbar installation/avinstallation.
Hur: definiera install/uninstall-kommandon, **detektionsregler** och

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

beroenden.
Om man låter bli: “zombie-appar”, dubbletter/konflikter, oklar klientstatus.

Autopilot + Enrollment Status Page (ESP)
Varför: “kartong → färdig” utan handpåläggning; ESP blockerar inloggning tills basen är klar.
Hur: registrera enheter i Autopilot; välj scenario (user-driven/self-deploying/pre-provisioned); aktivera ESP.
Om man låter bli: lång onboarding, fler manuella fel, användare börjar på halvfärdig klient.

RBAC och förändringsstyrning
Varför: minska risken för fel i stor skala och skapa spårbarhet.
Hur: rollstyr åtkomst (minsta privilegium), inför dev/test/prod-taggning och change-fönster.
Om man låter bli: svår root-cause, oavsiktliga ändringar, större påverkan vid misstag.

Skript & proaktiv remediering
Varför: hitta/laga små driftsavvikelser automatiskt, innan de blir ärenden.
Hur: PowerShell (Windows) / shell (macOS/Linux där stött); kör schemalagda kontroller + korrigeringar.
Om man låter bli: små fel byggs upp och skapar störningar och manuellt efterarbete.

Exempel (Exempel:) *“Bas-Windows”: dynamisk grupp för laptops, BitLocker-policy + compliance, Defender + ASR, WUfB-ringar, cert-/WLAN-profiler, Autopilot med ESP, samt kärnappar. Resten läggs på i lager.*

5.4.6 Konfigurationsexempel — utförliga typfall

Windows (företagsägt) — Autopilot user-driven
Flöde: klienten startas, kopplar till tenant, ESP visar förlopp; BitLocker, brandvägg och Defender/ASR slås på; WUfB-ring och cert/WLAN-profiler landar; Office + affärsappar installeras.
Resultat: användaren loggar in på en färdig, krypterad klient.
Utan detta: manuell “gör i ordning”-process, osäker säkerhetsnivå och ojämnhet mellan klienter.

macOS (företagsägt) — ADE + FileVault
Flöde: enheten binds i ABM, enrolleras automatiskt, FileVault krypterar,

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

systeminställningar låses där det behövs; cert/WLAN och appar (VPP/pkg) landar.
Resultat: enhetliga Apple-klienter; återställningsnycklar hanteras enligt policy.
Utan detta: kryptering hoppas över, lokala “hemfixar”, svår support och efterlevnad.

iOS/iPadOS — ADE, låst layout och selektiv wipe
Flöde: auto-enrollment, PIN-krav, WLAN, e-postprofil; appar pushas; hemskärm layoutas för arbetsflöde; vid förlust → fjärr-rensa.
Resultat: förutsägbart gränssnitt och låg supportbelastning.
Utan detta: spretig användarupplevelse, appar saknas, data blandas med privat.

Android — Work Profile (BYOD) och COPE/COBO (företagsägt)
Flöde: BYOD får arbetsprofil där jobbdata hålls separerat; företagsägt kan vara låst (COBO) eller blandat (COPE). Zero-touch/Knox underlättar.
Resultat: balans mellan kontroll och integritet.
Utan detta: jobbdata blandas med privat; svårt att rensa vid avslut.

Linux (utvalda distron) — agentbaserad compliance
Flöde: agent registrerar enheten; compliance kräver kryptering, brandvägg och minsta OS-nivå; skript åtgärdar avvikelser.
Resultat: Linux-klienter uppfyller samma “grönt-krav” innan de når resurser.
Utan detta: parallell värld med oklara regler och högre risk.

WLAN via certifikat (EAP-TLS)
Flöde: Intune delar ut klientcertifikat; en WLAN-profil ansluter mot SSID som kräver cert; tar man bort behörighet tappar enheten nätåtkomst utan lösenordsbyte i infrastrukturen.

Resultat: inga delade PSK; enkel offboarding; hög säkerhet.
Utan detta: PSK sprids och lever för länge, byten blir stökiga, onödig exponering.

5.4.7 Säkerhetsaspekter vid fjärrhantering — tre saker som ger störst effekt

1. **Kryptering + skärmlås** på alla bärbara/telefoner.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

2. **Compliance + villkorad åtkomst** så att bara “gröna” enheter når känsliga appar.
3. **Certifikat + segmentering** (EAP-TLS för WLAN/VPN, rätt VLAN/roll i nätet).

Lägg till **RBAC** för MDM-admins, tydlig **loggning** av fjärråtgärder och **sekretess** för BYOD (MDM ska inte kunna läsa privat data). Det bygger förtroende utan att sänka säkerheten.

Exempel (Exempel:) *En hittad men misstänkt komprometterad telefon blockeras direkt, jobbprofilen rensas, cert spärras. Åtkomst öppnas först efter ny enrollment och grön compliance.*

5.4.8 Jämförelse i korthet — Intune, Jamf, Workspace ONE, Miradore och Linux-vänliga alternativ

- **Microsoft Intune:** bred täckning (Windows, iOS/iPadOS, macOS, Android; visst Linux-stöd), stark koppling till Entra och villkorad åtkomst, Autopilot.
- **Jamf Pro:** djup och snabb för Apple-plattformar; kan samspela med Intune för compliance.
- **VMware Workspace ONE (AirWatch):** stark mobilhantering, brett OS-stöd, mogna appflöden.
- **Miradore/Hexnode/Kandji/Meraki SM:** lättare insteg; funktionellt djup varierar. **Miradore** är ett praktiskt multi-OS-alternativ.
- **Linux-inriktat:** JumpCloud, Canonical Landscape, SUSE Manager, Red Hat Satellite (agent/policy/patch/inventering i UEM-anda).

Exempel (Exempel:) *En blandad miljö kör Intune som “nav” (compliance, villkorad åtkomst, Autopilot). Apple-delen hanteras i Jamf men rapporterar compliance till Intune.*

5.4.9 Vanliga misstag — och hur man undviker dem

- Starta för brett: börja med **bas** (kryptering, skärmlås, uppdateringar, cert-WLAN).
- Otydliga grupper: bygg **dynamiska** målgrupper och testa på pilot.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- Blanda företagsägt och BYOD: håll isär; **selektiv wipe** för BYOD.
- Lösenords-WLAN i produktion: gå tidigt till **EAP-TLS**.
- Ingen återställningsplan: *öva* fjärr-wipe, återställning och om-enrollment.

Exempel (Exempel:) *En pilotgrupp får “Bas 1.0”. När den är stabil lägger man på appar och finjusteringar. Färre överraskningar och färre supportärenden.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. Förklara skillnaden mellan **MDM**, **EMM** och **UEM/UDM** och när man typiskt växlar upp.
2. Vad vinner man på att koppla **compliance** i MDM till **villkorad åtkomst**?
3. När passar **manuell** enrollment bäst, och när är **zero-touch** att föredra?
4. Vad är **Samsung Knox**, och varför spelar det roll för Android-enheter från Samsung?
5. Vilka **förutsättningar** behöver ofta vara på plats innan Intune kan användas fullt ut (per plattform)?
6. Hur samverkar **Intune** med **Windows AD** i **moln-först**, **hybrid-join** och **co-management**?
7. Varför är **EAP-TLS** för WLAN ett lyft jämfört med delade lösenord, och vad händer om man inte inför det?
8. Beskriv en **WUfB-ringdesign** (Pilot → Bred → Sen) och vad som kan gå fel utan ringar.
9. Hur hjälper **dynamiska grupper**, **RBAC** och **ESP** till att göra utrullningar robusta?
10. Välj ett av konfigurationsexemplen och förklara vad som händer “om man låter bli” det steget.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- Microsoft Intune (översikt och guider): `https://learn.microsoft.com/mem/intune/`
- Windows Autopilot (översikt): `https://learn.microsoft.com/mem/autopilot/`
- Apple Business Manager (ADE): `https://support.apple.com/guide/apple-business-manager/welcome/web`
- Android Enterprise (enrollment-alternativ): `https://developer.android.com/work`
- Samsung Knox (KME/Knox Suite): `https://www.samsungknox.com/`
- Jamf Pro: `https://learn.jamf.com/`
- VMware Workspace ONE: `https://www.vmware.com/products/workspace-one.html`
- Miradore: `https://www.miradore.com/`
- JumpCloud (device + identity): `https://jumpcloud.com/`
- Google Admin — ChromeOS enhetsregistrering: `https://support.google.com/chrome/a/answer/1360534`

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

5.5 Framtiden för nätverk

5.5.1 SDN – från lågnivåkonfiguration till intent och API:er

I dag bygger mycket nätverksdrift fortfarande på att tekniker konfigurerar enheter var för sig: port för port, ACL för ACL. **Software-Defined Networking (SDN)** byter perspektiv. Styrplanet flyttas upp i en **kontroller** som förstår hela topologin, och nätet styrs via **API:er** och deklarativa policys: “det här ska vara tillåtet mellan de här tjänsterna, med den här säkerhetsnivån”. Kontrollen fördelas sedan ned till hårdvaran (switchar, routrar, brandväggar) som ren dataplanslogik. I datacenter är **EVPN/VXLAN** och fabric-kontrollers vardag; i WAN är **SD-WAN** redan brett infört; i campus kommer “intentbaserad” segmentering och identitetsstyrd åtkomst steg för steg.

Konsekvensen är att **feltyperna förändras**. I ett klassiskt nät var fel ofta fysiska (länk ned, loop). I ett SDN blir fel oftare **logiska**: en policy träffar fel grupp, ett objekt saknar tagg, ett API-samtal misslyckas. Det ställer nya krav på **observability** (telemetri, flödesdata, modellinsyn) och på hur vi arbetar: versionering av förändringar, **Git som sanning**, **CI/CD** för nät, och **”pre-change”-tester** i sandlåda eller digital tvilling.

För användare märks skiftet som **snabbare förändring** (nya VLAN/WLAN, ny QoS, ny site) och **jämnare kvalitet**. För driftteamet innebär det att **CLI-hantverket** kompletteras – inte försvinner – av **automation**: YANG/NETCONF/RESTCONF, gNMI, Ansible/Terraform, Python. Rollen glider mot **policy-, data- och livscykelarbete** snarare än “klicka i rätt ruta”. Den som kan läsa logik, förstå modellering och skriva tester blir lika viktig som den som känner varje kommando utantill.

Exempel (Exempel:**)* *En ny applikation ska ut i tre zoner. I stället för att skapa ACL:er och VLAN på femton switchar beskriver du i kontrollern vilka roller som får prata med vilka, och kontrollerna skapar underliggande flöden. En rollfelmappning gör att bara testzonen får trafik; telemetri och “intent verification” visar exakt var policyn stoppar.*

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

5.5.2 Zero Trust – identitet först, nätet som transport

Zero Trust utgår från antagandet att **inget** är pålitligt per automatik – inte ens “insidan”. Åtkomst beslutas **varje gång**, baserat på **identitet**, **enhetens skick (compliance)**, **kontext** och **risk**. Nätet blir mer av **transport**, medan kontrollen sker i kombination av **identitetstjänst**, **MDM/UEM**, **applikationsproxy** och **mikrosegmentering**. I campus/edge innebär det att **VLAN som grov kniv** ersätts av **finmaskiga policys** nära klient eller arbetsbelastning. I datahall och moln betyder det att arbetslaster isoleras per **tjänst/roll** snarare än per IP-subnät.

För användare betyder Zero Trust fler **tysta kontroller** (enhetens status, plats, beteende) och färre “stora” lösenordshinder – men **MFA** och **device posture** blir vardag. För driftteamet betyder det **färre platta nät**, **fler policys** och **mer samarbete** med identitetsteam och applikationsägare. Den praktiska utmaningen är **komplexitet**: policyexplosion om man inte modellerar roller rätt, och **friktion** om kraven inte är realistiskt satta. Nyckeln blir **enkla basregler**, tydliga undantagsprocesser och **telemetri** som visar *varför* något nekas.

Exempel (Exempel:**)* *En klient med gammalt OS tappar åtkomst till HR-systemet för att compliance-regeln kräver uppdaterad kernel och kryptering. Användaren får guidning i klientportalen; access öppnas igen när enheten är grön.*

5.5.3 IPv6-implementering – från dual-stack till IPv6-only

IPv6 har länge “varit på gång”, men de senaste åren har förutsättningarna mognat: operatörer kör IPv6 i ryggraden, stora innehållsleverantörer exponerar tjänster i IPv6, och klient-OS har stabil **privacy**-logik. Nästa steg är **IPv6-only** i allt fler miljöer, med **NAT64/DNS64** eller 464XLAT för det som ännu saknar IPv6. Vinsten är **enklare adressering i stor skala**, **färre NAT-lager** och möjlighet till **renare end-to-end-säkerhet**. Samtidigt blir **säkerhetsmodellen tydligare**: NAT var aldrig en brandvägg – IPv6 kräver **uttalade filter**, god **ICMPv6-hygien** (annars dör funktioner), och skydd mot **RA-spoofing** och felkonfigurerad **SLAAC/DHCPv6**.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

För användare märks lite – oftast **bättre reachability** och färre märkliga fel. För driftteamet märks mycket: **adressplanering i /64-tänk**, **brandväggsregler per tjänst**, uppdaterad **övervakning** (flow/NetFlow/IPFIX som förstår v6), och **felsökning** där man komfortabelt läser **Neighbor Discovery** och **ICMPv6**. Övergången kräver **dual-stack-period**, **piloter**, tydlig **inventering av v4-beroenden**, och planerad **avveckling** av gamla NAT-antaganden. I campus och WLAN handlar det om att RA-guard och DHCPv6-snooping fungerar lika bra som IPv4-kontrollerna. I datacenter och moln om att **säkerhetspolicys** (SG/NSG/ACL) migreras utan att varken över- eller under-skydda.

Exempel (Exempel:**)* *Ett elev-VLAN blir IPv6-only. DNS64/NAT64 ger fortfarande åtkomst till IPv4-webb. Loggarna visar lägre felsökningsbelastning, men brandväggen fick expanderade v6-regler för att ersätta gamla "NAT-skyddet".*

5.5.4 AI och automatisering – från NMS till beslutssystem

Automatisering har länge handlat om skript och verktyg som Ansible. Nästa steg är **AI-stödd drift**: system som **förutser** fel, **förklarar** avvikelser och i kontrollerade fall **åtgärdar** dem. I WLAN ser vi redan självlärande **radio-optimering** (kanal/effekt), i WAN **policy-styrd ruttval/QoS**, i DC **intent-verifiering** som säkerställer att dataplansflöden matchar avsikten. Med **AIOps** blir brusiga larm till **händelser med sammanhang**: "allt pekar på att DHCP-svar i VLAN 20 stoppas på denna trunk". Med **LLM-stöd** kan runbooks bli **konversationsbara**: "visa sannolik rotorsak och föreslå återställning".

Vinsten är **hastighet och kvalitet**. Risken är **överförtroende**: en modell kan "hallucinera", hamna i **feedbackloopar** eller agera på **partiska data**. Därför blir **skyddsräcken** centrala: förändringar ska gå genom **pull-requests**, testas i **sandlåda**, **stegvis rullas** och **mätas** efteråt. Data måste hanteras med **integritet**: nättelemetri kan innehålla personuppgifter eller känsliga mönster. Rollen för nätverksteknikern blir mer **ingenjör/mätare**: skriva **policyer**, definiera **SLO:er** (t.ex. anslutningslatens i WLAN), designa **mätningar** och granska **maskinens förslag**. För användare kan AI betyda **stabilare upplevelse** – om man samtidigt accepterar **mindre "magi" och mer transparens**: varför fattades ett beslut, och hur ångras det?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel (Exempel:)* *Ett AIOps-system ser mönstret “AP-roaming tar >300 ms” i en byggnad, kopplar det till en ny störkälla i 2,4 GHz och föreslår kanalplanering/effektjustering. Förslaget körs i nattfönster mot en testzon och sprids först efter mätbar förbättring.*

5.5.5 Etik och cybersäkerhet – data, styrka och ansvar

När nätet blir mer **mätbart** och mer **automatiserat** ökar både möjligheten och frestelsen att **se mer**. Telemetri, loggar och flöden kan avslöja arbetsmönster, platser och beteenden. **Integritet** måste därför vara en **första-klass-fråga**: samla bara det som behövs, **pseudonymisera** där det går, begränsa **åtkomst** och **lagringstid**, och var **öppen** med vad som samlas. I skola och offentlig sektor blir detta särskilt viktigt. På säkerhetssidan fortsätter **segmentering**, **Zero Trust-tänk**, **härdning** och **snabb patchning** att vara grund. Till det kommer **leverantörskedja** (firmware, SBOM, signering), **out-of-band-hantering** (så att man kan rädda även vid ransomware), och uppmärksamhet på **kryptoframtid** (post-kvant kommer påverka hur vi ser på nycklar och livslängd).

Yrkesrollen rör sig också etiskt: fler beslut om **övervakningens gränser**, **modellers skälighet**, **vendor-låsning** och **hållbarhet** (energi, värme, livslängd). Teknikern blir inte bara den som “får det att fungera”, utan den som **formar hur det bör fungera** – i balans mellan nytta, risk och rättigheter.

Sammanfattning att ta med sig

- **SDN** gör nät till **policy + API**; hantverk blir livscykel, test och modell.
- **Zero Trust** flyttar förtroendet till **identitet och enhetens skick**; nätet segmenteras finmaskigt.
- **IPv6** går från “parallellt” till **IPv6-only**; NAT ersätts av **tydlig filtrering** och god ICMPv6-hygien.
- **AI/AIOps** höjer hastighet och kvalitet – men kräver **skyddsräcken**, **mätning** och **mänsklig granskning**.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

- **Etik & säkerhet** blir inbyggt: minst data, tydliga roller, snabb patch, stark kedja från firmware till policy.

Frågor att fundera på

- Vilken **minsta uppsättning mätvärden** behöver vi för att kunna lita på ett AI-förslag i *vårt* nät?
- Var går gränsen mellan **nödvändig telemetri** och **onödig övervakning** i en skolmiljö?
- Om vi gör **IPv6-only** i ett delnät, vilka **tio saker** måste vara på plats för att det ska bli en förbättring – inte en risk?
- Vilka **färdigheter** (språk, verktyg, metoder) behöver en nätverkstekniker lägga till de kommande två åren för att må bra i SDN/Zero Trust-världen?
- Hur beskriver vi **intentionen** för nätet så att den är testbar före och efter varje förändring?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

📌 Kontrollfrågor

1. När ett nät styrs via en SDN-kontroller och “intent”, vilka nya feltyper uppstår jämfört med klassisk CLI-konfiguration – och hur märker man dem i drift?
2. Hur skulle du beskriva skillnaden mellan **mikrosegmentering** i ett Zero Trust-tänk och traditionell segmentering med VLAN/ACL – ur både säkerhets- och förvaltningsperspektiv?
3. Om ni inför **Zero Trust**: vilka tre datakällor (identitet, enhet, kontext) måste vara pålitliga för att besluten ska bli rimliga – och hur validerar ni dem?
4. Du får uppdraget att göra ett **IPv6-pilot** i ett campus-VLAN. Vilka tio praktiska kontroller vill du se “gröna” innan pilotens användare märker något negativt?
5. Vid **IPv6-only** med NAT64/DNS64: vilka applikationstyper brukar strula, och vilka mätetal sätter du upp för att snabbt fånga fel?
6. Nämn tre **skyddsräcken** du vill ha på plats innan ni låter automation/AI göra förändringar i nät (tänk kodflöde, test, övervakning).
7. Vilka **SLO:er** (service level objectives) är mest meningsfulla i ett modernt WLAN – och hur kopplas de till åtgärder i radionät/klientpolicy?
8. Vad är ett bra **minsta datalager** för AIOps (telemetri, flöden, loggar) så att analys blir möjlig utan att tumma på integriteten?
9. Hur förändras nätverksteknikerns **kompetensprofil** när SDN, Zero Trust och automatisering blir vardag – och hur tränar man det effektivt i skolan?
10. Välj en realistisk **etikfråga** (t.ex. klienttelemetri i skolmiljö). Var går gränsen mellan nödvändig mätning och onödig övervakning, och hur förklarar du det för användarna?

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

🔗 Fördjupningslänkar

- ONF – *Software-Defined Networking (SDN) Definition*: `https://opennetworking.org/sdn-definition/`
- RFC 7348 – *VXLAN*: `https://datatracker.ietf.org/doc/rfc7348/`
- RFC 7432 – *BGP EVPN*: `https://datatracker.ietf.org/doc/rfc7432/`
- NIST SP 800-207 – *Zero Trust Architecture*: `https://csrc.nist.gov/publications/detail/sp/800-207/final`
- RFC 8200 – *Internet Protocol, Version 6 (IPv6) Specification*: `https://datatracker.ietf.org/doc/rfc8200/`
- RFC 4861 / RFC 4862 – *Neighbor Discovery / SLAAC*: `https://datatracker.ietf.org/doc/rfc4861/` | `https://datatracker.ietf.org/doc/rfc4862/`
- RFC 4443 – *ICMPv6*: `https://datatracker.ietf.org/doc/rfc4443/`
- RFC 6146 / RFC 6147 – *NAT64 / DNS64*: `https://datatracker.ietf.org/doc/rfc6146/` | `https://datatracker.ietf.org/doc/rfc6147/`
- Google SRE – *Service Level Objectives*: `https://sre.google/sre-book/service-level-objectives/`
- NIST AI RMF 1.0 – *AI Risk Management Framework*: `https://www.nist.gov/itl/ai-risk-management-framework`

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

✍ Egna anteckningar

...

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Appendix I – Begrepp

Not: I brödtexten använder vi termen ”WLAN”. ”Wi-Fi” syftar här på certifieringsprogrammet/varumärket och används i denna begreppsruta.

ABM (Apple Business Manager) – Portal för enhets- och appinköp; grunden för automatisk Apple-enrollment (ADE) och VPP-licenser.
ACL – Åtkomstlista som avgör vilken trafik som tillåts eller nekas.
ACME – Protokoll för automatisk hantering av TLS-certifikat (t.ex. certbot).
AD (Windows Active Directory) – Katalogtjänst för identiteter, datorer och policyer.
AD CS – Certifikatinfrastruktur (CA) i Windowsmiljö.
AIDE – Filintegritetskontroll som larmar när filer ändras otillåtet.
AIOps – AI-stött drift som korrelerar larm, hittar orsaker och föreslår åtgärder.
Air-gap – System eller kopior som är fysiskt/logiskt isolerade från nätet.
Android zero-touch – Förregistrering av företagsägda Android-enheter för “zero-touch”.
Ansible – Automationsverktyg som beskriver systemtillstånd i YAML och kör idempotent.
Anycast – Samma IP annonseras från flera platser; trafik hamnar hos närmaste nod.
API – Programmeringsgränssnitt; i SDN/UEM används för styrning och automation.
AP (Accesspunkt) – Trådlös basstation för klientanslutning i ett **WLAN** (OS kallar ofta “Wi-Fi”).
APIPA – Automatisk IPv4-adress 169.254.0.0/16 när DHCP saknas.
AppArmor – MAC-säkerhet i Linux som begränsar programrättigheter.
ARP – Protokoll som översätter IPv4-adresser till MAC-adresser i L2.
Autentisering – Bevis av identitet (t.ex. lösenord, certifikat, MFA).
Autopilot (Windows) – Zero-touch-registrering och uppsättning av Windows-klienter.
Backup – Säkerhetskopia av data för återställning vid fel eller incident.
Bandbredd – Maximal datamängd per tidsenhet på en länk.
Bastion/Jump host – Härdad mellanvärd för säker SSH/RDP-åtkomst.
BGP – Externt routingprotokoll som utbyter vägar mellan autonoma system.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

BGP community – Attribut som styr policyn för hur rutter hanteras.
BGP ECMP – Lastdelning över flera likvärdiga vägar i BGP.
BGP route reflector – Minskar krav på full-mesh i iBGP.
BNC – Bajonettkontakt för koaxialkabel (historiskt/nischnät).
Bonjour/mDNS – Lokal namn- och tjänsteupptäckt i LAN.
BPDU – STP-kontrollramar som används för loopdetektering.
Bridge (brygga) – L2-koppling mellan segment; i servervärd: virtuell switch.
BSSID – MAC-adress för accesspunktens radiosändare (en SSID-cell).
BYOD – "Bring Your Own Device"; privatägd enhet i arbetsmiljö.
CA (Certificate Authority) – Utfärdare av digitala certifikat.
CAPA – Korrigerande och förebyggande åtgärder efter incident (Corrective/Preventive).
CASB – Cloud Access Security Broker; policy och kontroll för SaaS-åtkomst.
Cat6A (kabel) – Vanlig kopparkabel för 10G-Ethernet upp till 100 m.
Ceph – Distribuerad objekt/blocks/fil-lagring; vanligt i hypervisor-kluster.
Certificate pinning – Låsning av klient till specifik certifikatnyckel.
Certifikat – Kryptografisk legitimation kopplad till identitet/nyckel.
CIDR – Notation/metod för prefixlängd (t.ex. /24) och aggregering.
CI/CD – Automatiserat bygg, test och driftsättning (även för nätkonfig).
CLI – Kommandorad för konfiguration och felsökning.
Cloud-join (Entra-join) – Windows-klient ansluten direkt till Microsoft Entra ID.
Co-management – Samdrift av MECM/SCCM och Intune i samma klientpark.
COBO – Företagsägd, företagsanvänd Android (låst läge).
COPE – Företagsägd, personligen använd (balans jobb/privat).
Container – Isolerad applikationsmiljö (t.ex. Docker/LXC).
Controller (WLAN) – Central styrning/övervakning av många accesspunkter.
CoS (802.1p) – L2-prioritering i VLAN-taggen.
CRC – Felkontrollsumma för att upptäcka bitfel.
DAC-kabel – Direktkopplingskabel för SFP+/QSFP-port till korta avstånd.
DAD (Duplicate Address Detection) – IPv6-kontroll för att undvika adresskollision.
DAI (Dynamic ARP Inspection) – Skydd mot ARP-förfalskning i L2.
Data plane – Där paket faktiskt forwardas (jfr control/management

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

plane).
Deduplicering – Eliminering av dublettdata i backup/lagring.
Default route – Standardväg när ingen mer specifik rutt matchar.
DFS (WLAN) – Dynamic Frequency Selection; kanalreglering i 5/6 GHz.
DFS (Windows) – Distributed File System; virtuella filnamnrymder/pekare/replikering.
DHCP – Dynamisk tilldelning av IP-konfiguration till klienter.
DHCP relay – Vidarebefordrar DHCP mellan VLAN/subnät.
DHCP snooping – Switchskydd som stoppar falska DHCP-servrar.
DHCPv6 – IPv6-varianten av DHCP (komplement till SLAAC).
DNS – Namnuppslagning mellan domännamn och IP-adresser.
DNSSEC – Signerad DNS för att skydda mot förfalskning.
DoH/DoT – DNS över HTTPS/TLS för integritet.
Docker – Containerplattform för att paketera och köra applikationer.
Domän – Administrativ enhet i AD eller en DNS-zon.
DPI (Deep Packet Inspection) – Klassning/inspektion på applikationsnivå.
DTP – Cisco-protokoll för att förhandla trunk/access (bör undvikas i produktion).
DR (Disaster Recovery) – Återstart av kritiska system efter större avbrott.
DSCP – QoS-märkning i IP-huvud för prioritering.
EAP – Ramverk för nätverksautentisering (802.1X).
EAP-PEAP – 802.1X-metod som kapslar lösenord i TLS-tunnel.
EAP-TLS – Certifikatbaserad 802.1X-autentisering (WLAN/VPN).
EAP-TTLS – 802.1X-metod med tunnlad autentisering utan klientcert.
Edge – Nätets ytterkant (site/klient-nära lager).
EIGRP – Cisco-routingprotokoll (dynamiskt, hybridmodell).
ESP (Enrollment Status Page) – Intune-sida som blockerar tills baskrav är uppfyllda.
ESP (IPsec) – Encapsulating Security Payload; kryptering/integritet i IPsec.
EVPN – BGP-baserad L2/L3-VPN, ofta med VXLAN i datacenter.
Fail2ban – Blockerar upprepade felaktiga inloggningsförsök.
Failover – Automatisk övertagning när primär komponent faller.
FHRP – Protokoll för redundant gateway (t.ex. VRRP/HSRP/GLBP).
FileVault – Diskkryptering i macOS.
Firewall (tillståndsfull) – Brandvägg som förstår och spårar

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

sessionsflöden.
FreeIPA – Linuxidentitet (LDAP/Kerberos/CA/HBAC), likt AD.
FTP/SFTP – Filöverföring; SFTP går över SSH och är säkert.
Gateway – Standardväg ut ur ett subnät (L3-gräns).
Grafana – Visualisering av tidsserier (dashboards).
GRE – Tunnelprotokoll för inkapsling av trafik.
GFS (Grandfather-Father-Son) – Backup-retentionsmodell dag/vecka/månad.
Git – Versionshantering; grund för GitOps även i nät.
GNS3/EVE-NG – Emulering/virtualisering för nätlabbar.
GPO – Group Policy Objects; Windows-policyer kopplade till AD.
GPO-länkning – Att koppla GPO till Site/Domain/OU i AD-trädet.
GUA – Globala IPv6-adresser som routas på internet.
HA (High Availability) – Design för att undvika enkel felpunkt.
HBA (Host Bus Adapter) – Gränssnitt mellan server och SAN/lagring.
HCI (Hyper-Converged Infrastructure) – Kluster som förenar compute/lagring/nät.
Helm/Kubernetes – Orkestrering av containeriserade applikationer (paket/utrullning).
Honeypot – Avsiktligt lockbete för att observera angripare.
Hop – Ett routingsteg mellan källa och destination.
Host – Slutenhet/servrar/klienter som använder nätet.
Host-ID – Värddelen av en IP-adress (allt utanför Net-ID).
HSRP – Ciscos redundanta gateway (FHRP), liknande VRRP.
HTTP/HTTPS – Webprotokoll; HTTPS säkrat över TLS.
Hypervisor – Mjukvara som kör och isolerar virtuella maskiner.
IBN (Intent-Based Networking) – Nät styrt av deklarerad avsikt/policy.
ICMP – Kontrollprotokoll (ping, felmeddelanden m.m.).
ICMPv6 – IPv6-kontroll: Neighbor Discovery, fel, MTU m.m.
IdP (Identity Provider) – Identitetsleverantör (t.ex. Entra, ADFS).
IDS/IPS – Intrångsdetektering/-prevention i nätverk.
IGP/EGP – Routing inom ett AS (OSPF/IS-IS) respektive mellan AS (BGP).
Immutability (backup) – Skrivskyddad retention; skydd mot ransomware.
iSCSI – Blocklagring över IP (SAN).
Intune – Microsofts UEM/MDM-plattform.
IP – Internet Protocol; adressering och leverans av paket.
IP-adress – Identifierar ett nätverksinterface i IP-nät.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

IP Source Guard – Skydd mot IP-spoofing på accessportar.
IPFIX/NetFlow – Flödesmätning (käll/dest, portar, bytes).
IPsec – Krypterade tunnlar (site-to-site/klient-VPN).
IPv4 – 32-bitars IP; begränsat adressutrymme.
IPv6 – 128-bitars IP; stort adressutrymme och modern funktionalitet.
IS-IS – Länkstatusrouting (IGP), alternativ till OSPF.
JSON/YAML – Dataformat vanliga i API/automation.
Journald – systemd-loggtjänst på Linux.
JumpCloud – Moln-identitet/UEM för bl.a. Linux/Windows/macOS.
Kerberos – Biljettbaserad autentisering (AD/FreeIPA).
Kernel – Operativsystemets kärna.
Keyring (GPG) – Nyckelring för signering/validering av paket/repo.
Knox (Samsung) – Samsungägt ramverk/portal som ger extra MDM-djup.
KVM – Linux-hypervisor (KVM/QEMU) för VM-drift.
L2-switch – Växlar ramar via MAC-tabell inom samma broadcastdomän.
L3-switch – Switch med routingfunktioner mellan VLAN/subnät.
LACP – Länksammanslagning (802.3ad) för redundans/kapacitet.
LACP fast rate – Snabbare LACP-handtryck (~1 s intervall).
LAPS – Hantering av unika lokala adminlösenord (Windows).
Latens – Fördröjning från sändare till mottagare.
Layer 2/3/7 – OSI-lager: Datalänk/Network/Applikation.
LC/SC/MPO – Vanliga fiberkontakter (duplex/multifiber).
Link-local – Lokalt giltig adress (IPv4 APIPA; IPv6 fe80::/10).
Linux bridge – Programvarubrygga; virtuell switch i Linux.
Live migration – Flytt av VM mellan värdar utan driftstopp.
LLDP/CDP – Protokoll som visar grannar och portinformation.
LLM (språkmodell) – AI-modell som kan assistera felsökning/runbooks.
LTS – Långtidsstödd version av ett OS.
LUKS – Linux-diskkryptering (dm-crypt).
MAC-adress – Unik datalänksidentifierare (L2).
MAC-flapping – MAC växlar port ofta; tyder på loop/länkproblem.
MACsec – L2-kryptering på länknivå (802.1AE).
MAN – Metropolitan Area Network; stadsnät.
Management plane – API/CLI/SNMP-kanal för styrning/konfig.
MAB (MAC Authentication Bypass) – 802.1X-alternativ för “dumma” klienter.
MECM/SCCM – Microsofts klienthantering lokalt (f.d. SCCM).
MFA – Multifaktorautentisering (t.ex. lösenord + app).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Mesh (topologi) – Nät med flera redundanta vägar mellan noder.
Microsegmentering – Finmaskig policy per applikation/roll i stället för grova VLAN.
MIMO/MU-MIMO – Flerantennteknik i WLAN (flera samtidiga strömmar).
MPLS – Etikettbaserad forwarding; VPN/QoS i operatörsnät.
MSTP – STP-variant med multipla instanser per VLAN-grupp.
MTTR – Genomsnittlig återställningstid (Mean Time To Repair/Restore).
MTU – Maximal ramstorlek på länknivå.
MSS – Maximal TCP-nyttolast utan fragmentering.
NAC – Nätverksåtkomstkontroll (802.1X/MAB/posture).
NAT – Adressöversättning mellan privata och publika adresser.
NAT64/DNS64 – Överbryggar IPv6-klienter till IPv4-resurser.
NAT-T – IPsec över UDP för att passera NAT.
ND (Neighbor Discovery) – IPv6-grannprotokoll (ersätter ARP).
NDES – Microsofts SCEP-endpoint för certifikat via Intune.
Net-ID – Nätverksdelen i en IP-adress (prefix/mask).
NETCONF/RESTCONF – Modellbaserad konfig via YANG (SSH/HTTP).
Netplan – YAML-baserad nätverkskonfiguration i Ubuntu.
NetworkManager – Linux-tjänst/verktyg för nätverkskonfiguration.
NFS – Filresursprotokoll, främst Linux↔Linux.
Nginx – Webbserver/reverse proxy.
Node exporter – Agent som exponerar Linux-mätvärden för Prometheus.
NPTv6 – Prefixöversättning i IPv6 (utan portöversättning).
NTP/chrony – Tidssynkronisering i nät/operativsystem.
NTP-stratum – Nivå i tidssynkhierarki (avstånd till referens).
OOB (Out-of-Band) – Separat hanteringsnät, används även vid kris.
OpenSCAP – Efterlevnadskontroll mot säkerhetsprofiler.
OpenTelemetry – Standard för insamling av spår/logg/metrics.
Overlay/Underlay – Logiskt nät ovanpå fysisk fabric (t.ex. VXLAN över IP).
OSPF – Länkstatusrouting (IGP) i interna nät.
OSPF area – Hierarkisk uppdelning; Area 0 är ryggraden.
OSPF cost – OSPF-metric; baseras ofta på bandbredd.
OT (Operational Technology) – Produktions-/styrsystemsnät.
Packer – Bygger automatiserade OS-avbilder.
PAM – Pluggbar autentiseringsram i Linux/Unix.
PAT – Portbaserad NAT (många interna → en publik).
PE/CE – Provider Edge/Customer Edge i operatörsnät.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

PFsense/OPNsense – BSD-brandväggar för routing/NAT/VPN.
PHY – Fysiska lagret: kablage, modulering, optik.
PKI – Infrastruktur för nycklar/certifikat (CA, CRL, kedjor).
PKCS (Intune) – Direkt certifikatutfärdande till enheter/användare.
Polkit – Policy för privilegier i Linux (höjda rättigheter).
Port (L2) – Fysisk/logisk anslutning på switch/router.
Port forwarding – Vidarebefordran av portar från utsida till insida.
Portfast – STP-optimering för accessportar (snabb upp).
PoE – Ström över Ethernet till t.ex. AP/kameror.
PPPoE – Inkoppling över Ethernet (vanligt på WAN/operatör).
Prefixdelegation (PD) – Automatisk IPv6-prefixutdelning (CPE→LAN).
Private VLAN (PVLAN) – L2-isolering inom samma VLAN.
Prometheus/Alertmanager – Mätinsamlare och larmmotor.
Proxmox VE – Debian-baserad hypervisor (KVM/LXC) med ZFS-stöd.
Proxmox Backup Server (PBS) – Backup-lösning i Proxmox-ekosystemet.
Proxy (reverse) – Front som tar emot klienttrafik och vidarebefordrar till backend.
PXE – Nätboot av klienter (DHCP/TFTP).
QoS – Prioritering/klassning av trafik för bättre upplevelse.
QoS shaping/policing – Buffra jämnt vs hårt strypa trafikflöden.
RA (Router Advertisement) – IPv6-utdelning av prefix/standardväg.
RA-guard – Skydd mot falska IPv6-RA på access.
RADIUS – Central autentisering/auktorisering (t.ex. 802.1X).
RBAC – Rollbaserad behörighetskontroll för administratörer.
REST API – Webbaserat API (HTTP/JSON) för automation/integration.
Restic – Krypterande, deduplicerande backupklient.
RJ45 (8P8C) – Vanlig kontakt för koppar-Ethernet (twisted pair).
RMM (Remote Monitoring & Management) – Fjärrövervakning/klienthantering.
RPO – Hur mycket data som får förloras vid återställning.
RRM (Radio Resource Management) – Automatisk kanal/effekt-optimering i WLAN.
RTO – Hur snabbt system ska vara uppe efter fel.
RIP – Enkelt distans-vektor-routingprotokoll (mindre nät).
RSTP – Snabbare spanning-tree (802.1w).
Route summarization – Aggregera prefix för mindre routingtabeller.
Routing – Framledning av paket mellan nät/subnät (L3).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Router – Enhet som kopplar samman flera nät (L3-gräns).
RSSI/SNR – Signalstyrka och brusförhållande i trådlöst.
SASE/SSE – Säker nätarkitektur där skydd/åtkomst läggs i molnkant.
SBOM – Förteckning över programvarans komponenter (supply-chain).
SCEP – Förenklat protokoll för certifikatutfärdande (via NDES/Intune).
SDA (Software-Defined Access) – SDN i campus; policyn vid kanten.
SDN – Programvarustyrt nät; policy/intention via controller och API.
SD-WAN – Policy-styrt WAN över flera länkar/operatörer.
Security baseline – Minimikrav för konfiguration och säkerhet.
SELinux – MAC-säkerhet i Linux (RHEL-familjen).
sFlow – Provtagning av trafik för flödesanalys (alternativ till NetFlow).
SIEM – Central logg/säkerhetsanalys (korrelation och larm).
SLA – Service Level Agreement; avtalad tjänstenivå.
SLAAC – Automatisk IPv6-adressättning via RA (stateless).
SLO – Mätbar målnivå för en tjänst (t.ex. latens/anslutning).
SFP/SFP+/QSFP – Modulplatser/optik i switchar/routrar.
SMF/MMF (OS2/OM3/OM4) – Single/Multi-mode-fiber och deras klassningar.
SNAT/DNAT – Käll- och destinations-NAT.
SNMP – Övervakningsprotokoll för status/konfiguration.
Snapshot – Ögonblicksbild av data/VM.
SPAN (port mirroring) – Kopiera trafik till analysport (t.ex. Wireshark).
Spanning-tree guards – BPDU-/root-/loop-guard mot L2-fel.
SR-IOV – Delar upp NIC i virtuella funktioner för hög prestanda.
SSH – Krypterad fjärrterminal och filöverföring (scp/sftp).
SSL/TLS – Kryptering av applikationsprotokoll (t.ex. HTTPS).
SSID – Nätverksnamn som en AP annonserar i ett **WLAN**.
Stateful/stateless brandvägg – Med eller utan sessionskännedom.
Sticky MAC – Lås MAC-adress till en accessport.
Storm control – Begränsa broadcast/multicast-stormar.
Subnät – Del av nät med eget prefix/mask.
Switch – L2-enhet som kopplar ihop värdar i samma LAN.
Syslog – Standard för loggtransport (UDP/TCP/TLS).
Systemd – Init-/tjänstehanterare i moderna Linux.
TACACS+ – Autentisering/auktorisering för nätverksenheter.
Taggning (802.1Q) – VLAN-identifiering i Ethernet-ramar.
Terraform – “Infrastructure as Code” för moln och nät.
TFTP – Enkel filöverföring (UDP); används bl.a. vid PXE.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Throughput/Goodput – Total genomströmning vs faktisk nyttolast.
TLS – Krypteringsprotokoll för säker kommunikation.
TPM/UEFI Secure Boot – Plattformsskydd och nyckelmodul i klient/servrar.
TOTP – Engångslösenord i MFA-appar.
Trunkport – Switchport som bär flera VLAN (802.1Q-taggning).
Trunk pruning – Tillåt endast nödvändiga VLAN på en trunk.
TWT (Target Wake Time) – Strömsparfunktion i 802.11ax (WLAN).
UDLD – Upptäcker enkelriktade länkar för att undvika loopar.
UDP – O-tillförlitligt transportprotokoll (låg overhead, realtid).
UDM/UEM – Samlad enhetshantering för alla plattformar (endpoint-fokus).
UEFI Secure Boot – Startskydd som verifierar bootkomponenter.
UFW – Enkel brandväggsfrontend i Ubuntu/Debian.
ULA – Unika lokala IPv6-adresser (ej internet-routade).
Uplink – Överordnad anslutning (access→distribution→core).
UTM/NGFW – “Nästa generations” brandvägg (applikationsmedveten).
VLAN – Logisk L2-segmentering i ett LAN.
VM – Virtuell maskin (OS i hypervisor).
VPN – Krypterad tunnel mellan site/klient och nät.
VRRP – Standardiserat FHRP för redundant gateway.
vSwitch – Virtuell switch på hypervisor/värd.
VSS (Windows) – Volymskuggkopior; app-konsistens vid backup.
VXLAN – L2-overlay över IP/UDP (24-bitars VNI).
WAF – Web Application Firewall (lager-7-skydd).
WAN – Wide Area Network; nät över större avstånd.
Wazuh – Öppen plattform för säkerhet/övervakning/EDR.
WDS/MDT – Windows-avbildning/utrullning lokalt (om inte Autopilot används).
Wi-Fi (Wi-Fi Alliance) – Certifieringsprogram/varumärke för interoperabilitet enligt IEEE 802.11. Många operativsystem benämner trådlösa nätverksgränssnitt som “Wi-Fi”. I boken använder vi WLAN för själva tekniken (802.11) och reserverar “Wi-Fi” för certifieringen.
WLAN – Trådlöst lokalt nät (OS kallar ofta gränssnittet “Wi-Fi”).
WMI – Windows Management Instrumentation.
WMI-filter – GPO-filtrering baserat på systemegenskaper.
WPA2/WPA3 – Säkerhetsstandarder för **WLAN**-kryptering.
WPA2-Enterprise – 802.1X/EAP-baserad **WLAN**-autentisering.

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

WRED – Köhantering som droppar tidigt för att förebygga kollaps.
WUfB – Windows Update for Business; ringstyrda uppdateringar.
WireGuard – Modernt VPN-protokoll (snabbt, enkelt).
Wireshark – Protokollanalysator för nätverkstrafik.
ZFS – Avancerat filsystem/volymhantering (checksumma, snapshot, replika).
Zero Touch/ZTP – Automatisk provisionering/registrering utan handpåläggning.
Zero Trust – Åtkomst beslutas per begäran utifrån identitet, enhetens skick och kontext.
Zigbee/Z-Wave – Lågströmsprotokoll för IoT (hem/byggnad).

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Appendix II - CLI-REFERENS

Kodsnipplets att använda

Sammanfattning

När vi jobbar med nätverkskonfiguration handlar mycket av arbetet om planering, kontroll och felsökning. Är det något som inte fungerar, kontrollera paketets väg, från början till slut för att se var det har blivit fel.

Ta ALLTID för vana att placera ut saker organiserat och döp allting med exempelvis ip-adresser eller korrekta hostnames. Även det kommer göra felsökningen mycket enklare.

Många av de kommandon som används skrivs sällan ut som hela kommandon, då du kan använda förkortningar istället. Ofta när du söker hjälp på google m.m. så kommer förkortningarna upp. Exempel på detta är när du ansluter till en port eller skall sätta på en port. I de fallen skrivs bara förkortningar till kommandot ut

Den första fråga du får när du ansluter till enheten är: *Would you like to enter the initial configuration dialog? [yes/no]*, där svarar vi ALLTID no.

Att använda sig av ett frågetecken, ?, listar alla tänkbara kommandon som finns på nivån eller som fortsättning på kommandot du matat in. Det är bra att använda sig av om du är osäker.

För att avbryta pågående kommando trycker du **Ctrl+Shift+6**

Det sista tipset är att alltid ansluta en dator till enheten som du skall konfigurera. Detta för att minimera risken att du råkar göra ändringar i portinställningarna på enheten.

Lycka till

Grundläggande CLI-kommandon

enable

Används för att logga in i enheten som administratör. Om du har konfigurerat användarnamn och lösenord får du fylla i det efteråt.

syntax:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

enable

Exempel:

Router>enable

reload

Används för att starta om en enhet. Tänk på att alla kommandon du har lagt in kommer att nollställas, om du inte har sparat dem.

syntax:

reload

Exempel:

Router>reload

show

Är ett “grundkommando” som används för att visa inställningar och/eller anslutna enheter till både switch och router.

syntax:

Exempel:

Router>show [vad du vill visa]
Exempel:

Router>show interfaces status
Exempel:

Router>show running-config

exit

Används för att backa ur den nivå du befinner dig i. Om du har konfigurerat en port, och vill logga ut från det läget är detta kommandot du använder dig av

syntax:

exit

Exempel:

Router (config-if)# exit

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

configure terminal

Ändrar enhetens konfiguration, exempelvis portar m.m. Här behöver du vara för att göra alla inställningar.

syntax:

configure terminal eller
conf t

Exempel:

router#configure terminal eller
Router#conf t

hostname

Vi arbetar med många enheter i nätverket och något som är viktigt är att ha koll på vad de heter. Generellt döps routrar till R, och nummer i ordningen, till exempel: R1. Switchar döps på samma sätt och heter SW istället

syntax:

hostname [namn]

Exempel:

Router (config)#hostname R1

interface

Används för att ansluta till en port och göra inställningar, brukar också skrivas *int*. Här är det vanligt att använda förkortningar för porten istället för portens hela namn.

syntax:

interface [portnamn]

Exempel:

Router (config)#interface GigabitEthernet0/0
eller:
Router (config)#int g0/0

shutdown

Används för att stänga av en port. Skrivs no framför blir det omvänt, dvs no shutdown blir: sätt på porten

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

syntax:

shutdown eller sh

Exempel för att stänga av:

Router (config-if)#shutdown

eller:

Router (config-if)# sh

Exempel för att sätta på:

Router (config-if)#no shutdown

eller:

Router (config-if)# no sh

ip address

Används för att tilldela en IP-adress till porten. Vanligt att förkorta

syntax:

ip address [ip-adress subnätmask]

eller

ip ad [ip-adress subnätmask]

Exempel:

Router (config-if)#ip address 192.168.0.1 255.255.255.0

eller:

Router (config-if)#ip ad 192.168.0.1 255.255.255.0

ip route

Används för att stänga skapa en statisk route, det vill säga hur trafiken går från en routers nätverk till en annan routers nätverk. Tänk på att du oftast måste göra en statisk route åt båda håll så att datorerna i det andra nätverket kan svara också.

syntax:

ip route [nätverksadress subnätmask next_hop]

Exempel:

Router (config)#ip route 192.168.0.0 255.255.255.0 10.0.0.1

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

dhcp

Används för att skapa en dhcp-utdelning av ip-adresser. Den skapas i flera steg. Det första du anger är poolens namn. En pool är alla ip-adresser i ett subnät. Det andra du anger är nät-id för subnätet samt nätmask. Tredje saken är vilken gateway som nätverket skall ha. Avslutningsvis kan du (men är inget krav), lägga in DNS-serverns adress

syntax:

Router (config-if)#ip dhcp pool [poolnamn]

Router(dhcp-config)#network [nät-id] [subnätmask]

Router(dhcp-config)#default-router [gateway-ip]

Router(dhcp-config)#dns-server [dns ip]

Exempel:

Router (config-if)#ip dhcp pool polen

Router(dhcp-config)#network 192.168.0.0 255.255.255.0

Router(dhcp-config)#default-router 192.168.1.1

Router(dhcp-config)#dns-server 8.8.8.8

Precis som med ip-adresser, i ett VLAN, på en multilayerswitch, går det även att upprätta dhcp på en L3 switch. Kommandona är de samma, men du måste vara i VLAN-config-läge (se mer under rubriken VLAN)

dhcp-avancerat - Ranges

Om du använder dig av DHCP kan du vilja ta bort vissa adresser eller ranges

syntax:

Router(dhcp-config)# ip dhcp excluded-address [start-IP] [slut-IP]

Exempel:

Router(dhcp-config)# ip dhcp excluded-address 192.168.0.2 192.168.0.9

dhcp-avancerat - MAC-reservation

Om du använder dig av DHCP kan du vilja låsa en specifik mac-adress mot en viss ip

syntax:

Router(dhcp-config)# host [IP-adress] [subnetmask]

Router(dhcp-config)# client-identifier [MAC-adress]

Exempel:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Router(dhcp-config)# host 192.168.0.1 255.255.255.0

Router(dhcp-config)# client-identifier 0100.1A2B.3C4D.5E6F

DYNAMISKA ROUTINPROTOKOLL

Rip v2

router rip

v 2

network [IP för anslutning]

network [IP för LAN som delas ut]

no auto-summary

Exempel:

Router(config)# router rip

Router(config-router)# v 2

Router(config-router)# network 10.0.0.1

Router(config-router)# network 192.168.0.0

Router(config-router)# no auto-summary

OSPF

router ospf [process-id]

network [IP för anslutning] [Wildcardmask] area [nummer]

network [IP för anslutning] [Wildcardmask] area [nummer]

Exempel:

Router(config)# router ospf 1

Router(config-router)# network 192.168.0.0 0.0.0.255 area 0

Router(config-router)# network 200.0.0.0 0.0.0.3 area 0

Du kan styra cost för länken. Detta görs exempelvis i anslutande switch

interface [Port]

ip ospf cost [Värde]

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Exempel:

Switch(config)# interface g0/0
Switch(config-if)# ip ospf cost 1000

VLAN

Skapar ett virtuellt nätverk på en port, så kallade sub-portar. VLAN standard är att de döps till jämnt 10-tal

syntax:

int [portnamn.vlan-id]
encapsulation dot1q [vlanid]

Exempel:

Router (config)#int g0/0.10
Router(config-subif)# encap dot1q 10

Om du utför detta på en multilayerswitch behöver du inte skapa vare sig subport eller encapsulation. Där räcker det med att du skapar VLAN och tilldelar portar

syntax:

[vlan-id]
name [vlan-name]

Exempel:

Switch(config)#vlan 10
Switch(config-vlan)# name vlan10

Därefter måste du fortfarande tilldela portar (enligt nedan), ip adresser, dhcp m.m.

switchport

Används på switch för att hantera VLAN. Porten ställs antingen som access eller trunk. En trunk-port har hand om all trafik från samtliga VLAN och är ofta den port som leder tillbaka till routern. Access är den port som enheter ansluter till.

syntax:

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

switchport mode [typ av anslutning]

Exempel trunk:

Switch(config-if)#switchport mode trunk

Exempel access:

Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10

Skall du göra flera portar samtidigt kan du använda kommandot int range [startport-slutport]

ip routing

Vill du låta din multilayerswitch hantera trafiken måste du aktivera intern routing i den

syntax:

ip routing

Exempel:

Switch(config)#ip routing

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kommandon i Windows

ipconfig

Kommando för att se dina nätverksanslutningar i datorn. Den visar alla olika, ethernet, wlan, bluetooth m.m. Alla med en enkel översikt över ip adress m.m.

ipconfig /all

Kommando för att se dina nätverksanslutningar i datorn men, **all information**. Den visar alla olika, ethernet, wlan, bluetooth m.m. Alla med en enkel översikt över ip adress m.m.

ipconfig /release

Kommando för att släppa den ip-adress du har blivit tilldelad

ipconfig /renew

Kommando för att begära ny ip-adress

ipconfig /displaydns

Visar innehållet i DNS-cache

ipconfig /flushdns

Töm innehållet i DNS-cache

nslookup

Ger information om domännamn och ip-adresser

Syntax:

```
nslookup google.com
```

ping

Skickar ett ICMP-paket till en specifik ip adress, i syfte att kontrollera anslutningen

Syntax:

```
ping 192.168.0.1
```

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

tracert

Visar hur nätverkstrafiken routras till en viss ip. Du kan även (om du är ansluten till en DNS) skriva in en URL (webbadress)

Syntax:

```
tracert 192.168.0.1
```

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

Kommandon i Linux (Debian)

ip -brief addr
Kommando för att snabbt se aktiva nätverksgränssnitt och deras IP-adresser (kompakt översikt).

ip addr show
Kommando för att se dina nätverksgränssnitt i detalj: IPv4/IPv6-adresser, prefix, flags m.m.

ip link show
Visar länkstatus, MAC-adresser och MTU per interface (lager 2-vy).

ip route show
Visar routningstabellen (standardgateway, statiska rutter m.m.).
Syntax:
```
ip route show
ip route get 192.168.0.1
```

dhclient -r
Kommando för att släppa (release) din DHCP-lease på ett interface.
Syntax:
```
sudo dhclient -r -v eth0
```

dhclient
Kommando för att begära en ny DHCP-lease (renew) på ett interface.
Syntax:
```
sudo dhclient -v eth0
```

resolvectl status
Visar DNS-resolvrar och cache-statistik om **systemd-resolved** används.
Syntax:
```
resolvectl status
resolvectl query google.com
```

resolvectl flush-caches
Tömmer DNS-cache (gäller systemd-resolved; annars töm respektive

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

cache-tjänst).

dig

Ger detaljerad information om domännamn och IP-adresser (ersätter ofta nslookup).

Syntax:

```
dig +short google.com
dig -x 192.168.0.1 +short
```

ping

Skickar ICMP-ekon till en adress för att testa anslutning och rundtid.

tracepath

Visar vägen (hop för hop) som paket tar till en destination, utan root-krav.

mtr

Kombinerar ping och traceroute i realtid (fördröjning/förluster per hop).
Kan behöva installeras (`sudo apt install mtr`).

Syntax:

```
mtr google.com
mtr -6 2607:f8b0:4003:c00::6a
```